

Newton, *Principia*, with its Lean reconstruction interleaved

The 1687 Definitions, Laws and Book I Sections I-II in modern language, each
item followed by the theorems that formalize it

October 2026

Contents

About this rendering	1
How to read this version	2
How the editions differ	2
Definitions	3
Scholium	6
Axioms, or Laws of Motion	8
Scholium	17
Book I: On the Motion of Bodies	18
Section I. On the method of first and last ratios, by whose help what follows is demonstrated	18
Section II. On finding centripetal forces	33
Appendix A. Theorems anchored outside Sections I-II	115
Appendix B. The foundation: theorems with no source anchor	116

About this rendering

This is a rendering of the first edition (1687) of Newton's *Philosophiæ Naturalis Principia Mathematica*, made from the Newton Project TEI transcriptions kept in this repository: the Definitions and their Scholium (docs/m1/NATP00075.xml), the Axioms or Laws of Motion (NATP00076.xml) and Book I (NATP00077.xml). It covers the Definitions, the Laws with their Corollaries and Scholium, Book I Section I (Lemmas I-XI and the Scholium), and Book I Section II through Proposition IV and its Scholium. Propositions V-X, which close Section II, are not included.

Every item keeps Newton's number, order and concepts. Statements are put into modern mathematical language; proofs follow Newton's steps and add none. Where a mod-

ern formula helps, it follows Newton’s statement under *In modern terms*. Newton’s figures are not reproduced; the letters refer to them, and the descriptions are enough to redraw them. Square brackets mark the few notes added for a modern reader. Long physical or historical explanations are condensed, in Newton’s order.

The changes of the second edition (1713) and the third (1726) follow each item, marked **1713** and **1726**. They come from the Newton Project TEI of the 1713 Definitions, Laws and Book I (NATP00080–NATP00082, in this repository) and of the 1726 Definitions, Laws and Book I (NATP00085–NATP00087, in docs/m4). The three editions were aligned paragraph by paragraph; changes of spelling, punctuation and abbreviation are omitted, and only changes of content or of mathematical language are given. A summary of the pattern follows.

Vocabulary.

Newton	Modern meaning
ratio of equality; <i>ultimately equal</i>	$X/Y \rightarrow 1$ in the stated limit
ultimate ratio of vanishing quantities	$\lim X/Y$ as $X, Y \rightarrow 0$
first ratio of nascent quantities	$\lim X/Y$ as X, Y start from 0
duplicate, triplicate ratio	ratio of squares, of cubes
subduplicate, sesquuplicate ratio	ratio of square roots, of $3/2$ powers
<i>A applied to B</i>	A/B
<i>componendo</i>	summing proportional terms
<i>in consequentia</i>	in the direction of motion

Bold letters are vectors; $\times$ is the planar cross product $\mathbf{a} \times \mathbf{b} = a_1 b_2 - a_2 b_1$.

How to read this version

After each Definition, Law, Lemma, Proposition or Scholium, the theorems of the Lean reconstruction whose catalogued source passages cite that item are printed in a smaller monospace face, with their docstrings, statements and proofs, grouped by module in import order. The catalogue (research/formal-results.json) assigns each theorem to Newton Project paragraph anchors; a theorem citing several items is printed under the first. Items that no theorem cites say so. The Lean is a modern reconstruction in rational arithmetic with Lean 4 core only: it supplies no historical premise, and its appearance under an item records that the item motivated it, not that the item is thereby proved. Two appendices list, by statement only, the theorems anchored outside the rendered range and the foundation theorems with no source anchor, which are most of the code.

How the editions differ

1713, the limiting arguments are tightened. Lemma I now says that the quantities actually come nearer than any given difference before the end of the finite time. Lemma VI gets a new proof from the continuity of the curvature. The proofs of Lemmas VII–IX enlarge the vanishing figure so that its lines stay finite while the original shrinks.

1713, regularity premises enter the statements. Lemma X, which assumed a “regular” force, now assumes a finite force, either constant or continually increasing or decreasing. Lemma XI is restricted to curves with finite curvature at the point of contact.

1713, the sagitta becomes the measure of force. Lemma XI gains corollaries on sagittas; Proposition I gains six corollaries measuring force by the sagitta of arcs described in equal times; Proposition IV is proved anew through the versed sines of those arcs, and its Scholium compares centripetal force with gravity through the new Corollary IX.

1713, the language of proportion is defined, and an error is corrected. A new Scholium after Lemma X states what “A is as B directly and C inversely” means, and Lemma XI defines the sesquiplicate ratio. Proposition IV’s Corollary 6 is corrected and generalized to any power of the radius.

1713, the Definitions and Laws. Definition V gains a long explanation (the sling, the projectile fired from a mountain, the Moon, and the inverse problems of mathematics). Corollary I of the Laws specifies impulses impressed at one place, and Law III is extended explicitly to attractions, with a new argument for the mutual gravity of the parts of the Earth.

1726, products and quotients give way to compound ratios. Where 1713 said “multiplied into” or “applied to”, 1726 often says “jointly” or “in the ratio compounded of” (Definition VIII’s explanation, Proposition IV’s Corollaries 1 and 2, the reflection argument after Proposition IV). The Scholium to the Laws gains an explanation of Galileo’s results by equal impulses in equal particles of time.

Definitions

Definition I. *The quantity of matter is its measure, arising from its density and its volume jointly.*

In modern terms: $m \propto \rho V$.

Air of double density in double the space is quadruple; likewise snow or powders condensed by compression or melting, and all bodies condensed by any cause. A medium freely pervading the interstices of the parts, if there is one, is disregarded. This quantity is meant below by *body* or *mass*. It is known through each body’s weight, to which it is proportional, as very accurate pendulum experiments show (shown later).

1713. The example adds: in triple the space, sextuple.

No theorem of the Lean reconstruction cites this item.

Definition II. *The quantity of motion is its measure, arising from the velocity and the quantity of matter jointly.*

In modern terms: $\mathbf{p} = m\mathbf{v}$.

The motion of the whole is the sum of the motions of the parts; a body twice as large with equal velocity has double the motion, and with double velocity quadruple.

No theorem of the Lean reconstruction cites this item.

Definition III. *The inherent force of matter (vis insita) is a power of resisting, by which every body, as far as it can, perseveres in its state of rest or of uniform motion in a straight line.*

It is always proportional to the body and differs from the inertia of mass only in the way it is conceived; hence it may be called the *force of inertia*. A body exerts it only when another force impressed on it changes its state. The exertion is, from different points of view, *resistance* (the body resists the impressed force to keep its state) and *impetus* (the body, yielding with difficulty to an obstacle, tries to change the obstacle's state). Common usage gives resistance to bodies at rest and impetus to moving ones; but rest and motion, as commonly conceived, differ only relatively.

No theorem of the Lean reconstruction cites this item.

Definition IV. *Impressed force is an action exerted on a body to change its state of rest or of uniform motion in a straight line.*

It consists in the action alone and does not remain in the body afterwards; the body perseveres in any new state by the force of inertia alone. Impressed force has various origins: percussion, pressure, centripetal force.

No theorem of the Lean reconstruction cites this item.

Definition V. *Centripetal force is that by which a body is drawn, impelled, or in any way tends toward some point as to a centre.*

1713. *Bodies (plural) are drawn, impelled or tend toward the point from all sides.*

Examples: gravity, toward the centre of the Earth; magnetic force, by which iron seeks a magnet; and the force, whatever it is, by which the planets are continually drawn back from rectilinear motions and made to revolve in curved lines. Its quantity is of three kinds: absolute, accelerative and motive.

1713. The explanation is much expanded. A stone whirled in a sling endeavours to leave the hand, stretches the sling the more strongly the faster it revolves, and flies off when released; the contrary force by which the sling keeps drawing the stone back toward the hand, the centre of the orbit, is called centripetal. The same holds for all bodies moving in orbits: without a contrary force they would go off in straight lines with uniform motion. A projectile without gravity would go off in a straight line into the heavens; gravity bends it continually toward the Earth, more or less according to its gravity and velocity. A lead ball fired horizontally from a mountain top that lands two miles away would, with double or tenfold velocity, go about twice or ten times as far (air resistance removed); by increasing the velocity, the distance could be increased at will and the curvature of its path decreased, so that it would land ten, thirty or ninety degrees away, or go round the whole Earth before falling, or never fall and go off into the heavens. In the same way the Moon, by gravity or some other force urging it toward the Earth, can be drawn continually from its rectilinear course and bent into its orbit; with too small a force it would not be bent enough, with too great a force too much. It is for mathematicians to find the force that keeps a body exactly in a given

orbit with a given velocity, and conversely to find the curved path into which a body leaving a given place with a given velocity is bent by a given force.

1726. “The less its gravity *for its quantity of matter*”, and the last alternative reads simply “or go off into the heavens”.

No theorem of the Lean reconstruction cites this item.

Definition VI. *The absolute quantity of a centripetal force is its measure, greater or less according to the efficacy of the cause propagating it from the centre through the surrounding regions.*

For instance, magnetic power is greater in one magnet and less in another.

1713. “according to the size of the magnet or the intensity of its power”.

No theorem of the Lean reconstruction cites this item.

Definition VII. *The accelerative quantity of a centripetal force is its measure proportional to the velocity it generates in a given time.*

In modern terms: at a given place, $a \propto \Delta v$ for a given Δt .

The same magnet acts more strongly at a smaller distance; gravity is greater in valleys and less on high mountain tops (pendulum experiments), and less still at greater distances from the Earth (shown later). At equal distances it is the same on all sides, because all falling bodies, heavy or light, large or small, are equally accelerated once air resistance is removed.

No theorem of the Lean reconstruction cites this item.

Definition VIII. *The motive quantity of a centripetal force is its measure proportional to the motion it generates in a given time.*

In modern terms: $F \propto \Delta p$ for a given Δt .

Weight is greater in a greater body and less in a smaller; in the same body, greater near the Earth and less in the heavens. This quantity is the centripetency of the whole body, its propensity toward the centre, its weight; it is always known by the equal and contrary force that can prevent the body’s descent.

These quantities may be called briefly *absolute*, *accelerative* and *motive forces*, and referred respectively to the centre, to the place of the body, and to the body. The motive force belongs to the body as the endeavour of the whole toward the centre, compounded of the endeavours of all its parts; the accelerative force belongs to the place, as an efficacy diffused from the centre through each surrounding place to move the bodies there; the absolute force belongs to the centre, as endowed with some cause without which motive forces are not propagated, whether that cause is a central body or something else that does not appear. The concept is mathematical only; physical causes and seats of forces are not considered here.

Hence accelerative force is to motive force as velocity is to motion: the quantity of motion is velocity times quantity of matter, and motive force is accelerative force times the same quantity of matter.

In modern terms: $F = ma$ (as a proportionality).

1726. Motive force arises from the accelerative force *and* the quantity of matter *jointly*, and weight is as the body and the accelerative gravity *jointly*, where 1687 and 1713 say “multiplied into”.

The sum of the actions of the accelerative force on the single particles of a body is the motive force of the whole. Near the Earth’s surface, where accelerative gravity is the same in all bodies, weight is as the body; where accelerative gravity is smaller, weight decreases in proportion and is always as the body times accelerative gravity. Where accelerative gravity is half as large, a body half or a third as large weighs four or six times less.

Attractions and impulses are called accelerative and motive in the same sense. The words attraction, impulse or propensity toward a centre are used indifferently, the forces being considered mathematically and not physically. The reader should not take them to define a kind or mode of action or a physical cause, nor to attribute forces truly and physically to centres, which are mathematical points.

No theorem of the Lean reconstruction cites this item.

Scholium

Time, space, place and motion are not defined, being well known to all. But the common understanding conceives them only through their relation to sensible things, which breeds prejudices; to remove these, each is divided into absolute and relative, true and apparent, mathematical and common.

I. *Absolute, true and mathematical time*, of itself and by its nature without relation to anything external, flows equably, and is also called *duration*. *Relative, apparent and common time* is any sensible, external measure of duration by motion, accurate or not, used in place of true time: an hour, a day, a month, a year.

II. *Absolute space*, by its nature without relation to anything external, remains always similar and immovable. *Relative space* is any movable measure or dimension of this space, determined by our senses through its position relative to bodies and commonly taken for immovable space, such as the dimension of a subterranean, aerial or celestial space fixed by its position relative to the Earth. Absolute and relative space are the same in kind and magnitude but do not always remain numerically the same: if the Earth moves, the space of our air, always the same relative to the Earth, is now one part of absolute space and now another.

III. *Place* is the part of space a body occupies, absolute or relative according to the space. It is a part of space, not the position of the body or its surrounding surface: equal solids always have equal places, while their surfaces, for dissimilar figures, are mostly unequal; positions, properly speaking, have no quantity and are affections of places rather than places. Since the motion of the whole equals the sum of the motions of the parts, the place of the whole equals the sum of the places of the parts, and is internal and in the whole body.

IV. *Absolute motion* is the translation of a body from one absolute place to another; *relative motion*, from one relative place to another. In a ship under sail, a body’s relative place is the region of the ship it occupies, which moves with the ship; relative rest is remaining in that region; true rest is remaining in the same part

of immovable space. If the Earth truly rests, a body resting relative to the ship moves truly with the ship's velocity on the Earth; if the Earth also moves, the true motion arises partly from the Earth's true motion in immovable space and partly from the ship's relative motion; and if the body moves within the ship, from all three. Example: if the Earth's surface there moves truly east with 10010 parts of velocity, the ship west with 10 parts, and a sailor walks east with 1 part, the sailor moves truly east with 10001 parts and relatively to the Earth west with 9 parts.

Absolute time is distinguished from relative time in astronomy by the equation of time: natural days are unequal, and astronomers correct this to measure celestial motions by a truer time. Perhaps no equable motion exists by which time can be measured exactly; all motions can speed up or slow down, but the flow of absolute time cannot change. Duration is the same whether motions are fast, slow or absent. The need for the equation is shown by pendulum clocks and by the eclipses of Jupiter's satellites.

As the order of the parts of time is immutable, so is the order of the parts of space. Times and spaces are, as it were, the places of themselves and of all things: all things are placed in time as to order of succession and in space as to order of position. It is of their essence to be places, and for primary places to move is absurd. These are absolute places, and only translations from them are absolute motions.

These parts of space cannot be seen or distinguished by our senses, so sensible measures are used instead: places are defined by positions and distances from some body regarded as immovable, and motions estimated with respect to those places. In human affairs this serves; in philosophy one must abstract from the senses, since possibly no body truly rests to which places and motions could be referred.

Absolute and relative rest and motion are distinguished by their properties, causes and effects.

Properties. Bodies truly at rest rest among themselves; but since some body far away, among or beyond the fixed stars, may rest absolutely while the positions of bodies in our regions cannot tell us whether any of them keeps a given position to it, true rest cannot be defined from their mutual positions. Parts that keep given positions in their wholes share the wholes' motions: the parts of a revolving body endeavour to recede from the axis, and the impetus of a progressing body arises from the joint impetus of its parts. Hence when surrounding bodies move, bodies at rest relative to them move too, and true motion cannot be defined by translation from the neighbourhood of bodies regarded as resting; those bodies must truly rest. Likewise, when a place moves, what is in it moves with it; a body moving from a moving place shares its place's motion. Every whole and absolute motion is compounded of the motion of the body from its first place, of that place from its place, and so on, until an immovable place is reached. Whole and absolute motions can therefore be defined only by immovable places, and immovable places are only those that keep given positions to each other from infinity to infinity, which constitute immovable space.

Causes. True motion is neither generated nor changed except by forces impressed on the moving body itself; relative motion can be generated or changed without forces impressed on it, by forces impressed only on the bodies of reference. True motion always changes under forces impressed on the moving body; relative motion need not, if the same forces act on the bodies of reference so as to keep the relative position.

Effects. The forces of receding from the axis of circular motion distinguish absolute from relative motion: in purely relative circular motion they are null, in true motion greater or less according to the quantity of motion. A bucket hung by a long cord is turned until the cord is stiffly twisted, filled with water, and released. At first the surface of the water is flat, as before the motion; as the bucket gradually communicates its motion to the water, the water recedes from the middle and rises at the sides, becoming concave, and rises more as its motion grows, until it revolves in equal times with the bucket and rests relative to it. The rise shows the endeavour to recede from the axis, and by it the true and absolute circular motion of the water is known and measured, quite contrary to the relative motion: at the start, when the relative motion was greatest, there was no such endeavour and no true circular motion.

Relative quantities are therefore not the quantities whose names they bear but their sensible measures. Those who read these words in Scripture as the measured quantities do it violence, and those who confuse true quantities with their relations and common measures corrupt mathematics and philosophy.

To know the true motions of single bodies and distinguish them from apparent ones is very difficult, because the parts of immovable space do not come under the senses. But the matter is not hopeless: arguments are available partly from apparent motions, which are differences of true motions, and partly from forces, which are causes and effects of true motions. Two globes joined by a cord at a given distance and revolving about their common centre of gravity show, by the cord's tension, their endeavour to recede from the axis, from which the quantity of circular motion can be computed. Impressing equal forces on alternate faces to increase or decrease the motion, the change in tension shows which faces must be pushed to increase the motion most, namely the trailing faces, and hence the direction of the motion. In this way the quantity and direction of the circular motion could be found even in an immense vacuum with nothing external and sensible to compare. How true motions are inferred from their causes, effects and apparent differences, and conversely, is taught more fully in what follows; the treatise was composed for this purpose.

No theorem of the Lean reconstruction cites this item.

Axioms, or Laws of Motion

Law I. *Every body perseveres in its state of rest or of uniform motion in a straight line, except insofar as it is compelled by impressed forces to change that state.*

In modern terms: with no impressed force, $\mathbf{v}$ is constant.

Projectiles persevere in their motions except as air resistance retards them and gravity impels them downward. A top, whose cohering parts continually draw each other back from rectilinear motions, keeps spinning except as the air retards it. The greater bodies of planets and comets keep their progressive and circular motions longer in less resisting spaces.

Lean reconstruction: 37 theorems in 3 modules cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/Polygon/InertialControl.lean

```

theorem radius_positive (eps : Fraction) (v : Point)
  (heps : Fraction.positive eps) : Fraction.positive (radius eps v) := heps

private theorem scalar_bound (eps h x : Fraction) (K : Nat)
  (heps : Fraction.positive eps) (hK : x.num.natAbs < K)
  (hh : absLt h (eps.num, eps.den * (K : Int)),
    Int.mul_pos eps.den_pos (Int.ofNat_pos.mpr (by omega)))) :
  absLt (Fraction.mul h x) eps := by
  have hK' : (x.num.natAbs : Int) ≤ K := Int.ofNat_le.mpr (Nat.le_of_lt hK)
  have hden : 1 ≤ x.den := by
    have := x.den_pos
    omega
  have hleft : 0 ≤ (h.num.natAbs : Int) * eps.den :=
    Int.mul_nonneg (Int.ofNat_zero_le _) (Int.le_of_lt eps.den_pos)
  have hmul := Int.mul_le_mul_of_nonneg_left hK' hleft
  have hright : 0 ≤ eps.num * h.den :=
    Int.mul_nonneg (Int.le_of_lt heps) (Int.le_of_lt h.den_pos)
  have hdenmul := Int.mul_le_mul_of_nonneg_left hden hright
  have hsmall : (h.num.natAbs : Int) * eps.den * (K : Int) < eps.num * h.den := by
    unfold absLt at hh
    dsimp at hh
    simpa only [Int.mul_assoc] using hh
  have hchain : (h.num.natAbs : Int) * (x.num.natAbs : Int) * eps.den <
    eps.num * (h.den * x.den) := by
    calc
      (h.num.natAbs : Int) * (x.num.natAbs : Int) * eps.den
      = ((h.num.natAbs : Int) * eps.den) * (x.num.natAbs : Int) := by ac_rfl
      ≤ ((h.num.natAbs : Int) * eps.den) * (K : Int) := hmul
      < eps.num * h.den := hsmall
      ≤ (eps.num * h.den) * x.den := by simpa using hdenmul
      = eps.num * (h.den * x.den) := by ac_rfl
  unfold absLt Fraction.mul
  dsimp
  rw [Int.natAbs_mul]
  exact hchain

/-- For each positive rational tolerance, one explicit radius controls both
    coordinates of every rational drift at fixed velocity. -/
theorem drift_small (eps : Fraction) (v : Point)
  (heps : Fraction.positive eps) (h : Fraction)
  (hh : absLt h (radius eps v)) :
  absLt (pointScale h v).1 eps ∧ absLt (pointScale h v).2 eps := by
  have hK1 : v.1.num.natAbs < velocityBound v := by
    unfold velocityBound
    omega
  have hK2 : v.2.num.natAbs < velocityBound v := by
    unfold velocityBound
    omega
  constructor
  · exact scalar_bound eps h v.1 (velocityBound v) heps hK1 hh
  · exact scalar_bound eps h v.2 (velocityBound v) heps hK2 hh

/-- The controlled drift is the increment in the rational inertial map at
    every rational base time. This is an equivalence of represented positions,
    not an assumed curve or a limit theorem. -/
theorem inertialAt_small_increment (eps : Fraction) (p v : Point)
  (heps : Fraction.positive eps) (t h : Fraction)
  (hh : absLt h (radius eps v)) :
  pointEquiv (inertialAt (inertialAt p v t) v h)
    (inertialAt p v (Fraction.add t h)) ∧
  absLt (pointScale h v).1 eps ∧ absLt (pointScale h v).2 eps := by
  exact (inertialAt_add p v t h, drift_small eps v heps h hh)

/-- The same estimate applies to an actual zero-force end-kick cell begun at
    any rational inertial time. -/
theorem endKick_zero_small_increment (eps : Fraction) (p v : Point)
  (heps : Fraction.positive eps) (t h : Fraction)
  (hh : absLt h (radius eps v)) :
  pointEquiv (endKick h (inertialAt p v t, v) zeroPoint).1
    (inertialAt p v (Fraction.add t h)) ∧
  absLt (pointScale h v).1 eps ∧ absLt (pointScale h v).2 eps := by

```

```

have hcell := (endKick_zero h (inertialAt p v t, v)).1
have hmap := inertialAt_add p v t h
exact ((Fraction.equiv_trans hcell.1 hmap.1,
  Fraction.equiv_trans hcell.2 hmap.2), drift_small eps v heps h hh)

/-- Quantified small-time form. The witness is `radius eps v`, independent of
the base time and initial position. -/
theorem exists_uniform_inertial_radius (eps : Fraction) (v : Point)
  (heps : Fraction.positive eps) :
  ∃ delta : Fraction, Fraction.positive delta ∧
    ∀ (p : Point) (t h : Fraction), absLt h delta →
      pointEquiv (inertialAt (inertialAt p v t) v h)
        (inertialAt p v (Fraction.add t h)) ∧
        absLt (pointScale h v).1 eps ∧ absLt (pointScale h v).2 eps := by
  exact (radius eps v, radius_positive eps v heps,
    fun p t h hh => inertialAt_small_increment eps p v heps t h hh)

NewtonLimitDynamics/Polygon/InertialDefect.lean

/-- Three samples of the same finite zero-force comparison map have an
additive directed determinant. The identity is finite Fraction arithmetic. -/
theorem inertialEdge_compose (p v : Point) (s t u : Fraction) :
  Fraction.equiv
    (Fraction.add (inertialEdge p v s t) (inertialEdge p v t u))
    (inertialEdge p v s u) := by
  unfold Fraction.equiv Fraction.add inertialEdge det inertialAt pointAdd pointScale
  Fraction.mul Fraction.add
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Determinant of a degenerate connector. -/
theorem inertialEdge_self (p v : Point) (s : Fraction) :
  Fraction.equiv (inertialEdge p v s s) (Fraction.ofInt 0) := by
  unfold Fraction.equiv inertialEdge det Fraction.add Fraction.mul Fraction.ofInt
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Every finite collinear walk telescopes to its end connector. -/
theorem inertialWalk_eq_edge (p v : Point) (a b : Fraction) (times : List Fraction) :
  Fraction.equiv (inertialWalk p v a b times) (inertialEdge p v a b) := by
  induction times generalizing a with
  | nil => exact Fraction.equiv_refl _
  | cons t ts ih =>
    exact Fraction.equiv_trans (Fraction.add_equiv_left _ (ih t))
      (inertialEdge_compose p v a t b)

/-- Any finite closed polygon sampled from an inertial recurrence has zero
signed doubled determinant sum, for any start, times, and velocity. -/
theorem inertialWalk_closed (p v : Point) (a : Fraction) (times : List Fraction) :
  Fraction.equiv (inertialWalk p v a a times) (Fraction.ofInt 0) := by
  exact Fraction.equiv_trans (inertialWalk_eq_edge p v a a times)
    (inertialEdge_self p v a)

/-- The four-vertex boundary used by the finite scheduling diagnostic is a
special case of the arbitrary closed walk. -/
theorem inertial_closedBoundaryTwice (p v : Point) (a b c d : Fraction) :
  Fraction.equiv
    (closedBoundaryTwice (inertialAt p v a) (inertialAt p v b)
      (inertialAt p v c) (inertialAt p v d)) (Fraction.ofInt 0) := by
  exact Fraction.equiv_trans
    (Fraction.add_assoc (inertialEdge p v a b) (inertialEdge p v b c)
      (Fraction.add (inertialEdge p v c d) (inertialEdge p v d a)))
    (inertialWalk_closed p v a [b, c, d])

private theorem fracNeg_congr {a b : Fraction} (h : Fraction.equiv a b) :
  Fraction.equiv (fracNeg a) (fracNeg b) := by
  unfold Fraction.equiv fracNeg at *

```

```

dsimp at *
simp only [Int.neg_mul] using congrArg Neg.neg h

/-- The represented directed determinant respects point equivalence. -/
theorem det_congr {a a' b b' : Point} (ha : pointEquiv a a')
  (hb : pointEquiv b b') : Fraction.equiv (det a b) (det a' b') := by
  unfold det
  exact Fraction.add_equiv (Fraction.mul_equiv ha.1 hb.2) (fracNeg_congr (Fraction.mul_equiv ha.2 hb.1))

theorem closedBoundaryTwice_congr {a a' b b' c c' d d' : Point}
  (ha : pointEquiv a a') (hb : pointEquiv b b')
  (hc : pointEquiv c c') (hd : pointEquiv d d') :
  Fraction.equiv (closedBoundaryTwice a b c d) (closedBoundaryTwice a' b' c' d') := by
  unfold closedBoundaryTwice
  exact Fraction.add_equiv (Fraction.add_equiv (det_congr ha hb) (det_congr hb hc))
    (Fraction.add_equiv (det_congr hc hd) (det_congr hd ha))

/-- Four arbitrary actual zero-force schedules (possibly with different
partitions) have a vanishing signed determinant boundary. -/
theorem partitionMotion_closedBoundaryTwice (D : Nat) (hD : 0 < D)
  (p v : Point) (w₀ w₁ w₂ w₃ : List Nat) :
  Fraction.equiv
    (closedBoundaryTwice
      (partitionMotion D hD p v zeroPoint w₀).1
      (partitionMotion D hD p v zeroPoint w₁).1
      (partitionMotion D hD p v zeroPoint w₂).1
      (partitionMotion D hD p v zeroPoint w₃).1)
    (Fraction.ofInt 0) := by
  exact Fraction.equiv_trans
    (closedBoundaryTwice_congr
      (partitionMotion_zero_force D hD p v w₀).1
      (partitionMotion_zero_force D hD p v w₁).1
      (partitionMotion_zero_force D hD p v w₂).1
      (partitionMotion_zero_force D hD p v w₃).1)
    (inertial_closedBoundaryTwice p v
      (duration D (total w₀) hD) (duration D (total w₁) hD)
      (duration D (total w₂) hD) (duration D (total w₃) hD))

NewtonLimitDynamics/Polygon/ZeroForce.lean

private theorem pointEquiv_refl (p : Point) : pointEquiv p p :=
  (Fraction.equiv_refl _, Fraction.equiv_refl _)

private theorem inertial_add_scalar (p v s t : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add p (Fraction.mul s v)) (Fraction.mul t v))
    (Fraction.add p (Fraction.mul (Fraction.add s t) v)) := by
  have hsum : Fraction.equiv (Fraction.add (Fraction.mul s v) (Fraction.mul t v))
    (Fraction.mul (Fraction.add s t) v) := by
    exact Fraction.equiv_trans
      (Fraction.equiv_trans
        (Fraction.add_equiv_right (Fraction.mul t v) (Fraction.mul_comm s v))
        (Fraction.add_equiv_left (Fraction.mul v s) (Fraction.mul_comm t v)))
      (Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_add v s t))
        (Fraction.equiv_symm (Fraction.mul_comm (Fraction.add s t) v)))
    exact Fraction.equiv_trans (Fraction.add_assoc p (Fraction.mul s v) (Fraction.mul t v))
      (Fraction.add_equiv_left p hsum)

/-- Inertial rational-time evolution joins by addition of elapsed times. -/
theorem inertialAt_add (p v : Point) (s t : Fraction) :
  pointEquiv (inertialAt (inertialAt p v s) v t) (inertialAt p v (Fraction.add s t)) := by
  constructor <=> apply inertial_add_scalar

private theorem zero_scale (d : Fraction) : pointEquiv (pointScale d zeroPoint) zeroPoint := by
  constructor <=> unfold pointScale zeroPoint Fraction.equiv Fraction.mul Fraction.ofInt <=> dsimp <=>
  ↪ simp

private theorem pointAdd_zero (p : Point) : pointEquiv (pointAdd p zeroPoint) p := by
  constructor <=> unfold pointAdd zeroPoint Fraction.equiv Fraction.add Fraction.ofInt <=> dsimp <=> simp

/-- One actual zero-force cell is exactly a drift at its stated Fraction time,

```

```

    and leaves velocity unchanged. -/
theorem endKick_zero (d : Fraction) (state : Point × Point) :
  pointEquiv (endKick d state zeroPoint).1 (inertialAt state.1 state.2 d) ∧
  pointEquiv (endKick d state zeroPoint).2 state.2 := by
  constructor
  · exact pointEquiv_refl _
  · change pointEquiv (pointAdd state.2 (pointScale d zeroPoint)) state.2
    exact pointEquiv_trans (pointAdd_congr (pointEquiv_refl _) (zero_scale d)) (pointAdd_zero state.2)

/-- Replacing a rational time by an equivalent fraction leaves its inertial
position unchanged. -/
theorem inertialAt_time_congr (p v : Point) {s t : Fraction} (h : Fraction.equiv s t) :
  pointEquiv (inertialAt p v s) (inertialAt p v t) := by
  apply pointAdd_congr (pointEquiv_refl _)
  constructor
  · exact Fraction.equiv_trans (Fraction.mul_comm s v.1)
    (Fraction.equiv_trans (Fraction.mul_equiv_left v.1 h) (Fraction.equiv_symm (Fraction.mul_comm t
    ↪ v.1)))
  · exact Fraction.equiv_trans (Fraction.mul_comm s v.2)
    (Fraction.equiv_trans (Fraction.mul_equiv_left v.2 h) (Fraction.equiv_symm (Fraction.mul_comm t
    ↪ v.2)))

private theorem encodedPosition_zero (D : Nat) (hD : 0 < D) (s : PartitionStats) (p v : Point) :
  pointEquiv (encodedPosition D hD s p v zeroPoint)
    (inertialAt p v (duration D s.T hD)) := by
  unfold encodedPosition inertialAt
  exact pointEquiv_trans
    (pointAdd_congr (pointEquiv_refl _)
      (zero_scale (squareDuration D s.A hD)))
    (pointAdd_zero _)

private theorem encodedVelocity_zero (D : Nat) (hD : 0 < D) (s : PartitionStats) (v : Point) :
  pointEquiv (encodedVelocity D hD s v zeroPoint) v := by
  unfold encodedVelocity
  exact pointEquiv_trans
    (pointAdd_congr (pointEquiv_refl _)
      (zero_scale (duration D s.T hD)))
    (pointAdd_zero _)

/-- Every actual finite zero-force schedule reaches the inertial map at its
elapsed rational time and retains its incoming velocity. -/
theorem partitionMotion_zero_force (D : Nat) (hD : 0 < D) (p v : Point) (weights : List Nat) :
  pointEquiv (partitionMotion D hD p v zeroPoint weights).1
    (inertialAt p v (duration D (total weights) hD)) ∧
  pointEquiv (partitionMotion D hD p v zeroPoint weights).2 v := by
  have hformula := partitionMotion_formula D hD p v zeroPoint weights
  constructor
  · exact pointEquiv_trans hformula.1
    (encodedPosition_zero D hD (stats weights) p v)
  · exact pointEquiv_trans hformula.2
    (encodedVelocity_zero D hD (stats weights) v)

/-- Addition of two common-denominator elapsed times represents their summed
numerator. -/
theorem duration_add (D a b : Nat) (hD : 0 < D) :
  Fraction.equiv (Fraction.add (duration D a hD) (duration D b hD))
    (duration D (a + b) hD) := by
  unfold Fraction.equiv Fraction.add duration
  dsimp
  simp only [Int.ofNat_add, Int.mul_add, Int.add_mul]
  ac_rfl

/-- The finite recurrence itself restarts exactly: this is `foldl_append`, not
an assumption about a background curve. -/
theorem partitionMotion_append (D : Nat) (hD : 0 < D) (p v : Point)
  (ws xs : List Nat) :
  partitionMotion D hD p v zeroPoint (ws ++ xs) =
    partitionMotion D hD (partitionMotion D hD p v zeroPoint ws).1
      (partitionMotion D hD p v zeroPoint ws).2 zeroPoint xs := by
  unfold partitionMotion

```

```

rw [List.foldl_append]

private theorem inertialAt_state_congr {p p' v v' : Point} (hp : pointEquiv p p')
  (hv : pointEquiv v v') (t : Fraction) :
  pointEquiv (inertialAt p v t) (inertialAt p' v' t) :=
  pointAdd_congr hp (pointScale_congr t hv)

/-- Drifting for a rational amount `r` from the actual prefix state agrees
  with the inertial map at elapsed prefix time plus `r`. -/
theorem withinCell_position (D : Nat) (hD : 0 < D) (p v : Point)
  (pre : List Nat) (r : Fraction) :
  pointEquiv (endKick r (partitionMotion D hD p v zeroPoint pre) zeroPoint).1
    (inertialAt p v (Fraction.add (duration D (total pre) hD) r)) := by
  have hp := partitionMotion_zero_force D hD p v pre
  have hkick := endKick_zero r (partitionMotion D hD p v zeroPoint pre)
  exact pointEquiv_trans hkick.1
    (pointEquiv_trans (inertialAt_state_congr hp.1 hp.2 r)
      (inertialAt_add p v (duration D (total pre) hD) r))

/-- The same algebra applies to an in-cell physical drift; the displayed
  inequalities express that `r` lies between the prefix vertex and the next
  cell endpoint and are not used as algebraic premises. -/
theorem withinCell_position_bounded (D w : Nat) (hD : 0 < D) (p v : Point)
  (pre : List Nat) (r : Fraction)
  (_hr0 : Fraction.le (Fraction.ofInt 0) r)
  (_hrCell : Fraction.le r (duration D w hD)) :
  pointEquiv (endKick r (partitionMotion D hD p v zeroPoint pre) zeroPoint).1
    (inertialAt p v (Fraction.add (duration D (total pre) hD) r)) :=
  withinCell_position D hD p v pre r

/-- Rest is the zero-velocity specialization of the actual finite recurrence. -/
theorem partitionMotion_rest (D : Nat) (hD : 0 < D) (p : Point) (weights : List Nat) :
  pointEquiv (partitionMotion D hD p zeroPoint zeroPoint weights).1 p ∧
  pointEquiv (partitionMotion D hD p zeroPoint zeroPoint weights).2 zeroPoint := by
  have h := partitionMotion_zero_force D hD p zeroPoint weights
  constructor
  · exact pointEquiv_trans h.1 (by
    unfold inertialAt
    exact pointEquiv_trans
      (pointAdd_congr (pointEquiv_refl _) (zero_scale (duration D (total weights) hD)))
      (pointAdd_zero _))
  · exact h.2

/-- Equal rational elapsed times give equal positions even for schedules with
  different positive common denominators and different partitions. -/
theorem partitionMotion_cross_partition (D E : Nat) (hD : 0 < D) (hE : 0 < E)
  (p v : Point) (ws xs : List Nat)
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total xs) hE)) :
  pointEquiv (partitionMotion D hD p v zeroPoint ws).1
    (partitionMotion E hE p v zeroPoint xs).1 := by
  have hleft := partitionMotion_zero_force D hD p v ws
  have hright := partitionMotion_zero_force E hE p v xs
  exact pointEquiv_trans hleft.1
    (pointEquiv_trans (inertialAt_time_congr p v ht)
      (Fraction.equiv_symm hright.1.1, Fraction.equiv_symm hright.1.2))

/-- Cross-partition agreement includes the unchanged terminal velocity. -/
theorem partitionMotion_cross_partition_state (D E : Nat) (hD : 0 < D) (hE : 0 < E)
  (p v : Point) (ws xs : List Nat)
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total xs) hE)) :
  pointEquiv (partitionMotion D hD p v zeroPoint ws).1
    (partitionMotion E hE p v zeroPoint xs).1 ∧
  pointEquiv (partitionMotion D hD p v zeroPoint ws).2
    (partitionMotion E hE p v zeroPoint xs).2 := by
  constructor
  · exact partitionMotion_cross_partition D E hD hE p v ws xs ht
  · have hleft := partitionMotion_zero_force D hD p v ws
    have hright := partitionMotion_zero_force E hE p v xs
    exact pointEquiv_trans hleft.2
      (Fraction.equiv_symm hright.2.1, Fraction.equiv_symm hright.2.2)

```

```

/-- Same endpoint with different elapsed times: `slow(1) = fast(1/2)`. -/
theorem slow_fast_equal_endpoint : pointEquiv (slow scalarOne) (fast scalarHalf) := by
  decide

/-- At a common half-time the two velocity choices give different positions. -/
theorem slow_fast_different_half_time : ¬ pointEquiv (slow scalarHalf) (fast scalarHalf) := by
  decide

/-- Collinear samples of these distinct motions close with zero directed area.
  This is a geometric diagnostic only, not fixed-data nonuniqueness. -/
theorem slow_fast_collinear_closedBoundary :
  Fraction.equiv (closedBoundaryTwice zeroPoint (slow scalarHalf) (slow scalarOne)
    (fast scalarHalf)) scalarZero := by
  decide

/-- A concrete two-cell actual schedule has the expected inertial endpoint. -/
theorem slow_two_cell_schedule :
  pointEquiv (partitionMotion 2 (by decide) zeroPoint (slow scalarOne) zeroPoint [1, 1]).1
    (slow scalarOne) := by
  decide

```

Law II. *The change of motion is proportional to the motive force impressed, and takes place along the straight line in which that force is impressed.*

In modern terms: $\Delta \mathbf{p} \propto \mathbf{J}$, with $\mathbf{J}$ the impressed motive force (an impulse) and $\Delta \mathbf{p}$ parallel to it.

If a force generates some motion, double the force generates double the motion and triple the force triple, whether impressed all at once or gradually and successively. This motion, always directed with the generating force, is added to the body's earlier motion if they conspire, subtracted if contrary, and compounded obliquely if oblique, according to both directions.

No theorem of the Lean reconstruction cites this item.

Law III. *To every action there is always an equal and contrary reaction; or, the mutual actions of two bodies on each other are always equal and directed to contrary parts.*

In modern terms: $\mathbf{F}_{AB} = -\mathbf{F}_{BA}$.

Whatever presses or pulls another is pressed or pulled by it as much. If you press a stone with your finger, the finger is pressed by the stone. If a horse pulls a stone tied to a rope, the horse is pulled back equally toward the stone: the rope, stretched both ways, urges each toward the other with the same endeavour to relax. If a body striking another changes its motion, it undergoes the same change of its own motion in the contrary direction. The equal changes are of motions, not velocities (for bodies not otherwise impeded); the changes of velocities, also contrary, are inversely proportional to the bodies.

In modern terms: $m_A \Delta \mathbf{v}_A = -m_B \Delta \mathbf{v}_B$.

1713. The explanation ends: this law also holds in attractions, as will be proved in the next Scholium.

No theorem of the Lean reconstruction cites this item.

Corollary I. *A body acted on by two forces jointly describes the diagonal of a parallelogram in the same time in which it would describe the sides by the forces separately.*

Proof. Let the body in a given time be carried by force M alone from A to B , and by force N alone from A to C ; complete the parallelogram $ABDC$. Since N acts along AC , parallel to BD , it does not change the velocity of approach to the line BD generated by M . So the body reaches the line BD in the same time whether N acts or not, and at the end of that time it is somewhere on BD . By the same argument it is somewhere on CD , so it is at their intersection D .

In modern terms: $\overrightarrow{AD} = \overrightarrow{AB} + \overrightarrow{AC}$.

1713. The forces M and N are *impressed at the place A*, the motion from A to B is *uniform*, the invariance of the velocity of approach is *by Law II*, and the proof ends: the body proceeds in a straight line from A to D by Law I. The corollary is thereby stated for impulses given at one place, followed by inertial motion.

No theorem of the Lean reconstruction cites this item.

Corollary II. *Hence the composition of a direct force AD out of any oblique forces AB and BD, and conversely the resolution of any direct force AD into any oblique forces AB and BD. This is abundantly confirmed by mechanics.*

Weights A and P hung by threads from unequal spokes OM , ON of a wheel balance when they are inversely as the perpendicular distances OK , OL of their threads from the centre (the property of the balance, the lever and the wheel and axle): resolving the weight A , represented by AD , into AC along the spoke OD , which does nothing to turn the wheel, and DC perpendicular to it, gives $P : A = DC : DA = OK : OL$ by the similar triangles ADC , DOK . A weight p resting partly on an inclined plane and partly hung by a thread resolves likewise, which gives the tensions of oblique threads, the forces of the wedge between the faces of a split body, of the mallet, and of the screw, which is a wedge driven by a lever. From these follow the forces of all machines built from wheels, drums, pulleys, levers, taut strings and weights, and of the muscles moving the bones of animals.

No theorem of the Lean reconstruction cites this item.

Corollary III. *The quantity of motion, obtained by taking the sum of the motions directed to the same side and the difference of those directed to contrary sides, is not changed by the actions of bodies on each other.*

In modern terms: $\sum_i m_i \mathbf{v}_i$ is invariant under mutual interactions.

Proof. Action and reaction are equal (Law III), so by Law II they produce equal changes of motion toward contrary sides. If the motions are to the same side, whatever is added to the motion of the body fleeing is taken from the body pursuing, so the sum stays the same; if the bodies meet, equal amounts are taken from both, so the difference stays the same.

Example: a sphere A three times as large as a sphere B moves with 2 parts of velocity, and B follows on the same line with 10, so the motions are 6 and 10, total 16. Whatever A gains in the collision, 3, 4 or 5 parts, B loses, leaving A with 9, 10 or 11 and B with 7, 6 or 5; if A gains 9 to 12, B continues with 1, stops, or recoils with 1 or 2. The sum of conspiring motions or the difference of contrary ones is always 16. For non-spherical bodies or oblique collisions, find the plane touching both at the point of

contact, resolve each motion (Corollary II) into components perpendicular and parallel to that plane, keep the parallel components, and give the perpendicular components equal and contrary changes so that the sum or difference is unchanged. Collisions also produce rotations about the bodies' own centres, which are not considered here.

No theorem of the Lean reconstruction cites this item.

Corollary IV. *The common centre of gravity of two or more bodies does not change its state of motion or rest by the actions of the bodies among themselves; therefore the common centre of gravity of all bodies acting on each other (external actions and impediments excluded) either rests or moves uniformly in a straight line.*

In modern terms: with no external forces, $\mathbf{R} = \sum_i m_i \mathbf{r}_i / \sum_i m_i$ satisfies $\ddot{\mathbf{R}} = 0$.

1713. "The common centre of gravity of two or more bodies". The proof treats points moving in one plane and adds that the same holds if their motions are not in one plane. **1726** cites Lemma XXIII and its corollary.

Proof. If two points move uniformly in straight lines and their distance is divided in a given ratio, the dividing point rests or moves uniformly in a straight line (proved later in Lemma XXIII for the plane; the same holds in space). So if any number of bodies move uniformly in straight lines, the common centre of any two rests or moves uniformly, since it divides in a given ratio the line joining their centres; likewise the common centre of these two and any third, and so on to infinity. Hence in a system of bodies free of mutual actions and of all external forces, each moving uniformly in a straight line, the common centre of all rests or moves uniformly in a straight line. Further, for two bodies acting on each other, their distances from the common centre are inversely as the bodies, so their relative motions of approach to or recession from the centre are equal; the centre is neither advanced nor retarded by equal changes of motion in contrary directions, that is, by the mutual actions. In a system of many bodies, the common centre of any two acting on each other keeps its state, and the common centre of the rest is unaffected by that action; the distance between these two centres is divided by the common centre of all inversely as the total bodies whose centres they are; so the common centre of all keeps its state. All mutual actions are between pairs of bodies or compounded of such, so the common centre of all is never changed by them.

No theorem of the Lean reconstruction cites this item.

Corollary V. *The motions of bodies contained in a given space are the same among themselves whether that space rests or moves uniformly in a straight line without circular motion.*

Proof. The differences of motions toward the same side and the sums of motions toward contrary sides are the same at the start in both cases (by hypothesis), and from them arise the collisions and impulses by which the bodies strike each other; so by Law II the effects are equal in both cases, and the motions among themselves stay equal. A clear experiment confirms it: all motions behave the same in a ship at rest or moving uniformly in a straight line.

In modern terms: the laws are invariant under $\mathbf{r}' = \mathbf{r} - \mathbf{u}t$, $t' = t$, with $\mathbf{u}$ constant.

No theorem of the Lean reconstruction cites this item.

Corollary VI. *If bodies move in any way among themselves and are urged along parallel lines by equal accelerative forces, they all continue to move among themselves as if those forces did not act.*

Proof. The forces, acting equally (in proportion to the quantities of the bodies) along parallel lines, move all bodies equally as to velocity (Law II), and so never change their positions and motions among themselves.

No theorem of the Lean reconstruction cites this item.

Scholium

These principles are received by mathematicians and confirmed by many experiments. By the first two Laws and Corollaries Galileo found that the descent of heavy bodies is in the duplicate ratio of the time and that the motion of projectiles is in a parabola, as experiment agrees except for the small retardation by air. On the same Laws and Corollaries depend the demonstrations about the times of oscillating pendulums, confirmed daily by clocks. From these and the third Law, Wren, Wallis and Huygens independently found the rules of the collision and reflection of two bodies and communicated them to the Royal Society at about the same time, in full agreement. Wren confirmed them by an experiment with pendulums before the Society, and Mariotte soon devoted a whole book to it. For exact agreement one must allow for the air's resistance and for the elastic force of the colliding bodies. Newton describes how to correct a pendulum-collision experiment for air resistance by measuring the loss over swings, and reports his own trials with bodies of various hardness, including tightly wound wool balls: the bodies separate with a relative velocity in a given ratio to the relative velocity of approach, and the rules hold for soft bodies as well as hard ones.

1713. The description of the pendulum experiment is made exact: the arcs are marked so that $RS = TV$ and $RS : ST = 3 : 2$, and the equal changes of motion are stated for the bodies.

1726. A passage is inserted after the mention of Galileo. Uniform gravity, acting equally in equal particles of time, impresses equal forces on a falling body and generates equal velocities; in the whole time it impresses the whole force and generates the whole velocity, proportional to the time; and the spaces described in proportional times are as the velocities and times jointly, that is, in the duplicate ratio of the times. For a body thrown upward, uniform gravity impresses forces and takes away velocities proportional to the times; the times of ascent to the greatest heights are as the velocities to be taken away, and the heights as the velocities and times jointly, or in the duplicate ratio of the velocities. A body thrown along any straight line compounds the motion from the projection with the motion from gravity: if by projection alone it would describe AB in a given time and by falling alone the height AC , completing the parallelogram $ABDC$ puts it at D at the end of the time, and the curve AED it describes is a parabola touching AB at A whose ordinate BD is as AB^2 .

For attractions, suppose two bodies A and B attract each other and an obstacle is placed between them. If A were drawn toward B more than B toward A , the obstacle would be pressed more by A than by B and would not stay in equilibrium; the stronger

pressure would make the system of the two bodies and the obstacle move toward B and, accelerating forever in free space, go off to infinity, which is absurd and contrary to the first Law. So the bodies press the obstacle equally and attract each other equally. Newton tried this with a lodestone and iron floating in separate vessels on still water: neither pushed the other, and they came to rest in equilibrium.

1713. A new paragraph: gravity between the Earth and its parts is mutual. Cut the Earth by any plane EG into parts EGF and EGI . Cut the larger part EGI by a parallel plane HK into $EGKH$ and HKI , with HKI equal to EGF . The middle part $EGKH$ inclines by its own weight toward neither extreme part and rests between them in equilibrium; the extreme part HKI presses with its whole weight on the middle part and urges it toward EGF . So the force with which EGI tends toward EGF equals the weight of HKI , that is, of EGF , and the two parts weigh equally on each other. Otherwise the whole Earth, floating in the free aether, would yield to the greater weight and go off to infinity.

As bodies are equipollent in collision when their velocities are inversely as their inherent forces, so in mechanical instruments agents are equipollent and sustain each other when their velocities, estimated along the forces, are inversely as the forces. So for the balance, the pulley, trains of wheels, the screw and the wedge. The use of machines lies only in increasing force by diminishing velocity and conversely; this solves, for every suitable instrument, the problem of moving a given weight by a given force. If the action of the agent is estimated by its force and velocity jointly, and the reaction of the resistance by the velocities of its parts and their forces of resisting (from friction, cohesion, weight and acceleration), action and reaction are always equal in every use of instruments. Mechanics is not the subject here; this only shows how widely and how surely the third Law holds.

No theorem of the Lean reconstruction cites this item.

Book I: On the Motion of Bodies

Section I. On the method of first and last ratios, by whose help what follows is demonstrated

Lemma I. *Quantities, and also ratios of quantities, which in a given finite time tend constantly to equality, and before the end of that time approach nearer to each other than by any given difference, become ultimately equal.*

In modern terms: if $X(t), Y(t)$ on $[0, T)$ approach each other so that for every $\varepsilon > 0$ there is $t < T$ beyond which $|X - Y| < \varepsilon$, then their ultimate values at T are equal.

Proof. If not, let their ultimate difference be $D > 0$. Then they cannot approach nearer to equality than by the given difference D , contrary to the hypothesis.

1713. The statement reads “in any finite time” and replaces “can approach nearer” by “before the end of that time approach nearer than by any given difference”: the approach is asserted to happen within the time, not merely to be possible. The proof begins “If you deny it, let them be ultimately unequal”.

No theorem of the Lean reconstruction cites this item.

Lemma II. *If in any figure $AacE$, bounded by the straight lines Aa , AE and the curve acE , any number of parallelograms Ab , Bc , Cd , ... are inscribed on equal bases AB , BC , CD , ... with sides Bb , Cc , Dd , ... parallel to the side Aa , and the parallelograms $aKbl$, $bLcm$, $cMdn$, ... are completed; and if then the width of the parallelograms is diminished and their number increased indefinitely: the ultimate ratios which the inscribed figure $AKbLcMdD$, the circumscribed figure $AalbmcndoE$ and the curvilinear figure $AabcdE$ have to one another are ratios of equality.*

In modern terms: for the region under a monotone curve over $[a, b]$, with lower and upper step sums L_n , U_n on n equal subintervals and area $\mathcal{A}$, the ratios $L_n : U_n : \mathcal{A}$ tend to $1 : 1 : 1$. [In Newton's figure the curve is monotone, and the curvilinear figure's area is taken as given.]

Proof. The difference of the inscribed and circumscribed figures is the sum of the parallelograms $Kl + Lm + Mn + Do$, that is (the bases being equal) the rectangle under one base Kb and the sum of the heights Aa , the rectangle $ABla$. Since its width AB is diminished indefinitely, it becomes less than any given rectangle. So by Lemma I the inscribed and circumscribed figures, and still more the curvilinear figure between them, become ultimately equal.

In modern terms: $U_n - L_n = \Delta x (f(a) - f(b)) \rightarrow 0$.

No theorem of the Lean reconstruction cites this item.

Lemma III. *The same ultimate ratios are also ratios of equality when the widths AB , BC , CD , ... of the parallelograms are unequal and all are diminished indefinitely.*

Proof. Let AF equal the greatest width and complete the parallelogram $FAaf$. It is greater than the difference of the inscribed and circumscribed figures, and as its width AF is diminished indefinitely it becomes less than any given rectangle.

In modern terms: $U - L \leq \max_i \Delta x_i (f(a) - f(b)) \rightarrow 0$ as the mesh tends to 0.

Corollary 1. Hence the ultimate sum of the vanishing parallelograms coincides in every part with the curvilinear figure.

Corollary 2. Still more does the rectilinear figure bounded by the chords ab , bc , cd , ... of the vanishing arcs coincide ultimately with the curvilinear figure.

Corollary 3. So does the rectilinear figure circumscribed by the tangents of the same arcs.

Corollary 4. And therefore these ultimate figures (as to their perimeters acE) are not rectilinear, but curvilinear limits of rectilinear figures.

Lean reconstruction: 6 theorems in 1 module cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/Polygon/Enclosure.lean

```
theorem isum_mono (f g : Nat → Int) (n : Nat) (h : ∀ i, i < n → f i ≤ g i) :
  isum f n ≤ isum g n := by
  induction n with
  | zero => exact Int.le_refl _
  | succ n ih =>
    simp only [isum]
```

```

    exact Int.add_le_add (ih (fun i hi => h i (by omega))) (h n (by omega))

theorem isum_mul (f : Nat → Int) (c : Int) (n : Nat) :
  isum (fun i => c * f i) n = c * isum f n := by
  induction n with
  | zero => simp [isum]
  | succ n ih => simp [isum, ih, Int.mul_add]

theorem telescoping (height : Nat → Int) (n : Nat) :
  isum (fun i => height (i+1) - height i) n = height n - height 0 := by
  induction n with
  | zero => simp [isum]
  | succ n ih => simp only [isum, ih]; omega

/-- Lemmas II/III's finite rectangle estimate, for a monotone patch. Widths
    may be unequal; every width must obey the SAME maximum. Multiplication
    measures rectangle area. A curve enclosed by these rectangles is an
    additional geometric hypothesis, not supplied by this arithmetic result. -/
theorem rectangle_gap_bound (width height : Nat → Int) (maxWidth : Int) (n : Nat)
  (hw : ∀ i, i < n → width i ≤ maxWidth)
  (hh : ∀ i, i < n → height i ≤ height (i+1)) :
  isum (fun i => width i * (height (i+1) - height i)) n ≤
    maxWidth * (height n - height 0) := by
  have h := isum_mono (fun i => width i * (height (i+1) - height i))
  (fun i => maxWidth * (height (i+1) - height i)) n (by
    intro i hi
    exact Int.mul_le_mul_of_nonneg_right (hw i hi) (by have := hh i hi; omega))
  rw [isum_mul, telescoping] at h
  exact h

theorem enclosed_gap_vanishes (gap budget : Fraction → Fraction)
  (hbudget : Vanishes budget)
  (henclose : Near Fraction.magnitudes (fun mesh => Fraction.le (gap mesh) (budget mesh))) :
  Vanishes gap := by
  intro epsilon hepsilon
  obtain {d, hd, h} := near_and Fraction.magnitudes _ _ henclose (hbudget epsilon hepsilon)
  exact {d, hd, fun mesh hm hmd =>
    Fraction.magnitudes.lt_of_le_lt (h mesh hm hmd).1 (h mesh hm hmd).2}

/-- Conditional transfer of polygon area ratios to enclosed sector area ratios.
    Lower and upper limits are geometric premises. No trajectory existence or
    identification with continuous force follows from this type. -/
theorem sector_ratio_reconstruction (sector inner outer : Fraction → Fraction)
  (c : Fraction) (hin : Ultimate Fraction.magnitudes inner c)
  (hout : Ultimate Fraction.magnitudes outer c)
  (henclose : Near Fraction.magnitudes (fun mesh =>
    Fraction.le (inner mesh) (sector mesh) ∧ Fraction.le (sector mesh) (outer mesh))) :
  Ultimate Fraction.magnitudes sector c :=
  enclosure_reconstruction Fraction.magnitudes sector inner outer c hin hout henclose

```

Lemma IV. *If in two figures $AacE$, $PprT$ two series of parallelograms are inscribed (as above), equal in number, and when the widths are diminished indefinitely the ultimate ratios of the parallelograms of one figure to those of the other, each to each, are the same: the two figures are to each other in that same ratio.*

Proof. As the parallelograms are to each other one by one, so (*componendo*) is the sum of all to the sum of all, and so is figure to figure, each figure being to its sum in the ratio of equality (Lemma III).

In modern terms: if $a_i/b_i \rightarrow r$ for all i as the mesh tends to 0, then $\lim \sum a_i / \lim \sum b_i = r$. [The convergence is tacitly uniform in i .]

Corollary. Hence if two quantities of any kind are divided into the same number of parts, and those parts, as their number is increased and their magnitude diminished indefinitely, have a given ratio to each other, first to first, second to second and so on

in order, the wholes are to each other in that given ratio. For if parallelograms are taken in the figures of this Lemma proportional to the parts, the sums of the parts are always as the sums of the parallelograms, and so, in the limit, in the ultimate ratio of parallelogram to parallelogram, that is (by hypothesis) of part to part.

No theorem of the Lean reconstruction cites this item.

Lemma V. *All corresponding sides of similar figures, curvilinear as well as rectilinear, are proportional, and the areas are in the duplicate ratio of the sides.*

In modern terms: under a similarity of ratio λ , lengths scale by λ and areas by λ^2 .

No theorem of the Lean reconstruction cites this item.

Lemma VI. *If any arc AB , given in position, is subtended by its chord AB , and at some point A in the middle of a continuous curvature it is touched by a straight line AD produced both ways; and if then the points A and B approach each other and coincide: the angle BAD contained by the chord and the tangent is diminished indefinitely and ultimately vanishes.*

In modern terms: at a point where the curve has a tangent, the direction of the chord AB tends to the direction of the tangent as $B \rightarrow A$.

Proof. Produce AB to b and AD to d . As A and B coincide, no part AB of the line Ab lies any longer inside the curve, so the line Ab either coincides with the tangent Ad or passes between the tangent and the curve. The latter is contrary to the nature of curvature, so the former holds.

1713. A new proof replaces this one: if the angle does not vanish, the arc AB will contain with the tangent AD an angle equal to a rectilinear angle, and so the curvature at A will not be continuous, contrary to the hypothesis. **1726** names the arc ACB , through an intermediate point C , here and in Lemmas VII and VIII.

No theorem of the Lean reconstruction cites this item.

Lemma VII. *Under the same suppositions, the ultimate ratio of the arc, the chord and the tangent to each other is the ratio of equality.*

Proof. Produce AB and AD to b and d , draw bd parallel to the secant BD , and let the arc Ab be similar to the arc AB . As A and B coincide, the angle dAb vanishes (Lemma VI), so the straight lines Ab , Ad and the arc between them coincide and are equal. Hence AB , AD and the arc AB , always proportional to these, ultimately have the ratio of equality.

In modern terms: $\text{arc } AB / \text{chord } AB \rightarrow 1$ and $AD / \text{chord } AB \rightarrow 1$ as $B \rightarrow A$.

1713. The proof is reorganized as a rescaling: while B approaches A , the lines AB , AD are imagined always produced to *distant* points b , d , so that the enlarged lines Ab , Ad and the arc between them stay *always finite* while the angle dAb vanishes; they coincide and are equal, and the proportional small lines AB , AD and the arc AB *vanish* with ultimate ratio of equality.

Corollary 1. If through B a line BF is drawn parallel to the tangent, meeting any straight line AF through A in F , then BF ultimately has the ratio of equality to the

vanishing arc AB , because, completing the parallelogram $AFBD$, it always has that ratio to AD .

Corollary 2. If through B and A more lines BE , BD , AF , AG are drawn cutting the tangent AD and its parallel BF , the ultimate ratio of all the abscissas AD , AE , BF , BG , and of the chord and arc AB , to each other is the ratio of equality.

Corollary 3. Therefore all these lines may be used for one another in any argument about ultimate ratios.

No theorem of the Lean reconstruction cites this item.

Lemma VIII. *If the given straight lines AR , BR form with the arc AB , the chord AB and the tangent AD three triangles RAB , RAB , RAD (the first with the arc as a side), and then the points A and B approach each other: the ultimate form of the vanishing triangles is that of similarity, and their ultimate ratio is that of equality.*

Proof. Produce AB , AD , AR to b , d , r ; draw rbd parallel to RD , and the arc Ab similar to the arc AB . As A and B coincide, the angle bAd vanishes, so the three triangles rAb , rAb , rAd coincide and are therefore similar and equal. Hence RAB , RAB , RAD , always similar and proportional to these, become ultimately similar and equal to each other.

1713. The same rescaling: AB , AD , AR are always produced to distant points b , d , r , and the three triangles rAb , rAb , rAd stay *always finite* while they coincide.

Corollary. Hence those triangles may be used for one another in any argument about ultimate ratios.

No theorem of the Lean reconstruction cites this item.

Lemma IX. *If a straight line AE and a curve AC , given in position, cut each other at a given angle A , and to that line, at another given angle, ordinates BD , EC are applied meeting the curve in B , C , and then the points B , C approach the point A : the areas of the triangles ADB , AEC will be ultimately to each other in the duplicate ratio of the sides.*

In modern terms: $\text{area}(ADB)/\text{area}(AEC) \rightarrow (AD/AE)^2$ as $B, C \rightarrow A$, the triangles having the curved side.

Proof. On AD produced take Ad , Ae proportional to AD , AE , and erect ordinates db , ec parallel and proportional to DB , EC . Produce AC to c , draw the curve Abc similar to ABC , and the straight line Ag touching both curves at A and cutting the ordinates in F , G , f , g . Then let B and C coincide with A : the angle cAg vanishes, the curvilinear areas Abd , Ace coincide with the rectilinear Afd , Age , and so (Lemma V) are in the duplicate ratio of the sides Ad , Ae . But the areas ABD , ACE are always proportional to these, and the sides AD , AE to these sides. So the areas ABD , ACE are ultimately in the duplicate ratio of the sides AD , AE .

1713. The enlarged figure is again kept finite: while B , C approach A , the line AD is always produced to distant points d , e , and the points B , C coalesce with A while the length Ae stays fixed.

Lean reconstruction: 1 theorem in 1 module cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/Principial687/ConstructedRatio.lean

```
theorem constructed_quadratic_bridge (s : Fraction → Fraction) (c : Fraction)
  (hc : positive c) (p : ContactEnclosure s c) :
  DeMotu1684.QuadraticInitialDeflection magnitudes (ratio s) c := by
  refine (hc, enclosure_reconstruction magnitudes _ _ c ? _ ? p.mechanical_enclosure)
  · exact constructed_triangle_limit p.lowerSlope c p.lower_contact
  · exact constructed_triangle_limit p.upperSlope c p.upper_contact
```

Lemma X. *The spaces which a body describes, urged by any regular force, are at the very beginning of the motion in the duplicate ratio of the times.*

1713. *The spaces which a body describes, urged by any finite force, whether that force is determinate and immutable or is continually increased or continually decreased, are at the very beginning of the motion in the duplicate ratio of the times.* The 1687 “regular force” becomes a finite force that is constant or monotone; Corollaries 1 and 2 are reworded accordingly (“by any equal forces”, “measured by the distances of the bodies from those places of the similar figures”).

In modern terms: $s(t_1)/s(t_2) \rightarrow (t_1/t_2)^2$ as $t_1, t_2 \rightarrow 0$; that is, $s(t) \sim c t^2$ at the start.

Proof. Represent the times by the lines AD , AE and the velocities generated by the ordinates DB , EC . The spaces described with these velocities are as the areas ABD , ACE described by these ordinates, that is, at the very beginning of the motion (Lemma IX), in the duplicate ratio of the times AD , AE .

In modern terms: $s(t) = \int_0^t v(\tau) d\tau$, the area under the velocity graph, whose initial piece is the curvilinear triangle of Lemma IX.

Corollary 1. Hence the errors of bodies describing similar parts of similar figures in proportional times, generated by equal forces similarly applied to the bodies in those parts, and measured from the places of the figures which the bodies would reach in the same proportional times without those forces, are very nearly as the squares of the times in which they are generated.

Corollary 2. The errors generated by proportional forces similarly applied are as the forces and the squares of the times jointly.

In modern terms: the deviation from the force-free path is $\delta \approx \frac{1}{2} a t^2$, so $\delta \propto F t^2$ for a given body.

1713. Three new corollaries and a Scholium.

Corollary 3. The same holds for any spaces that bodies describe under different forces: at the very beginning of the motion they are as the forces and the squares of the times jointly, $s \propto F t^2$.

Corollary 4. So the forces are as the spaces described at the very beginning directly and the squares of the times inversely, $F \propto s/t^2$.

Corollary 5. And the squares of the times are as the spaces directly and the forces inversely, $t^2 \propto s/F$.

Scholium. When indeterminate quantities of different kinds are compared, and one is said to be as another directly or inversely, the meaning is that the first increases or decreases in the same ratio as the second or as its reciprocal. When one is said to be as two or more others directly or inversely, it increases or decreases in the ratio compounded of the ratios in which the others, or their reciprocals, increase or decrease. So “ A is as B directly and C directly and D inversely” means that A changes in the same ratio as $B \cdot C/D$, that is, A and BC/D are in a given ratio.

Lean reconstruction: 15 theorems in 5 modules cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/Contact/AreaCoefficient.lean

```
private theorem zero_of_scaled_bounds (delta C : Int)
  (h : ∀ n : Nat, 0 < n → (n : Int)*delta ≤ C ∧ (n : Int)*(-delta) ≤ C) : delta = 0 := by
  let n := C.natAbs + 1
  have hn : 0 < n := by omega
  obtain (hu, hl) := h n hn
  have habs : C ≤ (C.natAbs : Int) := Int.le_natAbs
  have hnlarge : C < (n : Int) := by dsimp [n]; omega
  have hnp : 0 ≤ (n : Int) := by omega
  by_cases hp : 0 < delta
  · have hd : 1 ≤ delta := by omega
    have hm := Int.mul_le_mul_of_nonneg_left hd hnp
    simp only [Int.mul_one] at hm
    omega
  · by_cases hm : delta < 0
    · have hd : 1 ≤ -delta := by omega
      have hb := Int.mul_le_mul_of_nonneg_left hd hnp
      simp only [Int.mul_one] at hb
      omega
    · omega

/-- Rectangle enclosure determines the normalized parabolic area as 1/3.
    Only finite sums and rational order are used. The geometric assertion that
    a selected curve's area obeys these rectangle enclosures is the premise. -/
theorem parabolic_area_coefficient (area : Fraction)
  (enclosed : ∀ n : Nat, (hn : 0 < n) →
    Fraction.le (lowerParabola n hn) area ∧ Fraction.le area (upperParabola n hn)) :
  Fraction.equiv area (1, 3, by decide) := by
  have heq : 3*area.num-area.den = 0 := by
    apply zero_of_scaled_bounds _ (2*area.den)
    intro n hn
    let N : Int := n
    let S : Int := nsum (fun i => i*i) n
    have hN : 0 < N := Int.ofNat_lt.mpr hn
    have hNN := Int.mul_pos hN hN
    obtain (hl, hu) := enclosed n hn
    change S*area.den ≤ area.num*(N*N*N) at hl
    change area.num*(N*N*N) ≤ (S+N*N)*area.den at hu
    have hi := congrArg (fun x : Nat => (x : Int)) (quadratic_rectangles n)
    simp only [Int.ofNat_add, Int.ofNat_mul] at hi
    have hid := congrArg (fun x : Int => x*area.den) hi
    have ident : 6*(S*area.den)+3*(N*N*area.den) =
      2*(N*N*N*area.den)+N*area.den := by
      simpa only [Int.add_mul, Int.mul_assoc] using hid
    have hlo := Int.mul_le_mul_of_nonneg_left hl (by decide : (0 : Int) ≤ 6)
    have hhi := Int.mul_le_mul_of_nonneg_left hu (by decide : (0 : Int) ≤ 6)
    simp only [Int.add_mul, Int.mul_add] at hhi
    have wpos := Int.mul_pos hN area.den_pos
    have wle : N*area.den ≤ N*N*area.den := by
      have hge : (1 : Int) ≤ N := by omega
      have ht := Int.mul_le_mul_of_nonneg_right hge (Int.le_of_lt wpos)
      simpa only [Int.one_mul, Int.mul_assoc] using ht
    have hupper : 6*(area.num*(N*N*N))-2*(N*N*N*area.den) ≤ 4*(N*N*area.den) := by omega
    have hlower : 2*(N*N*N*area.den)-6*(area.num*(N*N*N)) ≤ 4*(N*N*area.den) := by omega
```

```

have factor : 2*(N*(3*area.num-area.den))*(N*N) =
  6*(area.num*(N*N*N))-2*(N*N*N*area.den) := by
  simp only [Int.mul_sub, Int.sub_mul]
  have six : (6 : Int) = 2*3 := by decide
  rw [six]
  congr 1 <|> ac_rfl
have factor' : 2*(N*(area.den-3*area.num))*(N*N) =
  2*(N*N*N*area.den)-6*(area.num*(N*N*N)) := by
  simp only [Int.mul_sub, Int.sub_mul]
  have six : (6 : Int) = 2*3 := by decide
  rw [six]
  congr 1 <|> ac_rfl
have rhs : 4*(N*N*area.den) = (4*area.den)*(N*N) := by ac_rfl
rw [← factor, rhs] at hupper
rw [← factor', rhs] at hlower
have hu' := Int.le_of_mul_le_mul_right hupper hNN
have hl' := Int.le_of_mul_le_mul_right hlower hNN
have hnneg : N*(-(3*area.num-area.den)) = N*(area.den-3*area.num) := by
  congr 1
  omega
change N*(3*area.num-area.den) ≤ 2*area.den ∧ N*(-(3*area.num-area.den)) ≤ 2*area.den
rw [hnneg]
omega
unfold Fraction.equiv
dsimp
omega

/-- Normalized linear velocity area is 1/2, from finite rectangles alone. -/
theorem linear_area_coefficient (area : Fraction)
  (enclosed : ∀ n : Nat, (hn : 0 < n) →
    Fraction.le (lowerLinear n hn) area ∧ Fraction.le area (upperLinear n hn)) :
  Fraction.equiv area (1, 2, by decide) := by
  have heq : 2*area.num-area.den = 0 := by
    apply zero_of_scaled_bounds _ area.den
  intro n hn
  let N : Int := n
  let S : Int := nsum (fun i => i) n
  have hN : 0 < N := Int.ofNat_lt.mpr hn
  obtain {hl, hu} := enclosed n hn
  change S*area.den ≤ area.num*(N*N) at hl
  change area.num*(N*N) ≤ (S+N)*area.den at hu
  have hi := congrArg (fun x : Nat => (x : Int)) (linear_rectangles n)
  simp only [Int.ofNat_add, Int.ofNat_mul] at hi
  have hid := congrArg (fun x : Int => x*area.den) hi
  have ident : 2*(S*area.den)+N*area.den = N*N*area.den := by
    simp only [Int.add_mul, Int.mul_assoc] using hid
  have hlo := Int.mul_le_mul_of_nonneg_left hl (by decide : (0 : Int) ≤ 2)
  have hhi := Int.mul_le_mul_of_nonneg_left hu (by decide : (0 : Int) ≤ 2)
  simp only [Int.add_mul, Int.mul_add] at hhi
  have hupper : 2*(area.num*(N*N))-N*N*area.den ≤ N*area.den := by omega
  have hlower : N*N*area.den-2*(area.num*(N*N)) ≤ N*area.den := by omega
  have factor : (N*(2*area.num-area.den))*N = 2*(area.num*(N*N))-N*N*area.den := by
    simp only [Int.mul_sub, Int.sub_mul]
    congr 1 <|> ac_rfl
  have factor' : (N*(area.den-2*area.num))*N = N*N*area.den-2*(area.num*(N*N)) := by
    simp only [Int.mul_sub, Int.sub_mul]
    congr 1 <|> ac_rfl
  have rhs : N*area.den = area.den*N := by ac_rfl
  rw [← factor, rhs] at hupper
  rw [← factor', rhs] at hlower
  have hu' := Int.le_of_mul_le_mul_right hupper hN
  have hl' := Int.le_of_mul_le_mul_right hlower hN
  have hnneg : N*(-(2*area.num-area.den)) = N*(area.den-2*area.num) := by
    congr 1
    omega
  change N*(2*area.num-area.den) ≤ area.den ∧ N*(-(2*area.num-area.den)) ≤ area.den
  rw [hnneg]
  exact {hu', hl'}
  unfold Fraction.equiv
  dsimp

```

omega

```

/-- Constant-force example with unit tangential speed and zero initial normal
velocity. Similarity scales normalized rectangle areas. Mechanical
identification with displacement/defect is still separate from this
geometric coefficient theorem. -/
theorem constant_force_area_coefficients (acc time linearArea parabolaArea : Fraction)
  (hlinear : ∀ n : Nat, (hn : 0 < n) →
    Fraction.le (lowerLinear n hn) linearArea ∧ Fraction.le linearArea (upperLinear n hn))
  (hparabola : ∀ n : Nat, (hn : 0 < n) →
    Fraction.le (lowerParabola n hn) parabolaArea ∧ Fraction.le parabolaArea (upperParabola n hn)) :
  Fraction.equiv (Fraction.mul (Fraction.mul acc (Fraction.mul time time)) linearArea)
    (Fraction.half (Fraction.mul acc (Fraction.mul time time))) ∧
  Fraction.equiv (Fraction.mul (Fraction.half (Fraction.mul acc (Fraction.mul time (Fraction.mul time
    ↪ time)))) parabolaArea)
    (Fraction.mul (1, 6, by decide) (Fraction.mul acc (Fraction.mul time (Fraction.mul time time)))) :=
  ↪ by
constructor
· apply Fraction.equiv_trans (Fraction.mul_equiv_left _ (linear_area_coefficient linearArea hlinear))
  unfold Fraction.equiv Fraction.mul Fraction.half
  dsimp
  simp only [Int.one_mul, Int.mul_one]
  ac_rfl
· apply Fraction.equiv_trans (Fraction.mul_equiv_left _ (parabolic_area_coefficient parabolaArea
  ↪ hparabola))
  unfold Fraction.equiv Fraction.mul Fraction.half
  dsimp
  simp only [Int.one_mul, Int.mul_one]
  have six : (6 : Int) = 2*3 := by decide
  rw [six]
  ac_rfl

```

NewtonLimitDynamics/Polygon/MonotoneEnclosure.lean — Finite form of the enclosure behind Lemma X, under the force clause that 1713 added (NATP00082 par28: *Vi finita, sive Vis illa determinata & immutabilis sit, sive eadem continuo augetur vel continuo diminuatur*; 1687 NATP00077 par27 has *vi regulari*). One-dimensional impulse construction with unit time cells and integer magnitudes, starting at rest: drift with the current velocity, then the impulse a_i of cell i . If every impulse lies between l_0 and h_i , the space described lies between the spaces described under the constant forces l_0 and h_i . For a monotone force history these ...

```

/-- Under a constant force the space described is `force * tri n`. -/
theorem pos_const (c : Int) : (n : Nat) → pos (fun _ => c) n = c * tri n ∧ vel (fun _ => c) n = c * n
| 0 => by simp [pos, vel, tri]
| n + 1 => by
  obtain (hp, hv) := pos_const c n
  simp only [pos, vel, tri, hp, hv, Int.mul_add, Int.mul_one, Int.ofNat_add]
  refine (trivial, ?_)
  rw [Int.ofNat_one, Int.mul_one]

theorem vel_bounds (a : Nat → Int) (lo hi : Int) (n : Nat)
  (h : ∀ i, i < n → lo ≤ a i ∧ a i ≤ hi) :
  ∀ k, k ≤ n → lo * k ≤ vel a k ∧ vel a k ≤ hi * k := by
intro k
induction k with
| zero => intro _; simp [vel]
| succ k ih =>
  intro hk
  obtain (h1, h2) := ih (Nat.le_of_succ_le hk)
  obtain (h3, h4) := h k (Nat.lt_of_succ_le hk)
  simp only [vel, Int.ofNat_add, Int.mul_add, Int.mul_one]
  constructor <|> omega

/-- Enclosure: impulses between `lo` and `hi` put the space described between
the spaces described under the constant forces `lo` and `hi`. -/
theorem enclosure (a : Nat → Int) (lo hi : Int) (n : Nat)
  (h : ∀ i, i < n → lo ≤ a i ∧ a i ≤ hi) :

```

```

    lo * tri n ≤ pos a n ∧ pos a n ≤ hi * tri n := by
have hv := vel_bounds a lo hi n h
have key : ∀ m, m ≤ n → lo * tri m ≤ pos a m ∧ pos a m ≤ hi * tri m := by
  intro m
  induction m with
  | zero => intro _; simp [pos, tri]
  | succ m ih =>
    intro hm
    obtain ⟨h1, h2⟩ := ih (Nat.le_of_succ_le hm)
    obtain ⟨h3, h4⟩ := hv m (Nat.le_of_succ_le hm)
    simp only [pos, tri, Int.mul_add]
    constructor <=> omega
exact key n (Nat.le_refl n)

/-- The 1713 clause: a force that continually increases (monotone history)
    gives the enclosure between the initial and final forces. -/
theorem monotone_enclosure (a : Nat → Int) (n : Nat)
  (hmono : ∀ i j, i ≤ j → j ≤ n → a i ≤ a j) :
  a 0 * tri (n + 1) ≤ pos a (n + 1) ∧ pos a (n + 1) ≤ a n * tri (n + 1) :=
enclosure a (a 0) (a n) (n + 1) (fun i hi =>
  (hmono 0 i (Nat.zero_le i) (Nat.le_of_lt_succ hi),
   hmono i n (Nat.le_of_lt_succ hi) (Nat.le_refl n)))

/-- And a force that continually decreases, symmetrically. -/
theorem antitone_enclosure (a : Nat → Int) (n : Nat)
  (hanti : ∀ i j, i ≤ j → j ≤ n → a j ≤ a i) :
  a n * tri (n + 1) ≤ pos a (n + 1) ∧ pos a (n + 1) ≤ a 0 * tri (n + 1) :=
enclosure a (a n) (a 0) (n + 1) (fun i hi =>
  (hanti i n (Nat.le_of_lt_succ hi) (Nat.le_refl n),
   hanti 0 i (Nat.zero_le i) (Nat.le_of_lt_succ hi)))

/-- A finite force that rises and falls (impulses 0, 5, 0) describes space 5,
    outside the enclosure `0 ≤ pos ≤ 0` fixed by its initial and final forces. -/
theorem nonmonotone_escapes :
  pos (fun i => if i = 1 then 5 else 0) 3 = 5 ∧
  (fun i => if i = 1 then (5 : Int) else 0) 0 * tri 3 = 0 ∧
  (fun i => if i = 1 then (5 : Int) else 0) 2 * tri 3 = 0 := by
  decide

```

NewtonLimitDynamics/Principial687/LemmaX.lean

```

theorem lemmaX_reconstruction {Q : Type} (g : Magnitudes Q)
  (ratio : Q → Q) (c : Q) (p : LemmaXPremises g ratio c) :
  Ultimate g ratio c := by
  have heq : ratio = p.areaRatio := funext p.velocity_area
  rw [heq]
  exact enclosure_reconstruction g _ _ c p.lower_limit p.upper_limit p.enclosure

/-- Conditional bridge; not a completed historical discharge of Hypothesis 4. -/
theorem discharges_quadraticPremise_reconstruction {Q : Type}
  (g : Magnitudes Q) (ratio : Q → Q) (c : Q)
  (hc : g.positive c) (p : LemmaXPremises g ratio c) :
  DeMotul684.QuadraticInitialDeflection g ratio c :=
  (hc, lemmaX_reconstruction g ratio c p)

```

NewtonLimitDynamics/Principial1713/ForceComparison.lean

```

theorem corollary4_coefficient_reconstruction (k f t : Fraction)
  (hk : positive k) (ht : positive t) :
  equiv (quotient (generated k f t) (mul k (mul t t)))
    (positive_mul k _ hk (positive_mul t t ht ht)) f := by
  unfold equiv quotient generated mul
  dsimp
  ac_rfl

/-- Unlike Corollary 4, this rearrangement excludes zero force. -/
theorem corollary5_coefficient_reconstruction (k f t : Fraction)
  (hk : positive k) (hf : positive f) :
  equiv (quotient (generated k f t) (mul k f) (positive_mul k f hk hf))
    (mul t t) := by
  unfold equiv quotient generated mul

```

dsimp
ac rfl

NewtonLimitDynamics/Principia1713/LemmaX.lean

```
theorem lemmaX_reconstruction {Q : Type} (g : Magnitudes Q)
  (ratio : Q → Q) (c : Q) (p : LemmaXPremises g ratio c) :
  Ultimate g ratio c :=
  Principia1687.lemmaX_reconstruction g ratio c p.geometry
```

Lemma XI. *The vanishing subtense of the angle of contact is ultimately in the duplicate ratio of the subtense of the conterminous arc.*

In modern terms: with BD the distance from B on the curve to the tangent at A , measured in a fixed direction, $BD \propto AB^2$ ultimately; for finite curvature κ , $BD \approx \frac{1}{2}\kappa AB^2$.

1713. The statement is restricted: *in all curves having finite curvature at the point of contact.*

Case 1. Let the arc be AB , its tangent AD , the subtense BD of the angle of contact perpendicular to the tangent, and AB the subtense of the arc. Erect AG perpendicular to the subtense AB and BG perpendicular to the tangent AD , meeting in G . Then let the points D, B, G go to d, b, g , and let I be the intersection of the lines BG, AG ultimately formed when D, B reach A . The distance GI can be made less than any assigned length. By the nature of circles through A, B, G and through A, b, g , $AB^2 = AG \cdot BD$ and $Ab^2 = Ag \cdot bd$, so the ratio $AB^2 : Ab^2$ is compounded of $AG : Ag$ and $BD : bd$. Since GI can be taken less than any assigned length, the ratio $AG : Ag$ can differ from equality by less than any assigned difference, and so $AB^2 : Ab^2$ can differ from $BD : bd$ by less than any assigned difference. By Lemma I the ultimate ratio $AB^2 : Ab^2$ equals the ultimate ratio $BD : bd$.

Case 2. Let BD be inclined to AD at any given angle; the ultimate ratio $BD : bd$ is the same as before, and so the same as $AB^2 : Ab^2$.

Case 3. Even if the angle D is not given, the angles D, d always tend to equality and approach each other nearer than by any assigned difference, so they are ultimately equal (Lemma I), and the lines BD, bd are in the same ratio as before.

1713. Case 3 also covers the line BD converging to a given point, or placed by any other law, provided the angles D, d are placed by a common law.

Corollary 1. Since the tangents AD, Ad , the arcs AB, Ab and their sines BC, bc become ultimately equal to the chords AB, Ab , their squares too are ultimately as the subtenses BD, bd .

1713. Two new corollaries follow, and the old Corollaries 2 and 3 become Corollaries 4 and 5.

Corollary 2. Their squares are also ultimately as the sagittas of the arcs that bisect the chords and converge to a given point, for those sagittas are as the subtenses BD, bd .

Corollary 3. Hence the sagitta is in the duplicate ratio of the time in which a body describes the arc with a given velocity.

Corollary 2. The rectilinear triangles ADB , Adb are ultimately in the triplicate ratio of the sides AD , Ad , and in the sesquiplicate ratio of the sides DB , db , being in the ratio compounded of AD and DB , Ad and db . Likewise the triangles ABC , Abc are ultimately in the triplicate ratio of the sides BC , bc .

1713 (now Corollary 4) adds a definition: the sesquiplicate ratio is the subduplicate of the triplicate, compounded of the simple and the subduplicate ratio, which others call sesquialteral; that is, the ratio of $3/2$ powers.

Corollary 3. Since DB , db are ultimately parallel and in the duplicate ratio of AD , Ad , the ultimate curvilinear areas ADB , Adb are (by the nature of the parabola) two thirds of the rectilinear triangles ADB , Adb , and the segments AB , Ab one third of the same triangles. Hence these areas and segments are in the triplicate ratio both of the tangents AD , Ad and of the chords and arcs AB , Ab .

Lean reconstruction: 13 theorems in 2 modules cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/Contact/Bounds.lean

```
theorem le_quotient_iff (a b c : Fraction) (hc : positive c) :
  le a (Principal713.quotient b c hc) ↔ le (mul a c) b := by
  unfold le Principal713.quotient mul
  dsimp
  have e1 : a.num * (b.den * c.num) = a.num * c.num * b.den := by ac_rfl
  have e2 : b.num * c.den * a.den = b.num * (a.den * c.den) := by ac_rfl
  rw [e1, e2]

/-- Circle identity  $AB^2=AG*BD$  imported from Lemma XI case 1. A uniform
    positive lower bound on  $AG$  is essential; the identity alone is insufficient.
    This proves an inequality, not existence of the osculating configuration. -/
theorem normal_subtense_bound (chord subtense diameter minDiameter : Fraction)
  (hs : positive subtense) (hd : positive minDiameter)
  (hmin : le minDiameter diameter)
  (circle : equiv (mul diameter subtense) (mul chord chord)) :
  le subtense (Principal713.quotient (mul chord chord) minDiameter hd) := by
  apply (le_quotient_iff _ _ hd).mpr
  have hc := (equiv_iff_mutual_le _ _).mp (mul_comm subtense minDiameter)
  have hm := mul_le_mul_positive hmin subtense hs
  have he := (equiv_iff_mutual_le _ _).mp circle
  exact magnitudes.le_trans hc.1 (magnitudes.le_trans hm he.1)

/-- Explicit finite cubic inequality from a rectangle enclosing the defect.
    K is a proved/assumed quadratic bound valid on the SAME neighbourhood.
    No assertion of contact or enclosure is hidden in power notation. -/
theorem cubic_rectangle_bound (base departure defect K : Fraction)
  (hb : positive base)
  (hquad : le departure (mul K (mul base base)))
  (henclose : le defect (mul departure base)) :
  le defect (mul K (mul base (mul base base))) := by
  have hm := mul_le_mul_positive hquad base hb
  have he : equiv (mul (mul K (mul base base)) base) (mul K (mul base (mul base base))) := by
    unfold equiv mul
    dsimp
    ac_rfl
  exact magnitudes.le_trans henclose (magnitudes.le_trans hm ((equiv_iff_mutual_le _ _).mp he).1)

/-- Coordinate construction for Lemma XI case 1:  $A=(0,0)$ ,  $B=(x,y)$ ,
     $G=(0,D)$ , with  $AB$  perpendicular to  $BG$ . The corresponding right-triangle
    relation yields  $AB^2=AG*BD$ ; existence of  $G$  and its limiting position is
    deliberately not inferred. This is a reconstruction of circle geometry. -/
theorem circle_identity_from_perpendicular (x y D : Int)
  (perpendicular : x*x + y*(y-D) = 0) : x*x+y*y = D*y := by
  rw [Int.mul_sub] at perpendicular
  have h : y*D = D*y := Int.mul_comm _ _
```

omega

NewtonLimitDynamics/Contact/FiniteSums.lean

```
-- Lower rectangles for a linear velocity diagram. With n equal cells the
doubled lower sum differs from  $n^2$  by n, yielding coefficient 1/2. -/
theorem linear_rectangles (n : Nat) : 2 * nsum (fun i => i) n + n = n*n := by
  induction n with
  | zero => decide
  | succ n ih =>
    simp only [nsum, Nat.mul_add, Nat.add_mul, Nat.mul_one, Nat.one_mul]
    omega

-- Lower rectangles for  $y=x^2$ . Division by  $6n^3$  yields the area coefficient
1/3 with explicit corrections  $-1/(2n)+1/(6n^2)$ , not an integral theorem. -/
theorem quadratic_rectangles (n : Nat) :
  6 * nsum (fun i => i*i) n + 3*(n*n) = 2*(n*n*n)+n := by
  induction n with
  | zero => decide
  | succ n ih =>
    simp only [nsum, Nat.mul_add, Nat.add_mul, Nat.mul_one, Nat.one_mul]
    omega

theorem upper_lower_gap (n : Nat) :
  nsum (fun i => (i+1)*(i+1)) n = nsum (fun i => i*i) n + n*n := by
  induction n with
  | zero => decide
  | succ n ih => simp only [nsum, ih]

-- Error of the lower quadratic rectangle sum in units of  $6n^3$  is  $\leq 3n^2$ ;
after positive division this is  $\leq 1/(2n)$ . -/
theorem quadratic_lower_error (n : Nat) :
  2*(n*n*n)  $\leq$  6*nsum (fun i => i*i) n + 3*(n*n) := by
  have := quadratic_rectangles n
  omega

theorem sum_bound (f : Nat → Nat) (C n : Nat) (hf :  $\forall i, i < n \rightarrow f i \leq C$ ) :
  nsum f n  $\leq$  n*C := by
  induction n with
  | zero => simp [nsum]
  | succ n ih =>
    have h := ih (fun i hi => hf i (by omega))
    have hn := hf n (by omega)
    simp only [nsum, Nat.add_mul, Nat.one_mul]
    omega

-- Uniform partition error, expressed without division:  $n^2 * \sum \text{error}_i \leq C$ .
All errors use a common unit; the local bound is  $n^3 * \text{error}_i \leq C$  for EVERY
cell of the fixed interval. C must include the fixed interval's  $T^3$  factor.
Positivity of n permits cancellation; pointwise local scaling is not enough. -/
theorem uniform_partition_error (error : Nat → Nat) (C n : Nat) (hn :  $0 < n$ )
(hlocal :  $\forall i, i < n \rightarrow n*n*n*\text{error } i \leq C$ ) :
  n*n*nsum error n  $\leq$  C := by
  have hs := sum_bound (fun i => n*n*n*error i) C n hlocal
  have hmul :  $\forall k, \text{nsum (fun i => n*n*n*error i) } k = n*n*n*\text{nsum error } k$  := by
    intro k
    induction k with
    | zero => simp [nsum]
    | succ k ih => simp [nsum, ih, Nat.mul_add]
  rw [hmul] at hs
  have he :  $n*n*n*\text{nsum error } n = n*(n*n*\text{nsum error } n)$  := by ac_rfl
  rw [he] at hs
  exact Nat.le_of_mul_le_mul_left hs hn

theorem nsum_mul (f : Nat → Nat) (k n : Nat) :
  nsum (fun i => f i*k) n = nsum f n*k := by
  induction n with
  | zero => simp [nsum]
  | succ n ih => simp [nsum, ih, Nat.add_mul]

-- Rational version: errors have a common denominator D within this partition,
```

```

    and the fixed uniform coefficient is Knum/Kden. D may vary with refinement.
    Hence this is not restricted to integer errors eventually becoming zero. -/
theorem rational_uniform_partition_error (error : Nat → Nat)
  (Knum Kden D n : Nat) (hk : 0 < Kden) (hd : 0 < D) (hn : 0 < n)
  (hlocal : ∀ i, i < n → n*n*(error i*Kden) ≤ Knum*D) :
  Fraction.le
    ((nsum error n : Int), (D : Int), Int.ofNat_lt.mpr hd)
    ((Knum : Int), (Kden*(n*n) : Nat),
      Int.ofNat_lt.mpr (Nat.mul_pos hk (Nat.mul_pos hn hn))) := by
  have h := uniform_partition_error (fun i => error i*Kden) (Knum*D) n hn hlocal
  rw [nsum_mul] at h
  have he : n*n*(nsum error n*Kden) = nsum error n*(Kden*(n*n)) := by ac_rfl
  rw [he] at h
  unfold Fraction.le
  dsimp
  simpa only [Int.ofNat_mul] using (Int.ofNat_le.mpr h)

/-- An explicit Archimedean argument for rational error budgets C/n.
    This imports no analytic convergence theorem. -/
theorem reciprocal_budget_vanishes (error : Nat → Fraction) (C : Nat)
  (hbound : ∀ n, (hn : 0 < n) → Fraction.le (error n)
    ((C : Int), (n : Int), Int.ofNat_lt.mpr hn)) : SeqVanishes error := by
  intro e he
  refine (C * e.den.natAbs + 1, ?_)
  intro n hn
  have hnpos : 0 < n := by omega
  have hd : (e.den.natAbs : Int) = e.den := Int.natAbs_of_nonneg (Int.le_of_lt e.den_pos)
  have hlarge : (C : Int) * e.den < (n : Int) := by
    have hnats : C * e.den.natAbs < n := by omega
    have hi := Int.ofNat_lt.mpr hnats
    simpa only [Int.ofNat_mul, hd] using hi
  have hen : 1 ≤ e.num := by unfold Fraction.positive at he; omega
  have hm := Int.mul_le_mul_of_nonneg_right hen (Int.le_of_lt (Int.ofNat_lt.mpr hnpos))
  have hbracket : Fraction.lt ((C : Int), (n : Int), Int.ofNat_lt.mpr hnpos) e := by
    unfold Fraction.lt
    dsimp
    simp only [Int.one_mul] at hm
    omega
  exact Fraction.magnitudes.lt_of_le_lt (hbound n hnpos) hbracket

```

Scholium

In all this we suppose the angle of contact neither infinitely greater than the angles of contact which circles make with their tangents, nor infinitely less; that is, the curvature at the point A neither infinitely small nor infinitely great, or the interval AI of finite magnitude. For DB can be taken as AD^3 , in which case no circle can be drawn through A between the tangent AD and the curve AB , and the angle of contact is infinitely less than circular ones. By a similar argument, taking DB successively as AD^4 , AD^5 , AD^6 , AD^7 , ... gives a series of angles of contact going on to infinity, each infinitely less than the one before. Taking DB successively as AD^2 , $AD^{3/2}$, $AD^{4/3}$, $AD^{5/4}$, $AD^{6/5}$, $AD^{7/6}$, ... gives another infinite series, the first of the same kind as the circular ones, the second infinitely greater, and each infinitely greater than the one before. Between any two of these angles a series going on to infinity both ways can be inserted, each infinitely greater than the one before: for instance, between the terms AD^2 and AD^3 the series $AD^{13/6}$, $AD^{11/5}$, $AD^{9/4}$, $AD^{7/3}$, $AD^{5/2}$, $AD^{8/3}$, $AD^{11/4}$, $AD^{14/5}$, $AD^{17/6}$, And again between any two angles of this series a new series of intermediate angles can be inserted, differing from each other by infinite intervals. Nature knows no limit.

In modern terms: $DB \propto AD^p$ near A has curvature 0 for $p > 2$, finite nonzero curvature for $p = 2$, and infinite curvature for $1 < p < 2$; the orders p are dense.

What has been shown about curved lines and the surfaces they enclose applies easily to curved surfaces and contents of solids. These lemmas were set down first to avoid the tedium of long proofs by reduction to absurdity in the manner of the ancient geometers. Proofs are shorter by the method of indivisibles; but the hypothesis of indivisibles is harsher, and that method is therefore reckoned less geometrical. So the proofs of what follows are reduced to the ultimate sums and ratios of vanishing quantities and the first ratios of nascent ones, that is, to the limits of sums and ratios, and the proofs of those limits are given first as briefly as possible. This achieves what the method of indivisibles achieves, with principles now proved. Hence in what follows, whenever quantities are considered as made of particles, or small curved lines are used for straight ones, what is meant is not indivisibles but vanishing divisible quantities, not sums and ratios of determinate parts but always limits of sums and ratios, and the force of such proofs always rests on the method of the preceding lemmas.

Objection: vanishing quantities have no ultimate proportion, since before they vanish it is not ultimate, and when they have vanished there is none. By the same argument a body arriving at a place would have no ultimate velocity: before it arrives the velocity is not ultimate, and when it has arrived there is none. The answer is easy. The ultimate velocity is that with which the body moves neither before it reaches the last place and the motion stops, nor after, but at the moment it arrives. Likewise the ultimate ratio of vanishing quantities is the ratio with which they vanish, neither before nor after; the first ratio of nascent quantities is the ratio with which they begin; and the first and last sum is that with which they begin and cease to be (or to increase and decrease). There is a limit which the velocity can reach at the end of the motion but not exceed; this is the ultimate velocity. The same holds for the limit of all beginning and ceasing quantities and proportions. Since this limit is certain and definite, finding it is a truly geometrical problem, and geometrical things may legitimately be used to determine and prove other geometrical things.

Objection: if the ultimate ratios of vanishing quantities are given, their ultimate magnitudes are given too, and so every quantity consists of indivisibles, contrary to what Euclid proved about incommensurables in Book X of the *Elements*. This objection rests on a false hypothesis. The ultimate ratios with which quantities vanish are not ratios of ultimate quantities, but limits which the ratios of quantities decreasing without limit always approach, nearer than by any given difference, never passing them and never reaching them before the quantities are diminished indefinitely. This is clearer with infinitely great quantities: if two quantities with a given difference are increased indefinitely, their ultimate ratio, equality, is given, though no ultimate or greatest quantities of which it is the ratio are given. So in what follows, whenever for ease of imagination quantities are called least, vanishing or ultimate, do not understand quantities of determinate magnitude, but think of them as always to be diminished without limit.

In modern terms: an ultimate ratio is $\lim X/Y$, and Lemma I supplies its defining property.

No theorem of the Lean reconstruction cites this item.

Section II. On finding centripetal forces

Proposition I. Theorem I. *The areas which bodies made to move in orbits describe by radii drawn to an immovable centre of forces lie in immovable planes and are proportional to the times.*

In modern terms: for a centripetal force toward a fixed point S , the motion stays in one plane and $\frac{1}{2} \mathbf{r} \times \dot{\mathbf{r}}$ is constant, so swept area is proportional to time.

Proof. Divide the time into equal parts, and in the first part let the body describe the straight line AB by its inherent force. In the second part, if nothing impeded it, it would go straight on to c (Law I), describing Bc equal to AB , so that with radii AS , BS , cS drawn to the centre, the areas ASB and BSc would be equal. But when the body reaches B , let a centripetal force act with a single but great impulse and make the body deviate from the line Bc and go on in the line BC . Draw cC parallel to BS , meeting BC in C ; at the end of the second part of time the body (Corollary I of the Laws) will be at C , in the same plane as the triangle ASB . Join SC ; the triangle SBC equals the triangle SBC , because of the parallels SB , Cc , and so it also equals the triangle SAB . By a similar argument, if the centripetal force acts successively at C , D , E , ..., making the body describe in the single particles of time the single straight lines CD , DE , EF , ..., these lie in the same plane, and the triangle SCD equals the triangle SBC , SDE equals SCD , and SEF equals SDE . So in equal times equal areas are described in an immovable plane, and *componendo*, any sums of areas $SADS$, $SAFS$ are to each other as the times of description. Now let the number of triangles be increased and their width diminished indefinitely: their ultimate perimeter ADF (Corollary 4 of Lemma III) will be a curved line; so the centripetal force, by which the body is perpetually drawn back from the tangent of this curve, will act uninterruptedly, and any areas described, $SADS$, $SAFS$, being always proportional to the times of description, will be proportional to those times in this case too.

In modern terms: with vertices $\mathbf{r}_k$ and impulses toward S , $\mathbf{r}_{k+1} = 2\mathbf{r}_k - \mathbf{r}_{k-1} + \lambda_k \mathbf{r}_k$. Then $\mathbf{r}_k \times \mathbf{r}_{k+1} = \mathbf{r}_{k-1} \times \mathbf{r}_k$, so every triangle $S \mathbf{r}_k \mathbf{r}_{k+1}$ has the same area (Newton: triangles on the same base between parallels, Euclid I.37–38). The final step passes to the limit of the polygons.

Corollary 1. In non-resisting media, if the areas are not proportional to the times, the forces do not tend to the point where the radii meet.

Corollary 2. In all media, if the description of areas is accelerated, the forces do not tend to the point where the radii meet, but decline from it *in consequentia*.

1713. These two corollaries move, expanded, to Proposition II. In their place come six new ones.

Corollary 1. The velocity of a body attracted to an immovable centre is, in non-resisting spaces, inversely as the perpendicular let fall from the centre onto the rectilinear tangent of the orbit; for the velocities at A , B , C , D , E are as the bases AB , BC , CD , DE , EF of equal triangles, and these bases are inversely as the perpendiculars on them. In modern terms: $v \cdot p$ is constant, with p the distance from S to the tangent.

Corollary 2. If the chords AB , BC of two arcs described successively in

equal times by the same body in non-resisting spaces are completed into a parallelogram $ABCU$, its diagonal BU , in the position it has ultimately when the arcs are diminished indefinitely, passes through the centre of forces when produced both ways.

Corollary 3. If the chords AB , BC and DE , EF of arcs described in equal times are completed into parallelograms $ABCU$, $DEFZ$, the forces at B and E are in the ultimate ratio of the diagonals BU , EZ as the arcs are diminished indefinitely; for the motions BC , EF are compounded (Corollary I of the Laws) of Bc , BU and Ef , EZ , and BU , EZ , equal to Cc , Ff , were generated in the proof of this Proposition by the impulses of the centripetal force at B and E , and so are proportional to those impulses.

Corollary 4. The forces by which any bodies in non-resisting spaces are drawn from rectilinear motions and bent into curved orbits are to each other as the sagittas of arcs described in equal times that converge to the centre of forces and bisect the chords, as the arcs are diminished indefinitely; for these sagittas are halves of the diagonals of Corollary 3.

Corollary 5. So these forces are to the force of gravity as these sagittas are to the vertical sagittas of the parabolic arcs that projectiles describe in the same time.

Corollary 6. All the same holds, by Corollary V of the Laws, when the planes in which the bodies move, together with the centres of forces in them, are not at rest but move uniformly in a straight line.

Lean reconstruction: 435 theorems in 24 modules cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/DeMotu1684/AreaLaw.lean — Finite reconstruction of De Motu **Theorem 1**, not a retrospectively numbered Principia proposition. Witness NATP00089 par8-9 (De motu corporum in gyrum, <https://www.newtonproject.ox.ac.uk/view/texts/diplomatic/NA>) and witness NATP00090 par16-17 (De motu sphæricorum corporum in fluidis, <https://www.newtonproject.ox.ac.uk/view/texts/diplomatic/NATP00090#par17>) remain separate. Both explicitly construct equal-time central-impulse polygons and say that equal areas are described in equal times. The arbitrary-block comparison below is a derived finite reconstruction; these passages do ...

```
/-- NATP00089 par9: finite equal areas, conditional on the named construction
    premises. No limiting conclusion or reconstructed marginal citation. -/
theorem natp00089_finite_equal_areas (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
  all_unsigned_cell_areas g p q impulse n

/-- NATP00090 par17: separately named witness-local finite equal-area result. -/
theorem natp00090_finite_equal_areas (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
  all_unsigned_cell_areas g p q impulse n

/-- Derived block comparison from NATP00089's finite equal-area construction.
    Positive time cell and counts rule out division by zero total times. -/
theorem natp00089_finite_block_comparison (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start₁ start₂ m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
```

```

      unsignedBlock g p q impulse start1 m * (n * dt) =
      unsignedBlock g p q impulse start2 n * (m * dt) :=
positive_unsigned_area_comparison g p q impulse dt hdt start1 start2 m n hm hn

/-- Derived comparison for NATP00090 alone; it imports no printed-edition
    limiting lemma or premise from NATP00089's unresolved revision layer. -/
theorem natp00090_finite_block_comparison (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start1 start2 m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
    unsignedBlock g p q impulse start1 m * (n * dt) =
    unsignedBlock g p q impulse start2 n * (m * dt) :=
positive_unsigned_area_comparison g p q impulse dt hdt start1 start2 m n hm hn

/-- NATP00089's polygon-to-motion obligation, as a conditional reconstruction:
    D_mesh is the nonnegative polygon-trajectory region, NOT its Kepler area.
    No numbered limiting lemma is imported into this witness. -/
theorem natp00089_polygon_trajectory_defect_control
  (polygonTrajectoryArea budget : Fraction → Fraction) (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea :=
polygon_trajectory_defect_vanishes polygonTrajectoryArea budget hbudget hgeometry

/-- Separately named NATP00090 obligation with the same explicit geometric
    premises; its final infinitely-small-triangle assertion remains unproved. -/
theorem natp00090_polygon_trajectory_defect_control
  (polygonTrajectoryArea budget : Fraction → Fraction) (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea :=
polygon_trajectory_defect_vanishes polygonTrajectoryArea budget hbudget hgeometry

```

NewtonLimitDynamics/Diagnostic/PhaseArea.lean — Action diagnostic layer (research/action-arguments Arg004). One-dimensional phase plane (q , v) for the cell of Proposition I's finite construction: inertial drift (Law I), then the impulse of the force at the arrival point (Law II, laws' Corollary I). The construction is the same in De Motu (NATP00089 par9, NATP00090 par17), 1687 (NATP00077 par45) and 1713 (NATP00082 par51). Modern rational reconstruction; no limit is taken. Each step moves one coordinate by an amount depending only on the other (a shear), so each line of fixed velocity (drift) or fixed position (impulse) is translated ...

```

/-- Cavalieri, drift: two phase points with the same velocity keep their
    position difference. Holds for every field. -/
theorem drift_rigid (d q q' v : Fraction) :
  Fraction.equiv (fsub (drift d (q', v)).1 (drift d (q, v)).1) (fsub q' q) := by
  unfold fsub negF drift Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Cavalieri, impulse: two phase points at the same position keep their
    velocity difference, for an arbitrary field `F`. -/
theorem kick_rigid (F : Fraction → Fraction) (d q v v' : Fraction) :
  Fraction.equiv (fsub (kick F d (q, v')).2 (kick F d (q, v)).2) (fsub v' v) := by
  unfold fsub negF kick Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

theorem drift_area2 (d : Fraction) (z0 z1 z2 : Phase) :
  Fraction.equiv (area2 (drift d z0) (drift d z1) (drift d z2)) (area2 z0 z1 z2) := by
  unfold area2 det pointSub pointNeg pointAdd drift Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

```

```

theorem kick_area2 (c w d : Fraction) (z0 z1 z2 : Phase) :
  Fraction.equiv (area2 (kick (affine c w) d z0) (kick (affine c w) d z1) (kick (affine c w) d z2))
    (area2 z0 z1 z2) := by
  unfold area2 det pointSub pointNeg pointAdd kick affine negF Fraction.equiv Fraction.add
  Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- One cell preserves the doubled phase area of every triangle, for every
duration. -/
theorem cell_area2 (c w d : Fraction) (z0 z1 z2 : Phase) :
  Fraction.equiv
    (area2 (cell (affine c w) d z0) (cell (affine c w) d z1) (cell (affine c w) d z2))
    (area2 z0 z1 z2) :=
  Fraction.equiv_trans (kick_area2 c w d _ _ _) (drift_area2 d z0 z1 z2)

/-- Every equal-cell schedule, at every mesh, preserves it. -/
theorem cells_area2 (c w d : Fraction) :
  (n : Nat) → (z0 z1 z2 : Phase) →
  Fraction.equiv
    (area2 (cells (affine c w) d n z0) (cells (affine c w) d n z1) (cells (affine c w) d n z2))
    (area2 z0 z1 z2)
  | 0, _, _, _ => Fraction.equiv_refl _
  | n + 1, z0, z1, z2 =>
    Fraction.equiv_trans (cells_area2 c w d n _ _ _) (cell_area2 c w d z0 z1 z2)

theorem cell_energyD (a d : Fraction) (z : Phase) :
  Fraction.equiv (energyD a d (cell (fun _ => a) d z)) (energyD a d z) := by
  unfold energyD fsub negF cell kick drift Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- The duration-dependent invariant differs between meshes at the same state
(`a = 1`, state `(0, 1)`): `0` for `d = 1`, `1/2` for `d = 1/2`. -/
theorem energyD_depends_on_mesh :
  ¬ Fraction.equiv (energyD one one (zero, one)) (energyD one half (zero, one)) := by
  decide

/-- Refinement example, force as distance (`c = 0`, `w = 1`): one cell of
duration 1 and two cells of duration 1/2 send the phase point `(1, 0)` to
different places, while both keep the unit triangle's doubled area 1. -/
theorem refinement_moves_points_keeps_area :
  ¬ pointEquiv (cells (affine zero one) one 1 (one, zero))
    (cells (affine zero one) half 2 (one, zero)) ∧
  Fraction.equiv
    (area2 (cells (affine zero one) one 1 (zero, zero)) (cells (affine zero one) one 1 (one, zero))
      (cells (affine zero one) one 1 (zero, one))) one ∧
  Fraction.equiv
    (area2 (cells (affine zero one) half 2 (zero, zero)) (cells (affine zero one) half 2 (one, zero))
      (cells (affine zero one) half 2 (zero, one))) one := by
  decide

```

NewtonLimitDynamics/Polygon/CentralSchedule.lean — Finite impulse polygons for an arbitrary position-dependent central force, with arbitrary (possibly unequal) rational time cells. Modern rational coordinate reconstruction motivated by the finite part of Proposition I (1687 NATP00077 par45, 1713 NATP00082 par51), which divides time into *equal* parts and applies each impulse at the vertex reached by inertial motion. The centre S is the origin. No curve, limit or force regularity is assumed; the field is an arbitrary function on represented points.

```

theorem det_drift (x v : Point) (d : Fraction) :
  Fraction.equiv (det (pointAdd x (pointScale d v)) v) (det x v) := by
  unfold Fraction.equiv det pointAdd pointScale Fraction.add Fraction.mul

```

```

dsimp
simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
ac_nf
omega

theorem det_kick_split (x v w : Point) (d : Fraction) :
  Fraction.equiv (det x (pointAdd v (pointScale d w)))
    (Fraction.add (det x v) (Fraction.mul d (det x w))) := by
  unfold Fraction.equiv det pointAdd pointScale Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- The doubled triangle swept in one drift is duration times areal velocity. -/
theorem det_cell_area (x v : Point) (d : Fraction) :
  Fraction.equiv (det x (pointAdd x (pointScale d v))) (Fraction.mul d (det x v)) := by
  unfold Fraction.equiv det pointAdd pointScale Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- A central impulse at `x` leaves `det(x, ·)` unchanged. -/
theorem central_kick (a : Field) (hc : central a) (x v : Point) (d : Fraction) :
  Fraction.equiv (det x (pointAdd v (pointScale d (a x)))) (det x v) :=
  Fraction.equiv_trans (det_kick_split x v (a x) d)
    (Fraction.equiv_trans
      (Fraction.add_equiv_left (det x v)
        (Fraction.equiv_trans (Fraction.mul_equiv_left d (hc x)) (Fraction.mul_zero d)))
      (Fraction.add_zero (det x v)))

/-- Each cell (drift, then central impulse at the arrival vertex) preserves the
areal velocity exactly, for any cell duration. -/
theorem cell_momentum (a : Field) (hc : central a) (d : Fraction) (s : Point × Point) :
  Fraction.equiv (momentum (cell a d s)) (momentum s) :=
  Fraction.equiv_trans (central_kick a hc _ s.2 d) (det_drift s.1 s.2 d)

theorem schedule_momentum (a : Field) (hc : central a) :
  (ds : List Fraction) → (s : Point × Point) →
  Fraction.equiv (momentum (schedule a ds s)) (momentum s)
| [], _ => Fraction.equiv_refl _
| d :: ds, s =>
  Fraction.equiv_trans (schedule_momentum a hc ds (cell a d s)) (cell_momentum a hc d s)

/-- Areas proportional to times, for arbitrary unequal rational cells and any
central field: the swept doubled area equals elapsed time times the initial
doubled areal velocity. -/
theorem swept_eq (a : Field) (hc : central a) :
  (ds : List Fraction) → (s : Point × Point) →
  Fraction.equiv (swept a ds s) (Fraction.mul (elapsed ds) (momentum s))
| [], _ => by
  unfold swept elapsed Fraction.equiv Fraction.mul Fraction.ofInt
  simp
| d :: ds, s => by
  have hcell : Fraction.equiv (det s.1 (cell a d s).1) (Fraction.mul d (momentum s)) :=
    det_cell_area s.1 s.2 d
  have hrest : Fraction.equiv (swept a ds (cell a d s))
    (Fraction.mul (elapsed ds) (momentum s)) :=
    Fraction.equiv_trans (swept_eq a hc ds (cell a d s))
      (Fraction.mul_equiv_left (elapsed ds) (cell_momentum a hc d s))
  exact Fraction.equiv_trans
    (Fraction.equiv_trans (Fraction.add_equiv_right _ hcell) (Fraction.add_equiv_left _ hrest))
    (Fraction.equiv_symm (Fraction.add_mul d (elapsed ds) (momentum s)))

/-- Refinement of one cell `h+k` into `h` then `k`, for an arbitrary field:
the fine endpoint exceeds the coarse one by exactly `h*k` times the force at
the intermediate vertex. No regularity of the field is used. -/
theorem refine_position (a : Field) (h k : Fraction) (s : Point × Point) :
  pointEquiv (cell a k (cell a h s)).1

```

```

      (pointAdd (cell a (Fraction.add h k) s).1
        (pointScale (Fraction.mul h k) (a (cell a h s).1))) := by
constructor <=>
· unfold cell pointAdd pointScale Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

private theorem velocity_scalar (v ay az aX h k : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add v (Fraction.mul h ay)) (Fraction.mul k az))
    (Fraction.add (Fraction.add v (Fraction.mul (Fraction.add h k) aX))
      (Fraction.add (Fraction.mul h (fsub ay aX)) (Fraction.mul k (fsub az aX)))) := by
  unfold fsub Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Velocity mismatch of the same refinement, exactly: the fine velocity equals
the coarse one plus `h*(a y - a X) + k*(a z - a X)`, where `y`, `z` are the
fine vertices and `X` the coarse vertex. It vanishes for a constant field;
otherwise its control is a premise about the field, not a consequence of
the construction. -/
theorem refine_velocity (a : Field) (h k : Fraction) (s : Point × Point) :
  pointEquiv (cell a k (cell a h s)).2
    (pointAdd (cell a (Fraction.add h k) s).2
      (pointAdd
        (pointScale h (pointSub (a (cell a h s).1) (a (cell a (Fraction.add h k) s).1)))
        (pointScale k (pointSub (a (cell a k (cell a h s)).1) (a (cell a (Fraction.add h k) s).1))))) :=
  (velocity_scalar _ _ _ _ _ , velocity_scalar _ _ _ _ _)

private theorem expand_left (d d' L K : Fraction) :
  Fraction.equiv (Fraction.mul d (Fraction.mul d' (Fraction.add L K)))
    (Fraction.add (Fraction.mul (Fraction.mul d d') L) (Fraction.mul (Fraction.mul d d') K)) := by
  unfold Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

private theorem expand_right (d d' L : Fraction) :
  Fraction.equiv (Fraction.mul d' (Fraction.mul d L)) (Fraction.mul (Fraction.mul d d') L) := by
  unfold Fraction.equiv Fraction.mul
  dsimp
  ac_nf

private theorem add_cancel_num {P Q : Fraction} (h : Fraction.equiv (Fraction.add P Q) P) :
  Q.num = 0 := by
  unfold Fraction.equiv Fraction.add at h
  dsimp at h
  have hpd := P.den_pos
  have e1 : P.num * Q.den * P.den = P.num * (P.den * Q.den) := by ac_rfl
  have hz : Q.num * P.den * P.den = 0 := by
    rw [Int.add_mul] at h
    omega
  rcases Int.mul_eq_zero.mp hz with h1 | h1
  · rcases Int.mul_eq_zero.mp h1 with h2 | h2
    · exact h2
    · omega
  · omega

/-- Converse for unequal cells (motivated by Proposition II, whose statement
speaks of areas proportional to times): drift `d` from `x` with velocity
`v`, an arbitrary impulse `J` at the arrival vertex `x`, then drift `d`.
If the two doubled areas are proportional to the nonzero durations, the
impulse is parallel to the radius `Sx`. -/
theorem unequal_cells_converse (x v J : Point) (d d' : Fraction)
  (hd : d.num ≠ 0) (hd' : d'.num ≠ 0)
  (h : Fraction.equiv
    (Fraction.mul d (det (pointAdd x (pointScale d v))

```

```

    (pointAdd (pointAdd x (pointScale d v)) (pointScale d' (pointAdd v J))))
    (Fraction.mul d' (det x (pointAdd x (pointScale d v)))) :
  Fraction.equiv (det (pointAdd x (pointScale d v)) J) (Fraction.ofInt 0) := by
-- second triangle: d' * (L + K), with L = det x v and K = det x' J
have hA : Fraction.equiv
  (det (pointAdd x (pointScale d v))
    (pointAdd (pointAdd x (pointScale d v)) (pointScale d' (pointAdd v J))))
  (Fraction.mul d' (Fraction.add (det x v) (det (pointAdd x (pointScale d v)) J))) :=
  Fraction.equiv_trans (det_cell_area _ _ d')
  (Fraction.mul_equiv_left d'
    (Fraction.equiv_trans (det_add_right _ v J)
      (Fraction.add_equiv_right _ (det_drift x v d))))
have hB := det_cell_area x v d
have h1 := Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_equiv_left d hA))
  (Fraction.equiv_trans h (Fraction.mul_equiv_left d' hB))
have h2 := Fraction.equiv_trans (Fraction.equiv_symm (expand_left d d' _))
  (Fraction.equiv_trans h1 (expand_right d d' _))
have hq := add_cancel_num h2
have hq' : d.num * d'.num * (det (pointAdd x (pointScale d v)) J).num = 0 := hq
have hK : (det (pointAdd x (pointScale d v)) J).num = 0 := by
  rcases Int.mul_eq_zero.mp hq' with h3 | h3
  · rcases Int.mul_eq_zero.mp h3 with h4 | h4
  · exact absurd h4 hd
  · exact absurd h4 hd'
  · exact h3
unfold Fraction.equiv Fraction.ofInt
dsimp
rw [hK]
simp

theorem harmonic_central : central harmonic := by
  intro p
  unfold harmonic pointNeg det Fraction.equiv Fraction.add Fraction.mul Fraction.ofInt
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg, Int.mul_one, Int.zero_mul]
  ac_nf
  omega

/-- In the harmonic example, refining a unit cell into two half cells changes the
terminal velocity as well as the endpoint: a varying force breaks the
exact velocity agreement that holds for constant force. -/
theorem harmonic_refinement_changes_velocity :
  ~ pointEquiv (cell harmonic half (cell harmonic half exampleState)).2
    (cell harmonic (Fraction.add half half) exampleState).2 ^
  ~ pointEquiv (cell harmonic half (cell harmonic half exampleState)).1
    (cell harmonic (Fraction.add half half) exampleState).1 := by
  decide

/-- Both schedules nevertheless sweep the same doubled area, as `swept_eq`
requires: elapsed time times the initial areal velocity. -/
theorem harmonic_equal_swept :
  Fraction.equiv (swept harmonic [half, half] exampleState)
    (swept harmonic [Fraction.add half half] exampleState) := by
  decide

```

NewtonLimitDynamics/Polygon/Contact.lean

```

theorem velocityContact_position (left right : FiniteSegment Point Velocity)
  (h : velocityContact left right) : positionContact left right :=
  h.1

theorem impulseContact_position (advance : Velocity → Impulse → Velocity)
  (left right : FiniteSegment Point Velocity) (j : Impulse)
  (h : impulseContact advance left right j) : positionContact left right :=
  h.1

theorem glue_first (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) : (glue left right h).firstPoint = left.first :=
  rfl

theorem glue_last (left right : FiniteSegment Point Velocity)

```

```

    (h : positionContact left right) : (glue left right h).lastPoint = right.last :=
    rfl

theorem glue_middle_from_left (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) : (glue left right h).middlePoint = left.last :=
  h.symm

theorem glue_middle_from_right (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) : (glue left right h).middlePoint = right.first :=
  rfl

theorem glued_position_contact (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) :
  positionContact (glue left right h).left (glue left right h).right :=
  rfl

theorem restriction_cell (path : SampledPath Point Velocity) (offset i : Nat) :
  (path.restrict offset).cell i = path.cell (offset+i) := by
  simp [SampledPath.restrict, SampledPath.cell, Nat.add_assoc]

theorem adjacent_position_contact (path : SampledPath Point Velocity) (i : Nat) :
  positionContact (path.cell i) (path.cell (i+1)) :=
  rfl

theorem adjacent_velocityContact_iff (path : SampledPath Point Velocity) (i : Nat) :
  velocityContact (path.cell i) (path.cell (i+1)) ↔
  path.arriving (i+1) = path.departing (i+1) := by
  simp [velocityContact, positionContact, SampledPath.cell]

theorem adjacent_impulseContact_iff (advance : Velocity → Impulse → Velocity)
  (path : SampledPath Point Velocity) (i : Nat) (j : Impulse) :
  impulseContact advance (path.cell i) (path.cell (i+1)) j ↔
  path.departing (i+1) = advance (path.arriving (i+1)) j := by
  simp [impulseContact, positionContact, SampledPath.cell]

/-- Restarting the finite construction from its kth constructed pair, with the
    shifted impulse sequence, gives the original construction after k+n cells.
    This is a theorem about the existing recursive polygonal motion only. -/
theorem motion_restart (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (k n : Nat) :
  motion g (motion g p q impulse k).1 (motion g p q impulse k).2
  (fun i => impulse (k+i)) n = motion g p q impulse (k+n) := by
  induction n with
  | zero => simp [motion]
  | succ n ih =>
    rw [show k + (n+1) = (k+n)+1 by omega]
    simp only [motion]
    rw [ih]

/-- Consecutive pairs created by `motion` share their middle vertex. -/
theorem motion_adjacent_pair_position_contact
  (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) :
  (motion g p q impulse n).2 = (motion g p q impulse (n+1)).1 :=
  rfl

/-- The actual lattice step changes discrete velocity by the radial impulse.
    Unit time is built into the use of adjacent vertex differences. -/
theorem lattice_step_velocity_jump (p q : LatticePoint) (j : Int) :
  latticeVelocity q (step lattice p q j) =
  latticeAdd (latticeVelocity p q) (latticeScale j q) := by
  apply Prod.ext <=>
  simp [latticeVelocity, latticeAdd, latticeScale, step, lattice, kick, extend] <=>
  omega

theorem motion_lattice_velocity_jump (p q : LatticePoint)
  (impulse : Nat → Int) (n : Nat) :
  latticeVelocity (motion lattice p q impulse n).2
  (motion lattice p q impulse (n+1)).2 =
  latticeAdd

```

```

      (latticeVelocity (motion lattice p q impulse n).1
        (motion lattice p q impulse n).2)
      (latticeScale (impulse n) (motion lattice p q impulse n).2) := by
change latticeVelocity (motion lattice p q impulse n).2
  (step lattice (motion lattice p q impulse n).1
    (motion lattice p q impulse n).2 (impulse n)) = _
exact lattice_step_velocity_jump _ _ _

theorem latticeMotionCell_adjacent_position_contact (p q : LatticePoint)
  (impulse : Nat → Int) (n : Nat) :
  positionContact (latticeMotionCell p q impulse n)
    (latticeMotionCell p q impulse (n+1)) :=
  motion_adjacent_pair_position_contact lattice p q impulse n

theorem latticeMotionCell_impulseContact (p q : LatticePoint)
  (impulse : Nat → Int) (n : Nat) :
  impulseContact latticeAdd (latticeMotionCell p q impulse n)
    (latticeMotionCell p q impulse (n+1))
    (latticeScale (impulse n) (motion lattice p q impulse n).2) := by
  constructor
  · exact latticeMotionCell_adjacent_position_contact p q impulse n
  · change latticeVelocity (motion lattice p q impulse (n+1)).1
    (motion lattice p q impulse (n+1)).2 = _
    rw [← motion_adjacent_pair_position_contact lattice p q impulse n]
    exact motion_lattice_velocity_jump p q impulse n

theorem latticeMotionCell_zero_impulse_velocityContact (p q : LatticePoint)
  (impulse : Nat → Int) (n : Nat) (hzero : impulse n = 0) :
  velocityContact (latticeMotionCell p q impulse n)
    (latticeMotionCell p q impulse (n+1)) := by
  constructor
  · exact latticeMotionCell_adjacent_position_contact p q impulse n
  · change latticeVelocity (motion lattice p q impulse n).1
    (motion lattice p q impulse n).2 =
      latticeVelocity (motion lattice p q impulse (n+1)).1
    (motion lattice p q impulse (n+1)).2
    rw [← motion_adjacent_pair_position_contact lattice p q impulse n]
    rw [motion_lattice_velocity_jump, hzero]
    simp [latticeAdd, latticeScale]

theorem inward_impulses_same_swept (n : Nat) :
  swept lattice (1, 0) (1, 1) inwardOneRadialImpulse n =
  swept lattice (1, 0) (1, 1) inwardTwoRadialImpulse n := by
  rw [swept_eq, swept_eq]

theorem inward_impulses_distinct_next_vertex :
  (motion lattice (1, 0) (1, 1) inwardOneRadialImpulse 1).2 ≠
  (motion lattice (1, 0) (1, 1) inwardTwoRadialImpulse 1).2 := by
  decide

theorem equal_swept_area_does_not_identify_next_vertex :
  (∀ n, swept lattice (1, 0) (1, 1) inwardOneRadialImpulse n =
    swept lattice (1, 0) (1, 1) inwardTwoRadialImpulse n) ∧
  (motion lattice (1, 0) (1, 1) inwardOneRadialImpulse 1).2 ≠
  (motion lattice (1, 0) (1, 1) inwardTwoRadialImpulse 1).2 :=
  (inward_impulses_same_swept, inward_impulses_distinct_next_vertex)

```

NewtonLimitDynamics/Polygon/Finite.lean

```

theorem central_step_area (g : EuclideanConstruction Point Impulse)
  (p q : Point) (j : Impulse) : g.area q (step g p q j) = g.area p q := by
  rw [step, g.same_base_parallel, g.equal_base_altitude]

theorem all_cell_areas (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) :
  g.area (motion g p q impulse n).1 (motion g p q impulse n).2 = g.area p q := by
  induction n with
  | zero => rfl
  | succ n ih =>
    simp only [motion]
    rw [central_step_area]

```

```

exact ih

theorem sum_constant (f : Nat → Int) (c : Int) (hf : ∀ i, f i = c) (n : Nat) :
  isum f n = (n : Int) * c := by
  induction n with
  | zero => simp [isum]
  | succ n ih => simp [isum, ih, hf, Int.add_mul]

theorem swept_eq (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) : swept g p q impulse n = (n : Int) * g.area p q :=
  sum_constant _ _ (all_cell_areas g p q impulse) n

/-- Cross-multiplied area/time ratio. Positive common dt is stated; division by
total times additionally requires positive counts. No curve is produced. -/
theorem equal_time_area_reconstruction (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Int) (_hdt : 0 < dt) (m n : Nat) :
  swept g p q impulse m * ((n : Int) * dt) =
  swept g p q impulse n * ((m : Int) * dt) := by
  rw [swept_eq, swept_eq]
  ac_rfl

theorem nsum_constant (f : Nat → Nat) (c : Nat) (hf : ∀ i, f i = c) (n : Nat) :
  nsum f n = n * c := by
  induction n with
  | zero => simp [nsum]
  | succ n ih => simp [nsum, ih, hf, Nat.add_mul]

theorem all_unsigned_cell_areas (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
  congrArg Int.natAbs (all_cell_areas g p q impulse n)

theorem unsigned_block_eq (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (start count : Nat) :
  unsignedBlock g p q impulse start count = count * (g.area p q).natAbs :=
  nsum_constant _ _ (fun i => all_unsigned_cell_areas g p q impulse (start + i)) count

/-- Algebraic cross multiplication, valid also for zero counts. A time-ratio
interpretation additionally requires a positive cell and positive counts. -/
theorem unsigned_block_time_cross (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (dt start₁ start₂ m n : Nat) :
  unsignedBlock g p q impulse start₁ m * (n * dt) =
  unsignedBlock g p q impulse start₂ n * (m * dt) := by
  rw [unsigned_block_eq, unsigned_block_eq]
  ac_rfl

/-- Positive total times and the finite unsigned-area/time comparison, with
geometric interpretation and force direction still supplied separately. -/
theorem positive_unsigned_area_comparison (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start₁ start₂ m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
  unsignedBlock g p q impulse start₁ m * (n * dt) =
  unsignedBlock g p q impulse start₂ n * (m * dt) :=
  (Nat.mul_pos hm hdt, Nat.mul_pos hn hdt,
  unsigned_block_time_cross g p q impulse dt start₁ start₂ m n)

theorem extension_identity (p q : LatticePoint) : det q (extend p q) = det p q := by
  simp only [det, extend, Int.mul_sub, Int.mul_assoc]
  have h : q.1 * (2 * q.2) = q.2 * (2 * q.1) := by ac_rfl
  have h1 : q.1 * p.2 = p.2 * q.1 := Int.mul_comm _ _
  have h2 : q.2 * p.1 = p.1 * q.2 := Int.mul_comm _ _
  omega

theorem parallel_identity (q x : LatticePoint) (j : Int) : det q (kick q x j) = det q x := by
  simp only [det, kick, Int.mul_add]
  have h : q.1 * (j * q.2) = q.2 * (j * q.1) := by ac_rfl
  omega

/-- Checked orientation control: signed and unsigned sums agree here. -/

```

```

theorem positive_orientation_unsigned_example :
  swept lattice (1, 0) (1, 1) (fun _ => 0) 3 = 3 ∧
  unsignedBlock lattice (1, 0) (1, 1) (fun _ => 0) 0 3 = 3 := by
  decide

/-- Reversing orientation preserves unsigned magnitude, not the signed sum. -/
theorem negative_orientation_unsigned_example :
  swept lattice (1, 0) (1, -1) (fun _ => 0) 3 = -3 ∧
  unsignedBlock lattice (1, 0) (1, -1) (fun _ => 0) 0 3 = 3 ∧
  swept lattice (1, 0) (1, -1) (fun _ => 0) 3 ≠
  (unsignedBlock lattice (1, 0) (1, -1) (fun _ => 0) 0 3 : Int) := by
  decide

/-- Radial degeneracy, rest, and empty blocks need no area division. -/
theorem degenerate_unsigned_examples :
  unsignedBlock lattice (1, 0) (2, 0) (fun _ => -1) 2 3 = 0 ∧
  unsignedBlock lattice (1, 0) (1, 0) (fun _ => 0) 2 3 = 0 ∧
  unsignedBlock lattice (1, 0) (1, 1) (fun _ => 0) 4 0 = 0 := by
  decide

/-- Even inward radial impulses can revisit triangles. The unsigned cell sum
counts repeated coverage and therefore cannot identify a sector union. -/
theorem repeated_triangle_coverage_example :
  motion lattice (1, 0) (0, 1) (fun _ => -2) 4 = ((1, 0), (0, 1)) ∧
  unsignedBlock lattice (1, 0) (0, 1) (fun _ => -2) 0 4 = 4 ∧
  unsignedBlock lattice (1, 0) (0, 1) (fun _ => -2) 0 8 = 8 := by
  decide

```

NewtonLimitDynamics/Polygon/HarmonicAccumulation.lean — Finite global comparison of actual harmonic end-kick schedules with common elapsed time. This is a coordinate L1 state budget only. It constructs no limiting trajectory and makes no claim about the nonnegative region between polygonal paths or the separate Kepler swept areas. The De Motu, 1687, and 1713 historical stages remain distinct from this modern rational estimate.

```

/-- The actual fine-minus-coarse state displacement, as represented rational
values. The first component is the position error, the second the velocity
error. -/
theorem local_error_identity (w h : Fraction) (s : Point × Point) :
  stateEquiv (stateSub (HarmonicRefinement.fine w h s) (HarmonicRefinement.coarse w h s))
    (pointScale (negF (localA w h)) (middle w h s),
     pointAdd (pointScale (localA w h) s.2)
      (pointScale (localC w h) (middle w h s))) := by
  constructor <=> constructor <=>
  simp only [stateSub, localA, localC, HarmonicRefinement.fine,
    HarmonicRefinement.coarse, middle,
    pointEquiv, pointSub, pointNeg, cell, linearField, negF,
    pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

private theorem localA_abs (w h : Fraction) :
  Fraction.equiv (localA w h).abs
    (Fraction.mul (Fraction.mul h.abs h.abs) w.abs) := by
  simp only [localA, Fraction.equiv, Fraction.abs, Fraction.mul,
    Int.natAbs_mul, Int.ofNat_mul]

private theorem neg_localA_abs (w h : Fraction) :
  Fraction.equiv (negF (localA w h)).abs
    (Fraction.mul (Fraction.mul h.abs h.abs) w.abs) :=
  Fraction.equiv_trans (Fraction.abs_neg (localA w h)) (localA_abs w h)

private theorem localC_abs (w h : Fraction) :
  Fraction.equiv (localC w h).abs
    (Fraction.mul (Fraction.mul (Fraction.mul h.abs h.abs) w.abs)
      (Fraction.mul h.abs w.abs)) := by
  simp only [localC, Fraction.equiv, Fraction.abs, Fraction.mul,
    Int.natAbs_mul, Int.ofNat_mul]

```

```

ac_nf

private theorem middle_norm_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le (pointNorm (middle w h s))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) h.abs) (stateNorm s)) :=
  Fraction.magnitudes.le_trans (point_le_state (drift h s)) (drift_bound h s)

private theorem amplitude_nonnegative (w h : Fraction) :
  0 ≤ (amplitude w h).num :=
  Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _))
    (Fraction.abs_num_nonnegative _)

private theorem kickMagnitude_nonnegative (w h : Fraction) :
  0 ≤ (kickMagnitude w h).num :=
  Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _)

/-- Triangle and scaling estimate for the explicit local mismatch. -/
theorem local_error_expanded_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le
    (stateNorm (stateSub (HarmonicRefinement.fine w h s)
      (HarmonicRefinement.coarse w h s)))
    (Fraction.add
      (Fraction.mul (amplitude w h) (pointNorm (middle w h s)))
      (Fraction.add
        (Fraction.mul (amplitude w h) (pointNorm s.2))
        (Fraction.mul (Fraction.mul (amplitude w h) (kickMagnitude w h))
          (pointNorm (middle w h s)))))) := by
  let y := middle w h s
  let a := pointScale (negF (localA w h)) y
  let b := pointScale (localA w h) s.2
  let c := pointScale (localC w h) y
  have he := stateNorm_equiv (local_error_identity w h s)
  have ht := Fraction.add_le_add_left (pointNorm_add_le b c) (pointNorm a)
  have hp : Fraction.equiv (pointNorm a)
    (Fraction.mul (amplitude w h) (pointNorm y)) :=
    Fraction.equiv_trans (pointNorm_scale _ _)
      (Fraction.mul_equiv (neg_localA_abs w h) (Fraction.equiv_refl _))
  have hv : Fraction.equiv (pointNorm b)
    (Fraction.mul (amplitude w h) (pointNorm s.2)) :=
    Fraction.equiv_trans (pointNorm_scale _ _)
      (Fraction.mul_equiv (localA_abs w h) (Fraction.equiv_refl _))
  have hc : Fraction.equiv (pointNorm c)
    (Fraction.mul (Fraction.mul (amplitude w h) (kickMagnitude w h))
      (pointNorm y)) :=
    Fraction.equiv_trans (pointNorm_scale _ _)
      (Fraction.mul_equiv (localC_abs w h) (Fraction.equiv_refl _))
  exact Fraction.le_equiv_left he
    (Fraction.le_equiv_right ht (Fraction.add_equiv hp (Fraction.add_equiv hv hc)))

/-- The local defect of two actual half-cells against one full cell is bounded
by  $|h|^2|w|(\kappa+1)$  times the current state magnitude. -/
theorem local_error_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le
    (stateNorm (stateSub (HarmonicRefinement.fine w h s)
      (HarmonicRefinement.coarse w h s)))
    (Fraction.mul (localFactor w h) (stateNorm s)) := by
  have h₀ := local_error_expanded_bound w h s
  have hy := middle_norm_bound w h s
  have hv := velocity_le_state s
  have ha := amplitude_nonnegative w h
  have hat := Fraction.nonnegative_mul (amplitude w h) (kickMagnitude w h) ha
    (kickMagnitude_nonnegative w h)
  have h₁ := Fraction.mul_le_mul_nonnegative_left hy (amplitude w h) ha
  have h₂ := Fraction.mul_le_mul_nonnegative_left hv (amplitude w h) ha
  have h₃ := Fraction.mul_le_mul_nonnegative_left hy
    (Fraction.mul (amplitude w h) (kickMagnitude w h)) hat
  have hs := Fraction.add_le_add h₁ (Fraction.add_le_add h₂ h₃)
  have hchain := Fraction.magnitudes.le_trans h₀ hs
  apply Fraction.le_equiv_right hchain

```

```

simp only [localFactor, amplitude, kickMagnitude, kappa, Fraction.equiv,
  Fraction.add, Fraction.mul, Fraction.ofInt]
simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
ac_nf

private theorem coarseAt_comm (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    coarseAt w h (HarmonicRefinement.coarse w h s) n =
      HarmonicRefinement.coarse w h (coarseAt w h s n)
| 0 => rfl
| n + 1 => by
  simp only [coarseAt]
  rw [coarseAt_comm w h s n]

private theorem fineAt_comm (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    fineAt w h (HarmonicRefinement.fine w h s) n =
      HarmonicRefinement.fine w h (fineAt w h s n)
| 0 => rfl
| n + 1 => by
  simp only [fineAt]
  rw [fineAt_comm w h s n]

/-- The coarse recurrence is the actual list schedule of `n` full cells. -/
theorem coarseAt_schedule (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    schedule (linearField w) (List.replicate n (Fraction.add h h)) s =
      coarseAt w h s n
| 0 => rfl
| n + 1 => by
  simp only [List.replicate_succ, schedule]
  change schedule (linearField w) (List.replicate n (Fraction.add h h))
    (HarmonicRefinement.coarse w h s) = coarseAt w h s (n + 1)
  rw [coarseAt_schedule w h (HarmonicRefinement.coarse w h s) n]
  exact coarseAt_comm w h s n

/-- The fine recurrence is the actual schedule of `2n` half-cells. -/
theorem fineAt_schedule (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    schedule (linearField w) (fineDurations h n) s = fineAt w h s n
| 0 => rfl
| n + 1 => by
  simp only [fineDurations, schedule]
  change schedule (linearField w) (fineDurations h n)
    (HarmonicRefinement.fine w h s) = fineAt w h s (n + 1)
  rw [fineAt_schedule w h (HarmonicRefinement.fine w h s) n]
  exact fineAt_comm w h s n

/-- Both actual lists have the same represented elapsed duration. -/
theorem schedules_common_time (w h : Fraction) :
  (n : Nat) →
    Fraction.equiv
      (elapsed (List.replicate n (Fraction.add h h)))
      (elapsed (fineDurations h n))
| 0 => Fraction.equiv_refl _
| n + 1 => by
  simp only [List.replicate_succ, fineDurations, elapsed]
  exact Fraction.equiv_trans
    (Fraction.add_assoc h h (elapsed (List.replicate n (Fraction.add h h))))
    (Fraction.add_equiv (Fraction.equiv_refl h)
      (Fraction.add_equiv (Fraction.equiv_refl h) (schedules_common_time w h n)))

theorem kappa_nonnegative (w h : Fraction) : 0 ≤ (kappa w h).num := by
  unfold kappa
  apply Fraction.nonnegative_mul
  · exact Fraction.nonnegative_add _ _ (by decide) (Fraction.abs_num_nonnegative _)
  · exact Fraction.nonnegative_add _ _ (by decide) (kickMagnitude_nonnegative w h)

theorem localFactor_nonnegative (w h : Fraction) :
  0 ≤ (localFactor w h).num :=

```

```

    Fraction.nonnegative_mul _ _ (amplitude_nonnegative w h)
      (Fraction.nonnegative_add _ _ (kappa_nonnegative w h) (by decide))

theorem fineFactor_nonnegative (w h : Fraction) :
  0 ≤ (fineFactor w h).num :=
  Fraction.nonnegative_mul _ _ (kappa_nonnegative w h) (kappa_nonnegative w h)

private theorem coarseFactor_nonnegative (w h : Fraction) :
  0 ≤ (coarseFactor w h).num := kappa_nonnegative w (Fraction.add h h)

theorem errorBudget_nonnegative (w h : Fraction) (s : Point × Point) :
  (n : Nat) → 0 ≤ (errorBudget w h s n).num
| 0 => by simp [errorBudget, Fraction.ofInt]
| n + 1 =>
  Fraction.nonnegative_add _ _
    (Fraction.nonnegative_mul _ _ (fineFactor_nonnegative w h)
      (errorBudget_nonnegative w h s n))
    (Fraction.nonnegative_mul _ _
      (Fraction.nonnegative_mul _ _ (localFactor_nonnegative w h)
        (fpower_nonnegative _ (coarseFactor_nonnegative w h) n))
      (stateNorm_nonnegative s))

/-- Two actual fine cells carry an input perturbation by at most `kappa^2`. -/
theorem fine_perturbation (w h : Fraction) (s t : Point × Point) :
  Fraction.le
    (stateNorm (stateSub (HarmonicRefinement.fine w h s)
      (HarmonicRefinement.fine w h t)))
    (Fraction.mul (fineFactor w h) (stateNorm (stateSub s t))) := by
  have h1 := cell_perturbation w h (cell (linearField w) h s)
  (cell (linearField w) h t)
  have h2 := cell_perturbation w h s t
  have hm := Fraction.mul_le_mul_nonnegative_left h2 (kappa w h)
    (kappa_nonnegative w h)
  have hc := Fraction.magnitudes.le_trans h1 hm
  apply Fraction.le_equiv_right hc
  simp only [fineFactor, Fraction.equiv, Fraction.mul]
  ac_nf

/-- Every actual coarse state is bounded by `b^n` times the initial state
magnitude, with `b = kappa(w,h+h)`. -/
theorem coarse_norm_bound (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    Fraction.le (stateNorm (coarseAt w h s n))
      (Fraction.mul (fpower (coarseFactor w h) n) (stateNorm s))
| 0 => by
  apply Fraction.le_of_equiv
  simp only [coarseAt, fpower, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp only [Int.one_mul, Int.mul_one]
| n + 1 => by
  have hc := cell_bound w (Fraction.add h h) (coarseAt w h s n)
  have hi := coarse_norm_bound w h s n
  have hm := Fraction.mul_le_mul_nonnegative_left hi (coarseFactor w h)
    (coarseFactor_nonnegative w h)
  have hchain := Fraction.magnitudes.le_trans hc hm
  apply Fraction.le_equiv_right hchain
  simp only [coarseAt, fpower, coarseFactor, Fraction.equiv, Fraction.mul]
  ac_nf

/-- The actual endpoints after `n` common blocks obey the recursively
constructed finite error budget. The recurrence uses the coarse state at each
block for the local defect and propagates the previous actual endpoint error
through two actual fine cells. -/
theorem actual_error_bound (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    Fraction.le (stateNorm (stateSub (fineAt w h s n) (coarseAt w h s n)))
      (errorBudget w h s n)
| 0 => Fraction.le_of_equiv (stateSub_self_norm_zero s)
| n + 1 => by
  let F := fineAt w h s n
  let C := coarseAt w h s n

```

```

have ht := stateSub_triangle
  (HarmonicRefinement.fine w h F)
  (HarmonicRefinement.fine w h C)
  (HarmonicRefinement.coarse w h C)
have hp := fine_perturbation w h F C
have hl := local_error_bound w h C
have hraw := Fraction.magnitudes.le_trans ht (Fraction.add_le_add hp hl)
have hi := actual_error_bound w h s n
have hc := coarse_norm_bound w h s n
have hbi := Fraction.mul_le_mul_nonnegative_left hi (fineFactor w h)
  (fineFactor_nonnegative w h)
have hbc := Fraction.mul_le_mul_nonnegative_left hc (localFactor w h)
  (localFactor_nonnegative w h)
have hbudget := Fraction.add_le_add hbi hbc
have hchain := Fraction.magnitudes.le_trans hraw hbudget
apply Fraction.le_equiv_right hchain
simp only [errorBudget, Fraction.equiv, Fraction.add, Fraction.mul]
simp only [Int.add_mul, Int.mul_add]
ac_nf

theorem sample_zero_blocks :
  stateEquiv (fineAt one zero sample 1) (coarseAt one zero sample 1) := by decide

theorem sample_zero_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one zero sample 1) (coarseAt one zero sample 1)))
  zero := by decide

theorem sample_zero_budget :
  Fraction.equiv (errorBudget one zero sample 1) zero := by decide

theorem sample_initial_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one half sample 0) (coarseAt one half sample 0)))
  zero := by decide

theorem sample_local_factor :
  Fraction.equiv (localFactor one half) (13, 16, by decide) := by decide

/-- Exact one-block state error. Its four component magnitudes are `1/4`,
`1/8`, `1/8`, and `5/16`; their sum is `13/16`. -/
theorem sample_one_block_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one half sample 1) (coarseAt one half sample 1)))
  (13, 16, by decide) := by decide

theorem sample_one_block_budget :
  Fraction.equiv (errorBudget one half sample 1) (13, 8, by decide) := by decide

theorem sample_two_block_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one half sample 2) (coarseAt one half sample 2)))
  (173, 256, by decide) := by decide

theorem sample_one_block_distinct :
  ~ stateEquiv (fineAt one half sample 1) (coarseAt one half sample 1) := by decide

```

NewtonLimitDynamics/Polygon/HarmonicBinaryPrefix.lean — Actual intermediate prefixes of one dyadic harmonic polygon family, indexed by binary addresses. The approximants are finite schedules only. No completed point, continuum trajectory, or region between polygon and trajectory is assumed; Kepler swept area remains a distinct quantity.

```

theorem prefix_coarse (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat) :
  stateEquiv (prefixState b w T s j)
    (coarseAt w (duration T (j + 1)) s (ticks b j)) := by
  let h := duration T (j + 1)
  have hc := schedule_replicate_congr w (duration T j) (Fraction.add h h)
  (duration_halving T j) (ticks b j) s s

```

```

    ({Fraction.equiv_refl _, Fraction.equiv_refl _},
     {Fraction.equiv_refl _, Fraction.equiv_refl _})
  simpa only [prefixState, coarseAt_schedule] using hc

theorem prefix_state_le_two (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le (stateNorm (prefixState b w T s j))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  let h := duration T (j + 1)
  have hprefix := smallTime_prefix w h (ticks b j) (blocks j) hT
  (ticks_le_blocks b j) (dyadic_smallTime w T j hs)
  have hc := coarse_state_le_two w h s (ticks b j) hT hprefix
  exact Fraction.le_equiv_left
    (stateNorm_equiv (prefix_coarse b w T s j)) hc

theorem elapsed_replicate (d : Fraction) :
  (n : Nat) → Fraction.equiv (elapsed (List.replicate n d))
    (Fraction.mul (Fraction.ofInt (n : Int)) d)
| 0 => by
  simpa only [elapsed, List.replicate_zero, Fraction.equiv,
    Fraction.ofInt, Fraction.mul]
  simpa
| n + 1 => by
  have ih := elapsed_replicate d n
  have he := Fraction.add_equiv (Fraction.equiv_refl d) ih
  apply Fraction.equiv_trans he
  simpa only [elapsed, List.replicate_succ, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.ofInt, Int.natCast_add]
  simpa only [Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

theorem prefix_elapsed (b : Nat → Bool) (T : Fraction) (j : Nat) :
  Fraction.equiv
    (elapsed (List.replicate (ticks b j) (duration T j)))
    (Fraction.mul (Fraction.ofInt (ticks b j : Int)) (duration T j)) :=
  elapsed_replicate (duration T j) (ticks b j)

theorem prefix_elapsed_le_time (b : Nat → Bool) (T : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.le (elapsed (List.replicate (ticks b j) (duration T j))) T := by
  let d := duration T j
  have hcount : Fraction.le (Fraction.ofInt (ticks b j : Int))
    (Fraction.ofInt (blocks j : Int)) := by
    unfold Fraction.le Fraction.ofInt
    dsimp
    simpa only [Int.mul_one]
    exact Int.ofNat_le_mpr (ticks_le_blocks b j)
  have hm := Fraction.mul_le_mul_nonnegative hcount d hT
  have hfull := endpoint_elapsed T j
  have he := elapsed_replicate d (blocks j)
  have hbound := Fraction.le_equiv_left
    (prefix_elapsed b T j) hm
  exact Fraction.le_equiv_right hbound
    (Fraction.equiv_trans (Fraction.equiv_symm he) hfull)

theorem schedule_replicate_step (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
  schedule (linearField w) (List.replicate (n + 1) h) s =
    cell (linearField w) h
    (schedule (linearField w) (List.replicate n h) s)
| 0 => rfl
| n + 1 => by
  simpa only [List.replicate_succ, schedule]
  change schedule (linearField w) (List.replicate (n + 1) h)
    (cell (linearField w) h s) =
    cell (linearField w) h
    (schedule (linearField w) (List.replicate n h)
      (cell (linearField w) h s))
  rw [schedule_replicate_step w h (cell (linearField w) h s) n]

```

```

theorem prefix_next (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat) :
  prefixState b w T s (j + 1) =
    if b j then
      cell (linearField w) (duration T (j + 1))
        (fineAt w (duration T (j + 1)) s (ticks b j))
    else fineAt w (duration T (j + 1)) s (ticks b j) := by
  let h := duration T (j + 1)
  by_cases hb : b j
  · simp only [prefixState, ticks_next]
    simp only [bit, hb, ↑reduceIte]
    change schedule (linearField w)
      (List.replicate ((ticks b j + ticks b j) + 1) h) s = _
    rw [schedule_replicate_step]
    rw [← fineDurations_replicate]
    exact congrArg (cell (linearField w) h)
      (fineAt_schedule w h s (ticks b j))
  · simp only [prefixState, ticks_next]
    simp [bit, hb] at *
    rw [← fineDurations_replicate]
    exact fineAt_schedule w h s (ticks b j)

private theorem scalar_one_bound (a b c : Fraction)
  (ha : 0 ≤ a.num) :
  Fraction.le
    (Fraction.add b (Fraction.mul c (Fraction.add a b)))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) c)
      (Fraction.add a b)) := by
  let M := Fraction.add a b
  have hbM : Fraction.le b M := by
    unfold Fraction.le M Fraction.add
    dsimp
    rw [Int.add_mul]
    have hnon := Int.mul_nonneg
      (Int.mul_nonneg ha (Int.le_of_lt b.den_pos)) (Int.le_of_lt b.den_pos)
    have he : b.num * (a.den * b.den) = b.num * a.den * b.den := by ac_rfl
    rw [he]
    omega
  have hfirst := Fraction.add_le_add_right hbM (Fraction.mul c M)
  apply Fraction.le_equiv_right hfirst
  simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  simp only [M, Fraction.add]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem cell_parameter_bound_one (w sigma tau : Fraction) (s : Point × Point)
  (hs : 0 ≤ sigma.num) (ht : 0 ≤ tau.num)
  (hsum : Fraction.le (Fraction.add sigma tau) (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub (cell (linearField w) tau s)
      (cell (linearField w) sigma s)))
    (Fraction.mul (durationDifference sigma tau).abs
      (Fraction.mul
        (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm s))) := by
  let d := durationDifference sigma tau
  have hp := short_sum_point_bound sigma tau s.1 s.2 hs ht hsum
  have hw := Fraction.mul_le_mul_nonnegative_left hp w.abs
  (Fraction.abs_num_nonnegative w)
  have ha := Fraction.add_le_add_left hw (pointNorm s.2)
  have hd := Fraction.mul_le_mul_nonnegative_left ha d.abs
  (Fraction.abs_num_nonnegative d)
  have hc := scalar_one_bound (pointNorm s.1) (pointNorm s.2) w.abs
  (pointNorm_nonnegative s.1)
  have hdc := Fraction.mul_le_mul_nonnegative_left hc d.abs
  (Fraction.abs_num_nonnegative d)
  have hf := cell_parameter_norm_formula w sigma tau s
  exact Fraction.le_equiv_right
    (Fraction.magnitudes.le_trans (Fraction.le_equiv_left hf hd) hdc)

```

```

    (by simp only [stateNorm]; exact Fraction.equiv_refl _)

theorem cell_increment_bound (w h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) (hsmall : Fraction.le h (Fraction.ofInt 1)) :
  Fraction.le (stateNorm (stateSub (cell (linearField w) h s) s))
    (Fraction.mul h.abs
      (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm s))) := by
  have hsum : Fraction.le (Fraction.add zero h) (Fraction.ofInt 1) :=
    Fraction.le_equiv_left (by
      simp only [zero, Fraction.equiv, Fraction.add]
      simp) hsmall
  have hb := cell_parameter_bound_one w zero h s (by decide) hh hsum
  have he : stateEquiv
    (stateSub (cell (linearField w) h s) s)
    (stateSub (cell (linearField w) h s) (cell (linearField w) zero s)) :=
    stateSub_congr
      ((Fraction.equiv_refl _, Fraction.equiv_refl _),
       (Fraction.equiv_refl _, Fraction.equiv_refl _))
      ((Fraction.equiv_symm (zero_step w s).1.1,
        Fraction.equiv_symm (zero_step w s).1.2),
       (Fraction.equiv_symm (zero_step w s).2.1,
        Fraction.equiv_symm (zero_step w s).2.2))
  have hfirst := Fraction.le_equiv_left (stateNorm_equiv he) hb
  apply Fraction.le_equiv_right hfirst
  simp only [durationDifference, zero, negF, Fraction.equiv,
    Fraction.abs, Fraction.add, Fraction.mul]
  simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
    Int.natAbs_zero, Int.ofNat_zero, Int.mul_one, Int.one_mul,
    Int.neg_zero]

theorem fine_prefix_state_le_two (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (fineAt w (duration T (j + 1)) s (ticks b j)))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  let h := duration T (j + 1)
  have hpref := smallTime_prefix w h (ticks b j) (blocks j) hT
  (ticks_le_blocks b j) (dyadic_smallTime w T j hs)
  exact fine_state_le_two w h s (ticks b j) hT hpref

theorem prefix_totalTime_le (b : Nat → Bool) (T : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.le (totalTime (duration T (j + 1)) (ticks b j)) T := by
  let h := duration T (j + 1)
  have he : Fraction.equiv (totalTime h (ticks b j))
    (elapsed (List.replicate (ticks b j) (duration T j))) :=
    Fraction.equiv_trans
      (Fraction.equiv_symm (coarse_elapsed_totalTime h (ticks b j)))
      (elapsed_replicate_congr
        (Fraction.equiv_symm (duration_halving T j)) (ticks b j))
  exact Fraction.le_equiv_left he (prefix_elapsed_le_time b T j hT)

theorem fine_optional_increment (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub
      (cell (linearField w) (duration T (j + 1))
        (fineAt w (duration T (j + 1)) s (ticks b j)))
        (fineAt w (duration T (j + 1)) s (ticks b j))))
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (duration T (j + 1))
        (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs)
          (stateNorm s)))) := by
  let h := duration T (j + 1)
  let q := fineAt w h s (ticks b j)
  have hle : Fraction.le h (Fraction.ofInt 1) := by
    have h1 := duration_le_time T (j + 1) hT
    have h2 := dyadic_time_le_half w T hT hs

```

```

    have h3 : Fraction.le ((1, 2, by decide) : Fraction)
      (Fraction.ofInt 1) := by unfold Fraction.le; decide
    exact Fraction.magnitudes.le_trans h1
      (Fraction.magnitudes.le_trans h2 h3)
  have hb := cell_increment_bound w h q hT hle
  have hq := fine_prefix_state_le_two b w T s j hT hs
  have hm1 := Fraction.mul_le_mul_nonnegative_left hq
    (Fraction.add (Fraction.ofInt 1) w.abs) (by
      unfold Fraction.add Fraction.ofInt Fraction.abs
      dsimp
      have hw := Int.ofNat_nonneg w.num.natAbs
      have hd := w.den_pos
      omega)
  have hm2 := Fraction.mul_le_mul_nonnegative_left hm1 h hT
  have habs := Fraction.abs_of_nonnegative h hT
  have he := Fraction.mul_equiv habs
    (Fraction.equiv_refl
      (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm q)))
  have hchain := Fraction.magnitudes.le_trans
    (Fraction.le_equiv_right hb he) hm2
  apply Fraction.le_equiv_right hchain
  simp only [h]
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

theorem prefix_refinement_error (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub
      (fineAt w (duration T (j + 1)) s (ticks b j))
      (prefixState b w T s j)))
    (refinementCap w T s j) := by
  let h := duration T (j + 1)
  let n := ticks b j
  have hs' := smallTime_prefix w h n (blocks j) hT
    (ticks_le_blocks b j) (dyadic_smallTime w T j hs)
  have hb := actual_uniform_error w h s n hT hs'
  have he : stateEquiv
    (stateSub (fineAt w h s n) (prefixState b w T s j))
    (stateSub (fineAt w h s n) (coarseAt w h s n)) :=
  stateSub_congr
    ((Fraction.equiv_refl _, Fraction.equiv_refl _),
      (Fraction.equiv_refl _, Fraction.equiv_refl _))
    (prefix_coarse b w T s j)
  have hfirst := Fraction.le_equiv_left (stateNorm_equiv he) hb
  have htime := prefix_totalTime_le b T j hT
  have hnon : 0 ≤ (Fraction.mul h (Fraction.mul w.abs (stateNorm s))).num :=
  Int.mul_nonneg hT
    (Int.mul_nonneg (Fraction.abs_num_nonnegative w)
      (stateNorm_nonnegative s))
  have hm1 := Fraction.mul_le_mul_nonnegative htime
    (Fraction.mul h (Fraction.mul w.abs (stateNorm s))) hnon
  have hm2 := Fraction.mul_le_mul_nonnegative_left hm1
    (Fraction.ofInt 3) (by decide)
  exact Fraction.magnitudes.le_trans hfirst hm2

private theorem optionalCap_nonnegative (w T : Fraction) (s : Point × Point)
  (j : Nat) (hT : 0 ≤ T.num) :
  0 ≤ (optionalCap w T s j).num := by
  unfold optionalCap
  have hfactor : 0 ≤ (Fraction.add (Fraction.ofInt 1) w.abs).num := by
  unfold Fraction.add Fraction.ofInt Fraction.abs
  dsimp
  have hw := Int.ofNat_nonneg w.num.natAbs
  have hd := w.den_pos
  omega
  exact Int.mul_nonneg (by decide)
    (Int.mul_nonneg hT
      (Int.mul_nonneg hfactor (stateNorm_nonnegative s)))

```

```

private theorem le_add_optional (a c : Fraction) (hc : 0 ≤ c.num) :
  Fraction.le a (Fraction.add c a) := by
  have hz : Fraction.le (Fraction.ofInt 0) c := by
    unfold Fraction.le Fraction.ofInt
    dsimp
    simp only [Int.zero_mul, Int.mul_one]
    exact hc
  have h := Fraction.add_le_add_left hz a
  have he : Fraction.equiv (Fraction.add a (Fraction.ofInt 0)) a := by
    simp only [Fraction.equiv, Fraction.add, Fraction.ofInt]
    simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
      Int.mul_one, Int.one_mul]
  exact Fraction.le_equiv_right
    (Fraction.le_equiv_left (Fraction.equiv_symm he) h)
    (Fraction.add_comm a c)

theorem adjacent_error_le_add (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub (prefixState b w T s (j + 1))
      (prefixState b w T s j)))
    (adjacentCap w T s j) := by
  have hnext := prefix_next b w T s j
  have href := prefix_refinement_error b w T s j hT hs
  by_cases hb : b j
  · rw [hnext, if_pos hb]
    have htri := stateSub_triangle
      (cell (linearField w) (duration T (j + 1))
        (fineAt w (duration T (j + 1)) s (ticks b j)))
      (fineAt w (duration T (j + 1)) s (ticks b j))
      (prefixState b w T s j)
    exact Fraction.magnitudes.le_trans htri
      (Fraction.add_le_add (fine_optional_increment b w T s j hT hs) href)
  · rw [hnext, if_neg hb]
    exact Fraction.magnitudes.le_trans href
      (le_add_optional _ _ (optionalCap_nonnegative w T s j hT))

theorem adjacentCap_tail (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv (adjacentCap w T s j) (tailCap w T s (j + 1)) := by
  simp only [adjacentCap, optionalCap, refinementCap, tailCap,
    coefficient, duration, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt, Fraction.abs,
    Int.pow_succ]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf

theorem adjacent_error_le (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub (prefixState b w T s (j + 1))
      (prefixState b w T s j)))
    (tailCap w T s (j + 1)) :=
  Fraction.le_equiv_right (adjacent_error_le_add b w T s j hT hs)
    (adjacentCap_tail w T s j)

theorem coefficient_nonnegative (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) : 0 ≤ (coefficient w T s).num := by
  have hOneW : 0 ≤ (Fraction.add (Fraction.ofInt 1) w.abs).num := by
    unfold Fraction.add Fraction.ofInt Fraction.abs
    dsimp
    have hw := Int.ofNat_nonneg w.num.natAbs
    have hd := w.den_pos
    omega
  have h1 : 0 ≤ (Fraction.mul (Fraction.ofInt 2)
    (Fraction.add (Fraction.ofInt 1) w.abs)).num :=
    Int.mul_nonneg (by decide) hOneW
  have h2 : 0 ≤ (Fraction.mul (Fraction.ofInt 3)

```

```

    (Fraction.mul T w.abs)).num :=
    Int.mul_nonneg (by decide)
    (Int.mul_nonneg hT (Fraction.abs_num_nonnegative w))
  unfold coefficient
  exact Int.mul_nonneg hT
    (Int.mul_nonneg (stateNorm_nonnegative s)
      (Fraction.nonnegative_add _ _ h1 h2))

theorem tail_halving (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv
    (Fraction.add (tailCap w T s (j + 1)) (tailCap w T s (j + 1)))
    (tailCap w T s j) := by
  exact GeometricTail.tail_halving (coefficient w T s) j

theorem tail_double (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv (Fraction.add (tailCap w T s j) (tailCap w T s j))
    (doubleTail w T s j) := by
  exact GeometricTail.tail_double (coefficient w T s) j

theorem finite_gap_error (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  (k j : Nat) → Fraction.le
    (stateNorm (stateSub (prefixState b w T s (j + k))
      (prefixState b w T s j))) (tailCap w T s j) := by
  intro k j
  exact GeometricTail.finite_gap (prefixState b w T s) (coefficient w T s)
    (coefficient_nonnegative w T s hT)
    (fun i => adjacent_error_le b w T s i hT hs) k j

theorem two_sided_error (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (N m n : Nat) (hm : N ≤ m) (hn : N ≤ n) :
  Fraction.le
    (stateNorm (stateSub (prefixState b w T s m) (prefixState b w T s n)))
    (doubleTail w T s N) := by
  exact GeometricTail.two_sided (prefixState b w T s) (coefficient w T s)
    (coefficient_nonnegative w T s hT)
    (fun i => adjacent_error_le b w T s i hT hs) N m n hm hn

theorem doubleTail_lt_tolerance (w T : Fraction) (s : Point × Point)
  (eps : Fraction) (hT : 0 ≤ T.num) (heps : 0 < eps.num) :
  Fraction.lt (doubleTail w T s (modulus w T s eps)) eps := by
  exact GeometricTail.doubleTail_lt_tolerance (coefficient w T s) eps
    (coefficient_nonnegative w T s hT) heps

theorem prefix_cauchy (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  ∀ eps : Fraction, 0 < eps.num →
  ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
  Fraction.lt
    (stateNorm (stateSub (prefixState b w T s m)
      (prefixState b w T s n))) eps := by
  intro eps heps
  refine {modulus w T s eps, ?_}
  intro m n hm hn
  exact Fraction.magnitudes.lt_of_le_lt
    (two_sided_error b w T s hT hs _ m n hm hn)
    (doubleTail_lt_tolerance w T s eps hT heps)

theorem all_zero_prefix (w T : Fraction) (s : Point × Point) (j : Nat) :
  prefixState (fun _ => false) w T s j = s := by
  simp [prefixState, all_zero_ticks, schedule]

theorem zero_time_prefix (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat) (hT : T.num = 0) :
  stateEquiv (prefixState b w T s j) s := by
  exact zero_duration_schedule w (duration T j) (by exact hT) (ticks b j) s

theorem sample_ticks : ticks firstBit 1 = 1 ∧ ticks firstBit 2 = 2 := by decide

```

```

theorem sample_first_error :
  Fraction.equiv
    (stateNorm (stateSub
      (prefixState firstBit sampleOne sampleQuarter sampleState 1)
      (prefixState firstBit sampleOne sampleQuarter sampleState 0)))
    (17, 64, by decide) := by decide

theorem sample_coefficient :
  Fraction.equiv (coefficient sampleOne sampleQuarter sampleState)
    (19, 8, by decide) := by decide

theorem sample_second_error :
  Fraction.equiv
    (stateNorm (stateSub
      (prefixState firstBit sampleOne sampleQuarter sampleState 2)
      (prefixState firstBit sampleOne sampleQuarter sampleState 1)))
    (545, 65536, by decide) := by decide

```

NewtonLimitDynamics/Polygon/HarmonicComparison.lean — Finite perturbation comparisons for the actual harmonic end-kick cell. The L1 state magnitude is a chosen coordinate diagnostic and requires a unit calibration before interpreting position and velocity together physically.

```

private theorem zero_le_product (a b : Fraction) (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) :
  Fraction.le (Fraction.ofInt 0) (Fraction.mul a b) := by
  unfold Fraction.le Fraction.ofInt Fraction.mul
  dsimp
  simpa using Int.mul_nonneg ha hb

private theorem one_plus_bound_right (a b c : Fraction)
  (ha : 0 ≤ a.num) (hc : 0 ≤ c.num) :
  Fraction.le (Fraction.add (Fraction.add a b) (Fraction.mul c b))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) c) (Fraction.add a b)) := by
  let lhs := Fraction.add (Fraction.add a b) (Fraction.mul c b)
  have h := Fraction.add_le_add_left (zero_le_product c a hc ha) lhs
  have h' : Fraction.le lhs (Fraction.add lhs (Fraction.mul c a)) :=
    Fraction.le_equiv_left (Fraction.equiv_symm (Fraction.add_zero lhs)) h
  apply Fraction.le_equiv_right h'
  simp only [lhs, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf

private theorem one_plus_bound_left (a b c : Fraction)
  (hb : 0 ≤ b.num) (hc : 0 ≤ c.num) :
  Fraction.le (Fraction.add (Fraction.add a b) (Fraction.mul c a))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) c) (Fraction.add a b)) := by
  let lhs := Fraction.add (Fraction.add a b) (Fraction.mul c a)
  have h := Fraction.add_le_add_left (zero_le_product c b hc hb) lhs
  have h' : Fraction.le lhs (Fraction.add lhs (Fraction.mul c b)) :=
    Fraction.le_equiv_left (Fraction.equiv_symm (Fraction.add_zero lhs)) h
  apply Fraction.le_equiv_right h'
  simp only [lhs, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf

theorem cell_eq_kick_drift (w h : Fraction) (s : Point × Point) :
  cell (linearField w) h s = kick w h (drift h s) := rfl

theorem drift_bound (h : Fraction) (s : Point × Point) :
  Fraction.le (stateNorm (drift h s))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) h.abs) (stateNorm s)) := by
  have h₁ := Fraction.add_le_add_right (pointNorm_add_le s.1 (pointScale h s.2))
    (pointNorm s.2)
  have h₂ : Fraction.equiv
    (Fraction.add (Fraction.add (pointNorm s.1) (pointNorm (pointScale h s.2)))
      (pointNorm s.2))
    (Fraction.add (Fraction.add (pointNorm s.1) (pointNorm s.2))
      (Fraction.mul h.abs (pointNorm s.2))) := by
  have hs := pointNorm_scale h s.2

```

```

    have hh := Fraction.add_equiv (Fraction.equiv_refl (pointNorm s.1)) hs
    have hhh := Fraction.add_equiv hh (Fraction.equiv_refl (pointNorm s.2))
    exact Fraction.equiv_trans hhh (by
      simp only [Fraction.equiv, Fraction.add, Fraction.mul]
      simp only [Int.add_mul, Int.mul_add]
      ac_nf)
    have h₃ := Fraction.le_equiv_right h₁ h₂
    exact Fraction.magnitudes.le_trans h₃
      (one_plus_bound_right (pointNorm s.1) (pointNorm s.2) h.abs
        (pointNorm_nonnegative _) (Fraction.abs_num_nonnegative _))

private theorem kick_scale_norm (w h : Fraction) (p : Point) :
  Fraction.equiv (pointNorm (pointScale h (linearField w p)))
    (Fraction.mul (Fraction.mul h.abs w.abs) (pointNorm p)) := by
  simp only [Fraction.equiv, pointNorm, pointScale, linearField, negF,
    Fraction.abs, Fraction.add, Fraction.mul, Int.natAbs_mul,
    Int.natAbs_neg, Int.ofNat_mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem kick_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le (stateNorm (kick w h s))
    (Fraction.mul
      (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs))
      (stateNorm s)) := by
  have h₁ := Fraction.add_le_add_left
    (pointNorm_add_le s.2 (pointScale h (linearField w s.1))) (pointNorm s.1)
  have h₂ : Fraction.equiv
    (Fraction.add (pointNorm s.1)
      (Fraction.add (pointNorm s.2)
        (pointNorm (pointScale h (linearField w s.1)))))
    (Fraction.add (Fraction.add (pointNorm s.1) (pointNorm s.2))
      (Fraction.mul (Fraction.mul h.abs w.abs) (pointNorm s.1))) := by
  have hf := kick_scale_norm w h s.1
  have hh := Fraction.add_equiv (Fraction.equiv_refl (pointNorm s.2)) hf
  have hhh := Fraction.add_equiv (Fraction.equiv_refl (pointNorm s.1)) hh
  exact Fraction.equiv_trans hhh (by
    simp only [Fraction.equiv, Fraction.add, Fraction.mul]
    simp only [Int.add_mul, Int.mul_add]
    ac_nf)
  have h₃ := Fraction.le_equiv_right h₁ h₂
  exact Fraction.magnitudes.le_trans h₃
    (one_plus_bound_left (pointNorm s.1) (pointNorm s.2)
      (Fraction.mul h.abs w.abs) (pointNorm_nonnegative _)
      (Int.mul_nonneg (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _)))

/-- One actual harmonic end-kick cell amplifies the coordinate L1 state
magnitude by at most  $(1+|h|)(1+|h||w|)$ , including signed and zero data. -/
theorem cell_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le (stateNorm (cell (linearField w) h s))
    (Fraction.mul (kappa w h) (stateNorm s)) := by
  rw [cell_eq_kick_drift]
  have hk := kick_bound w h (drift h s)
  have hd := drift_bound h s
  have hc : 0 ≤ (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs)).num := by
    have hp := Int.mul_pos h.abs.den_pos w.abs.den_pos
    have hn := Int.mul_nonneg (Fraction.abs_num_nonnegative h)
      (Fraction.abs_num_nonnegative w)
    unfold Fraction.add Fraction.ofInt Fraction.mul
    dsimp
    omega
  have hm := Fraction.mul_le_mul_nonnegative_left hd
    (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs)) hc
  have hchain := Fraction.magnitudes.le_trans hk hm
  apply Fraction.le_equiv_right hchain
  simp only [kappa, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf

/-- Subtracting the outputs of two actual cells equals applying the same

```

```

linear harmonic cell to their input difference, as rational values. -/
theorem cell_difference (w h : Fraction) (s t : Point × Point) :
  stateEquiv (stateSub (cell (linearField w) h s) (cell (linearField w) h t))
    (cell (linearField w) h (stateSub s t)) := by
  constructor <=> constructor <=>
  simp only [zero, stateSub, pointEquiv, pointSub, pointNeg, cell, linearField,
    negF, pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- A one-step perturbation estimate for two actual cells under the same
linear field and duration. The input difference is a represented state. -/
theorem linearField_comparison_contract (w : Fraction) :
  FiniteEstimates.comparisonContract (linearField w) (linearField w)
    w.abs (Fraction.ofInt 0) := by
  intro p q
  have hs := FiniteEstimates.difference_scale (negF w) p q
  have he := Fraction.equiv_trans hs
  (Fraction.mul_equiv (Fraction.abs_neg w) (Fraction.equiv_refl _))
  exact Fraction.le_of_equiv (Fraction.equiv_trans he
    (Fraction.equiv_symm (Fraction.add_zero _)))

theorem cell_perturbation (w h : Fraction) (s t : Point × Point) :
  Fraction.le
    (stateNorm (stateSub (cell (linearField w) h s) (cell (linearField w) h t)))
    (Fraction.mul (kappa w h) (stateNorm (stateSub s t))) := by
  have hg := FiniteEstimates.cell_amplification (linearField w) (linearField w)
    h w.abs (Fraction.ofInt 0) s t (Fraction.abs_num_nonnegative w)
  (linearField_comparison_contract w)
  apply Fraction.le_equiv_right hg
  exact Fraction.equiv_trans (Fraction.add_equiv (Fraction.equiv_refl _)
    (Fraction.mul_zero h.abs)) (Fraction.add_zero _)

/-- Zero duration leaves the represented state unchanged in rational value. -/
theorem zero_step (w : Fraction) (s : Point × Point) :
  stateEquiv (cell (linearField w) zero s) s := by
  constructor <=> constructor <=>
  simp only [zero, stateSub, pointEquiv, pointSub, pointNeg, cell, linearField,
    negF, pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
    Int.zero_mul, Int.mul_zero, Int.add_zero] <=>
  ac_nf <=> omega

theorem zero_step_norm (w : Fraction) (s : Point × Point) :
  Fraction.equiv (stateNorm (cell (linearField w) zero s)) (stateNorm s) :=
  stateNorm_equiv (zero_step w s)

theorem zero_kappa (w : Fraction) : Fraction.equiv (kappa w zero) one := by
  simp only [kappa, zero, one, Fraction.equiv, Fraction.abs, Fraction.ofInt,
    Fraction.add, Fraction.mul]
  dsimp
  simp only [Int.zero_mul, Int.mul_zero, Int.add_zero, Int.mul_one, Int.one_mul]

theorem sample_kappa : Fraction.equiv (kappa one half) (9, 4, by decide) := by decide

theorem sample_initial_norm : Fraction.equiv (stateNorm sample) (Fraction.ofInt 2) := by decide

theorem sample_cell_norm :
  Fraction.equiv (stateNorm (cell (linearField one) half sample)) (11, 4, by decide) := by decide

theorem sample_cell_bound :
  Fraction.le (stateNorm (cell (linearField one) half sample))
    (Fraction.mul (kappa one half) (stateNorm sample)) := by
  unfold Fraction.le
  decide

```

NewtonLimitDynamics/Polygon/HarmonicCover.lean — Explicit finite square covers for matched pieces of the actual harmonic coarse and fine polygonal paths. The square area is counted with multiplicity over blocks. This is a geometric covering budget,

not the area of the union or the area between a polygon and a realized continuum trajectory. It is separate from Kepler's centre-swept area.

```

private theorem totalTime_nonnegative (h : Fraction) (n : Nat) (hh : 0 ≤ h.num) :
  0 ≤ (totalTime h n).num := by
  unfold totalTime Fraction.mul Fraction.ofInt
  exact Int.mul_nonneg (Int.mul_nonneg (by decide) (Int.ofNat_nonneg _)) hh

private theorem maxError_nonnegative (w h : Fraction) (s : Point × Point)
  (n : Nat) (hh : 0 ≤ h.num) : 0 ≤ (maxError w h s n).num :=
  Fraction.nonnegative_mul _ _ (by decide)
  (Fraction.nonnegative_mul _ _ (totalTime_nonnegative h n hh)
    (Fraction.nonnegative_mul _ _ hh
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w) (stateNorm_nonnegative s))))

private theorem halfDriftBudget_nonnegative (h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) : 0 ≤ (halfDriftBudget h s).num :=
  Fraction.nonnegative_mul _ _ (by decide) (Fraction.nonnegative_mul _ _ hh (stateNorm_nonnegative s))

private theorem fullDriftBudget_nonnegative (h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) : 0 ≤ (fullDriftBudget h s).num :=
  Fraction.nonnegative_mul _ _ (by decide) (Fraction.nonnegative_mul _ _ hh (stateNorm_nonnegative s))

theorem radius_nonnegative (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) : 0 ≤ (radius w h s n).num :=
  Fraction.nonnegative_add _ _ (fullDriftBudget_nonnegative h s hh)
  (maxError_nonnegative w h s n hh)

theorem squareArea_nonnegative (R : Fraction) (hR : 0 ≤ R.num) :
  0 ≤ (squareArea R).num := SquareOuterContent.squareArea_nonnegative R hR

theorem coverBudget_nonnegative (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) : 0 ≤ (coverBudget w h s n).num :=
  Fraction.nonnegative_mul _ _ (Int.ofNat_nonneg _)
  (squareArea_nonnegative _ (radius_nonnegative w h s n hh))

private theorem totalTime_le (h : Fraction) (i n : Nat)
  (hh : 0 ≤ h.num) (hin : i ≤ n) :
  Fraction.le (totalTime h i) (totalTime h n) := by
  have hi : (i : Int) ≤ (n : Int) := Int.ofNat_le.mpr hin
  have hcoef : 0 ≤ 2 * h.num * h.den :=
    Int.mul_nonneg (Int.mul_nonneg (by decide) hh) (Int.le_of_lt h.den_pos)
  have hm := Int.mul_le_mul_of_nonneg_right hi hcoef
  unfold Fraction.le totalTime Fraction.mul Fraction.ofInt
  dsimp
  simp only [Int.one_mul]
  calc
    2 * (i : Int) * h.num * h.den ≤
      2 * (n : Int) * h.num * h.den := by
        calc
          _ = (i : Int) * (2 * h.num * h.den) := by ac_rfl
          _ ≤ (n : Int) * (2 * h.num * h.den) := hm
          _ = _ := by ac_rfl

/-- Every earlier actual endpoint error is bounded by the final-count cap. -/
private theorem prefix_error_le_max (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le
    (stateNorm (stateSub (fineAt w h s i) (coarseAt w h s i)))
    (maxError w h s n) := by
  have hsmall := smallTime_prefix w h i n hh hin hs
  have he := actual_uniform_error w h s i hh hsmall
  have ht := totalTime_le h i n hh hin
  have hfactor : 0 ≤ (Fraction.mul h (Fraction.mul w. abs (stateNorm s))).num :=
    Fraction.nonnegative_mul _ _ hh
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w) (stateNorm_nonnegative s))
  have hm := Fraction.mul_le_mul_nonnegative ht
  (Fraction.mul h (Fraction.mul w. abs (stateNorm s))) hfactor
  have hm' := Fraction.mul_le_mul_nonnegative_left hm (Fraction.ofInt 3) (by decide)

```

```

exact Fraction.magnitudes.le_trans he hm'

private theorem drift_offset_le_state (d : Fraction) (hd : 0 ≤ d.num)
  (t : Point × Point) :
  Fraction.le
    (pointNorm (pointSub (pointAdd t.1 (pointScale d t.2)) t.1))
    (Fraction.mul d (stateNorm t)) := by
  have he := pointNorm_equiv (drift_offset d t.1 t.2)
  have hs := pointNorm_scale d t.2
  have hdabs := Fraction.abs_of_nonnegative d hd
  have hmul := Fraction.mul_equiv hdabs (Fraction.equiv_refl (pointNorm t.2))
  have hstart := Fraction.equiv_trans he (Fraction.equiv_trans hs hmul)
  have hv := velocity_le_state t
  have hm := Fraction.mul_le_mul_nonnegative_left hv d hd
  exact Fraction.le_equiv_left hstart hm

private theorem two_le_four (h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) :
  Fraction.le (halfDriftBudget h s) (fullDriftBudget h s) := by
  have ht : Fraction.le two four := by
    unfold Fraction.le two four Fraction.ofInt
    decide
  exact Fraction.mul_le_mul_nonnegative ht (Fraction.mul h (stateNorm s))
  (Fraction.nonnegative_mul _ _ hh (stateNorm_nonnegative s))

private theorem half_drift_le (h : Fraction) (s t : Point × Point)
  (hh : 0 ≤ h.num)
  (ht : Fraction.le (stateNorm t) (Fraction.mul two (stateNorm s))) :
  Fraction.le
    (pointNorm (pointSub (pointAdd t.1 (pointScale h t.2)) t.1))
    (halfDriftBudget h s) := by
  have h₀ := drift_offset_le_state h hh t
  have h₁ := Fraction.mul_le_mul_nonnegative_left ht h hh
  have hc := Fraction.magnitudes.le_trans h₀ h₁
  apply Fraction.le_equiv_right hc
  simp only [halfDriftBudget, two, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

private theorem full_drift_le (h : Fraction) (s t : Point × Point)
  (hh : 0 ≤ h.num)
  (ht : Fraction.le (stateNorm t) (Fraction.mul two (stateNorm s))) :
  Fraction.le
    (pointNorm (pointSub (pointAdd t.1 (pointScale (Fraction.add h h) t.2)) t.1))
    (fullDriftBudget h s) := by
  have hsum : 0 ≤ (Fraction.add h h).num :=
    Fraction.nonnegative_add h h hh hh
  have h₀ := drift_offset_le_state (Fraction.add h h) hsum t
  have h₁ := Fraction.mul_le_mul_nonnegative_left ht (Fraction.add h h) hsum
  have hc := Fraction.magnitudes.le_trans h₀ h₁
  apply Fraction.le_equiv_right hc
  simp only [fullDriftBudget, two, four, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt]
  simp only [show (4 : Int) = 2 + 2 by rfl,
    Int.add_mul, Int.mul_add]
  ac_nf

private theorem coarse_mid_le_half (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (coarseMid w h s i) (coarseStart w h s i)))
    (halfDriftBudget h s) :=
  half_drift_le h s (coarseAt w h s i) hh
  (coarse_state_le_two w h s i hh (smallTime_prefix w h i n hh hin hs))

private theorem fine_mid_own_le_half (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (fineMid w h s i) (fineStart w h s i)))
    (halfDriftBudget h s) :=
  half_drift_le h s (fineAt w h s i) hh

```

```

(fine_state_le_two w h s i hh (smallTime_prefix w h i n hh hin hs))

private theorem coarse_end_le_full (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (coarseEnd w h s i) (coarseStart w h s i)))
    (fullDriftBudget h s) :=
  full_drift_le h s (coarseAt w h s i) hh
  (coarse_state_le_two w h s i hh (smallTime_prefix w h i n hh hin hs))

private theorem fine_start_error_le (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (fineStart w h s i) (coarseStart w h s i)))
    (maxError w h s n) :=
  Fraction.magnitudes.le_trans
    (point_le_state (stateSub (fineAt w h s i) (coarseAt w h s i)))
    (prefix_error_le_max w h s i n hh hin hs)

private theorem fine_mid_le (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (fineMid w h s i) (coarseStart w h s i)))
    (Fraction.add (maxError w h s n) (halfDriftBudget h s)) := by
  have ht := pointSub_triangle (fineMid w h s i) (fineStart w h s i)
    (coarseStart w h s i)
  have h1 := fine_mid_own_le_half w h s i n hh hin hs
  have h2 := fine_start_error_le w h s i n hh hin hs
  have hc := Fraction.magnitudes.le_trans ht (Fraction.add_le_add h1 h2)
  exact Fraction.le_equiv_right hc
  (Fraction.add_comm (halfDriftBudget h s) (maxError w h s n))

private theorem fine_end_le (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i < n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (fineEnd w h s i) (coarseStart w h s i)))
    (Fraction.add (maxError w h s n) (fullDriftBudget h s)) := by
  have hnext : i + 1 ≤ n := Nat.succ_le_of_lt hin
  have ht := pointSub_triangle (fineEnd w h s i) (coarseEnd w h s i)
    (coarseStart w h s i)
  have h1 := fine_start_error_le w h s (i + 1) n hh hnext hs
  have h2 := coarse_end_le_full w h s i n hh (Nat.le_of_lt hin) hs
  exact Fraction.magnitudes.le_trans ht (Fraction.add_le_add h1 h2)

private theorem zero_le (R : Fraction) (hR : 0 ≤ R.num) :
  Fraction.le (Fraction.ofInt 0) R := by
  unfold Fraction.le Fraction.ofInt
  simp using hR

/-- Six actual vertices, each measured from the coarse block start, fit in
the same coordinate L1 ball of radius `4hM + Emax`. The midpoint is only a
subdivision of the coarse drift; it receives no impulse. -/
theorem actual_corners_in_ball (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i < n)
  (hs : SmallTime w h n) :
  let x := coarseStart w h s i
  let R := radius w h s n
  Fraction.le (pointNorm (pointSub x x)) R ∧
  Fraction.le (pointNorm (pointSub (coarseMid w h s i) x)) R ∧
  Fraction.le (pointNorm (pointSub (coarseEnd w h s i) x)) R ∧
  Fraction.le (pointNorm (pointSub (fineStart w h s i) x)) R ∧
  Fraction.le (pointNorm (pointSub (fineMid w h s i) x)) R ∧
  Fraction.le (pointNorm (pointSub (fineEnd w h s i) x)) R := by
  dsimp
  have hi : i ≤ n := Nat.le_of_lt hin
  have hE := maxError_nonnegative w h s n hh
  have hH := fullDriftBudget_nonnegative h s hh
  have hH2 := two_le_four h s hh
  have hHR : Fraction.le (fullDriftBudget h s) (radius w h s n) :=
    Fraction.le_add_nonnegative _ _ hE

```

```

have hER : Fraction.le (maxError w h s n) (radius w h s n) :=
  Fraction.le_equiv_right (Fraction.le_add_nonnegative _ _ hH)
    (Fraction.add_comm (maxError w h s n) (fullDriftBudget h s))
have hsum : Fraction.le
  (Fraction.add (maxError w h s n) (halfDriftBudget h s))
  (radius w h s n) :=
  Fraction.le_equiv_right
    (Fraction.add_le_add_left hH2 (maxError w h s n))
    (Fraction.add_comm (maxError w h s n) (fullDriftBudget h s))
have hlast : Fraction.equiv
  (Fraction.add (maxError w h s n) (fullDriftBudget h s))
  (radius w h s n) :=
  Fraction.add_comm _ _
refine (?!_, ?!, ?!, ?!, ?!, ?!)
· exact Fraction.le_equiv_left (pointSub_self_zero _)
  (zero_le _ (radius_nonnegative w h s n hh))
· exact Fraction.magnitudes.le_trans (coarse_mid_le_half w h s i n hh hi hs)
  (Fraction.magnitudes.le_trans hH2 hHR)
· exact Fraction.magnitudes.le_trans (coarse_end_le_full w h s i n hh hi hs) hHR
· exact Fraction.magnitudes.le_trans (fine_start_error_le w h s i n hh hi hs) hER
· exact Fraction.magnitudes.le_trans (fine_mid_le w h s i n hh hi hs) hsum
· exact Fraction.le_equiv_right (fine_end_le w h s i n hh hin hs) hlast

theorem firstPatch_square (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i < n)
  (hs : SmallTime w h n) (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda) :
  SquareContains (coarseStart w h s i) (radius w h s n)
    (firstPatch w h s i theta lambda) := by
  obtain (hc0, hc1, _, hf0, hf1, _) := actual_corners_in_ball w h s i n hh hin hs
  exact matchedPatch_square theta lambda ht hl _ _ _ _ _ hc0 hc1 hf0 hf1

theorem secondPatch_square (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i < n)
  (hs : SmallTime w h n) (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda) :
  SquareContains (coarseStart w h s i) (radius w h s n)
    (secondPatch w h s i theta lambda) := by
  obtain (_, hc0, hc1, _, hf0, hf1) := actual_corners_in_ball w h s i n hh hin hs
  exact matchedPatch_square theta lambda ht hl _ _ _ _ _ hc0 hc1 hf0 hf1

/-- Compact radius formula: `R=h*M*(4+3*T*|w|)`. -/
theorem radius_formula (w h : Fraction) (s : Point × Point) (n : Nat) :
  Fraction.equiv (radius w h s n)
    (Fraction.mul (Fraction.mul h (stateNorm s)) (shapeFactor w h n)) := by
  simp only [radius, fullDriftBudget, maxError, shapeFactor, four,
    Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

/-- The summed square budget is `2*T*h*M*(4+3*T*|w|)^2`. It counts one
square per coarse block; no union-area or disjointness assertion is used. -/
theorem coverBudget_formula (w h : Fraction) (s : Point × Point) (n : Nat) :
  Fraction.equiv (coverBudget w h s n)
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (totalTime h n)
        (Fraction.mul h
          (Fraction.mul (Fraction.mul (stateNorm s) (stateNorm s))
            (Fraction.mul (shapeFactor w h n) (shapeFactor w h n)))))) := by
  let R := radius w h s n
  let Q := shapeFactor w h n
  let M := stateNorm s
  have hr := radius_formula w h s n
  have hsq := Fraction.mul_equiv hr hr
  have harea := Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt 4)) hsq
  have hbudget := Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt (n : Int))) harea
  apply Fraction.equiv_trans hbudget
  simp only [coverBudget, squareArea, R, Q, M, totalTime,
    Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp only [show (4 : Int) = 2 * 2 by rfl]

```

```

ac_nf

theorem sample_small_time : SmallTime one eighth 1 := by
  unfold SmallTime Fraction.le
  decide

theorem sample_radius :
  Fraction.equiv (radius one eighth sample 1) (19, 16, by decide) := by decide

theorem sample_square_area :
  Fraction.equiv (squareArea (radius one eighth sample 1))
    (361, 64, by decide) := by decide

theorem sample_cover_budget :
  Fraction.equiv (coverBudget one eighth sample 1)
    (361, 64, by decide) := by decide

theorem sample_all_corners :
  let x := coarseStart one eighth sample 0
  let R := radius one eighth sample 1
  Fraction.le (pointNorm (pointSub x x)) R ^
  Fraction.le (pointNorm (pointSub (coarseMid one eighth sample 0) x)) R ^
  Fraction.le (pointNorm (pointSub (coarseEnd one eighth sample 0) x)) R ^
  Fraction.le (pointNorm (pointSub (fineStart one eighth sample 0) x)) R ^
  Fraction.le (pointNorm (pointSub (fineMid one eighth sample 0) x)) R ^
  Fraction.le (pointNorm (pointSub (fineEnd one eighth sample 0) x)) R :=
  actual_corners_in_ball one eighth sample 0 1 (by decide) (by decide) sample_small_time

theorem sample_first_patch_square (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda) :
  SquareContains (coarseStart one eighth sample 0) (radius one eighth sample 1)
    (firstPatch one eighth sample 0 theta lambda) :=
  firstPatch_square one eighth sample 0 1 (by decide) (by decide)
  sample_small_time theta lambda ht hl

theorem sample_second_patch_square (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda) :
  SquareContains (coarseStart one eighth sample 0) (radius one eighth sample 1)
    (secondPatch one eighth sample 0 theta lambda) :=
  secondPatch_square one eighth sample 0 1 (by decide) (by decide)
  sample_small_time theta lambda ht hl

theorem sample_zero_blocks_budget :
  Fraction.equiv (coverBudget one eighth sample 0) zero := by decide

theorem sample_zero_duration_budget :
  Fraction.equiv (coverBudget one zero sample 1) zero := by decide

theorem sample_quarter_ball_too_small :
  ~ Fraction.le
    (pointNorm (pointSub (fineEnd one eighth sample 0)
      (coarseStart one eighth sample 0))) quarter := by
  unfold Fraction.le
  decide

/-- A separate coordinate-square control: the coarse endpoint's vertical
offset is 1/4, so a square of radius 1/8 cannot cover it. The preceding
L1-ball control does not assert failure of a square of radius 1/4. -/
theorem sample_eighth_square_too_small :
  ~ SquareContains (coarseStart one eighth sample 0) eighth
    (coarseEnd one eighth sample 0) := by
  unfold SquareContains Fraction.le
  decide

```

NewtonLimitDynamics/Polygon/HarmonicDyadic.lean — Actual dyadic harmonic endpoint data at one fixed represented rational time. The Cauchy estimates are derived from the finite end-kick cells. A Cauchy name is not a limit point, continuous trajectory, or geometric region between polygon and trajectory. Kepler swept area remains a

separate quantity.

```

theorem totalTime_dyadic (T : Fraction) (j : Nat) :
  Fraction.equiv (totalTime (duration T (j + 1)) (blocks j)) T := by
  simp only [totalTime, duration, blocks, Fraction.equiv, Fraction.mul,
    Fraction.ofInt, Int.pow_succ, Int.natCast_pow]
  ac_nf

private theorem pointScale_congr {a b : Fraction} {p q : Point}
  (ha : Fraction.equiv a b) (hp : pointEquiv p q) :
  pointEquiv (pointScale a p) (pointScale b q) :=
  (Fraction.mul_equiv ha hp.1, Fraction.mul_equiv ha hp.2)

private theorem cell_congr {d e : Fraction} {s t : Point × Point}
  (hd : Fraction.equiv d e) (hs : stateEquiv s t) (w : Fraction) :
  stateEquiv (cell (linearField w) d s) (cell (linearField w) e t) := by
  have hpos := pointAdd_congr hs.1 (pointScale_congr hd hs.2)
  have hfield : pointEquiv (linearField w (cell (linearField w) d s).1)
    (linearField w (cell (linearField w) e t).1) :=
    pointScale_congr (Fraction.equiv_refl _) hpos
  exact (hpos, pointAdd_congr hs.2 (pointScale_congr hd hfield))

theorem schedule_replicate_congr (w d e : Fraction)
  (hd : Fraction.equiv d e) :
  (n : Nat) → (s t : Point × Point) → stateEquiv s t →
  stateEquiv (schedule (linearField w) (List.replicate n d) s)
    (schedule (linearField w) (List.replicate n e) t)
| 0, _, _, hs => hs
| n + 1, _, _, hs =>
  schedule_replicate_congr w d e hd n _ _ (cell_congr hd hs w)

theorem fineDurations_replicate (h : Fraction) :
  (n : Nat) → fineDurations h n = List.replicate (n + n) h
| 0 => rfl
| n + 1 => by
  have ih := fineDurations_replicate h n
  have hn : (n + 1) + (n + 1) = 2 + (n + n) := by omega
  rw [hn]
  have hc : 2 + (n + n) = (n + n) + 2 := by omega
  rw [hc]
  simp only [fineDurations, Nat.add_succ, List.replicate_succ]
  rw [ih]

/-- The coarser dyadic endpoint is value-equivalent to the actual coarse
block schedule at half the next level's duration. -/
theorem endpoint_coarse (w T : Fraction) (s : Point × Point) (j : Nat) :
  stateEquiv (endpoint w T s j)
    (coarseAt w (duration T (j + 1)) s (blocks j)) := by
  have hc := schedule_replicate_congr w (duration T j)
    (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
    (duration_halving T j) (blocks j) s s
  ((Fraction.equiv_refl _, Fraction.equiv_refl _),
  (Fraction.equiv_refl _, Fraction.equiv_refl _))
  simpa only [endpoint, coarseAt_schedule] using hc

/-- The next dyadic endpoint is the actual two-half-cell schedule. -/
theorem endpoint_fine (w T : Fraction) (s : Point × Point) (j : Nat) :
  endpoint w T s (j + 1) =
  fineAt w (duration T (j + 1)) s (blocks j) := by
  unfold endpoint
  rw [blocks_succ]
  rw [← fineDurations_replicate]
  exact fineAt_schedule w (duration T (j + 1)) s (blocks j)

theorem elapsed_replicate_congr {d e : Fraction}
  (hd : Fraction.equiv d e) :
  (n : Nat) →
  Fraction.equiv (elapsed (List.replicate n d)) (elapsed (List.replicate n e))
| 0 => Fraction.equiv_refl _
| n + 1 => Fraction.add_equiv hd (elapsed_replicate_congr hd n)

```

```

theorem endpoint_elapsed (T : Fraction) (j : Nat) :
  Fraction.equiv (elapsed (List.replicate (blocks j) (duration T j))) T :=
  Fraction.equiv_trans
    (elapsed_replicate_congr (duration_halving T j) (blocks j))
    (Fraction.equiv_trans
      (coarse_elapsed_totalTime (duration T (j + 1)) (blocks j))
      (totalTime_dyadic T j))

theorem endpoint_next_elapsed (T : Fraction) (j : Nat) :
  Fraction.equiv
    (elapsed (List.replicate (blocks (j + 1)) (duration T (j + 1)))) T := by
  rw [blocks_succ, ← fineDurations_replicate]
  exact Fraction.equiv_trans
    (fine_elapsed_totalTime (duration T (j + 1)) (blocks j))
    (totalTime_dyadic T j)

theorem dyadic_smallTime (w T : Fraction) (j : Nat)
  (hs : DyadicSmallTime w T) :
  SmallTime w (duration T (j + 1)) (blocks j) := by
  have ht := totalTime_dyadic T j
  have he := Fraction.mul_equiv ht
    (Fraction.equiv_refl (Fraction.add (Fraction.ofInt 1) w.abs))
  exact Fraction.le_equiv_left he hs

theorem adjacent_error_le (w T : Fraction) (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub (endpoint w T s (j + 1)) (endpoint w T s j)))
    (adjacentCap w T s j) := by
  let h := duration T (j + 1)
  let n := blocks j
  have hf := endpoint_fine w T s j
  have hc := endpoint_coarse w T s j
  have he : stateEquiv (stateSub (endpoint w T s (j + 1)) (endpoint w T s j))
    (stateSub (fineAt w h s n) (coarseAt w h s n)) :=
    stateSub_congr
      (by rw [hf]; exact ⟨(Fraction.equiv_refl _, Fraction.equiv_refl _),
        (Fraction.equiv_refl _, Fraction.equiv_refl _)⟩)
      hc
  have hbound := actual_uniform_error w h s n hT (dyadic_smallTime w T j hs)
  have hfirst := Fraction.le_equiv_left (stateNorm_equiv he) hbound
  apply Fraction.le_equiv_right hfirst
  exact Fraction.mul_equiv (Fraction.equiv_refl _)
    (Fraction.mul_equiv (totalTime_dyadic T j) (Fraction.equiv_refl _))

theorem adjacentCap_tail (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv (adjacentCap w T s j) (tailCap w T s (j + 1)) := by
  simp only [adjacentCap, tailCap, coefficient, duration, Fraction.equiv,
    Fraction.mul, Fraction.ofInt, Int.pow_succ]
  ac_nf

theorem tail_halving (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv
    (Fraction.add (tailCap w T s (j + 1)) (tailCap w T s (j + 1)))
    (tailCap w T s j) := by
  exact GeometricTail.tail_halving (coefficient w T s) j

theorem tail_double (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv (Fraction.add (tailCap w T s j) (tailCap w T s j))
    (doubleTail w T s j) := by
  exact GeometricTail.tail_double (coefficient w T s) j

theorem coefficient_nonnegative (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) : 0 ≤ (coefficient w T s).num :=
  Int.mul_nonneg (by decide)
    (Int.mul_nonneg hT (Int.mul_nonneg hT
      (Int.mul_nonneg (Fraction.abs_num_nonnegative w) (stateNorm_nonnegative s))))

/-- Any finite separation of dyadic levels has error within the tail at its
coarser endpoint. The proof uses actual neighboring schedules. -/

```

```

theorem finite_gap_error (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  (k j : Nat) → Fraction.le
    (stateNorm (stateSub (endpoint w T s (j + k))
      (endpoint w T s j))) (tailCap w T s j) := by
  intro k j
  exact GeometricTail.finite_gap (endpoint w T s) (coefficient w T s)
    (coefficient_nonnegative w T s hT)
    (fun i => Fraction.le_equiv_right (adjacent_error_le w T s i hT hs)
      (adjacentCap_tail w T s i)) k j

/-- Both later endpoints are compared to the same earlier actual endpoint. -/
theorem two_sided_error (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (N m n : Nat) (hm : N ≤ m) (hn : N ≤ n) :
  Fraction.le
    (stateNorm (stateSub (endpoint w T s m) (endpoint w T s n)))
    (doubleTail w T s N) := by
  exact GeometricTail.two_sided (endpoint w T s) (coefficient w T s)
    (coefficient_nonnegative w T s hT)
    (fun i => Fraction.le_equiv_right (adjacent_error_le w T s i hT hs)
      (adjacentCap_tail w T s i)) N m n hm hn

theorem doubleTail_lt_tolerance (w T : Fraction) (s : Point × Point)
  (eps : Fraction) (hT : 0 ≤ T.num) (heps : 0 < eps.num) :
  Fraction.lt (doubleTail w T s (modulus w T s eps)) eps := by
  exact GeometricTail.doubleTail_lt_tolerance (coefficient w T s) eps
    (coefficient_nonnegative w T s hT) heps

/-- A Cauchy name stores finite rational endpoint approximants and a proved
positive-tolerance condition. It does not supply a limit point. -/
theorem endpoint_cauchy (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  ∀ eps : Fraction, 0 < eps.num →
  ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
  Fraction.lt
    (stateNorm (stateSub (endpoint w T s m) (endpoint w T s n))) eps := by
  intro eps heps
  refine {modulus w T s eps, ?_}
  intro m n hm hn
  exact Fraction.magnitudes.lt_of_le_lt
    (two_sided_error w T s hT hs _ m n hm hn)
    (doubleTail_lt_tolerance w T s eps hT heps)

private theorem stateEquiv_refl (s : Point × Point) : stateEquiv s s :=
  ({Fraction.equiv_refl _, Fraction.equiv_refl _},
  {Fraction.equiv_refl _, Fraction.equiv_refl _})

private theorem stateEquiv_trans {a b c : Point × Point}
  (hab : stateEquiv a b) (hbc : stateEquiv b c) : stateEquiv a c :=
  ({Fraction.equiv_trans hab.1.1 hbc.1.1,
    Fraction.equiv_trans hab.1.2 hbc.1.2},
  {Fraction.equiv_trans hab.2.1 hbc.2.1,
    Fraction.equiv_trans hab.2.2 hbc.2.2})

private theorem zero_duration_cell (w d : Fraction) (s : Point × Point)
  (hd : d.num = 0) : stateEquiv (cell (linearField w) d s) s := by
  let z : Fraction := {0, 1, by decide}
  have he : Fraction.equiv d z := by
    unfold Fraction.equiv z
    simp [hd]
  exact stateEquiv_trans (cell_congr he (stateEquiv_refl s) w)
    (zero_step w s)

theorem zero_duration_schedule (w d : Fraction) (hd : d.num = 0) :
  (n : Nat) → (s : Point × Point) →
  stateEquiv (schedule (linearField w) (List.replicate n d) s) s
| 0, s => stateEquiv_refl s
| n + 1, s =>
  stateEquiv_trans

```

```

    (zero_duration_schedule w d hd n (cell (linearField w) d s))
    (zero_duration_cell w d s hd)

theorem zero_time_endpoint (w T : Fraction) (s : Point × Point) (j : Nat)
  (hT : T.num = 0) : stateEquiv (endpoint w T s j) s := by
  exact zero_duration_schedule w (duration T j) (by exact hT) (blocks j) s

theorem sample_dyadic_small_time : DyadicSmallTime sampleOne sampleTime := by
  unfold DyadicSmallTime Fraction.le
  decide

theorem sample_adjacent_error :
  Fraction.equiv
    (stateNorm (stateSub (endpoint sampleOne sampleTime sampleState 1)
      (endpoint sampleOne sampleTime sampleState 0)))
    (145, 4096, by decide) := by decide

theorem sample_adjacent_cap :
  Fraction.equiv (adjacentCap sampleOne sampleTime sampleState 0)
    (3, 16, by decide) := by decide

theorem sample_tail_cap :
  Fraction.equiv (tailCap sampleOne sampleTime sampleState 0)
    (3, 8, by decide) := by decide

```

NewtonLimitDynamics/Polygon/HarmonicRefinement.lean — Finite common-time comparison for the linear central field. The two actual fine cells have duration h ; the one actual coarse cell has duration $h+h$. Their terminal positions need not agree. The directed connector from the fine endpoint to the coarse endpoint closes a polygon comparison and is not a mechanical cell. All statements are finite rational arithmetic. In particular, this file constructs no limiting curve and makes no global area estimate. Modern reconstruction motivated by the polygon arguments in De Motu NATP00089 par9 / NATP00090 par17, 1687 NATP00077 par45 and 1713 NATP00082 ...

```

/-- The fine endpoint is the coarse endpoint plus  $-h^2 w y$ . -/
theorem position_mismatch (w h : Fraction) (s : Point × Point) :
  pointEquiv (fine w h s).1
    (pointAdd (coarse w h s).1
      (pointScale (negF (Fraction.mul (Fraction.mul h h) w)) (middle w h s)))) := by
  constructor <=>
    simp only [fine, coarse, middle, cell, linearField, negF, pointAdd,
      pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
      Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
    ac_nf <=> omega

/-- Fine-minus-coarse velocity:  $h^2 w v + h^3 w^2 y$ . -/
theorem velocity_mismatch (w h : Fraction) (s : Point × Point) :
  pointEquiv (fine w h s).2
    (pointAdd (coarse w h s).2
      (pointAdd (pointScale (Fraction.mul (Fraction.mul h h) w) s.2)
        (pointScale (Fraction.mul (Fraction.mul (Fraction.mul h h) h)
          (Fraction.mul w w)) (middle w h s))))) := by
  constructor <=>
    simp only [fine, coarse, middle, cell, linearField, negF, pointAdd,
      pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
      Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
    ac_nf <=> omega

/-- The local closed boundary reduces to the single triangle  $y, z, X$ . -/
theorem closed_eq_triangle (w h : Fraction) (s : Point × Point) :
  Fraction.equiv (closedDefect w h s)
    (det (pointSub (fine w h s).1 (middle w h s))
      (pointSub (coarse w h s).1 (middle w h s))) := by
  simp only [closedDefect, closedBoundaryTwice, fine, coarse, middle, cell,
    linearField, negF, pointSub, pointNeg, pointAdd, pointScale, det,
    Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>

```

```

ac_nf <=> omega

/-- Exact cubic signed doubled gap for the common-time local refinement. -/
theorem closed_defect_cubic (w h : Fraction) (s : Point × Point) :
  Fraction.equiv (closedDefect w h s)
    (negF (Fraction.mul (Fraction.mul (Fraction.mul h h) h)
      (Fraction.mul w (det s.1 s.2)))) := by
  simp only [closedDefect, closedBoundaryTwice, fine, coarse, middle, cell,
    linearField, negF, pointAdd, pointScale, det,
    Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

theorem absoluteClosedGap_nonnegative (w h : Fraction) (s : Point × Point) :
  0 ≤ (absoluteClosedGap w h s).num := by
  exact Fraction.abs_num_nonnegative _

/-- Nonnegative local gap as the magnitude of the exact cubic coefficient.
  This is one triangle, so no cancellation of distinct lobes is involved. -/
theorem absolute_closed_defect_cubic (w h : Fraction) (s : Point × Point) :
  Fraction.equiv (absoluteClosedGap w h s)
    (Fraction.mul (Fraction.mul (Fraction.mul h h) h)
      (Fraction.mul w (det s.1 s.2))).abs :=
  Fraction.equiv_trans (Fraction.abs_equiv (closed_defect_cubic w h s))
    (Fraction.abs_neg _)

/-- Central-force Kepler sums agree for these two schedules. -/
theorem swept_equal (w h : Fraction) (s : Point × Point) :
  Fraction.equiv (swept (linearField w) [h, h] s)
    (swept (linearField w) [Fraction.add h h] s) := by
  simp only [swept, fine, coarse, middle, cell, linearField, negF,
    pointAdd, pointScale, det, Fraction.equiv, Fraction.add, Fraction.mul,
    Fraction.ofInt, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
    Int.mul_one, Int.one_mul, Int.zero_mul, Int.mul_zero] <=>
  ac_nf <=> omega

theorem sample_middle : pointEquiv (middle one half sample) (one, half) := by decide

theorem sample_fine : pointEquiv (fine one half sample).1
  ((3, 4, by decide), (7, 8, by decide)) := by decide

theorem sample_coarse : pointEquiv (coarse one half sample).1 (one, one) := by decide

theorem sample_fine_swept : Fraction.equiv (swept (linearField one) [half, half] sample) one := by decide

theorem sample_coarse_swept : Fraction.equiv (swept (linearField one) [Fraction.add half half] sample)
  ↪ one := by decide

theorem sample_closed_defect : Fraction.equiv (closedDefect one half sample) (-1, 8, by decide) := by
  ↪ decide

theorem sample_absolute_closed_gap :
  Fraction.equiv (absoluteClosedGap one half sample) (1, 8, by decide) := by decide

theorem sample_closed_defect_nonzero :
  ¬ Fraction.equiv (closedDefect one half sample) zero := by decide

```

NewtonLimitDynamics/Polygon/HarmonicTimeComparison.lean — Finite comparisons of actual harmonic endpoint schedules at two rational times. The cell counts agree; their durations differ. These estimates concern Cauchy data only, without a limit point, trajectory, or intervening-area content.

```

theorem cell_parameter_difference (w sigma tau : Fraction) (s : Point × Point) :
  stateEquiv
    (stateSub (cell (linearField w) tau s) (cell (linearField w) sigma s))
    (pointScale (durationDifference sigma tau) s.2,
      pointScale (negF (Fraction.mul (durationDifference sigma tau) w))
        (pointAdd s.1 (pointScale (Fraction.add sigma tau) s.2))) := by
  constructor <=> constructor <=>

```

```

simp only [durationDifference, stateSub, pointEquiv, pointSub, pointNeg,
  cell, linearField, negF, pointAdd, pointScale, Fraction.equiv,
  Fraction.add, Fraction.mul,
  Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <;>
ac_nf <;> omega

theorem cell_parameter_norm_formula (w sigma tau : Fraction) (s : Point × Point) :
  Fraction.equiv
    (stateNorm (stateSub (cell (linearField w) tau s)
      (cell (linearField w) sigma s)))
    (Fraction.mul (durationDifference sigma tau).abs
      (Fraction.add (pointNorm s.2)
        (Fraction.mul w.abs
          (pointNorm (pointAdd s.1
            (pointScale (Fraction.add sigma tau) s.2)))))) := by
  have hs := stateNorm_equiv (cell_parameter_difference w sigma tau s)
  apply Fraction.equiv_trans hs
  simp only [stateNorm, pointNorm, pointScale, negF, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.abs, Int.natAbs_mul,
    Int.natAbs_neg, Int.ofNat_mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem short_sum_point_bound (sigma tau : Fraction) (x v : Point)
  (hσ : 0 ≤ sigma.num) (ht : 0 ≤ tau.num)
  (hsum : Fraction.le (Fraction.add sigma tau) (Fraction.ofInt 1)) :
  Fraction.le
    (pointNorm (pointAdd x (pointScale (Fraction.add sigma tau) v)))
    (Fraction.add (pointNorm x) (pointNorm v)) := by
  let q := Fraction.add sigma tau
  have hq : 0 ≤ q.num := Fraction.nonnegative_add sigma tau hσ ht
  have hs := pointNorm_scale q v
  have hqabs := Fraction.abs_of_nonnegative q hq
  have hscale : Fraction.le (pointNorm (pointScale q v)) (pointNorm v) := by
    have hq' : Fraction.le q.abs (Fraction.ofInt 1) :=
      Fraction.le_equiv_left hqabs hsum
    have hm := Fraction.mul_le_mul_nonnegative hq' (pointNorm v)
      (pointNorm_nonnegative v)
    apply Fraction.le_equiv_left hs
    apply Fraction.le_equiv_right hm
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp
  exact Fraction.magnitudes.le_trans (pointNorm_add_le x (pointScale q v))
    (Fraction.add_le_add_left hscale (pointNorm x))

private theorem scalar_local_bound (a b c : Fraction)
  (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) (hc : 0 ≤ c.num) :
  Fraction.le
    (Fraction.add b (Fraction.mul c (Fraction.add a b)))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1)
      (Fraction.mul (Fraction.ofInt 2) c)) (Fraction.add a b)) := by
  let M := Fraction.add a b
  let cM := Fraction.mul c M
  have hbM : Fraction.le b M := by
    unfold Fraction.le M Fraction.add
    dsimp
    rw [Int.add_mul]
    have hnon := Int.mul_nonneg
      (Int.mul_nonneg ha (Int.le_of_lt b.den_pos)) (Int.le_of_lt b.den_pos)
    have he : b.num * (a.den * b.den) = b.num * a.den * b.den := by ac_rfl
    rw [he]
    omega
  have hfirst := Fraction.add_le_add_right hbM cM
  have hcM : 0 ≤ cM.num := Int.mul_nonneg hc (Fraction.nonnegative_add a b ha hb)
  have hz : Fraction.le (Fraction.ofInt 0) cM := by
    unfold Fraction.le Fraction.ofInt
    dsimp
    simp using hcM
  have hzero : Fraction.equiv (Fraction.add M (Fraction.ofInt 0)) M := by
    unfold Fraction.equiv Fraction.add Fraction.ofInt

```

```

    simp only [Int.mul_one, Int.zero_mul, Int.add_zero, Int.mul_zero]
  have hsecond : Fraction.le (Fraction.add M cM)
    (Fraction.add (Fraction.add M cM) cM) := by
    have hh := Fraction.add_le_add_left hz (Fraction.add M cM)
    exact Fraction.le_equiv_left (by
      simp only [Fraction.equiv, Fraction.add, Fraction.ofInt]
      simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
        Int.mul_one, Int.one_mul]) hh
  have he : Fraction.equiv (Fraction.add (Fraction.add M cM) cM)
    (Fraction.mul (Fraction.add (Fraction.ofInt 1)
      (Fraction.mul (Fraction.ofInt 2) c)) M) := by
    simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
    simp only [show (2 : Int) = 1 + 1 by rfl]
    simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
    simp only [cM, Fraction.mul]
    ac_nf
  exact Fraction.le_equiv_right (Fraction.magnitudes.le_trans hfirst hsecond) he

/-- The exact duration mismatch of one actual end-kick cell, bounded under a
short nonnegative combined duration. -/
theorem cell_parameter_bound (w sigma tau : Fraction) (s : Point × Point)
  (ho : 0 ≤ sigma.num) (ht : 0 ≤ tau.num)
  (hsum : Fraction.le (Fraction.add sigma tau) (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub (cell (linearField w) tau s)
      (cell (linearField w) sigma s)))
    (Fraction.mul (durationDifference sigma tau).abs
      (Fraction.mul
        (Fraction.add (Fraction.ofInt 1)
          (Fraction.mul (Fraction.ofInt 2) w.abs)) (stateNorm s))) := by
  let d := durationDifference sigma tau
  have hp := short_sum_point_bound sigma tau s.1 s.2 ho ht hsum
  have hw := Fraction.mul_le_mul_nonnegative_left hp w.abs
  (Fraction.abs_num_nonnegative w)
  have ha := Fraction.add_le_add_left hw (pointNorm s.2)
  have hd := Fraction.mul_le_mul_nonnegative_left ha d.abs
  (Fraction.abs_num_nonnegative d)
  have hc := scalar_local_bound (pointNorm s.1) (pointNorm s.2) w.abs
  (pointNorm_nonnegative s.1) (pointNorm_nonnegative s.2)
  (Fraction.abs_num_nonnegative w)
  have hdc := Fraction.mul_le_mul_nonnegative_left hc d.abs
  (Fraction.abs_num_nonnegative d)
  have hf := cell_parameter_norm_formula w sigma tau s
  exact Fraction.le_equiv_right
    (Fraction.magnitudes.le_trans (Fraction.le_equiv_left hf hd) hdc)
    (by simp only [stateNorm]; exact Fraction.equiv_refl _)

private theorem parameterFactor_nonnegative (w : Fraction) :
  0 ≤ (parameterFactor w).num := by
  unfold parameterFactor
  exact Fraction.nonnegative_add _ _ (by decide)
  (Int.mul_nonneg (by decide) (Fraction.abs_num_nonnegative w))

private theorem localParameterBudget_nonnegative (w sigma tau : Fraction)
  (s : Point × Point) :
  0 ≤ (localParameterBudget w sigma tau s).num := by
  unfold localParameterBudget
  exact Int.mul_nonneg (Fraction.abs_num_nonnegative _)
    (Int.mul_nonneg (parameterFactor_nonnegative w)
      (Int.mul_nonneg (by decide) (stateNorm_nonnegative s)))

private theorem parameterErrorBudget_nonnegative (w ho ht : Fraction)
  (s : Point × Point) :
  (i : Nat) → 0 ≤ (parameterErrorBudget w ho ht s i).num
| 0 => by simp [parameterErrorBudget, Fraction.ofInt]
| i + 1 =>
  Fraction.nonnegative_add _ _
    (Int.mul_nonneg (kappa_nonnegative w (Fraction.add ht ht))
      (parameterErrorBudget_nonnegative w ho ht s i))
    (localParameterBudget_nonnegative w _ _ s)

```

```

theorem coarse_parameter_step (w sigma tau : Fraction) (a b : Point × Point)
  (hσ : 0 ≤ sigma.num) (hτ : 0 ≤ tau.num)
  (hsum : Fraction.le (Fraction.add sigma tau) (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub (cell (linearField w) tau a)
      (cell (linearField w) sigma b)))
    (Fraction.add
      (Fraction.mul (kappa w tau) (stateNorm (stateSub a b)))
      (Fraction.mul (durationDifference sigma tau).abs
        (Fraction.mul (parameterFactor w) (stateNorm b)))))) := by
  have htri := stateSub_triangle (cell (linearField w) tau a)
    (cell (linearField w) tau b) (cell (linearField w) sigma b)
  have h₁ := cell_perturbation w tau a b
  have h₂ := cell_parameter_bound w sigma tau b hσ hτ hsum
  exact Fraction.magnitudes.le_trans htri (Fraction.add_le_add h₁ h₂)

theorem actual_coarse_parameter_error (w hσ hτ : Fraction) (s : Point × Point)
  (n : Nat) (hho : 0 ≤ hσ.num) (hht : 0 ≤ hτ.num)
  (hsum : Fraction.le
    (Fraction.add (Fraction.add hσ hσ) (Fraction.add hτ hτ))
    (Fraction.ofInt 1)) (hsmall : SmallTime w hσ n) :
  (i : Nat) → i ≤ n →
  Fraction.le
    (stateNorm (stateSub (coarseAt w hτ s i) (coarseAt w hσ s i)))
    (parameterErrorBudget w hσ hτ s i)
| 0, _ => Fraction.le_of_equiv (stateSub_self_norm_zero s)
| i + 1, hi => by
  let sigma := Fraction.add hσ hσ
  let tau := Fraction.add hτ hτ
  let a := coarseAt w hτ s i
  let b := coarseAt w hσ s i
  have hi' : i ≤ n := by omega
  have honon := Fraction.nonnegative_add hσ hσ hho hho
  have htnon := Fraction.nonnegative_add hτ hτ hht hht
  have hstep := coarse_parameter_step w sigma tau a b
  have honon htnon hsum
  have hprev := actual_coarse_parameter_error w hσ hτ s n hho hht
  have hA := Fraction.mul_le_mul_nonnegative_left hprev
    (coarseFactor w hτ) (kappa_nonnegative w tau)
  have hprefix := coarse_state_le_two w hσ s i hho
    (smallTime_prefix w hσ i n hho hi' hsmall)
  have hB₁ := Fraction.mul_le_mul_nonnegative_left hprefix
    (parameterFactor w) (parameterFactor_nonnegative w)
  have hB₂ := Fraction.mul_le_mul_nonnegative_left hB₁
    (durationDifference sigma tau).abs
    (Fraction.abs_num_nonnegative _)
  have hsum' := Fraction.add_le_add hA hB₂
  have hchain := Fraction.magnitudes.le_trans hstep hsum'
  simpa only [coarseAt, HarmonicRefinement.coarse,
    parameterErrorBudget, localParameterBudget] using hchain

theorem parameter_budget_power (w hσ hτ : Fraction) (s : Point × Point) :
  (i : Nat) → Fraction.le (parameterErrorBudget w hσ hτ s i)
    (parameterPowerBudget w hσ hτ s i)
| 0 => Fraction.le_of_equiv (by
  simp only [parameterErrorBudget, parameterPowerBudget, fpower,
    Fraction.equiv, Fraction.ofInt, Fraction.mul]
  simp)
| i + 1 => by
  let d := localParameterBudget w (Fraction.add hσ hσ) (Fraction.add hτ hτ) s
  let b := coarseFactor w hτ
  have hd := localParameterBudget_nonnegative w
    (Fraction.add hσ hσ) (Fraction.add hτ hτ) s
  have hb := kappa_nonnegative w (Fraction.add hτ hτ)
  have h₁ := Fraction.mul_le_mul_nonnegative_left
    (parameter_budget_power w hσ hτ s i) b hb
  have hD : Fraction.le d (Fraction.mul d (fpower b (i + 1))) := by
    have hpow := one_le_power b hb (one_le_kappa w (Fraction.add hτ hτ))

```

```

      (i + 1)
      have hm := Fraction.mul_le_mul_nonnegative_left hpow d hd
      apply Fraction.le_equiv_left (by
        simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
        simp) hm
      have hsum := Fraction.add_le_add h1 hD
      apply Fraction.le_equiv_right hsum
      simp only [d, b, parameterErrorBudget, parameterPowerBudget,
        fpower, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
      simp only [Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add,
        Int.one_mul, Int.mul_one]
      ac_nf

theorem actual_coarse_parameter_uniform (w hσ hτ : Fraction)
  (s : Point × Point) (n : Nat)
  (hho : 0 ≤ hσ.num) (hht : 0 ≤ hτ.num)
  (hsum : Fraction.le
    (Fraction.add (Fraction.add hσ hσ) (Fraction.add hτ hτ))
    (Fraction.ofInt 1))
  (hsmallσ : SmallTime w hσ n) (hsmallτ : SmallTime w hτ n) :
  Fraction.le
    (stateNorm (stateSub (coarseAt w hτ s n) (coarseAt w hσ s n)))
    (Fraction.mul (Fraction.ofInt (2 * (n : Int)))
      (localParameterBudget w (Fraction.add hσ hσ)
        (Fraction.add hτ hτ) s)) := by
  let d := localParameterBudget w (Fraction.add hσ hσ)
    (Fraction.add hτ hτ) s
  have h1 := actual_coarse_parameter_error w hσ hτ s n hho hht
    hsum hsmallσ n (Nat.le_refl n)
  have h2 := parameter_budget_power w hσ hτ s n
  have hp := coarse_power_le_two w hτ n hht hsmallτ
  have hd := localParameterBudget_nonnegative w
    (Fraction.add hσ hσ) (Fraction.add hτ hτ) s
  have hm := Fraction.mul_le_mul_nonnegative_left hp d hd
  have hn := Fraction.mul_le_mul_nonnegative_left hm
    (Fraction.ofInt (n : Int)) (Int.ofNat_nonneg n)
  have hchain := Fraction.magnitudes.le_trans
    (Fraction.magnitudes.le_trans h1 h2) hn
  apply Fraction.le_equiv_right hchain
  simp only [parameterPowerBudget, d, Fraction.equiv, Fraction.mul,
    Fraction.ofInt]
  ac_nf

theorem dyadic_time_le_half (w T : Fraction)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le T half := by
  have hw : 0 ≤ (w.num.natAbs : Int) := Int.ofNat_nonneg _
  have hnon : 0 ≤ 2 * T.num * (w.num.natAbs : Int) :=
    Int.mul_nonneg (Int.mul_nonneg (by decide) hT) hw
  have hraw :
    2 * T.num * (w.den + (w.num.natAbs : Int)) ≤ T.den * w.den := by
    unfold DyadicSmallTime Fraction.le Fraction.mul Fraction.add Fraction.ofInt
    Fraction.abs at hs
    dsimp [HarmonicDyadic.halfThreshold] at hs
    simp only [Int.one_mul, Int.mul_one] at hs
    calc
      2 * T.num * (w.den + (w.num.natAbs : Int)) =
        T.num * (w.den + (w.num.natAbs : Int)) * 2 := by ac_rfl
      _ ≤ T.den * w.den := hs
  have hmul : (2 * T.num) * w.den ≤ T.den * w.den := by
    rw [Int.mul_add] at hraw
    omega
  have hbase := Int.le_of_mul_le_mul_right hmul w.den_pos
  unfold Fraction.le half
  dsimp
  omega

theorem duration_le_time (T : Fraction) (j : Nat) (hT : 0 ≤ T.num) :
  Fraction.le (duration T j) T := by
  have hpow : 1 ≤ (2 : Int) ^ j := by

```

```

    have h := two_pow_ge_succ j
    omega
  have hd := Int.mul_nonneg hT (Int.le_of_lt T.den_pos)
  have hm := Int.mul_le_mul_of_nonneg_left hpow hd
  unfold Fraction.le duration
  dsimp
  have he : T.num * (T.den * (2 : Int) ^ j) =
    (T.num * T.den) * (2 : Int) ^ j := by ac_rfl
  rw [he]
  omega

theorem short_duration_pair (w T U : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) (hU : 0 ≤ U.num)
  (hsT : DyadicSmallTime w T) (hsU : DyadicSmallTime w U) :
  Fraction.le
    (Fraction.add
      (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
      (Fraction.add (duration U (j + 1)) (duration U (j + 1))))
    (Fraction.ofInt 1) := by
  have hT' := duration_le_time T j hT
  have hU' := duration_le_time U j hU
  have hT'' := Fraction.le_equiv_left
    (Fraction.equiv_symm (duration_halving T j)) hT'
  have hU'' := Fraction.le_equiv_left
    (Fraction.equiv_symm (duration_halving U j)) hU'
  have hhalf := Fraction.add_le_add (dyadic_time_le_half w T hT hsT)
    (dyadic_time_le_half w U hU hsU)
  have hsum := Fraction.magnitudes.le_trans
    (Fraction.add_le_add hT'' hU'') hhalf
  apply Fraction.le_equiv_right hsum
  decide

/-- The common count cancels the per-cell signed duration difference in
rational value, even though the representatives differ. -/
theorem count_duration_difference (T U : Fraction) (j : Nat) :
  Fraction.equiv
    (Fraction.mul (Fraction.ofInt (blocks j : Int))
      (durationDifference
        (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
        (Fraction.add (duration U (j + 1)) (duration U (j + 1)))))
    (durationDifference T U) := by
  have hdur : Fraction.equiv
    (durationDifference
      (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
      (Fraction.add (duration U (j + 1)) (duration U (j + 1)))))
    (durationDifference (duration T j) (duration U j)) :=
  Fraction.add_equiv
    (Fraction.equiv_symm (duration_halving U j))
    (HarmonicDyadic.neg_equiv
      (Fraction.equiv_symm (duration_halving T j)))
  have hm := Fraction.mul_equiv
    (Fraction.equiv_refl (Fraction.ofInt (blocks j : Int))) hdur
  apply Fraction.equiv_trans hm
  simp only [blocks, duration, durationDifference, negF, Fraction.equiv,
    Fraction.ofInt, Fraction.add, Fraction.mul, Int.natCast_pow]
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
    Int.one_mul, Int.mul_one]
  ac_nf

theorem count_abs_duration_difference (T U : Fraction) (j : Nat) :
  Fraction.equiv
    (Fraction.mul (Fraction.ofInt (blocks j : Int))
      (durationDifference
        (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
        (Fraction.add (duration U (j + 1)) (duration U (j + 1)))).abs)
    (durationDifference T U).abs := by
  have hs := count_duration_difference T U j
  have habs := Fraction.abs_equiv hs
  have hm := Fraction.abs_mul (Fraction.ofInt (blocks j : Int))
    (durationDifference

```

```

    (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
    (Fraction.add (duration U (j + 1)) (duration U (j + 1))))
have hcount : Fraction.equiv (Fraction.ofInt (blocks j : Int)).abs
  (Fraction.ofInt (blocks j : Int)) :=
  Fraction.abs_of_nonnegative _ (Int.ofNat_nonneg _)
have hmul := Fraction.mul_equiv hcount
  (Fraction.equiv_refl
    (durationDifference
      (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
      (Fraction.add (duration U (j + 1)) (duration U (j + 1)))).abs)
exact Fraction.equiv_trans (Fraction.equiv_symm hmul)
  (Fraction.equiv_trans (Fraction.equiv_symm hm) habs)

/-- Uniform rational-time variation of the actual dyadic endpoint schedules.
The same level has the same count and two different cell durations. -/
theorem endpoint_time_bound (w T U : Fraction) (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hU : 0 ≤ U.num)
  (hsT : DyadicSmallTime w T) (hsU : DyadicSmallTime w U) :
  Fraction.le
    (stateNorm (stateSub (endpoint w U s j) (endpoint w T s j)))
    (Fraction.mul (timeLipschitz w s) (durationDifference T U).abs) := by
  let hσ := duration T (j + 1)
  let hτ := duration U (j + 1)
  let n := blocks j
  have honon : 0 ≤ hσ.num := hT
  have htnon : 0 ≤ hτ.num := hU
  have hbound := actual_coarse_parameter_uniform w hσ hτ s n
  honon htnon (short_duration_pair w T U j hT hU hsT hsU)
  (dyadic_smallTime w T j hsT) (dyadic_smallTime w U j hsU)
  have he : stateEquiv
    (stateSub (endpoint w U s j) (endpoint w T s j))
    (stateSub (coarseAt w hτ s n) (coarseAt w hσ s n)) :=
    stateSub_congr (endpoint_coarse w U s j) (endpoint_coarse w T s j)
  have hfirst := Fraction.le_equiv_left (stateNorm_equiv he) hbound
  let d := (durationDifference
    (Fraction.add hσ hσ) (Fraction.add hτ hτ)).abs
  have hrewrite : Fraction.equiv
    (Fraction.mul (Fraction.ofInt (2 * (n : Int)))
      (localParameterBudget w (Fraction.add hσ hσ)
        (Fraction.add hτ hτ) s))
    (Fraction.mul
      (Fraction.mul (Fraction.ofInt 4)
        (Fraction.mul (parameterFactor w) (stateNorm s)))
      (Fraction.mul (Fraction.ofInt (n : Int)) d)) := by
    simp only [localParameterBudget, d, Fraction.equiv, Fraction.mul,
      Fraction.ofInt]
  ac_nf
  have hc := count_abs_duration_difference T U j
  have hsecond := Fraction.mul_equiv
    (Fraction.equiv_refl (timeLipschitz w s)) hc
  exact Fraction.le_equiv_right hfirst
    (Fraction.equiv_trans hrewrite hsecond)

theorem timeLipschitz_nonnegative (w : Fraction) (s : Point × Point) :
  0 ≤ (timeLipschitz w s).num :=
  Int.mul_nonneg (by decide)
  (Int.mul_nonneg (parameterFactor_nonnegative w) (stateNorm_nonnegative s))

private theorem timeDenominator_positive (w : Fraction) (s : Point × Point) :
  0 < (timeDenominator w s).num := by
  unfold timeDenominator Fraction.add Fraction.ofInt
  dsimp
  have hL := timeLipschitz_nonnegative w s
  have hd := (timeLipschitz w s).den_pos
  omega

theorem timeDelta_positive (w : Fraction) (s : Point × Point) (eps : Fraction)
  (heps : 0 < eps.num) : 0 < (timeDelta w s eps).num :=
  Int.mul_pos heps (timeDenominator w s).den_pos

```

```

private theorem mul_lt_mul_positive_left {a b : Fraction}
  (hab : Fraction.lt a b) (c : Fraction) (hc : 0 < c.num) :
  Fraction.lt (Fraction.mul c a) (Fraction.mul c b) := by
  have hm := Int.mul_lt_mul_of_pos_right hab
  (Int.mul_pos hc c.den_pos)
  unfold Fraction.lt Fraction.mul at *
  dsimp at *
  have h1 : c.num * a.num * (c.den * b.den) =
    (a.num * b.den) * (c.num * c.den) := by ac_rfl
  have h2 : c.num * b.num * (c.den * a.den) =
    (b.num * a.den) * (c.num * c.den) := by ac_rfl
  rw [h1, h2]
  exact hm

private theorem timeLipschitz_le_denominator (w : Fraction) (s : Point × Point) :
  Fraction.le (timeLipschitz w s) (timeDenominator w s) := by
  unfold timeDenominator Fraction.le Fraction.add Fraction.ofInt
  dsimp
  have hd := (timeLipschitz w s).den_pos
  have hsq : 0 ≤ (timeLipschitz w s).den * (timeLipschitz w s).den :=
    Int.mul_nonneg (Int.le_of_lt hd) (Int.le_of_lt hd)
  simp only [Int.one_mul, Int.mul_one, Int.add_mul]
  omega

private theorem delta_product_equiv (w : Fraction) (s : Point × Point)
  (eps : Fraction) :
  Fraction.equiv
    (Fraction.mul (timeDenominator w s) (timeDelta w s eps)) eps := by
  unfold Fraction.equiv Fraction.mul timeDelta
  dsimp
  ac_nf

theorem parameter_delta_control (w : Fraction) (s : Point × Point)
  (eps d : Fraction) (h_eps : 0 < eps.num)
  (hd : 0 ≤ d.num) (hdelta : Fraction.lt d (timeDelta w s eps)) :
  Fraction.lt (Fraction.mul (timeLipschitz w s) d) eps := by
  have hweak := Fraction.mul_le_mul_nonnegative
    (timeLipschitz_le_denominator w s) d hd
  have hstrict := mul_lt_mul_positive_left hdelta
    (timeDenominator w s) (timeDenominator_positive w s)
  have htrans := Fraction.magnitudes.lt_of_le_lt hweak hstrict
  have heq := delta_product_equiv w s eps
  exact Fraction.magnitudes.lt_of_le_le htrans
    ((Fraction.equiv_iff_mutual_le _ _).mp heq).1

/-- One explicit delta controls every approximant level at once. -/
theorem timeName_uniform_continuity (w : Fraction) (s : Point × Point)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ delta : Fraction, 0 < delta.num ∧
    ∀ T U : ShortRationalTime w,
      Fraction.lt (durationDifference T.val U.val).abs delta →
        ∀ j : Nat,
          Fraction.lt
            (stateNorm (stateSub ((timeName w s U).approx j)
              ((timeName w s T).approx j))) eps := by
  refine {timeDelta w s eps, timeDelta_positive w s eps heps, ?_}
  intro T U hdelta j
  have hb := endpoint_time_bound w T.val U.val s j
  T.property.1 U.property.1 T.property.2 U.property.2
  have hd := Fraction.abs_num_nonnegative (durationDifference T.val U.val)
  have hstrict := parameter_delta_control w s eps
    (durationDifference T.val U.val).abs heps hd hdelta
  exact Fraction.magnitudes.lt_of_le_lt hb hstrict

theorem same_time_error_zero (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv
    (stateNorm (stateSub (endpoint w T s j) (endpoint w T s j)))
    (Fraction.ofInt 0) :=
  stateSub_self_norm_zero _

```

```

theorem zero_state_error_zero (w T U : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) (hU : 0 ≤ U.num)
  (hsT : DyadicSmallTime w T) (hsU : DyadicSmallTime w U) :
  Fraction.equiv
    (stateNorm (stateSub (endpoint w U zeroState j)
      (endpoint w T zeroState j))) (Fraction.ofInt 0) := by
  have hb := endpoint_time_bound w T U zeroState j hT hU hsT hsU
  have hz : Fraction.equiv
    (Fraction.mul (timeLipschitz w zeroState)
      (durationDifference T U).abs) (Fraction.ofInt 0) := by
  simp only [timeLipschitz, parameterFactor, zeroState,
    zeroFraction, stateNorm, pointNorm, Fraction.equiv,
    Fraction.abs, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp
  have hle := Fraction.le_equiv_right hb hz
  have hother : Fraction.le (Fraction.ofInt 0)
    (stateNorm (stateSub (endpoint w U zeroState j)
      (endpoint w T zeroState j))) := by
  unfold Fraction.le Fraction.ofInt
  dsimp
  simp only [Int.zero_mul, Int.mul_one]
  exact stateNorm_nonnegative _
  exact (Fraction.equiv_iff_mutual_le _ _).mpr (hle, hother)

theorem sample_short_times :
  DyadicSmallTime sampleOne sampleQuarter ∧
  DyadicSmallTime sampleOne sampleEighth := by
  constructor <=> unfold DyadicSmallTime Fraction.le <=> decide

theorem sample_parameter_error :
  Fraction.equiv
    (stateNorm (stateSub
      (endpoint sampleOne sampleEighth sampleState 0)
      (endpoint sampleOne sampleQuarter sampleState 0)))
    (19, 64, by decide) := by decide

theorem sample_time_lipschitz :
  Fraction.equiv (timeLipschitz sampleOne sampleState)
    (Fraction.ofInt 24) := by decide

theorem sample_time_budget :
  Fraction.equiv
    (Fraction.mul (timeLipschitz sampleOne sampleState)
      (durationDifference sampleQuarter sampleEighth).abs)
    (Fraction.ofInt 3) := by decide

theorem sample_parameter_bound :
  Fraction.le
    (stateNorm (stateSub
      (endpoint sampleOne sampleEighth sampleState 0)
      (endpoint sampleOne sampleQuarter sampleState 0)))
    (Fraction.mul (timeLipschitz sampleOne sampleState)
      (durationDifference sampleQuarter sampleEighth).abs) :=
  endpoint_time_bound sampleOne sampleQuarter sampleEighth sampleState 0
  (by decide) (by decide) sample_short_times.1 sample_short_times.2

```

NewtonLimitDynamics/Polygon/HarmonicTimeRealization.lean — Actual harmonic state values indexed by the constructed binary-time quotient. All comparisons use finite same-grid end-kick schedules. The quotient is not identified with an external real interval; force identification and actual intervening-region area remain separate.

```

theorem countState_coarse (w T : Fraction) (s : Point × Point)
  (j n : Nat) :
  stateEquiv (countState w T s j n)
    (coarseAt w (duration T (j + 1)) s n) := by
  let h := duration T (j + 1)
  have hc := schedule_replicate_congr w (duration T j) (Fraction.add h h)
  (duration_halving T j) n s
  (⟨Fraction.equiv_refl _, Fraction.equiv_refl _⟩,

```

```

    (Fraction.equiv_refl _, Fraction.equiv_refl _))
  simp only [countState, coarseAt_schedule] using hc

theorem countState_le_two (w T : Fraction) (s : Point × Point)
  (j n : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hn : n ≤ blocks j) :
  Fraction.le (stateNorm (countState w T s j n))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  let h := duration T (j + 1)
  have hprefix := smallTime_prefix w h n (blocks j) hT hn
  (dyadic_smallTime w T j hs)
  exact Fraction.le_equiv_left (stateNorm_equiv (countState_coarse w T s j n))
    (coarse_state_le_two w h s n hT hprefix)

theorem countState_step_bound (w T : Fraction) (s : Point × Point)
  (j n : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hn : n ≤ blocks j) :
  Fraction.le (distance (countState w T s j (n + 1)))
    (countState w T s j n))
  (Fraction.mul (duration T j) (stateTimeFactor w s)) := by
  let h := duration T j
  let q := countState w T s j n
  have hle : Fraction.le h (Fraction.ofInt 1) := by
    have h1 := duration_le_time T j hT
    have h2 := dyadic_time_le_half w T hT hs
    have h3 : Fraction.le ((1, 2), by decide) : Fraction)
      (Fraction.ofInt 1) := by unfold Fraction.le; decide
    exact Fraction.magnitudes.le_trans h1
      (Fraction.magnitudes.le_trans h2 h3)
  have hb := cell_increment_bound w h q hT hle
  have hq := countState_le_two w T s j n hT hs hn
  have hfac : 0 ≤ (Fraction.add (Fraction.ofInt 1) w.abs).num := by
    unfold Fraction.add Fraction.ofInt Fraction.abs
    dsimp
    have hw := Int.ofNat_nonneg w.num.natAbs
    have hd := w.den_pos
    omega
  have hm1 := Fraction.mul_le_mul_nonnegative_left hq
    (Fraction.add (Fraction.ofInt 1) w.abs) hfac
  have hm2 := Fraction.mul_le_mul_nonnegative_left hm1 h hT
  have habs := Fraction.abs_of_nonnegative h hT
  have he := Fraction.mul_equiv habs
    (Fraction.equiv_refl
      (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm q)))
  have hchain := Fraction.magnitudes.le_trans
    (Fraction.le_equiv_right hb he) hm2
  have hstep : countState w T s j (n + 1) = cell (linearField w) h q :=
    schedule_replicate_step w h s n
  rw [hstep]
  change Fraction.le (stateNorm (stateSub (cell (linearField w) h q) q))
    (Fraction.mul h (stateTimeFactor w s))
  apply Fraction.le_equiv_right hchain
  simp only [stateTimeFactor, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

theorem countState_gap (w T : Fraction) (s : Point × Point)
  (j : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  (n k : Nat) → n + k ≤ blocks j →
  Fraction.le (distance (countState w T s j (n + k)))
    (countState w T s j n))
  (Fraction.mul (Fraction.ofInt (k : Int))
    (Fraction.mul (duration T j) (stateTimeFactor w s)))
  | n, k, hnk => FiniteSequenceGap.finite_gap distance stateSub_self_norm_zero
  stateSub_triangle (countState w T s j) (blocks j)
  (Fraction.mul (duration T j) (stateTimeFactor w s))
  (fun i hi => countState_step_bound w T s j i hT hs (Nat.le_of_lt hi)) n k hnk

theorem stateTimeFactor_nonnegative (w : Fraction) (s : Point × Point) :
  0 ≤ (stateTimeFactor w s).num := by
  unfold stateTimeFactor Fraction.mul Fraction.add Fraction.ofInt Fraction.abs

```

```

dsimp
have hw := Int.ofNat_nonneg w.num.natAbs
have hd := w.den_pos
have hm := stateNorm_nonnegative s
exact Int.mul_nonneg (by decide)
(Int.mul_nonneg (by omega) hm)

theorem countState_ordered_bound (w T : Fraction) (s : Point × Point)
  (j n k : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hnk : n + k ≤ blocks j) :
  Fraction.le (distance (countState w T s j (n + k))
    (countState w T s j n))
    (Fraction.mul
      (durationDifference (countTime T j n) (countTime T j (n + k))).abs
      (stateTimeFactor w s)) := by
have hb := countState_gap w T s j hT hs n k hnk
apply Fraction.le_equiv_right hb
have he := countTime_abs_difference T j n k hT
apply Fraction.equiv_symm
apply Fraction.equiv_trans
(Fraction.mul_equiv he (Fraction.equiv_refl _))
simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
ac_nf

theorem countState_same_grid_bound (w T : Fraction) (s : Point × Point)
  (j m n : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hm : m ≤ blocks j) (hn : n ≤ blocks j) :
  Fraction.le (distance (countState w T s j m)
    (countState w T s j n))
    (Fraction.mul
      (durationDifference (countTime T j n) (countTime T j m)).abs
      (stateTimeFactor w s)) := by
rcases Nat.le_total n m with hnm | hmn
· have he : n + (m - n) = m := by omega
  have hb := countState_ordered_bound w T s j n (m - n) hT hs
  (by simp only [he] using hm)
  simp only [he] using hb
· have he : m + (n - m) = n := by omega
  have hb := countState_ordered_bound w T s j m (n - m) hT hs
  (by simp only [he] using hn)
  rw [he] at hb
have hd := stateSub_norm_symm (countState w T s j m)
(countState w T s j n)
have ht := durationDifference_abs_symm (countTime T j m)
(countTime T j n)
exact Fraction.le_equiv_right
(Fraction.le_equiv_left hd hb)
(Fraction.mul_equiv ht (Fraction.equiv_refl _))

theorem prefix_time_bound (b c : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) :
  Fraction.le (distance (prefixState b w T s j)
    (prefixState c w T s j))
    (Fraction.mul (distance (timeState b T j) (timeState c T j))
      (stateTimeFactor w s)) := by
have hb := countState_same_grid_bound w T s j (ticks b j) (ticks c j)
hT hs (ticks_le_blocks b j) (ticks_le_blocks c j)
apply Fraction.le_equiv_right hb
exact Fraction.mul_equiv
(Fraction.equiv_symm (scalarState_distance
  (timeApprox b T j) (timeApprox c T j)))
(Fraction.equiv_refl _)

theorem address_state_equiv (b c : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T)
  (hbc : AddressEquiv T hT b c) :
  NameEquiv (prefixName b w T s hT hs)
  (prefixName c w T s hT hs) := by

```

```

intro eps heps
let C := stateTimeFactor w s
have hC := stateTimeFactor_nonnegative w s
let delta := factorDelta C eps hC
obtain (N, hN) := hbc delta (factorDelta_positive C eps hC heps)
refine (N, ?_)
intro j hj
have hb := prefix_time_bound b c w T s j hT hs
have hsmall := factor_control C eps
  (distance (timeState b T j) (timeState c T j)) hC
  (stateNorm_nonnegative _) (hN j hj)
exact Fraction.magnitudes.lt_of_le_lt hb hsmall

theorem gammaValue_address (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) :
  gammaValue w T s hT hs (Quotient.mk _ b) =
  binaryValue b w T s hT hs := rfl

theorem gamma_within (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (x y : BinaryTime T hT) (R : Fraction)
  (_hR : 0 ≤ R.num) (hxy : TimeWithin T hT x y R) :
  Within (gammaValue w T s hT hs x)
    (gammaValue w T s hT hs y)
    (Fraction.mul R (stateTimeFactor w s)) := by
  induction x using Quotient.inductionOn with
  | _ b =>
    induction y using Quotient.inductionOn with
    | _ c =>
      change NameBound (BinaryTime.timeName b T hT)
        (BinaryTime.timeName c T hT) R at hxy
      change NameBound (prefixName b w T s hT hs)
        (prefixName c w T s hT hs)
        (Fraction.mul R (stateTimeFactor w s))
      apply nameBound_scale (prefixName b w T s hT hs)
        (prefixName c w T s hT hs)
        (BinaryTime.timeName b T hT) (BinaryTime.timeName c T hT)
        (stateTimeFactor w s) R (stateTimeFactor_nonnegative w s)
      · intro j
      · exact prefix_time_bound b c w T s j hT hs
      · exact hxy

theorem timeTolerance_positive (w : Fraction) (s : Point × Point)
  (eps : Fraction) (heps : 0 < eps.num) :
  0 < (timeTolerance w s eps).num :=
  factorDelta_positive _ _ _ heps

theorem gamma_uniform_continuity (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (eps : Fraction) (heps : 0 < eps.num)
  (x y : BinaryTime T hT)
  (hxy : TimeWithin T hT x y (timeTolerance w s eps)) :
  Within (gammaValue w T s hT hs x)
    (gammaValue w T s hT hs y) eps.half := by
  have hb := gamma_within w T s hT hs x y
  (timeTolerance w s eps)
  (Int.le_of_lt (timeTolerance_positive w s eps heps)) hxy
  exact within_mono _ _ _
  (factor_delta_weak _ _ (stateTimeFactor_nonnegative w s)
    (by simp only [Fraction.half] using Int.le_of_lt heps)) hb

theorem duration_factor_eventually_small (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ j : Nat, N ≤ j →
    Fraction.lt (Fraction.mul (duration T j) (stateTimeFactor w s))
      eps := by
  let C := stateTimeFactor w s
  let delta := factorDelta C eps (stateTimeFactor_nonnegative w s)

```

```

obtain (N, hN) := duration_eventually_small T delta hT
  (factorDelta_positive C eps (stateTimeFactor_nonnegative w s) heps)
refine (N, ?_)
intro j hj
exact factor_control C eps (duration T j)
  (stateTimeFactor_nonnegative w s) hT (hN j hj)

theorem left_endpoint_value (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  gammaValue w T s hT hs (leftTime T hT) = embed s := by
  apply Quotient.sound
  intro eps heps
  refine {0, ?_}
  intro j _
  change Fraction.lt (distance (prefixState leftAddress w T s j) s) eps
  rw [show prefixState leftAddress w T s j = s from
    all_zero_prefix w T s j]
  exact distance_self_lt s eps heps

theorem right_prefix_endpoint_bound (w T : Fraction)
  (s : Point × Point) (j : Nat) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) :
  Fraction.le (distance (endpoint w T s j)
    (prefixState rightAddress w T s j))
    (Fraction.mul (duration T j) (stateTimeFactor w s)) := by
  have hcount : ticks rightAddress j + 1 ≤ blocks j :=
    Nat.le_of_eq (right_ticks j)
  have hb := countState_gap w T s j hT hs (ticks rightAddress j) 1 hcount
  rw [right_ticks] at hb
  change Fraction.le (distance (endpoint w T s j)
    (prefixState rightAddress w T s j)) _ at hb
  apply Fraction.le_equiv_right hb
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt,
    Int.natCast_one,
    Int.one_mul, Int.mul_one]

theorem right_endpoint_value (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  gammaValue w T s hT hs (rightTime T hT) =
  endpointValue w T s hT hs := by
  apply Quotient.sound
  intro eps heps
  obtain (N, hN) := duration_factor_eventually_small w T s hT eps heps
  refine (N, ?_)
  intro j hj
  have hb := right_prefix_endpoint_bound w T s j hT hs
  have hs := stateSub_norm_symm
    (prefixState rightAddress w T s j) (endpoint w T s j)
  exact Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_equiv_left hs hb) (hN j hj)

theorem alias_ticks (j : Nat) :
  ticks firstAlias (j + 1) = blocks j ∧
  ticks secondAlias (j + 1) + 1 = blocks j := by
  induction j with
  | zero =>
    constructor <=> decide
  | succ j ih =>
    rcases ih with (hfirst, hsecond)
    constructor
    · change 2 * ticks firstAlias (j + 1) + bit firstAlias (j + 1) =
      blocks (j + 1)
      have hb : bit firstAlias (j + 1) = 0 := by
        simp [bit, firstAlias]
      rw [hb, blocks_succ]
      omega
    · change 2 * ticks secondAlias (j + 1) +
      bit secondAlias (j + 1) + 1 = blocks (j + 1)
      have hb : bit secondAlias (j + 1) = 1 := by
        simp [bit, secondAlias]

```

```

    rw [hb, blocks_succ]
    omega

theorem alias_time_distance (T : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.equiv
    (distance (timeState firstAlias T (j + 1))
      (timeState secondAlias T (j + 1)))
    (duration T (j + 1)) := by
  have hcounts : ticks secondAlias (j + 1) + 1 =
    ticks firstAlias (j + 1) := by
    have ha := alias_ticks j
    omega
  have htime := countTime_abs_difference T (j + 1)
    (ticks secondAlias (j + 1)) 1 hT
  rw [hcounts] at htime
  have hd := scalarState_distance
    (timeApprox firstAlias T (j + 1))
    (timeApprox secondAlias T (j + 1))
  apply Fraction.equiv_trans hd
  apply Fraction.equiv_trans htime
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt,
    Int.natCast_one, Int.one_mul, Int.mul_one]

theorem alias_address_equiv (T : Fraction) (hT : 0 ≤ T.num) :
  AddressEquiv T hT firstAlias secondAlias := by
  intro eps heps
  obtain (N, hN) := duration_eventually_small T eps hT heps
  refine (N + 1, ?_)
  intro j hj
  have hj' : j - 1 + 1 = j := by omega
  have hbound := alias_time_distance T (j - 1) hT
  rw [hj'] at hbound
  exact Fraction.magnitudes.lt_of_le_lt
    ((Fraction.equiv_iff_mutual_le _ _).mp hbound).1
    (hN j (by omega))

theorem alias_time_eq (T : Fraction) (hT : 0 ≤ T.num) :
  (Quotient.mk (addressSetoid T hT) firstAlias : BinaryTime T hT) =
  Quotient.mk (addressSetoid T hT) secondAlias :=
  Quotient.sound (alias_address_equiv T hT)

theorem alias_value_eq (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  binaryValue firstAlias w T s hT hs =
  binaryValue secondAlias w T s hT hs := by
  exact congrArg (gammaValue w T s hT hs) (alias_time_eq T hT)

theorem zero_time_value (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (hzero : T.num = 0) :
  gammaValue w T s hT hs (Quotient.mk _ b) = embed s := by
  apply Quotient.sound
  intro eps heps
  refine (0, ?_)
  intro j _
  have he := (distance_zero_iff_stateEquiv
    (prefixState b w T s j) s).mpr
    (zero_time_prefix b w T s j hzero)
  have hself := stateSub_self_norm_zero s
  have hle : Fraction.le (distance (prefixState b w T s j) s)
    (distance s s) := Fraction.le_of_equiv
    (Fraction.equiv_trans he (Fraction.equiv_symm hself))
  exact Fraction.magnitudes.lt_of_le_lt hle
    (distance_self_lt s eps heps)

theorem zero_state_norm_value (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (hzero : (stateNorm s).num = 0) :
  gammaValue w T s hT hs (Quotient.mk _ b) = embed s := by

```

```

have hC : (stateTimeFactor w s).num = 0 := by
  unfold stateTimeFactor Fraction.mul
  dsimp
  simp only [hzero, Int.mul_zero, Int.zero_mul]
calc
  gammaValue w T s hT hs (Quotient.mk _ b) =
    gammaValue w T s hT hs (leftTime T hT) := by
      apply Quotient.sound
      intro eps heps
      refine {0, ?_}
      intro j _
      have hb := prefix_time_bound b leftAddress w T s j hT hs
      have hz : (Fraction.mul
        (distance (timeState b T j) (timeState leftAddress T j))
        (stateTimeFactor w s)).num = 0 := by
        unfold Fraction.mul
        dsimp
        simp [hC]
      have hstrict : Fraction.lt
        (Fraction.mul
          (distance (timeState b T j) (timeState leftAddress T j))
          (stateTimeFactor w s)) eps := by
        unfold Fraction.lt
        simp only [hz, Int.zero_mul]
        exact Int.mul_pos heps
        (Fraction.mul
          (distance (timeState b T j) (timeState leftAddress T j))
          (stateTimeFactor w s)).den_pos
      exact Fraction.magnitudes.lt_of_le_lt hb hstrict
      _ = embed s := left_endpoint_value w T s hT hs

theorem sample_alias_counts :
  ticks firstAlias 2 = 2 ∧ ticks secondAlias 2 = 1 := by decide

theorem sample_alias_time_gap :
  Fraction.equiv
    (distance (timeState firstAlias sampleQuarter 2)
      (timeState secondAlias sampleQuarter 2))
    (1, 16, by decide) := by decide

theorem sample_state_time_factor :
  Fraction.equiv (StateTimeFactor sampleOne sampleState)
    (Fraction.ofInt 8) := by decide

theorem sample_alias_bound :
  Fraction.equiv
    (Fraction.mul (distance
      (timeState firstAlias sampleQuarter 2)
      (timeState secondAlias sampleQuarter 2))
      (stateTimeFactor sampleOne sampleState))
    (1, 2, by decide) := by decide

theorem sample_alias_actual_error :
  Fraction.equiv
    (distance
      (prefixState firstAlias sampleOne sampleQuarter sampleState 2)
      (prefixState secondAlias sampleOne sampleQuarter sampleState 2))
    (8927, 65536, by decide) := by decide

theorem sample_right_ne_left :
  gammaValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
    (rightTime sampleQuarter (by decide)) ≠
  gammaValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
    (leftTime sampleQuarter (by decide)) := by
  have hneq := CauchyValues.sample_endpoint_value_ne_initial
  change endpointValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide) ≠
  embed sampleState at hneq

```

```

intro h
apply hneg
calc
  endpointValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide) =
    gammaValue sampleOne sampleQuarter sampleState
      (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
      (rightTime sampleQuarter (by decide)) :=
      (right_endpoint_value _ _ _ _).symm
  _ = gammaValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
    (leftTime sampleQuarter (by decide)) := h
  _ = embed sampleState := left_endpoint_value _ _ _ _

```

NewtonLimitDynamics/Polygon/HarmonicUniform.lean — Mesh-uniform finite bounds for the actual harmonic coarse and fine schedules under a small total-time condition. These coordinate L1 estimates provide finite construction support only: no curve, completion, geometric region between paths, or historical limiting step is constructed here. The small-time threshold uses the chosen coordinate/unit calibration; it is not a universal physical time bound.

```

private theorem denom_pos (w h : Fraction) : 0 < denom w h :=
  Int.mul_pos h.den_pos w.den_pos

private theorem driftIncrement_nonnegative (w h : Fraction) (hh : 0 ≤ h.num) :
  0 ≤ driftIncrement w h :=
  Int.mul_nonneg hh (Int.le_of_lt w.den_pos)

private theorem kickIncrement_nonnegative (w h : Fraction) (hh : 0 ≤ h.num) :
  0 ≤ kickIncrement w h :=
  Int.mul_nonneg hh (Int.ofNat_nonneg _)

private theorem coarseWeights_nonnegative (w h : Fraction) (hh : 0 ≤ h.num) :
  (n : Nat) → Nonnegative (coarseWeights w h n)
| 0 => by simp [coarseWeights, Nonnegative]
| n + 1 => by
  simp only [coarseWeights, Nonnegative, List.mem_cons]
  intro a ha
  rcases ha with rfl | rfl | ht
  · exact Int.mul_nonneg (by decide) (driftIncrement_nonnegative w h hh)
  · exact Int.mul_nonneg (by decide) (kickIncrement_nonnegative w h hh)
  · exact coarseWeights_nonnegative w h hh n a ht

private theorem fineWeights_nonnegative (w h : Fraction) (hh : 0 ≤ h.num) :
  (n : Nat) → Nonnegative (fineWeights w h n)
| 0 => by simp [fineWeights, Nonnegative]
| n + 1 => by
  simp only [fineWeights, Nonnegative, List.mem_cons]
  intro a ha
  rcases ha with rfl | rfl | rfl | rfl | ht
  · exact driftIncrement_nonnegative w h hh
  · exact kickIncrement_nonnegative w h hh
  · exact driftIncrement_nonnegative w h hh
  · exact kickIncrement_nonnegative w h hh
  · exact fineWeights_nonnegative w h hh n a ht

private theorem common_weight_sum (w h : Fraction) :
  (n : Nat) →
  weightSum (coarseWeights w h n) =
  weightSum (fineWeights w h n) ∧
  weightSum (fineWeights w h n) =
  2 * (n : Int) * (driftIncrement w h + kickIncrement w h)
| 0 => by simp [coarseWeights, fineWeights, weightSum]
| n + 1 => by
  obtain {hc, hf} := common_weight_sum w h n
  simp only [coarseWeights, fineWeights, weightSum, Int.natCast_add,
    Int.natCast_one]
  constructor

```

```

      · rw [hc]
      · omega
      · rw [hf]
      · simp only [Int.add_mul, Int.mul_add]
      · omega

/-- The displayed total time is the elapsed time of the actual coarse list. -/
theorem coarse_elapsed_totalTime (h : Fraction) (n : Nat) :
  Fraction.equiv (elapsed (List.replicate n (Fraction.add h h)))
    (totalTime h n) := by
  induction n with
  | zero =>
    simp only [List.replicate_zero, elapsed, totalTime, Fraction.equiv,
      Fraction.mul, Fraction.ofInt]
    simp
  | succ n ih =>
    simp only [List.replicate_succ, elapsed]
    have ht := Fraction.equiv_trans
      (Fraction.add_comm (Fraction.add h h)
        (elapsed (List.replicate n (Fraction.add h h))))
      (Fraction.add_equiv_right (Fraction.add h h) ih)
    apply Fraction.equiv_trans ht
    simp only [totalTime, Fraction.equiv, Fraction.add, Fraction.mul,
      Fraction.ofInt, Int.natCast_add, Int.natCast_one,
      Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
    simp only [show (2 : Int) = 1 + 1 by rfl,
      Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
    ac_nf

/-- The fine list reaches the same displayed total time. -/
theorem fine_elapsed_totalTime (h : Fraction) (n : Nat) :
  Fraction.equiv (elapsed (fineDurations h n)) (totalTime h n) :=
  Fraction.equiv_trans
    (Fraction.equiv_symm (schedules_common_time (Fraction.ofInt 0) h n))
    (coarse_elapsed_totalTime h n)

private theorem smallTime_integer (w h : Fraction) (n : Nat)
  (hs : SmallTime w h n) :
  2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) ≤ denom w h := by
  unfold SmallTime totalTime halfThreshold Fraction.le Fraction.mul Fraction.add
  Fraction.ofInt Fraction.abs at hs
  dsimp at hs
  simp only [Int.mul_one, Int.one_mul, Int.add_zero] at hs
  change 2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) ≤
    h.den * w.den
  calc
    2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) =
      2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int)) * 2 := by ac_rfl
    _ ≤ h.den * w.den := hs

/-- The small-time condition is inherited by every earlier actual block. -/
theorem smallTime_prefix (w h : Fraction) (i n : Nat)
  (hh : 0 ≤ h.num) (hi : i ≤ n) (hs : SmallTime w h n) :
  SmallTime w h i := by
  have hn := smallTime_integer w h n hs
  have hcoef : 0 ≤ 4 * h.num * (w.den + (w.num.natAbs : Int)) := by
    exact Int.mul_nonneg
      (Int.mul_nonneg (by decide) hh)
      (Int.add_nonneg (Int.le_of_lt w.den_pos) (Int.ofNat_nonneg _))
  have hcast : (i : Int) ≤ (n : Int) := Int.ofNat_le.mpr hi
  have hm := Int.mul_le_mul_of_nonneg_right hcast hcoef
  have hpref :
    2 * (2 * (i : Int) * h.num * (w.den + (w.num.natAbs : Int))) ≤
      2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) := by
    calc
      _ = (i : Int) * (4 * h.num * (w.den + (w.num.natAbs : Int))) := by
        simp only [show (4 : Int) = 2 * 2 by rfl]
        ac_rfl
      _ ≤ (n : Int) * (4 * h.num * (w.den + (w.num.natAbs : Int))) := hm
      _ = _ := by

```

```

    simp only [show (4 : Int) = 2 * 2 by rfl]
    ac_rfl
  unfold SmallTime totalTime halfThreshold Fraction.le Fraction.mul Fraction.add
    Fraction.ofInt Fraction.abs
  dsimp
  simp only [Int.mul_one, Int.one_mul, Int.add_zero]
  calc
    2 * (i : Int) * h.num * (w.den + (w.num.natAbs : Int)) * 2 =
      2 * (2 * (i : Int) * h.num * (w.den + (w.num.natAbs : Int))) := by ac_rfl
    _ ≤ 2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) := hpref
    _ ≤ denom w h := hn

private theorem fineWeights_small (w h : Fraction) (n : Nat)
  (hs : SmallTime w h n) :
    2 * weightSum (fineWeights w h n) ≤ denom w h := by
  have hi := smallTime_integer w h n hs
  have hf := (common_weight_sum w h n).2
  calc
    2 * weightSum (fineWeights w h n) =
      2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) := by
      rw [hf]
      simp only [driftIncrement, kickIncrement, Int.mul_add]
      ac_nf
    _ ≤ denom w h := hi

private theorem coarseWeights_small (w h : Fraction) (n : Nat)
  (hs : SmallTime w h n) :
    2 * weightSum (coarseWeights w h n) ≤ denom w h := by
  rw [(common_weight_sum w h n).1]
  exact fineWeights_small w h n hs

theorem two_mul (x : Int) : 2 * x = x + x := by omega

private theorem coarse_block_equiv (w h : Fraction) (hh : 0 ≤ h.num) :
  Fraction.equiv
    (amplification (denom w h) (denom_pos w h)
      [2 * driftIncrement w h, 2 * kickIncrement w h])
    (coarseFactor w h) := by
  have hsum : 0 ≤ (Fraction.add h h).num := by
  unfold Fraction.add
  exact Int.add_nonneg
    (Int.mul_nonneg hh (Int.le_of_lt h.den_pos))
    (Int.mul_nonneg hh (Int.le_of_lt h.den_pos))
  simp only [amplification, factorProduct, denom, driftIncrement,
    kickIncrement, coarseFactor, kappa, Fraction.equiv, Fraction.abs,
    Fraction.add, Fraction.mul, Fraction.ofInt,
    List.length_cons, List.length_nil, Int.pow_succ,
    Int.pow_zero, Int.mul_one, Int.one_mul]
  change 0 ≤ h.num * h.den + h.num * h.den at hsum
  rw [Int.natAbs_of_nonneg hsum]
  simp only [two_mul, Int.add_mul, Int.mul_add]
  ac_nf

private theorem fine_block_equiv (w h : Fraction) (hh : 0 ≤ h.num) :
  Fraction.equiv
    (amplification (denom w h) (denom_pos w h)
      [driftIncrement w h, kickIncrement w h,
        driftIncrement w h, kickIncrement w h])
    (fineFactor w h) := by
  simp only [amplification, factorProduct, denom, driftIncrement,
    kickIncrement, fineFactor, kappa, Fraction.equiv, Fraction.abs,
    Fraction.add, Fraction.mul, Fraction.ofInt,
    List.length_cons, List.length_nil, Int.pow_succ,
    Int.pow_zero, Int.mul_one, Int.one_mul,
    Int.natAbs_of_nonneg hh]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

private theorem coarse_power_amplification (w h : Fraction) (hh : 0 ≤ h.num) :
  (n : Nat) →

```

```

    Fraction.equiv
      (amplification (denom w h) (denom_pos w h) (coarseWeights w h n))
      (fpower (coarseFactor w h) n)
  | 0 => amplification_empty _ _
  | n + 1 => by
    have ha := amplification_append (denom w h) (denom_pos w h)
      [2 * driftIncrement w h, 2 * kickIncrement w h] (coarseWeights w h n)
    have hb := coarse_block_equiv w h hh
    have hi := coarse_power_amplification w h hh n
    change Fraction.equiv
      (amplification (denom w h) (denom_pos w h)
        ([2 * driftIncrement w h, 2 * kickIncrement w h] ++ coarseWeights w h n))
      (Fraction.mul (coarseFactor w h) (fpower (coarseFactor w h) n))
    exact Fraction.equiv_trans ha (Fraction.mul_equiv hb hi)

private theorem fine_power_amplification (w h : Fraction) (hh : 0 ≤ h.num) :
  (n : Nat) →
    Fraction.equiv
      (amplification (denom w h) (denom_pos w h) (fineWeights w h n))
      (fpower (fineFactor w h) n)
  | 0 => amplification_empty _ _
  | n + 1 => by
    have ha := amplification_append (denom w h) (denom_pos w h)
      [driftIncrement w h, kickIncrement w h,
        driftIncrement w h, kickIncrement w h] (fineWeights w h n)
    have hb := fine_block_equiv w h hh
    have hi := fine_power_amplification w h hh n
    change Fraction.equiv
      (amplification (denom w h) (denom_pos w h)
        ([driftIncrement w h, kickIncrement w h,
          driftIncrement w h, kickIncrement w h] ++ fineWeights w h n))
      (Fraction.mul (fineFactor w h) (fpower (fineFactor w h) n))
    exact Fraction.equiv_trans ha (Fraction.mul_equiv hb hi)

/-- Uniform finite growth of the actual coarse amplification power. -/
theorem coarse_power_le_two (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (fpower (coarseFactor w h) n) (Fraction.ofInt 2) :=
  Fraction.le_equiv_left (Fraction.equiv_symm (coarse_power_amplification w h hh n))
    (uniform_amplification (denom w h) (denom_pos w h) (coarseWeights w h n)
      (coarseWeights_nonnegative w h hh n) (coarseWeights_small w h n hs))

/-- Uniform finite growth of the actual two-cell perturbation power. -/
theorem fine_power_le_two (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (fpower (fineFactor w h) n) (Fraction.ofInt 2) :=
  Fraction.le_equiv_left (Fraction.equiv_symm (fine_power_amplification w h hh n))
    (uniform_amplification (denom w h) (denom_pos w h) (fineWeights w h n)
      (fineWeights_nonnegative w h hh n) (fineWeights_small w h n hs))

/-- Actual coarse states stay inside twice the initial coordinate magnitude. -/
theorem coarse_state_le_two (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (stateNorm (coarseAt w h s n))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) :=
  Fraction.magnitudes.le_trans (coarse_norm_bound w h s n)
    (Fraction.mul_le_mul_nonnegative (coarse_power_le_two w h n hh hs)
      (stateNorm s) (stateNorm_nonnegative s))

private theorem fine_cell_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le (stateNorm (HarmonicRefinement.fine w h s))
    (Fraction.mul (fineFactor w h) (stateNorm s)) := by
  have h₁ := cell_bound w h (cell (linearField w) h s)
  have h₂ := cell_bound w h s
  have hm := Fraction.mul_le_mul_nonnegative_left h₂ (kappa w h)
    (kappa_nonnegative w h)
  have hc := Fraction.magnitudes.le_trans h₁ hm
  apply Fraction.le_equiv_right hc
  simp only [fineFactor, Fraction.equiv, Fraction.mul]
  ac_nf

```

```

theorem fine_norm_bound (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    Fraction.le (stateNorm (fineAt w h s n))
      (Fraction.mul (fpower (fineFactor w h) n) (stateNorm s))
| 0 => by
  apply Fraction.le_of_equiv
  simp only [fineAt, fpower, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp only [Int.one_mul, Int.mul_one]
| n + 1 => by
  have hf := fine_cell_bound w h (fineAt w h s n)
  have hi := fine_norm_bound w h s n
  have hm := Fraction.mul_le_mul_nonnegative_left hi (fineFactor w h)
    (fineFactor_nonnegative w h)
  have hc := Fraction.magnitudes.le_trans hf hm
  apply Fraction.le_equiv_right hc
  simp only [fineAt, fpower, Fraction.equiv, Fraction.mul]
  ac_nf

theorem fine_state_le_two (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (stateNorm (fineAt w h s n))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) :=
  Fraction.magnitudes.le_trans (fine_norm_bound w h s n)
    (Fraction.mul_le_mul_nonnegative (fine_power_le_two w h n hh hs)
      (stateNorm s) (stateNorm_nonnegative s))

private theorem square_dominates_double (D a : Int) (ha : 0 ≤ a) :
  D * (D + 2 * a) ≤ (D + a) * (D + a) := by
  have hp := Int.mul_nonneg ha ha
  have he : (D + a) * (D + a) = D * (D + 2 * a) + a * a := by
    simp only [two_mul, Int.add_mul, Int.mul_add]
  ac_nf
  omega

private theorem block_product_order (D A B : Int) (hD : 0 < D)
  (hA : 0 ≤ A) (hB : 0 ≤ B) :
  (D + 2 * A) * (D + 2 * B) * (D * D) ≤
    ((D + A) * (D + A)) * ((D + B) * (D + B)) := by
  have p := square_dominates_double D A hA
  have q := square_dominates_double D B hB
  have hp : 0 ≤ D * (D + 2 * A) :=
    Int.mul_nonneg (Int.le_of_lt hD)
      (Int.add_nonneg (Int.le_of_lt hD) (Int.mul_nonneg (by decide) hA))
  have hq : 0 ≤ (D + B) * (D + B) := by
    have hb : 0 ≤ D + B := Int.add_nonneg (Int.le_of_lt hD) hB
    exact Int.mul_nonneg hb hb
  have h1 := Int.mul_le_mul_of_nonneg_right p hq
  have h2 := Int.mul_le_mul_of_nonneg_left q hp
  calc
    (D + 2 * A) * (D + 2 * B) * (D * D) =
      (D * (D + 2 * A)) * (D * (D + 2 * B)) := by ac_rfl
    _ ≤ (D * (D + 2 * A)) * ((D + B) * (D + B)) := h2
    _ ≤ ((D + A) * (D + A)) * ((D + B) * (D + B)) := h1

private theorem block_amplification_order (w h : Fraction) (hh : 0 ≤ h.num) :
  Fraction.le
    (amplification (denom w h) (denom_pos w h)
      [2 * driftIncrement w h, 2 * kickIncrement w h])
    (amplification (denom w h) (denom_pos w h)
      [driftIncrement w h, kickIncrement w h,
        driftIncrement w h, kickIncrement w h]) := by
  have ho := block_product_order (denom w h) (driftIncrement w h)
    (kickIncrement w h) (denom_pos w h)
    (driftIncrement_nonnegative w h hh) (kickIncrement_nonnegative w h hh)
  unfold Fraction.le amplification
  simp only [factorProduct, List.length_cons, List.length_nil,
    Int.pow_succ, Int.pow_zero, Int.mul_one, Int.one_mul]
  have hd : 0 ≤ denom w h * denom w h :=
    Int.mul_nonneg (Int.le_of_lt (denom_pos w h)) (Int.le_of_lt (denom_pos w h))

```

```

have hm := Int.mul_le_mul_of_nonneg_right ho hd
calc
  _ =
    ((denom w h + 2 * driftIncrement w h) *
     (denom w h + 2 * kickIncrement w h) *
     (denom w h * denom w h)) * (denom w h * denom w h) := by ac_rfl
  _ ≤ ((denom w h + driftIncrement w h) *
       (denom w h + driftIncrement w h) *
       ((denom w h + kickIncrement w h) *
        (denom w h + kickIncrement w h))) *
       (denom w h * denom w h) := hm
  _ = _ := by ac_rfl

theorem coarseFactor_le_fineFactor (w h : Fraction) (hh : 0 ≤ h.num) :
  Fraction.le (coarseFactor w h) (fineFactor w h) :=
  Fraction.le_equiv_right
    (Fraction.le_equiv_left (Fraction.equiv_symm (coarse_block_equiv w h hh))
     (block_amplification_order w h hh))
    (fine_block_equiv w h hh)

private theorem one_le_one_add (a : Fraction) (ha : 0 ≤ a.num) :
  Fraction.le (Fraction.ofInt 1) (Fraction.add (Fraction.ofInt 1) a) := by
  unfold Fraction.le Fraction.add Fraction.ofInt
  dsimp
  have hp := Int.le_of_lt a.den_pos
  omega

theorem one_le_kappa (w h : Fraction) :
  Fraction.le (Fraction.ofInt 1) (kappa w h) := by
  let u := Fraction.add (Fraction.ofInt 1) h.abs
  let v := Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs)
  have hu := one_le_one_add h.abs (Fraction.abs_num_nonnegative h)
  have hv := one_le_one_add (Fraction.mul h.abs w.abs)
    (Int.mul_nonneg (Fraction.abs_num_nonnegative h) (Fraction.abs_num_nonnegative w))
  have h1 := Fraction.mul_le_mul_nonnegative hu (Fraction.ofInt 1) (by decide)
  have h2 := Fraction.mul_le_mul_nonnegative_left hv u (by
    unfold u Fraction.add Fraction.ofInt
    dsimp
    have hp := h.abs.den_pos
    have hn := Fraction.abs_num_nonnegative h
    omega)
  have hc := Fraction.magnitudes.le_trans h1 h2
  apply Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    decide) hc

private theorem one_le_fineFactor (w h : Fraction) :
  Fraction.le (Fraction.ofInt 1) (fineFactor w h) := by
  have hk := one_le_kappa w h
  have h1 := Fraction.mul_le_mul_nonnegative hk (Fraction.ofInt 1) (by decide)
  have h2 := Fraction.mul_le_mul_nonnegative_left hk (kappa w h)
    (kappa_nonnegative w h)
  have hc := Fraction.magnitudes.le_trans h1 h2
  apply Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    decide) hc

private theorem fpower_monotone (a b : Fraction) (ha : 0 ≤ a.num)
  (hb : 0 ≤ b.num) (hab : Fraction.le a b) :
  (n : Nat) → Fraction.le (fpower a n) (fpower b n)
| 0 => Fraction.magnitudes.le_refl _
| n + 1 => by
  have h1 := Fraction.mul_le_mul_nonnegative hab (fpower a n)
    (fpower_nonnegative a ha n)
  have h2 := Fraction.mul_le_mul_nonnegative_left
    (fpower_monotone a b ha hb hab n) b hb
  exact Fraction.magnitudes.le_trans h1 h2

private theorem coarse_power_le_fine_power (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) :

```

```

    Fraction.le (fpower (coarseFactor w h) n) (fpower (fineFactor w h) n) :=
    fpower_monotone _ _ (kappa_nonnegative w (Fraction.add h h))
    (fineFactor_nonnegative w h) (coarseFactor_le_fineFactor w h hh) n

private theorem budget_power_bound (w h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) :
  (n : Nat) → Fraction.le (errorBudget w h s n) (budgetCap w h s n)
| 0 => by
  apply Fraction.le_of_equiv
  simp only [errorBudget, budgetCap, count, fpower, Fraction.equiv,
    Fraction.ofInt, Fraction.mul]
  simp
| n + 1 => by
  let d := localFactor w h
  let r := fineFactor w h
  let M := stateNorm s
  let X := Fraction.mul (Fraction.mul d (fpower r n)) M
  have hi := budget_power_bound w h s hh n
  have hr := fineFactor_nonnegative w h
  have hd := localFactor_nonnegative w h
  have hpow := fpower_nonnegative r hr n
  have hX : 0 ≤ X.num :=
    Int.mul_nonneg (Int.mul_nonneg hd hpow) (stateNorm_nonnegative s)
  have h1 := Fraction.mul_le_mul_nonnegative_left hi r hr
  have hp := coarse_power_le_fine_power w h n hh
  have h2a := Fraction.mul_le_mul_nonnegative_left hp d hd
  have h2 := Fraction.mul_le_mul_nonnegative h2a M (stateNorm_nonnegative s)
  have h2b : Fraction.le X
    (Fraction.mul d (Fraction.mul (fpower r (n + 1)) M)) := by
    have hk := Fraction.mul_le_mul_nonnegative (one_le_fineFactor w h) X hX
    have he : Fraction.equiv X (Fraction.mul (Fraction.ofInt 1) X) := by
      simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp
    have hk' := Fraction.le_equiv_left he hk
    apply Fraction.le_equiv_right hk'
    simp only [d, r, M, X, fpower, Fraction.equiv, Fraction.mul]
    ac_nf
  have h2c := Fraction.magnitudes.le_trans h2 h2b
  have hsum := Fraction.add_le_add h1 h2c
  apply Fraction.le_equiv_right hsum
  simp only [d, r, M, errorBudget, budgetCap, count, fpower,
    Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add,
    Int.one_mul, Int.mul_one]
  ac_nf

private theorem budget_two_bound (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (errorBudget w h s n)
  (Fraction.mul (Fraction.ofInt 2)
    (Fraction.mul (count n) (Fraction.mul (localFactor w h) (stateNorm s)))) := by
  have h0 := budget_power_bound w h s hh n
  have hp := fine_power_le_two w h n hh hs
  have h1 := Fraction.mul_le_mul_nonnegative hp (stateNorm s) (stateNorm_nonnegative s)
  have h2 := Fraction.mul_le_mul_nonnegative_left h1 (localFactor w h)
    (localFactor_nonnegative w h)
  have h3 := Fraction.mul_le_mul_nonnegative_left h2 (count n) (by
    unfold count Fraction.ofInt
    exact Int.ofNat_nonneg n)
  have hc := Fraction.magnitudes.le_trans h0 h3
  apply Fraction.le_equiv_right hc
  simp only [budgetCap, count, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

theorem one_le_power (a : Fraction) (ha : 0 ≤ a.num)
  (h1 : Fraction.le (Fraction.ofInt 1) a) :
  (n : Nat) → Fraction.le (Fraction.ofInt 1) (fpower a n)
| 0 => Fraction.magnitudes.le_refl _
| n + 1 => by
  have hi := one_le_power a ha h1 n

```

```

    have hm := Fraction.mul_le_mul_nonnegative h1 (fpower a n)
      (fpower_nonnegative a ha n)
    have he : Fraction.equiv (fpower a n)
      (Fraction.mul (Fraction.ofInt 1) (fpower a n)) := by
      simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
      simp
    exact Fraction.magnitudes.le_trans hi (Fraction.le_equiv_left he hm)

private theorem factor_le_power_succ (a : Fraction) (ha : 0 ≤ a.num)
  (h1 : Fraction.le (Fraction.ofInt 1) a) (n : Nat) :
  Fraction.le a (fpower a (n + 1)) := by
  have hp := one_le_power a ha h1 n
  have hm := Fraction.mul_le_mul_nonnegative_left hp a ha
  exact Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp) hm

private theorem kappa_le_fineFactor (w h : Fraction) :
  Fraction.le (kappa w h) (fineFactor w h) := by
  have hk := one_le_kappa w h
  have hm := Fraction.mul_le_mul_nonnegative_left hk (kappa w h)
    (kappa_nonnegative w h)
  exact Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp) hm

private theorem kappa_le_two_of_positive_blocks (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h (n + 1)) :
  Fraction.le (kappa w h) (Fraction.ofInt 2) := by
  have h1 := kappa_le_fineFactor w h
  have h2 := factor_le_power_succ (fineFactor w h)
    (fineFactor_nonnegative w h) (one_le_fineFactor w h) n
  have h3 := fine_power_le_two w h (n + 1) hh hs
  exact Fraction.magnitudes.le_trans (Fraction.magnitudes.le_trans h1 h2) h3

private theorem meshAmplitude_nonnegative (w h : Fraction) :
  0 ≤ (meshAmplitude w h).num :=
  Int.mul_nonneg
    (Int.mul_nonneg (Fraction.abs_num_nonnegative h) (Fraction.abs_num_nonnegative h))
    (Fraction.abs_num_nonnegative w)

private theorem localFactor_le_three_amplitude (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h (n + 1)) :
  Fraction.le (localFactor w h)
    (Fraction.mul (Fraction.ofInt 3) (meshAmplitude w h)) := by
  have hk := kappa_le_two_of_positive_blocks w h n hh hs
  have hplus := Fraction.add_le_add_right hk (Fraction.ofInt 1)
  have hm := Fraction.mul_le_mul_nonnegative hplus (meshAmplitude w h)
    (meshAmplitude_nonnegative w h)
  have hleft : Fraction.equiv (localFactor w h)
    (Fraction.mul (Fraction.add (kappa w h) (Fraction.ofInt 1))
      (meshAmplitude w h)) := by
    unfold localFactor meshAmplitude
    exact Fraction.mul_comm _ _
  have hright : Fraction.equiv
    (Fraction.mul (Fraction.add (Fraction.ofInt 2) (Fraction.ofInt 1))
      (meshAmplitude w h))
    (Fraction.mul (Fraction.ofInt 3) (meshAmplitude w h)) := by
    simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
    simp only [Int.mul_one, Int.one_mul]
    ac_nf
  exact Fraction.le_equiv_right (Fraction.le_equiv_left hleft hm) hright

/-- Uniform finite error at common total time `T=2nh`: actual fine and coarse
endpoints differ in coordinate L1 magnitude by at most `3*T*h*|w|*M`.
The hypothesis includes `h≥0` and `T*(1+|w|)≤1/2`. This is an endpoint
estimate, with no limiting curve or intervening-area assertion. -/
theorem actual_uniform_error (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le

```

```

    (stateNorm (stateSub (fineAt w h s n) (coarseAt w h s n)))
    (Fraction.mul (Fraction.ofInt 3)
      (Fraction.mul (totalTime h n)
        (Fraction.mul h (Fraction.mul w.abs (stateNorm s)))))) := by
cases n with
| zero =>
  have h₀ := actual_error_bound w h s 0
  apply Fraction.le_equiv_right h₀
  simp only [errorBudget, totalTime, Fraction.equiv,
    Fraction.mul, Fraction.ofInt]
  simp
| succ m =>
  have h₀ := actual_error_bound w h s (m + 1)
  have h₁ := budget_two_bound w h s (m + 1) hh hs
  have h₆ := localFactor_le_three_amplitude w h m hh hs
  have h₂ := Fraction.mul_le_mul_nonnegative h₆ (stateNorm s)
    (stateNorm_nonnegative s)
  have h₃ := Fraction.mul_le_mul_nonnegative_left h₂ (count (m + 1)) (by
    unfold count Fraction.ofInt
    exact Int.ofNat_nonneg _)
  have h₄ := Fraction.mul_le_mul_nonnegative_left h₃ (Fraction.ofInt 2) (by decide)
  have hchain := Fraction.magnitudes.le_trans
    (Fraction.magnitudes.le_trans h₀ h₁) h₄
  apply Fraction.le_equiv_right hchain
  simp only [meshAmplitude, count, totalTime, Fraction.equiv,
    Fraction.mul, Fraction.abs, Fraction.ofInt,
    Int.natAbs_of_nonneg hh]
  ac_nf

theorem sample_small_time : SmallTime one eighth 1 := by
  unfold SmallTime Fraction.le
  decide

theorem sample_total_time : Fraction.equiv (totalTime eighth 1) (1, 4, by decide) := by
  decide

theorem sample_power_bounds :
  Fraction.le (fpower (coarseFactor one eighth) 1) (Fraction.ofInt 2) ^
  Fraction.le (fpower (fineFactor one eighth) 1) (Fraction.ofInt 2) :=
  (coarse_power_le_two one eighth 1 (by decide) sample_small_time,
    fine_power_le_two one eighth 1 (by decide) sample_small_time)

theorem sample_actual_error :
  Fraction.equiv
  (stateNorm (stateSub (fineAt one eighth sample 1) (coarseAt one eighth sample 1)))
  (145, 4096, by decide) := by decide

theorem sample_uniform_rhs :
  Fraction.equiv
  (Fraction.mul (Fraction.ofInt 3)
    (Fraction.mul (totalTime eighth 1)
      (Fraction.mul eighth (Fraction.mul one.abs (stateNorm sample)))))
  (3, 16, by decide) := by decide

theorem sample_uniform_error :
  Fraction.le
  (stateNorm (stateSub (fineAt one eighth sample 1) (coarseAt one eighth sample 1)))
  (Fraction.mul (Fraction.ofInt 3)
    (Fraction.mul (totalTime eighth 1)
      (Fraction.mul eighth (Fraction.mul one.abs (stateNorm sample))))) :=
  actual_uniform_error one eighth sample 1 (by decide) sample_small_time

theorem sample_zero_blocks :
  Fraction.equiv
  (stateNorm (stateSub (fineAt one eighth sample 0) (coarseAt one eighth sample 0)))
  zero := by decide

theorem sample_zero_duration :
  Fraction.equiv
  (stateNorm (stateSub (fineAt one zero sample 1) (coarseAt one zero sample 1)))

```

```

    zero := by decide

/-- Without the total-time hypothesis, the claimed factor-two bound fails:
`w=h=n=1` gives `fineFactor^1=16`. -/
theorem false_unrestricted_power :
  ¬ Fraction.le (fpower (fineFactor one one) 1) (Fraction.ofInt 2) := by
  unfold Fraction.le
  decide

NewtonLimitDynamics/Polygon/PartialCell.lean

/-- The partial position is the position component of the end-kick step. -/
theorem partialState_position (D : Nat) (hD : 0 < D) (p v a : Point)
  (weights : List Nat) (u : Nat) :
  (partialState D hD p v a weights u).1 = actualPartialPosition D hD p v a weights u := rfl

/-- The terminal kick does not alter the drift position: the position component
of an end-kick step is independent of the accelerative force applied at its end. -/
theorem endKick_position_kick_free (d : Fraction) (state : Point × Point) (a b : Point) :
  (endKick d state a).1 = (endKick d state b).1 := rfl

/-- The partial state is literally the recurrence on the appended schedule. -/
theorem partialState_append (D : Nat) (hD : 0 < D) (p v a : Point)
  (weights : List Nat) (u : Nat) :
  partialState D hD p v a weights u = partitionMotion D hD p v a (weights ++ [u]) := by
  unfold partialState partitionMotion
  rw [List.foldl_append]
  rfl

/-- Appending one weight applies the statistics recurrence once. -/
theorem stats_append (weights : List Nat) (u : Nat) :
  stats (weights ++ [u]) = next (stats weights) u := by
  unfold stats
  rw [List.foldl_append]
  rfl

theorem total_append (weights : List Nat) (u : Nat) :
  total (weights ++ [u]) = total weights + u := by
  unfold total
  rw [stats_append]
  rfl

theorem squares_append (weights : List Nat) (u : Nat) :
  squares (weights ++ [u]) = squares weights + u * u := by
  unfold squares
  rw [stats_append]
  rfl

/-- Exact residual at the sample time `(T+u)/D` inside the next cell: the
constructed candidate exceeds the actual partial position by
`((Q+u*u)/(2D^2))*a`. Derived from the appended schedule's statistics. -/
theorem candidate_partial_residual (D : Nat) (hD : 0 < D) (p v a : Point)
  (weights : List Nat) (u : Nat) :
  pointEquiv (candidate p v a (duration D (total weights + u) hD))
    (pointAdd (actualPartialPosition D hD p v a weights u)
      (pointScale (residualCoefficient D (squares weights + u * u) hD) a)) := by
  have h := candidate_partitionMotion_residual D hD p v a (weights ++ [u])
  rw [total_append, squares_append, ← partialState_append, partialState_position] at h
  exact h

/-- Within-cell mesh bound on the square statistic: a partial duration inside a
designated next cell of weight `w` (`0 ≤ u` is automatic in `Nat`). -/
theorem partial_squares_bound (M w u : Nat) (weights : List Nat)
  (hM : ∀ x ∈ weights, x ≤ M) (hw : w ≤ M) (hu : u ≤ w) :
  squares weights + u * u ≤ M * (total weights + u) := by
  have hall : ∀ x ∈ weights ++ [u], x ≤ M := by
    intro x hx
    rcases List.mem_append.mp hx with h | h
    · exact hM x h
    · rw [List.mem_singleton.mp h]
      exact Nat.le_trans hu hw

```

```

have hb := stats_bound M (weights ++ [u]) hall
rw [squares_append, total_append] at hb
exact hb

/-- The corresponding Fraction coefficient bound:
` (Q+u*u)/(2D^2) ≤ M*(T+u)/(2D^2)` . -/
theorem partial_residual_mesh_bound (D M w u : Nat) (hD : 0 < D) (weights : List Nat)
(hM : ∀ x ∈ weights, x ≤ M) (hw : w ≤ M) (hu : u ≤ w) :
  Fraction.le (residualCoefficient D (squares weights + u * u) hD)
  (meshCoefficient D M (total weights + u) hD) := by
  have hq := partial_squares_bound M w u weights hM hw hu
  have hi : ((squares weights + u * u : Nat) : Int) ≤ ((M * (total weights + u) : Nat) : Int) :=
    Int.ofNat_le.mpr hq
  unfold Fraction.le residualCoefficient meshCoefficient Fraction.half squareDuration
  dsimp
  have hp : 0 < (2 : Int) * ((D : Int) * (D : Int)) :=
    Int.mul_pos (by decide) (Int.mul_pos (by omega) (by omega))
  exact Int.mul_le_mul_of_nonneg_right hi (Int.le_of_lt hp)

/-- Boundary `u = w`: the partial position is the actual next vertex. -/
theorem partial_full_cell (D : Nat) (hD : 0 < D) (p v a : Point)
(weights : List Nat) (w : Nat) :
  actualPartialPosition D hD p v a weights w =
  (partitionMotion D hD p v a (weights ++ [w])).1 := by
  rw [← partialState_append]
  rfl

private theorem zero_drift_scalar (D : Nat) (hD : 0 < D) (x y : Fraction) :
  Fraction.equiv (Fraction.add x (Fraction.mul (duration D 0 hD) y)) x := by
  unfold Fraction.equiv Fraction.add Fraction.mul duration
  dsimp
  simp only [Int.ofNat_zero, Int.zero_mul, Int.add_zero]
  ac_rfl

/-- Boundary `u = 0`: the partial position is the actual prefix vertex. -/
theorem partial_zero (D : Nat) (hD : 0 < D) (p v a : Point) (weights : List Nat) :
  pointEquiv (actualPartialPosition D hD p v a weights 0)
  (partitionMotion D hD p v a weights).1 :=
  (zero_drift_scalar D hD _, zero_drift_scalar D hD _)

/-- Boundary `a = 0`: the actual partial position lies on the inertial map at
`(T+u)/D`. -/
theorem partial_zero_force (D : Nat) (hD : 0 < D) (p v : Point)
(weights : List Nat) (u : Nat) :
  pointEquiv (actualPartialPosition D hD p v zeroPoint weights u)
  (inertialAt p v (duration D (total weights + u) hD)) := by
  have h := (partitionMotion_zero_force D hD p v (weights ++ [u])).1
  rw [total_append, ← partialState_append, partialState_position] at h
  exact h

```

NewtonLimitDynamics/Polygon/PartitionComparison.lean

```

private theorem half_equiv {a b : Fraction} (h : Fraction.equiv a b) :
  Fraction.equiv (Fraction.half a) (Fraction.half b) := by
  unfold Fraction.equiv Fraction.half at *
  dsimp
  calc a.num * (2 * b.den) = 2 * (a.num * b.den) := by ac_rfl
  _ = 2 * (b.num * a.den) := by rw [h]
  _ = b.num * (2 * a.den) := by ac_rfl

private theorem candidate_scalar_congr {s t : Fraction} (h : Fraction.equiv s t)
(p v a : Fraction) :
  Fraction.equiv
  (Fraction.add (Fraction.add p (Fraction.mul s v)) (Fraction.mul (Fraction.half (Fraction.mul s s))
    ↪ a))
  (Fraction.add (Fraction.add p (Fraction.mul t v)) (Fraction.mul (Fraction.half (Fraction.mul t t))
    ↪ a)) :=
  Fraction.equiv_trans
  (Fraction.add_equiv_right _ (Fraction.add_equiv_left p (Fraction.mul_equiv_right v h)))
  (Fraction.add_equiv_left _ (Fraction.mul_equiv_right a (half_equiv (Fraction.mul_equiv h h))))

```

```

/-- The constructed candidate depends only on the represented rational time. -/
theorem candidate_time_congr (p v a : Point) {s t : Fraction} (h : Fraction.equiv s t) :
  pointEquiv (candidate p v a s) (candidate p v a t) :=
  (candidate_scalar_congr h p.1 v.1 a.1, candidate_scalar_congr h p.2 v.2 a.2)

/-- Two arbitrary finite schedules, with possibly different common denominators
and cells, reaching the same rational time: their actual positions, each
corrected by its own exact residual  $(Q/(2D^2))*a$ , agree. The candidate
serves only as the common algebraic comparison term. -/
theorem partition_comparison (D E : Nat) (hD : 0 < D) (hE : 0 < E) (p v a : Point)
  (ws ws' : List Nat)
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total ws') hE)) :
  pointEquiv
    (pointAdd (partitionMotion D hD p v a ws).1
      (pointScale (residualCoefficient D (squares ws) hD) a))
    (pointAdd (partitionMotion E hE p v a ws').1
      (pointScale (residualCoefficient E (squares ws') hE) a)) :=
  pointEquiv_trans (pointEquiv_symm (candidate_partitionMotion_residual D hD p v a ws))
    (pointEquiv_trans (candidate_time_congr p v a ht)
      (candidate_partitionMotion_residual E hE p v a ws'))

private theorem velocity_scalar_congr {s t : Fraction} (h : Fraction.equiv s t) (v a : Fraction) :
  Fraction.equiv (Fraction.add v (Fraction.mul s a)) (Fraction.add v (Fraction.mul t a)) :=
  Fraction.add_equiv_left v (Fraction.mul_equiv_right a h)

/-- Two arbitrary schedules reaching equivalent rational times have equivalent
actual velocities; no correction term is needed. -/
theorem velocity_comparison (D E : Nat) (hD : 0 < D) (hE : 0 < E) (p v a : Point)
  (ws ws' : List Nat)
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total ws') hE)) :
  pointEquiv (partitionMotion D hD p v a ws).2 (partitionMotion E hE p v a ws').2 :=
  pointEquiv_trans (partitionMotion_formula D hD p v a ws).2
    (pointEquiv_trans
      (show pointEquiv (encodedVelocity D hD (stats ws) v a)
        (encodedVelocity E hE (stats ws') v a) from
        (velocity_scalar_congr ht v.1 a.1, velocity_scalar_congr ht v.2 a.2))
      (pointEquiv_symm (partitionMotion_formula E hE p v a ws').2))

/-- Sample times inside cells: two schedules, each followed by a partial final
drift  $(u/D)$  and  $(u'/E)$ , reaching equivalent rational times. Their actual
partial positions agree after each is corrected by its own residual
 $((Q+u*u)/(2D^2))*a$ . Derived through the appended schedules. -/
theorem partial_comparison (D E : Nat) (hD : 0 < D) (hE : 0 < E) (p v a : Point)
  (ws ws' : List Nat) (u u' : Nat)
  (ht : Fraction.equiv (duration D (total ws + u) hD) (duration E (total ws' + u') hE)) :
  pointEquiv
    (pointAdd (PartialCell.actualPartialPosition D hD p v a ws u)
      (pointScale (residualCoefficient D (squares ws + u * u) hD) a))
    (pointAdd (PartialCell.actualPartialPosition E hE p v a ws' u')
      (pointScale (residualCoefficient E (squares ws' + u' * u') hE) a)) := by
  have h := partition_comparison D E hD hE p v a (ws ++ [u]) (ws' ++ [u'])
  (by rw [PartialCell.total_append, PartialCell.total_append]; exact ht)
  rw [PartialCell.squares_append, PartialCell.squares_append,
    ← PartialCell.partialState_append, ← PartialCell.partialState_append,
    PartialCell.partialState_position, PartialCell.partialState_position] at h
  exact h

/-- Packaged gap statement: two schedules reaching one rational time differ
only along  $a$ , through two nonnegative coefficients each bounded by its
own largest-cell coefficient  $M*T/(2D^2)$  (largest cell times elapsed time,
halved). -/
theorem partition_gap (D E M M' : Nat) (hD : 0 < D) (hE : 0 < E) (p v a : Point)
  (ws ws' : List Nat) (hw : ∀ w ∈ ws, w ≤ M) (hw' : ∀ w ∈ ws', w ≤ M')
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total ws') hE)) :
  ∃ c c' : Fraction,
    Fraction.le (Fraction.ofInt 0) c ∧ Fraction.le c (meshCoefficient D M (total ws) hD) ∧
    Fraction.le (Fraction.ofInt 0) c' ∧ Fraction.le c' (meshCoefficient E M' (total ws') hE) ∧
    pointEquiv (pointAdd (partitionMotion D hD p v a ws).1 (pointScale c a))
      (pointAdd (partitionMotion E hE p v a ws').1 (pointScale c' a)) :=

```

```
(_, _, residual_nonnegative D _ hD, residual_mesh_bound D M hD ws hw,
  residual_nonnegative E _ hE, residual_mesh_bound E M' hE ws' hw',
  partition_comparison D E hD hE p v a ws ws' ht)
```

NewtonLimitDynamics/Polygon/PartitionControl.lean

```
theorem next_identity (s : PartitionStats)
  (h : 2 * s.A + s.Q = s.T * s.T) (w : Nat) :
  2 * (next s w).A + (next s w).Q = (next s w).T * (next s w).T := by
  simp only [next]
  simp only [Nat.mul_add, Nat.add_mul]
  have htw : s.T * w = w * s.T := Nat.mul_comm _ _
  omega

theorem stats_identity_from (s : PartitionStats)
  (h : 2 * s.A + s.Q = s.T * s.T) : (weights : List Nat) ->
  2 * (weights.foldl next s).A + (weights.foldl next s).Q =
  (weights.foldl next s).T * (weights.foldl next s).T
| [] => h
| w :: ws => by
  simp only [List.foldl]
  exact stats_identity_from (next s w) (next_identity s h w) ws

/-- The ordered-pair and square decomposition for every finite schedule. -/
theorem stats_identity (weights : List Nat) :
  2 * cross weights + squares weights = total weights * total weights := by
  exact stats_identity_from initial (by decide) weights

theorem next_bound (M : Nat) (s : PartitionStats) (h : s.Q ≤ M * s.T)
  (w : Nat) (hw : w ≤ M) : (next s w).Q ≤ M * (next s w).T := by
  have hww : w * w ≤ M * w := Nat.mul_le_mul_right w hw
  simp only [next, Nat.mul_add]
  omega

theorem stats_bound_from (M : Nat) (s : PartitionStats) (h : s.Q ≤ M * s.T)
  (weights : List Nat) (hw : ∀ w ∈ weights, w ≤ M) :
  (weights.foldl next s).Q ≤ M * (weights.foldl next s).T := by
  induction weights generalizing s with
  | nil => exact h
  | cons w ws ih =>
    simp only [List.mem_cons] at hw
    simp only [List.foldl]
    exact ih (next s w) (next_bound M s h w (hw w (Or.inl rfl)))
    (fun x hx => hw x (Or.inr hx))

/-- A max-cell estimate for the finite square coefficient. -/
theorem stats_bound (M : Nat) (weights : List Nat) (hw : ∀ w ∈ weights, w ≤ M) :
  squares weights ≤ M * total weights := by
  exact stats_bound_from M initial (by change 0 ≤ M * 0; omega) weights hw

private theorem natDen_pos (D : Nat) (hD : 0 < D) : (0 : Int) < D := by omega

theorem duration_positive (D w : Nat) (hD : 0 < D) (hw : 0 < w) :
  Fraction.positive (duration D w hD) := by
  change 0 < (w : Int)
  omega

private theorem scalar_step_position (D A T w : Nat) (hD : 0 < D) (p v a : Fraction) :
  Fraction.equiv
  (Fraction.add
    (Fraction.add (Fraction.add p (Fraction.mul (duration D T hD) v))
      (Fraction.mul (squareDuration D A hD) a))
    (Fraction.mul (duration D w hD)
      (Fraction.add v (Fraction.mul (duration D T hD) a))))
  (Fraction.add
    (Fraction.add p (Fraction.mul (duration D (T + w) hD) v))
    (Fraction.mul (squareDuration D (A + w * T) hD) a)) := by
  unfold Fraction.equiv Fraction.add Fraction.mul duration squareDuration
  dsimp
  simp only [Int.ofNat_add, Int.ofNat_mul, Int.add_mul, Int.mul_add]
  ac_rfl
```

```

private theorem scalar_step_velocity (D T w : Nat) (hD : 0 < D) (v a : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add v (Fraction.mul (duration D T hD) a))
      (Fraction.mul (duration D w hD) a))
    (Fraction.add v (Fraction.mul (duration D (T + w) hD) a)) := by
  unfold Fraction.equiv Fraction.add Fraction.mul duration
  dsimp
  simp only [Int.ofNat_add, Int.add_mul]
  ac_rfl

private theorem endKick_congr (d : Fraction) {x y : Point × Point} (a : Point)
  (h : pointEquiv x.1 y.1 ∧ pointEquiv x.2 y.2) :
  pointEquiv (endKick d x a).1 (endKick d y a).1 ∧
  pointEquiv (endKick d x a).2 (endKick d y a).2 := by
  constructor
  · exact pointAdd_congr h.1 (pointScale_congr d h.2)
  · exact pointAdd_congr h.2 (Fraction.equiv_refl _, Fraction.equiv_refl _)

private theorem encoded_step (D : Nat) (hD : 0 < D) (s : PartitionStats) (w : Nat)
  (p v a : Point) :
  pointEquiv (endKick (duration D w hD) (encodedState D hD s p v a) a).1
    (encodedState D hD (next s w) p v a).1 ∧
  pointEquiv (endKick (duration D w hD) (encodedState D hD s p v a) a).2
    (encodedState D hD (next s w) p v a).2 := by
  constructor <=> constructor
  · exact scalar_step_position D s.A s.T w hD p.1 v.1 a.1
  · exact scalar_step_position D s.A s.T w hD p.2 v.2 a.2
  · exact scalar_step_velocity D s.T w hD v.1 a.1
  · exact scalar_step_velocity D s.T w hD v.2 a.2

private theorem fold_encoded (D : Nat) (hD : 0 < D) (s : PartitionStats)
  (state : Point × Point) (p v a : Point)
  (hstate : pointEquiv state.1 (encodedState D hD s p v a).1 ∧
    pointEquiv state.2 (encodedState D hD s p v a).2) : (weights : List Nat) ->
  pointEquiv (weights.foldl (fun x w => endKick (duration D w hD) x a) state).1
    (encodedState D hD (weights.foldl next s) p v a).1 ∧
  pointEquiv (weights.foldl (fun x w => endKick (duration D w hD) x a) state).2
    (encodedState D hD (weights.foldl next s) p v a).2
  | [] => hstate
  | w :: ws => by
    simp only [List.foldl]
    have hkick := endKick_congr (duration D w hD) a hstate
    have hencoded := encoded_step D hD s w p v a
    exact fold_encoded D hD (next s w) _ p v a
      (pointEquiv_trans hkick.1 hencoded.1, pointEquiv_trans hkick.2 hencoded.2) ws

private theorem initial_encoded (D : Nat) (hD : 0 < D) (p v a : Point) :
  pointEquiv p (encodedState D hD initial p v a).1 ∧
  pointEquiv v (encodedState D hD initial p v a).2 := by
  constructor <=> constructor <=>
  unfold encodedState encodedPosition encodedVelocity initial pointAdd pointScale duration
  <- squareDuration
  Fraction.equiv Fraction.add Fraction.mul <=> dsimp <=> simp <=> ac_rfl

/-- The finite end-kick schedule has velocity `v + (T/D)a` and position
  `p + (T/D)v + (A/D^2)a`, componentwise up to rational representation. -/
theorem partitionMotion_formula (D : Nat) (hD : 0 < D) (p v a : Point) (weights : List Nat) :
  pointEquiv (partitionMotion D hD p v a weights).1
    (encodedPosition D hD (stats weights) p v a) ∧
  pointEquiv (partitionMotion D hD p v a weights).2
    (encodedVelocity D hD (stats weights) v a) := by
  exact fold_encoded D hD initial (p, v) p v a (initial_encoded D hD p v a) weights

private theorem candidate_residual_scalar (D A Q T : Nat) (hD : 0 < D)
  (h : 2 * A + Q = T * T) (p v a : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add p (Fraction.mul (duration D T hD) v))
      (Fraction.mul (Fraction.half (Fraction.mul (duration D T hD) (duration D T hD))) a))
    (Fraction.add

```

```

      (Fraction.add (Fraction.add p (Fraction.mul (duration D T hD) v))
        (Fraction.mul (squareDuration D A hD) a))
      (Fraction.mul (residualCoefficient D Q hD) a)) := by
have hi : (2 : Int) * (A : Int) + (Q : Int) = (T : Int) * (T : Int) := by omega
unfold Fraction.equiv Fraction.add Fraction.mul duration squareDuration residualCoefficient
simp only [Fraction.half, squareDuration]
simp only [Int.ofNat_mul, Int.add_mul, Int.mul_add]
rw [← hi]
simp only [Int.add_mul, Int.mul_add]
ac rfl

/-- At the finite terminal time, the candidate differs from the constructed
    polygon position by exactly `Q/(2D²)` times the common acceleration. -/
theorem candidate_residual (D : Nat) (hD : 0 < D) (p v a : Point) (weights : List Nat) :
  pointEquiv (candidate p v a (duration D (total weights) hD))
    (pointAdd (encodedPosition D hD (stats weights) p v a)
      (pointScale (residualCoefficient D (squares weights) hD) a)) := by
  constructor
  · exact candidate_residual_scalar D (cross weights) (squares weights) (total weights) hD
    (stats_identity weights) p.1 v.1 a.1
  · exact candidate_residual_scalar D (cross weights) (squares weights) (total weights) hD
    (stats_identity weights) p.2 v.2 a.2

/-- The terminal residual stated against the actual recursively constructed
    polygon endpoint. -/
theorem candidate_partitionMotion_residual (D : Nat) (hD : 0 < D)
  (p v a : Point) (weights : List Nat) :
  pointEquiv (candidate p v a (duration D (total weights) hD))
    (pointAdd (partitionMotion D hD p v a weights).1
      (pointScale (residualCoefficient D (squares weights) hD) a)) := by
  have hcandidate := candidate_residual D hD p v a weights
  have hmotion := partitionMotion_formula D hD p v a weights
  exact pointEquiv_trans hcandidate
    (pointAdd_congr (pointEquiv_symm hmotion.1)
      (Fraction.equiv_refl _, Fraction.equiv_refl _))

theorem residual_nonnegative (D Q : Nat) (hD : 0 < D) :
  Fraction.le (Fraction.ofInt 0) (residualCoefficient D Q hD) := by
  unfold Fraction.le Fraction.ofInt residualCoefficient Fraction.half squareDuration
  dsimp
  omega

theorem residual_mesh_bound (D M : Nat) (hD : 0 < D) (weights : List Nat)
  (hw : ∀ w ∈ weights, w ≤ M) :
  Fraction.le (residualCoefficient D (squares weights) hD)
    (meshCoefficient D M (total weights) hD) := by
  have hq := stats_bound M weights hw
  have hi : (squares weights : Int) ≤ (M * total weights : Int) := by omega
  unfold Fraction.le residualCoefficient meshCoefficient Fraction.half squareDuration
  dsimp
  have hp : 0 < (2 : Int) * ((D : Int) * (D : Int)) :=
    Int.mul_pos (by decide) (Int.mul_pos (natDen_pos D hD) (natDen_pos D hD))
  apply Int.mul_le_mul_of_nonneg_right hi (Int.le_of_lt hp)

```

NewtonLimitDynamics/Polygon/PathDefect.lean — Area between paths, separated from Kepler area. The finite construction below compares coarse edges $A_i A_{i+1}$ with fine pairs $A_i B_i A_{i+1}$. Its signed closed-boundary area is the difference of the two signed Kepler sums; its nonnegative patch budget is the sum of absolute triangle areas and retains lobes of opposite orientation. These are finite paired polygons, not an actual trajectory. Common spatial endpoints of each patch are built into the data; matching mechanical data and time intervals are further obligations. For an actual trajectory γ and polygon P_{mesh} over one common time ...

```

/-- Radial closures cancel in the signed difference. This identity says nothing
    about the unsigned area of lobes between the two paths. -/
theorem signed_gap_eq_Kepler_difference (coarse inserted : Nat → LatticePoint) (n : Nat) :
  signedGap coarse inserted n = fineKeplerTwice coarse inserted n -

```

```

    coarseKeplerTwice coarse n := by
induction n with
| zero => rfl
| succ n ih =>
  simp only [signedGap, fineKeplerTwice, coarseKeplerTwice, isum,
    refinementStripTwice] at *
  omega

/-- The absolute patch budget dominates the absolute signed difference. The
reverse inequality need not hold because opposite lobes cancel. -/
theorem signed_gap_abs_le_budget (coarse inserted : Nat → LatticePoint) (n : Nat) :
  (signedGap coarse inserted n).natAbs ≤ absolutePatchBudget coarse inserted n := by
induction n with
| zero => exact Nat.le_refl _
| succ n ih =>
  change (signedGap coarse inserted n +
    refinementStripTwice (coarse n) (inserted n) (coarse (n + 1))).natAbs ≤
    absolutePatchBudget coarse inserted n +
    (refinementStripTwice (coarse n) (inserted n) (coarse (n + 1))).natAbs
  exact Nat.le_trans (Int.natAbs_add_le _ _) (Nat.add_le_add_right ih _)

/-- A uniform local triangle bound controls the entire finite absolute budget. -/
theorem absolute_budget_le_count_mul (coarse inserted : Nat → LatticePoint) (n b : Nat)
  (h : ∀ i, i < n → refinementDefect (coarse i) (inserted i) (coarse (i + 1)) ≤ b) :
  absolutePatchBudget coarse inserted n ≤ n * b := by
induction n with
| zero => simp [absolutePatchBudget, nsum]
| succ n ih =>
  have hold := ih (fun i hi => h i (by omega))
  have hlast := h n (by omega)
  change absolutePatchBudget coarse inserted n +
    refinementDefect (coarse n) (inserted n) (coarse (n + 1)) ≤ (n + 1) * b
  rw [Nat.add_mul, Nat.one_mul]
  exact Nat.add_le_add hold hlast

/-- Moving the origin does not change the area budget between these paths. -/
theorem absolute_budget_translation (origin : LatticePoint)
  (coarse inserted : Nat → LatticePoint) (n : Nat) :
  absolutePatchBudget (fun i => latticeAdd origin (coarse i))
    (fun i => latticeAdd origin (inserted i)) n = absolutePatchBudget coarse inserted n := by
induction n with
| zero => rfl
| succ n ih =>
  change absolutePatchBudget (fun i => latticeAdd origin (coarse i))
    (fun i => latticeAdd origin (inserted i)) n +
    (refinementStripTwice (latticeAdd origin (coarse n))
      (latticeAdd origin (inserted n)) (latticeAdd origin (coarse (n + 1)))).natAbs =
    absolutePatchBudget coarse inserted n +
    (refinementStripTwice (coarse n) (inserted n) (coarse (n + 1))).natAbs
  rw [ih, refinementStripTwice_translation]

/-- Both paths have the same signed AND unsigned Kepler sums, but a positive
area budget between them. Two adjacent, opposite-side triangle lobes have
doubled unsigned area 1 each; their signed contributions cancel. -/
theorem equal_Kepler_areas_positive_path_defect :
  coarseKeplerTwice exampleCoarse 2 = -6 ∧
  fineKeplerTwice exampleCoarse exampleInserted 2 = -6 ∧
  coarseKeplerUnsigned exampleCoarse 2 = 6 ∧
  fineKeplerUnsigned exampleCoarse exampleInserted 2 = 6 ∧
  signedGap exampleCoarse exampleInserted 2 = 0 ∧
  absolutePatchBudget exampleCoarse exampleInserted 2 = 2 := by
decide

/-- Conditional defect control: a vanishing geometric budget makes the actual
polygon-trajectory area small. No curve existence or geometric enclosure
is inferred from the finite Kepler area law. -/
theorem polygon_trajectory_defect_vanishes (polygonTrajectoryArea budget : Fraction → Fraction)
  (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea := by

```

```

    apply enclosed_gap_vanishes polygonTrajectoryArea budget hbudget
    obtain ⟨d, hd, h⟩ := hgeometry
    exact ⟨d, hd, fun mesh hm hmd => (h mesh hm hmd).2)

NewtonLimitDynamics/Polygon/RefinementStrip.lean

/-- The closed polygon difference is exactly the doubled area of its triangle.
    This is finite determinant algebra for Euclidean triangle decomposition;
    it uses no integration, derivatives, or limiting curve theorem. -/
theorem refinementStripTwice_eq_triangle (a b c : LatticePoint) :
  refinementStripTwice a b c = det (latticeSub b a) (latticeSub c a) := by
  simp [refinementStripTwice, det, latticeSub, Int.sub_mul, Int.mul_sub]
  have hab₁ : a.1*b.2 = b.2*a.1 := Int.mul_comm _ _
  have hab₂ : a.2*b.1 = b.1*a.2 := Int.mul_comm _ _
  have haa : a.1*a.2 = a.2*a.1 := Int.mul_comm _ _
  omega

theorem det_translation (origin a b : LatticePoint) :
  det (latticeAdd origin a) (latticeAdd origin b) =
  det a b + det origin b - det origin a := by
  simp [det, latticeAdd, Int.add_mul, Int.mul_add]
  have hoo : origin.1*origin.2 = origin.2*origin.1 := Int.mul_comm _ _
  have hao : a.1*origin.2 = origin.2*a.1 := Int.mul_comm _ _
  have hao' : a.2*origin.1 = origin.1*a.2 := Int.mul_comm _ _
  omega

/-- Translating both finite polygons leaves their enclosed signed strip area
    unchanged. -/
theorem refinementStripTwice_translation (origin a b c : LatticePoint) :
  refinementStripTwice (latticeAdd origin a) (latticeAdd origin b)
  (latticeAdd origin c) = refinementStripTwice a b c := by
  simp only [refinementStripTwice, det_translation]
  omega

theorem refinementDefect_eq_zero_iff (a b c : LatticePoint) :
  refinementDefect a b c = 0 ↔ refinementStripTwice a b c = 0 := by
  simp [refinementDefect]

theorem oneCellMotionRefinementCompatible_start
  (coarseStart coarseEnd fineStart fineMiddle : LatticePoint)
  (coarseImpulse fineImpulse : Nat → Int)
  (h : oneCellMotionRefinementCompatible coarseStart coarseEnd fineStart fineMiddle
  coarseImpulse fineImpulse) :
  (motion lattice coarseStart coarseEnd coarseImpulse 0).1 =
  (motion lattice fineStart fineMiddle fineImpulse 0).1 :=
  h.1

theorem oneCellMotionRefinementCompatible_end
  (coarseStart coarseEnd fineStart fineMiddle : LatticePoint)
  (coarseImpulse fineImpulse : Nat → Int)
  (h : oneCellMotionRefinementCompatible coarseStart coarseEnd fineStart fineMiddle
  coarseImpulse fineImpulse) :
  (motion lattice coarseStart coarseEnd coarseImpulse 0).2 =
  (motion lattice fineStart fineMiddle fineImpulse 1).2 :=
  h.2

/-- A nonzero local strip from an actual inward fine impulse and a spatially
    compatible coarse cell. This is not nonuniqueness for one fixed force law. -/
theorem inward_oneCell_refinement_compatible :
  oneCellMotionRefinementCompatible (1, 0) (0, 1) (1, 0) (1, 1)
  (fun _ => 0) inwardOneRadialImpulse := by
  constructor
  · rfl
  · decide

theorem inward_oneCell_refinement_defect :
  refinementDefect (1, 0) (1, 1)
  (motion lattice (1, 0) (1, 1) inwardOneRadialImpulse 1).2 = 1 := by
  decide

```

NewtonLimitDynamics/Polygon/StripArea.lean — Order 3 remainder (TASKS.md): the strip-area sum of the constant-force polygon. For a uniform rational cell h , initial velocity v and constant accelerative force a , the end-kick recurrence gives velocity $v_n = v + (n \cdot h) \cdot a$ and chord $c_n = h \cdot v_n$ (the drift of cell n). The two-cell chord triangle (p_n, p_{n+1}, p_{n+2}) has doubled area $\det c_n c_{n+1}$. Because $\det (w + s \cdot a) a = \det w a$, every such triangle has the same doubled area $h^3 \cdot \det(v, a)$, and the signed total over k triangles is $k \cdot h^3 \cdot \det(v, a)$. The common value can be negative: equality of the triangles does not ...

```

theorem pointSub_add_self_left_equiv (x y : Point) :
  pointEquiv (pointSub (pointAdd x y) x) y := by
  constructor <=>
    simp only [pointSub, pointAdd, pointNeg, Fraction.add, Fraction.mul, Fraction.equiv,
      Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
    ac_nf <=> omega

/-- The drift of a cell is exactly the position difference `p_{n+1} - p_n`. -/
theorem chord_is_position_diff (h : Fraction) (p v a : Point) (n : Nat) :
  pointEquiv (pointSub (posAt h p v a (n + 1)) (posAt h p v a n)) (chord h v a n) := by
  simp only [posAt, chord]
  exact pointSub_add_self_left_equiv (posAt h p v a n) (pointScale h (velAt h v a n))

private theorem det_add_left (q x y : Point) :
  Fraction.equiv (det (pointAdd x y) q) (Fraction.add (det x q) (det y q)) := by
  simp only [det, pointAdd, Fraction.add, Fraction.mul, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega

private theorem det_scale_left (d : Fraction) (q x : Point) :
  Fraction.equiv (det (pointScale d x) q) (Fraction.mul d (det x q)) := by
  simp only [det, pointScale, Fraction.add, Fraction.mul, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega

private theorem det_scale_right (d : Fraction) (p x : Point) :
  Fraction.equiv (det p (pointScale d x)) (Fraction.mul d (det p x)) := by
  simp only [det, pointScale, Fraction.add, Fraction.mul, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega

private theorem det_self (x : Point) : Fraction.equiv (det x x) (Fraction.ofInt 0) := by
  simp only [det, Fraction.add, Fraction.mul, Fraction.ofInt, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega

/-- The key lemma from the hand computation: `det (w + s·a) a = det w a`. -/
theorem det_kick_direction_constant (s : Fraction) (w a : Point) :
  Fraction.equiv (det (pointAdd w (pointScale s a)) a) (det w a) := by
  have h1 := det_add_left a w (pointScale s a)
  have h2 := det_scale_left s a a
  have h3 := det_self a
  have t1 : Fraction.equiv (det (pointAdd w (pointScale s a)) a)
    (Fraction.add (det w a) (Fraction.mul s (det a a))) :=
    Fraction.equiv_trans h1 (Fraction.add_equiv_left (det w a) h2)
  have t2 : Fraction.equiv (Fraction.mul s (det a a)) (Fraction.mul s (Fraction.ofInt 0)) :=
    Fraction.mul_equiv_left s h3
  have t3 : Fraction.equiv (Fraction.mul s (Fraction.ofInt 0)) (Fraction.ofInt 0) :=
    Fraction.mul_zero s
  have t4 : Fraction.equiv (Fraction.add (det w a) (Fraction.ofInt 0)) (det w a) := by
    simp only [Fraction.add, Fraction.ofInt, Fraction.equiv,
      Int.add_mul, Int.mul_add] <=> ac_nf <=> omega
  exact Fraction.equiv_trans t1
    (Fraction.equiv_trans (Fraction.add_equiv_left (det w a) (Fraction.equiv_trans t2 t3)) t4)

/-- Each velocity has the same determinant with `a` as the initial velocity. -/
theorem vel_det_constant (h : Fraction) (v a : Point) (n : Nat) :
  Fraction.equiv (det (velAt h v a n) a) (det v a) := by
  induction n with
  | zero => exact Fraction.equiv_refl _
  | succ n ih =>
    exact Fraction.equiv_trans (det_kick_direction_constant h (velAt h v a n) a) ih

```

```

/-- Every two-cell chord triangle has doubled area  $h^3 \cdot \det(v, a)$ . -/
theorem two_cell_triangle_constant (h : Fraction) (v a : Point) (n : Nat) :
  Fraction.equiv (twoCellTwice h v a n)
    (Fraction.mul h (Fraction.mul h (Fraction.mul h (det v a)))) := by
  have vn := velAt h v a n
  have vn1 : velAt h v a (n + 1) = pointAdd (velAt h v a n) (pointScale h a) := by simp [velAt]
  simp only [twoCellTwice, chord, vn1]
  -- det (h·v_n) (h·v_{n+1}) = h·det v_n (h·v_{n+1})
  have s1 := det_scale_left h (pointScale h (pointAdd (velAt h v a n) (pointScale h a))) (velAt h v a n)
  -- = h·(h·det v_n v_{n+1})
  have s2 := Fraction.mul_equiv_left h (det_scale_right h (velAt h v a n) (pointAdd (velAt h v a n)
    (pointScale h a)))
  -- det v_n v_{n+1} = det v_n v_n + h·det v_n a
  have s3 := det_add_right (velAt h v a n) (velAt h v a n) (pointScale h a)
  have s4 := Fraction.add_equiv (det_self (velAt h v a n)) (det_scale_right h (velAt h v a n) a)
  have s5 := Fraction.mul_equiv (Fraction.equiv_refl h) (vel_det_constant h v a n)
  have inner : Fraction.equiv (det (velAt h v a n) (pointAdd (velAt h v a n) (pointScale h a)))
    (Fraction.mul h (det v a)) :=
    Fraction.equiv_trans (Fraction.equiv_trans s3 s4)
    (Fraction.equiv_trans
      (Fraction.add_equiv_left (Fraction.ofInt 0) s5)
      (by simp only [Fraction.add, Fraction.ofInt, Fraction.mul, Fraction.equiv,
        Int.add_mul, Int.mul_add] <=> ac_nf <=> omega))
  have rhs := Fraction.mul_equiv_left h (Fraction.mul_equiv_left h inner)
  exact Fraction.equiv_trans (Fraction.equiv_trans s1 s2) rhs

/-- All two-cell triangles have the same signed doubled area. No absolute-area
operation or unsigned sum is asserted. -/
theorem all_triangles_equal (h : Fraction) (v a : Point) (m n : Nat) :
  Fraction.equiv (twoCellTwice h v a m) (twoCellTwice h v a n) :=
  Fraction.equiv_trans (two_cell_triangle_constant h v a m)
    (Fraction.equiv_symm (two_cell_triangle_constant h v a n))

/-- The signed strip sum over  $k$  triangles is  $k \cdot h^3 \cdot \det(v, a)$ . -/
theorem total_strip_area (h : Fraction) (v a : Point) (k : Nat) :
  Fraction.equiv (stripSum h v a k)
    (Fraction.mul (Fraction.ofInt k) (Fraction.mul h (Fraction.mul h (Fraction.mul h (det v a))))) := by
  induction k with
  | zero =>
    simp only [stripSum, Fraction.mul, Fraction.ofInt, Fraction.equiv] <=> omega
  | succ k ih =>
    let H := Fraction.mul h (Fraction.mul h (Fraction.mul h (det v a)))
    simp only [stripSum]
    exact Fraction.equiv_trans
      (Fraction.add_equiv ih (two_cell_triangle_constant h v a k))
      (by simp only [Fraction.mul, Fraction.add, Fraction.ofInt, Fraction.equiv,
        Int.add_mul, Int.mul_add, Int.ofNat_add, Int.mul_one, Int.one_mul] <=>
        ac_nf <=> omega)

```

NewtonLimitDynamics/Polygon/TimeSubdivision.lean

```

theorem totalDuration_positive (h k : Fraction)
  (hh : positiveDuration h) (hk : positiveDuration k) :
  positiveDuration (totalDuration h k) := by
  unfold positiveDuration totalDuration Fraction.positive Fraction.add
  dsimp
  exact Int.add_pos (Int.mul_pos hh k.den_pos) (Int.mul_pos hk h.den_pos)

private theorem fine_position_scalar (h k p v a : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add p (Fraction.mul h v))
      (Fraction.mul k (Fraction.add v (Fraction.mul h a))))
    (Fraction.add (Fraction.add p (Fraction.mul (Fraction.add h k) v))
      (Fraction.mul (Fraction.mul h k) a)) := by
  have hdist : Fraction.equiv (Fraction.mul k (Fraction.add v (Fraction.mul h a)))
    (Fraction.add (Fraction.mul k v) (Fraction.mul k (Fraction.mul h a))) :=
    Fraction.mul_add k v (Fraction.mul h a)
  have hkv : Fraction.equiv (Fraction.add (Fraction.mul h v) (Fraction.mul k v))
    (Fraction.mul (Fraction.add h k) v) := by
    exact Fraction.equiv_trans (Fraction.add_equiv (Fraction.mul_comm h v) (Fraction.mul_comm k v))

```

```

    (Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_add v h k))
      (Fraction.equiv_symm (Fraction.mul_comm (Fraction.add h k) v)))
  have hka : Fraction.equiv (Fraction.mul k (Fraction.mul h a))
    (Fraction.mul (Fraction.mul h k) a) := by
    exact Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_assoc k h a))
      (Fraction.mul_equiv_right a (Fraction.mul_comm k h))
  exact Fraction.equiv_trans
    (Fraction.equiv_trans (Fraction.add_assoc p (Fraction.mul h v)
      (Fraction.mul k (Fraction.add v (Fraction.mul h a))))
      (Fraction.add_equiv_left p (Fraction.add_equiv (Fraction.equiv_refl (Fraction.mul h v)) hdist)))
    (Fraction.equiv_trans
      (Fraction.add_equiv_left p (Fraction.equiv_symm (Fraction.add_assoc (Fraction.mul h v)
        (Fraction.mul k v) (Fraction.mul k (Fraction.mul h a)))))
      (Fraction.equiv_trans (Fraction.add_equiv_left p (Fraction.add_equiv hkv hka))
        (Fraction.equiv_symm (Fraction.add_assoc p (Fraction.mul (Fraction.add h k) v)
          (Fraction.mul (Fraction.mul h k) a)))))

/-- For the stated end-kick scheduling convention, time subdivision changes
the final position by `h*k*a`. This is finite Fraction arithmetic, not a
trajectory-existence theorem or a Newton central-force theorem. -/
theorem fine_position_eq_coarse_plus (h k : Fraction) (p v a : Point) :
  pointEquiv (fine h k p v a).1
    (pointAdd (coarse h k p v a).1 (pointScale (Fraction.mul h k) a)) := by
  constructor <=> apply fine_position_scalar

private theorem fine_velocity_scalar (h k v a : Fraction) :
  Fraction.equiv (Fraction.add (Fraction.add v (Fraction.mul h a)) (Fraction.mul k a))
    (Fraction.add v (Fraction.mul (Fraction.add h k) a)) := by
  exact Fraction.equiv_trans (Fraction.add_assoc v (Fraction.mul h a) (Fraction.mul k a))
    (Fraction.equiv_trans
      (Fraction.add_equiv_left v (Fraction.add_equiv (Fraction.mul_comm h a) (Fraction.mul_comm k a)))
      (Fraction.equiv_trans
        (Fraction.add_equiv_left v (Fraction.equiv_symm (Fraction.mul_add a h k)))
        (Fraction.add_equiv_left v (Fraction.equiv_symm (Fraction.mul_comm (Fraction.add h k) a)))))

theorem fine_velocity_eq_coarse (h k : Fraction) (p v a : Point) :
  pointEquiv (fine h k p v a).2 (coarse h k p v a).2 := by
  constructor <=>
    dsimp [fine, coarse, endKick, pointAdd, pointScale] <=>
    apply fine_velocity_scalar

theorem example_half_positive : positiveDuration half := by
  unfold positiveDuration Fraction.positive half
  decide

theorem example_total_positive : positiveDuration (totalDuration half half) :=
  totalDuration_positive half half example_half_positive example_half_positive

theorem example_coarse_position : pointEquiv (coarse half half exampleP exampleV exampleA).1 exampleC :=
  ↪ by
  decide

theorem example_fine_middle : pointEquiv (endKick half (exampleP, exampleV) exampleA).1 exampleB := by
  decide

theorem example_fine_position : pointEquiv (fine half half exampleP exampleV exampleA).1 exampleD := by
  decide

theorem example_endpoints_differ : ¬ pointEquiv exampleD exampleC := by
  intro h
  have := h.2
  change (1 : Int) * 1 = 0 * 4 at this
  omega

theorem example_fine_coarse_endpoints_differ :
  ¬ pointEquiv (fine half half exampleP exampleV exampleA).1
    (coarse half half exampleP exampleV exampleA).1 := by
  intro h
  exact example_endpoints_differ
    (Fraction.equiv_trans (Fraction.equiv_symm example_fine_position.1))

```

```

(Fraction.equiv_trans h.1 example_coarse_position.1),
Fraction.equiv_trans (Fraction.equiv_symm example_fine_position.2)
(Fraction.equiv_trans h.2 example_coarse_position.2))

theorem example_connector : pointEquiv (directedConnector exampleD exampleC) (zero, negQuarter) := by
decide

theorem example_closedBoundaryTwice :
  Fraction.equiv (closedBoundaryTwice exampleP exampleB exampleD exampleC) ({-1, 8, by decide}) := by
decide

```

NewtonLimitDynamics/Polygon/UniformRefinement.lean

```

private theorem unitCells_T_from (s : PartitionStats) : (n : Nat) ->
  ((unitCells n).foldl next s).T = s.T + n
| 0 => rfl
| n + 1 => by
  show ((unitCells n).foldl next (next s 1)).T = s.T + (n + 1)
  rw [unitCells_T_from (next s 1) n]
  simp only [next]
  omega

theorem unitCells_total (n : Nat) : total (unitCells n) = n := by
  unfold total stats
  rw [unitCells_T_from]
  exact Nat.zero_add n

theorem unitCells_le_one (n : Nat) : ∀ w ∈ unitCells n, w ≤ 1 := by
  intro w hw
  rw [List.eq_of_mem_replicate hw]
  exact Nat.le_refl 1

/-- Refining the rational time `N/E` by a factor `K` keeps the same time. -/
theorem refined_time (N E K : Nat) (hE : 0 < E) (hEK : 0 < E * K) :
  Fraction.equiv (duration (E * K) (total (unitCells (N * K)))) hEK (duration E N hE) := by
  rw [unitCells_total]
  unfold Fraction.equiv duration
  dsimp
  simp only [Int.ofNat_mul]
  ac_rfl

/-- Nat core of the estimate: `N*K*d ≤ n*2*(E*K)*(E*K)` once `K > N*d`,
  `n ≥ 1` and `E ≥ 1`. -/
private theorem nat_bound (N E n d : Nat) (hE : 0 < E) (hn : 0 < n) :
  N * (N * d + 1) * d ≤ n * (2 * ((E * (N * d + 1)) * (E * (N * d + 1)))) := by
  have hK : N * d ≤ N * d + 1 := Nat.le_succ _
  have h1 : N * (N * d + 1) * d ≤ (N * d + 1) * (N * d + 1) := by
    calc N * (N * d + 1) * d = (N * d) * (N * d + 1) := by ac_rfl
    _ ≤ (N * d + 1) * (N * d + 1) := Nat.mul_le_mul_right _ hK
  have hE1 : 1 ≤ E * E := Nat.mul_le_mul hE hE
  have h2 : (N * d + 1) * (N * d + 1) ≤ (E * E) * ((N * d + 1) * (N * d + 1)) := by
    calc (N * d + 1) * (N * d + 1) = 1 * ((N * d + 1) * (N * d + 1)) := (Nat.one_mul _).symm
    _ ≤ (E * E) * ((N * d + 1) * (N * d + 1)) := Nat.mul_le_mul_right _ hE1
  have h3 : (E * E) * ((N * d + 1) * (N * d + 1)) ≤
    n * (2 * ((E * (N * d + 1)) * (E * (N * d + 1)))) := by
    calc (E * E) * ((N * d + 1) * (N * d + 1))
      = 1 * (1 * ((E * (N * d + 1)) * (E * (N * d + 1)))) := by
        simp only [Nat.one_mul]; ac_rfl
    _ ≤ n * (2 * ((E * (N * d + 1)) * (E * (N * d + 1)))) :=
      Nat.mul_le_mul hn (Nat.mul_le_mul_right _ (by decide))
  exact Nat.le_trans h1 (Nat.le_trans h2 h3)

/-- For every positive rational tolerance and rational time `N/E`, an explicit
  uniform refinement (factor `K = N*den(ε)+1`, unit cells over `E*K`) makes
  the exact constant-force residual coefficient `Q/(2D²)` at most the
  tolerance. Together with `candidate_partitionMotion_residual`, the actual
  polygon position at `N/E` is within that coefficient (along `a`) of the
  constructed candidate. -/
theorem uniform_refinement_small (N E : Nat) (hE : 0 < E) (eps : Fraction)
  (heps : Fraction.positive eps) :
  ∃ K : Nat, ∃ hEK : 0 < E * K,

```

```

    Fraction.equiv (duration (E * K) (total (unitCells (N * K))) hEK) (duration E N hE) ∧
    Fraction.le (residualCoefficient (E * K) (squares (unitCells (N * K))) hEK) eps := by
have hdpes := eps.den_pos
have hnpos : 0 < eps.num := heps
let d := eps.den.toNat
let n := eps.num.toNat
have hd : (d : Int) = eps.den := Int.toNat_of_nonneg (Int.le_of_lt hdpes)
have hn : (n : Int) = eps.num := Int.toNat_of_nonneg (Int.le_of_lt hnpos)
have hn0 : 0 < n := by omega
have hEK : 0 < E * (N * d + 1) := Nat.mul_pos hE (Nat.succ_pos _)
refine {N * d + 1, hEK, refined_time N E _ hE hEK, ?_}
have hq := stats_bound 1 (unitCells (N * (N * d + 1))) (unitCells_le_one _)
rw [Nat.one_mul] at hq
change squares (unitCells (N * (N * d + 1))) ≤ total (unitCells (N * (N * d + 1))) at hq
rw [unitCells_total] at hq
have hnnt := nat_bound N E n d hE hn0
have hchain : squares (unitCells (N * (N * d + 1))) * d ≤
  n * (2 * ((E * (N * d + 1)) * (E * (N * d + 1)))) :=
  Nat.le_trans (Nat.mul_le_mul_right d hq) hnnt
have hint := Int.ofNat_le.mpr hchain
unfold Fraction.le residualCoefficient Fraction.half squareDuration
dsimp
rw [← hd, ← hn]
simp only [Int.ofNat_mul] at hint
have h2 : ((2 : Nat) : Int) = 2 := rfl
rw [h2] at hint
simp only [Int.ofNat_mul, Int.mul_assoc, Int.mul_comm, Int.mul_left_comm] using hint

```

NewtonLimitDynamics/Principia1687/PropositionI.lean — 1687 Proposition I, finite equal-cell reconstruction only. Source: NATP00077 par44–45, <https://www.newtonproject.ox.ac.uk/view/t>
The proof explicitly invokes Law I, the laws’ Corollary 1, then composes the equal triangle areas (componendo). The construction’s two named Euclidean preservation identities and geometric area semantics are supplied premises. Unsigned doubled triangle sums count cells with multiplicity. Inward sense, sector-union interpretation, and the final curve/uninterrupted-force passage through Lemma III Corollary 4 remain separate. No 1713 ...

```

theorem finite_equal_areas (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
  all_unsigned_cell_areas g p q impulse n

/-- Finite content of the same-edition `componendo` sentence; positive total
times are derived, not left implicit in a cross-multiplied identity. -/
theorem finite_componendo (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start₁ start₂ m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
  unsignedBlock g p q impulse start₁ m * (n * dt) =
  unsignedBlock g p q impulse start₂ n * (m * dt) :=
  positive_unsigned_area_comparison g p q impulse dt hdt start₁ start₂ m n hm hn

/-- Conditional control of the intervening polygon-trajectory region. The
curve, its relation to the construction, and the enclosure are supplied
geometric obligations; Lemma III Corollary 4 is not silently discharged. -/
theorem polygon_trajectory_defect_control
  (polygonTrajectoryArea budget : Fraction → Fraction) (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea :=
  polygon_trajectory_defect_vanishes polygonTrajectoryArea budget hbudget hgeometry

```

NewtonLimitDynamics/Principia1713/PropositionI.lean — 1713 Proposition I, finite equal-cell reconstruction only. Source: NATP00082 par50–51, <https://www.newtonproject.ox.ac.uk/view/t>
The proof invokes this edition’s Law I and laws’ Corollary 1 and explicitly composes equal triangle areas. The same mathematical kernel is reusable, but the historical

source and theorem namespace remain 1713-local. The two named Euclidean identities and area semantics are supplied. Sums of unsigned doubled triangle areas count repeated coverage. Inward dynamics, sector identification, Lemma III Corollary 4's curve passage, and ...

```
theorem finite_equal_areas (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
  all_unsigned_cell_areas g p q impulse n

/-- This edition's finite `componendo` consequence, before its limiting clause. -/
theorem finite_componendo (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start1 start2 m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
  unsignedBlock g p q impulse start1 m * (n * dt) =
  unsignedBlock g p q impulse start2 n * (m * dt) :=
  positive_unsigned_area_comparison g p q impulse dt hdt start1 start2 m n hm hn

/-- This edition's conditional polygon-trajectory defect interface. A realized
  trajectory and its geometric enclosure are premises, not conclusions of
  the exact finite Kepler triangle law. -/
theorem polygon_trajectory_defect_control
  (polygonTrajectoryArea budget : Fraction) (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea :=
  polygon_trajectory_defect_vanishes polygonTrajectoryArea budget hbudget hgeometry
```

Proposition II. Theorem II. *Every body that moves in some curved line and, by a radius drawn to a point either immovable or moving uniformly in a straight line, describes about that point areas proportional to the times, is urged by a centripetal force tending to that same point.*

In modern terms: if $\mathbf{r} \times \dot{\mathbf{r}}$ is constant about S , the force is parallel to $\mathbf{r}$.

Case 1. Every body moving in a curved line is turned from a rectilinear course by some force acting on it (Law I). The force by which the body is turned from its rectilinear course and made to describe, about the immovable point S , the least triangles SAB , SBC , SCD , ... equal in equal times, acts at the place B along a line parallel to cC (Euclid I.40 and Law II), that is, along the line BS ; and at the place C along a line parallel to dD , that is, along the line CS ; and so on. It therefore always acts along lines tending to that immovable point S .

Case 2. By Corollary V of the Laws it makes no difference whether the surface on which the body describes the curvilinear figure rests, or moves uniformly in a straight line together with the body, the figure described and its point S .

1713. The statement specifies a curved line *described in a plane*. Two corollaries are added, rewritten from the old corollaries of Proposition I.

Corollary 1. In non-resisting spaces or media, if the areas are not proportional to the times, the forces do not tend to the point where the radii meet, but decline from it *in consequentia*, toward the direction of motion, if the description of areas is accelerated, and *in antecedentia* if it is retarded.

Corollary 2. In resisting media too, if the description of areas is accelerated, the directions of the forces decline from the point where the radii meet toward the direction of motion.

Lean reconstruction: 12 theorems in 1 module cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/Polygon/Converse.lean — Finite converse of the equal-area polygon construction (motivated by 1687 Proposition II, NATP00077 par48–49, and 1713 Proposition II, NATP00082 par58–59). Modern integer-coordinate reconstruction with the centre S at the origin. Given three consecutive vertices p , q , C and the inertial continuation $c = \text{extend } p \ q$, equality of the **oriented** doubled areas Spq and SqC makes the deflection $C - c$ parallel to Sq , and the step is then a central kick with a rational impulse. Orientation (Euclid’s “same side”), a vertex distinct from S , and the inward sense are separate premises; ...

```
theorem det_sub (q x y : LatticePoint) : det q (sub x y) = det q x - det q y := by
  simp only [det, sub, Int.mul_sub]
  omega

/-- The deflection's determinant with the radius `Sq` is the change of oriented
area. -/
theorem det_deflection (p q C : LatticePoint) :
  det q (deflection p q C) = det q C - det p q := by
  unfold deflection
  rw [det_sub, extension_identity]

/-- Coordinate form of the Euclid I.39/I.40 step: equal oriented areas make the
deflection `cC` parallel to `Sq`. -/
theorem equal_area_parallel (p q C : LatticePoint) (h : det q C = det p q) :
  det q (deflection p q C) = 0 := by
  rw [det_deflection, h, Int.sub_self]

/-- And conversely a deflection parallel to `Sq` preserves the oriented area. -/
theorem parallel_equal_area (p q C : LatticePoint) (h : det q (deflection p q C) = 0) :
  det q C = det p q := by
  rw [det_deflection] at h
  omega

/-- A vector parallel to a nonzero radius is a rational multiple of it:
`b*d = a*q` with `b ≠ 0`. -/
theorem parallel_is_multiple (q d : LatticePoint) (hq : q ≠ (0, 0)) (h : det q d = 0) :
  ∃ a b : Int, b ≠ 0 ∧ b * d.1 = a * q.1 ∧ b * d.2 = a * q.2 := by
  unfold det at h
  by_cases h1 : q.1 = 0
  · have h2 : q.2 ≠ 0 := by
      intro h2
      apply hq
      exact Prod.ext h1 h2
    refine (d.2, q.2, h2, _, _)
    · rw [h1, Int.mul_zero]
      rw [h1, Int.zero_mul] at h
      have : q.2 * d.1 = 0 := by omega
      exact this
    · exact Int.mul_comm _ _
  · refine (d.1, q.1, h1, Int.mul_comm _ _, _)
    have : q.1 * d.2 = q.2 * d.1 := by omega
    rw [this, Int.mul_comm]
```

```
/-- Finite converse for one step: equal oriented areas and a vertex distinct
from S give a rational central impulse `a/b` with `b*C = b*c + a*q`. -/
theorem equal_area_central_step (p q C : LatticePoint) (hq : q ≠ (0, 0))
(h : det q C = det p q) :
  ∃ a b : Int, b ≠ 0 ∧
    b * C.1 = b * (extend p q).1 + a * q.1 ∧ b * C.2 = b * (extend p q).2 + a * q.2 := by
  obtain (a, b, hb, h1, h2) :=
    parallel_is_multiple q (deflection p q C) hq (equal_area_parallel p q C h)
  refine (a, b, hb, _, _)
  · simp only [deflection, sub, Int.mul_sub] at h1
    omega
  · simp only [deflection, sub, Int.mul_sub] at h2
```

```

omega

/-- A finite vertex sequence whose consecutive oriented triangles about S are
    all equal (equal areas in equal time cells) has every deflection parallel to
    its current radius. -/
theorem equal_areas_all_central (v : Nat → LatticePoint)
  (h : ∀ n, det (v (n + 1)) (v (n + 2)) = det (v n) (v (n + 1))) (n : Nat) :
  det (v (n + 1)) (deflection (v n) (v (n + 1)) (v (n + 2))) = 0 :=
  equal_area_parallel _ _ (h n)

/-- Orientation is needed: equal unsigned areas (`det q C = -det p q`) admit a
    deflection that is not parallel to `Sq`. -/
theorem unsigned_equal_area_not_central :
  det (1, 1) (2, 1) = -det (1, 0) (1, 1) ∧
  det (1, 1) (deflection (1, 0) (1, 1) (2, 1)) ≠ 0 := by
  decide

/-- A vertex at S is degenerate: both areas vanish for every next vertex, so the
    area data fix no direction. -/
theorem vertex_at_centre_degenerate (p C : LatticePoint) :
  det (0, 0) C = det p (0, 0) := by
  simp [det]

/-- The sense is not fixed by areas: an outward kick (`+1`) keeps the oriented
    area equal, as does the inward kick (`-1`). -/
theorem outward_kick_equal_area :
  det (1, 1) (kick (1, 1) (extend (1, 0) (1, 1)) 1) = det (1, 0) (1, 1) ∧
  det (1, 1) (kick (1, 1) (extend (1, 0) (1, 1)) (-1)) = det (1, 0) (1, 1) := by
  decide

/-- Finite Case-2 fact (motivated by the laws' Corollary V, cited in 1687 par50
    and 1713 par60): inertial continuation commutes with uniform translation of
    the reference centre, so relative vertices obey the same continuation. -/
theorem extend_relative (s0 w p q : LatticePoint) (n : Nat) :
  sub (extend p q) (centreAt s0 w (n + 2)) =
  extend (sub p (centreAt s0 w n)) (sub q (centreAt s0 w (n + 1))) := by
  simp only [sub, extend, centreAt]
  apply Prod.ext <;> dsimp <;> simp only [Int.ofNat_add, Int.add_mul] <;> omega

/-- Case 2, finite step: equal oriented areas about a uniformly moving centre
    make each deflection parallel to the current radius from that centre. -/
theorem moving_centre_equal_areas_central (s0 w : LatticePoint) (v : Nat → LatticePoint)
  (h : ∀ n, det (sub (v (n + 1)) (centreAt s0 w (n + 1))) (sub (v (n + 2)) (centreAt s0 w (n + 2))) =
  det (sub (v n) (centreAt s0 w n)) (sub (v (n + 1)) (centreAt s0 w (n + 1)))) (n : Nat) :
  det (sub (v (n + 1)) (centreAt s0 w (n + 1)))
  (sub (v (n + 2)) (extend (v n) (v (n + 1)))) = 0 := by
  have hpar := equal_area_parallel (sub (v n) (centreAt s0 w n))
  (sub (v (n + 1)) (centreAt s0 w (n + 1))) (sub (v (n + 2)) (centreAt s0 w (n + 2))) (h n)
  unfold deflection at hpar
  rw [← extend_relative] at hpar
  have hd : sub (sub (v (n + 2)) (centreAt s0 w (n + 2))) (sub (extend (v n) (v (n + 1))) (centreAt s0 w
  (n + 2))) =
  sub (v (n + 2)) (extend (v n) (v (n + 1))) := by
  simp only [sub]
  apply Prod.ext <;> dsimp <;> omega
  rw [hd] at hpar
  exact hpar

```

Scholium

A body may be urged by a centripetal force compounded of several forces. In that case the meaning of the Proposition is that the force compounded of them all tends to the point S. Further, if some force acts along a line perpendicular to the surface described, it will make the body deviate from the plane of its motion, but it will neither increase nor diminish the quantity of the surface described, and so it may be neglected in the composition of forces.

No theorem of the Lean reconstruction cites this item.

Proposition III. Theorem III. *Every body that, by a radius drawn to the centre of another body moving in any way, describes about that centre areas proportional to the times, is urged by a force compounded of the centripetal force tending to that other body and of all the accelerative force by which that other body is urged.*

In modern terms: if $\mathbf{r}_{12} \times \dot{\mathbf{r}}_{12}$ is constant for the relative position $\mathbf{r}_{12} = \mathbf{r}_1 - \mathbf{r}_2$, then $\mathbf{a}_1 - \mathbf{a}_2$ is parallel to $\mathbf{r}_{12}$.

Proof. By Corollary VI of the Laws, if both bodies are urged along parallel lines by a new force equal and contrary to the force urging the other body, the first body continues to describe the same areas about the other as before. The force urging the other body is now destroyed by its equal and contrary force, so (Law I) that other body either rests or moves uniformly in a straight line, and the first body, urged by the difference of the forces, continues to describe areas proportional to the times about it. So (Theorem II) the difference of the forces tends to that other body as its centre.

1713. The proof names the bodies L and T ; T , left to itself once its force is cancelled, rests or moves uniformly, and L moves under “the difference of the forces, that is, the remaining force”. The argument is unchanged.

Corollary 1. Hence if one body, by a radius drawn to another, describes areas proportional to the times, and from the total force urging the first (simple, or compounded of several forces by Corollary II of the Laws) the whole accelerative force urging the other is subtracted (by the same Corollary), all the remaining force urging the first tends to the other body as centre.

Corollary 2. And if those areas are very nearly proportional to the times, the remaining force tends very nearly to the other body.

Corollary 3. And conversely, if the remaining force tends very nearly to the other body, those areas are very nearly proportional to the times.

Corollary 4. If a body, by a radius drawn to another body, describes areas that are very unequal compared with the times, and that other body rests or moves uniformly in a straight line, the action of the centripetal force tending to that other body is either null or mixed and compounded with very powerful actions of other forces; and the total force compounded of them all, if there are several, is directed to another centre (immovable or moving) about which the description of areas is uniform. The same holds when the other body moves in any way, provided the centripetal force taken is what remains after subtracting the total force acting on that other body.

Lean reconstruction: 17 theorems in 1 module cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/Polygon/RelativeMotion.lean — Finite relative-motion step for Proposition III (1687 NATP00077 par53–54, 1713 NATP00082 par64–65). Both editions prove the proposition the same way: the laws’ Corollary VI adds an equal and contrary parallel acceleration to both bodies, Law I fixes the resulting motion of the reference body, and Proposition II then identifies the remaining force as centripetal. The reconstruction is the finite integer-coordinate polygon of Polygon.Finite, now for

two bodies in one pair recursion. Each body continues inertially and then takes its own deflection for the cell. Three results mirror the ...

```

theorem add_zero_right (a : LatticePoint) : add a (0, 0) = a := by
  simp [add]

/-- A summand common to both sides cancels in a difference. -/
theorem add_common_cancel (a b c : LatticePoint) : sub (add a c) (add b c) = sub a b := by
  simp only [add, sub]
  apply Prod.ext <=> dsimp <=> omega

theorem add_sub_distrib (a b c d : LatticePoint) :
  sub (add a b) (add c d) = add (sub a c) (sub b d) := by
  simp only [add, sub]
  apply Prod.ext <=> dsimp <=> omega

/-- Inertial continuation commutes with taking the difference of two pairs. -/
theorem extend_sub (a b c d : LatticePoint) :
  sub (extend a b) (extend c d) = extend (sub a c) (sub b d) := by
  simp only [sub, extend]
  apply Prod.ext <=> dsimp <=> omega

theorem det_add (q x y : LatticePoint) : det q (add x y) = det q x + det q y := by
  simp only [det, add, Int.mul_add]
  omega

theorem relP_succ (z : PairState) (d e : Nat → LatticePoint) (n : Nat) :
  relPAT z d e (n + 1) = relQAT z d e n :=
  rfl

/-- The relative polygon's step: inertial continuation of the relative pair plus
the **difference** of the two deflections. This is the finite content of
Proposition III's force composition (1687 par53, 1713 par64): the relative
motion is generated by the force toward the other body together with the
whole accelerative force of the other body, taken with the contrary
sense. -/
theorem relative_deflection_difference (z : PairState) (d e : Nat → LatticePoint) (n : Nat) :
  relQAT z d e (n + 1) =
  add (extend (relPAT z d e n) (relQAT z d e n)) (sub (d n) (e n)) := by
  simp only [pairMotion, relPAT, relQAT, relP, relQ, add_sub_distrib, extend_sub]

/-- A body's own deflection is exactly the gap between its next vertex and the
inertial continuation of its current pair. -/
theorem absolute_deflection (z : PairState) (d e : Nat → LatticePoint) (n : Nat) :
  sub (pairMotion z d e (n + 1)).q (extend (pairMotion z d e n).p (pairMotion z d e n).q) =
  d n := by
  simp only [pairMotion, sub, add]
  apply Prod.ext <=> dsimp <=> omega

/-- The laws' Corollary VI, finite form (1687 par54, 1713 par65): a deflection
history added to **both** bodies leaves both relative coordinates unchanged
at every stage. The common history is arbitrary, so this is the polygon
form of "equal accelerative forces along parallel lines". -/
theorem corVI_relative (z : PairState) (d e h : Nat → LatticePoint) (n : Nat) :
  relPAT z (fun i => add (d i) (h i)) (fun i => add (e i) (h i)) n = relPAT z d e n ∧
  relQAT z (fun i => add (d i) (h i)) (fun i => add (e i) (h i)) n = relQAT z d e n := by
  induction n with
  | zero => exact (rfl, rfl)
  | succ n ih =>
    refine {?, ?}
    · rw [relP_succ, relP_succ, ih.2]
    · rw [relative_deflection_difference, relative_deflection_difference, ih.1, ih.2]
    simp only [add_common_cancel]

theorem common_deflection_example :
  (relQAT exampleState (fun _ => (0, -2)) (fun _ => (0, 0)) 1 =
  relQAT exampleState (fun _ => (0, -1)) (fun _ => (0, 1)) 1) ∧
  (fun _ => (0, -2)) 0 ≠ (fun _ => (0, -1)) 0 ∧
  sub ((fun _ => (0, -2)) 0) ((fun _ => (0, 0)) 0) =
  sub ((fun _ => (0, -1)) 0) ((fun _ => (0, 1)) 0) := by

```

```

decide

/-- Law I, finite form (cited in 1687 par54, 1713 par65): with no deflection the
reference body's vertices are the uniform motion `centreAt` of its initial
pair. The first body's deflections are irrelevant to this claim. -/
theorem lawI_uniform (z : PairState) (d e : Nat → LatticePoint) (he : ∀ i, e i = (0, 0))
  (n : Nat) :
  (pairMotion z d e n).s = centreAt z.s (sub z.t z.s) n ∧
  (pairMotion z d e n).t = centreAt z.s (sub z.t z.s) (n + 1) := by
  induction n with
  | zero =>
    refine {?, ?}
    · simp only [pairMotion, centreAt]
      apply Prod.ext <=> dsimp <=> omega
    · simp only [pairMotion, centreAt, sub]
      apply Prod.ext <=> dsimp <=> omega
  | succ n ih =>
    refine {?, ?}
    · rw [show (pairMotion z d e (n + 1)).s = (pairMotion z d e n).t from rfl]
      exact ih.2
    · rw [show (pairMotion z d e (n + 1)).t =
        add (extend (pairMotion z d e n).s (pairMotion z d e n).t) (e n) from rfl]
      rw [he n, add_zero_right, ih.1, ih.2]
      simp only [extend, centreAt, sub]
      apply Prod.ext <=> dsimp <=> simp only [Int.ofNat_add, Int.add_mul] <=> omega

/-- With an inertial reference body, the relative radius from the uniform
`centreAt` is the relative coordinate of the constructed polygon: Newton's
moving centre is the reference body itself. -/
theorem centre_is_reference_body (z : PairState) (d e : Nat → LatticePoint)
  (he : ∀ i, e i = (0, 0)) (n : Nat) :
  sub (pairMotion z d e n).q (centreAt z.s (sub z.t z.s) (n + 1)) = relQAt z d e n := by
  rw [← (lawI_uniform z d e he n).2, relQAt, relQ]

/-- Newton's route for Proposition III: Corollary VI reduces to the uniformly
moving reference body, Law I makes that body the `centreAt` motion, and the
moving-centre converse of Proposition II (`Converse`) then makes the
deflection from the inertial continuation parallel to the relative
radius. -/
theorem propIII_via_moving_centre (z : PairState) (d e : Nat → LatticePoint)
  (h : ∀ n, det (relQAt z d e n) (relQAt z d e (n + 1))) =
  det (relPAT z d e n) (relQAt z d e n) (n : Nat) :
  det (relQAt z d e n) (sub (d n) (e n)) = 0 := by
  -- Corollary VI: cancel the reference body's deflection in both histories.
  let d' := fun i => add (d i) (sub (0, 0) (e i))
  let e' := fun i => add (e i) (sub (0, 0) (e i))
  have he' : ∀ i, e' i = (0, 0) := by
    intro i
    apply Prod.ext <=> dsimp [e', add, sub] <=> omega
  have hdiff : ∀ i, d' i = sub (d i) (e i) := by
    intro i
    apply Prod.ext <=> dsimp [d', add, sub] <=> omega
  have hrelative := corVI_relative z d e (fun i => sub (0, 0) (e i))
  -- Law I identifies .p with time i and .q with time i+1.
  have hp (i : Nat) :
    sub (pairMotion z d' e' i).p (centreAt z.s (sub z.t z.s) i) =
    relPAT z d e i := by
    rw [← (lawI_uniform z d' e' he' i).1]
    exact (hrelative i).1
  have hq (i : Nat) :
    sub (pairMotion z d' e' i).q (centreAt z.s (sub z.t z.s) (i + 1)) =
    relQAt z d e i :=
    (centre_is_reference_body z d' e' he' i).trans (hrelative i).2
  -- Proposition II applies to the reduced body's .p vertex sequence.
  have hareas : ∀ i,
    det (sub (pairMotion z d' e' (i + 1)).p (centreAt z.s (sub z.t z.s) (i + 1)))
      (sub (pairMotion z d' e' (i + 2)).p (centreAt z.s (sub z.t z.s) (i + 2))) =
    det (sub (pairMotion z d' e' i).p (centreAt z.s (sub z.t z.s) i))
      (sub (pairMotion z d' e' (i + 1)).p (centreAt z.s (sub z.t z.s) (i + 1))) := by
    intro i

```

```

    simp only [hp, relP_succ] using h i
    have hc := Converse.moving_centre_equal_areas_central z.s (sub z.t z.s)
    (fun i => (pairMotion z d' e' i).p) hareas n
    change det (sub (pairMotion z d' e' n).q (centreAt z.s (sub z.t z.s) (n + 1)))
      (sub (pairMotion z d' e' (n + 1)).q
        (extend (pairMotion z d' e' n).p (pairMotion z d' e' n).q)) = 0 at hc
    rw [hq, absolute_deflection, hdiff] at hc
    exact hc

/-- Regression for the moving-centre indexing: three consecutive true relative
radii have equal oriented areas, while pairing .q with time n instead of
n+1 gives unequal areas, even with an inertial reference body. -/
theorem moving_centre_alignment_example :
  let z : PairState := { p := (0, -1), q := (2, 0), s := (0, 0), t := (1, 0) }
  let d : Nat → LatticePoint := fun n => if n = 0 then (-2, 0) else (0, -2)
  let e : Nat → LatticePoint := fun _ => (0, 0)
  det (relQAt z d e 0) (relQAt z d e 1) =
    det (relQAt z d e 1) (relQAt z d e 2) ∧
  det (sub (pairMotion z d e 0).q (centreAt z.s (sub z.t z.s) 0))
    (sub (pairMotion z d e 1).q (centreAt z.s (sub z.t z.s) 1)) ≠
    det (sub (pairMotion z d e 1).q (centreAt z.s (sub z.t z.s) 1))
      (sub (pairMotion z d e 2).q (centreAt z.s (sub z.t z.s) 2)) := by
decide

/-- Proposition III, finite step in relative coordinates: equal relative
oriented areas in equal cells make the difference of the two deflections
parallel to the relative radius. -/
theorem relative_equal_area_central (z : PairState) (d e : Nat → LatticePoint)
  (h : ∀ n, det (relQAt z d e n) (relQAt z d e (n + 1)) =
    det (relPAT z d e n) (relQAt z d e n)) (n : Nat) :
  det (relQAt z d e n) (sub (d n) (e n)) = 0 := by
  have hn := h n
  rw [relative_deflection_difference, det_add, extension_identity] at hn
  omega

/-- With a nonzero relative radius the same step gives an explicit rational
multiple `a/b` of that radius. The inward sense is a separate premise. -/
theorem relative_rational_central (z : PairState) (d e : Nat → LatticePoint)
  (h : ∀ n, det (relQAt z d e n) (relQAt z d e (n + 1)) =
    det (relPAT z d e n) (relQAt z d e n))
  (hq : relQAt z d e 0 ≠ (0, 0)) :
  ∃ a b : Int, b ≠ 0 ∧ b * (sub (d 0) (e 0)).1 = a * (relQAt z d e 0).1 ∧
    b * (sub (d 0) (e 0)).2 = a * (relQAt z d e 0).2 :=
  Converse.parallel_is_multiple (relQAt z d e 0) (sub (d 0) (e 0)) hq
  (relative_equal_area_central z d e h 0)

/-- The parallelism is a statement about the difference: two different pairs
of absolute deflection histories with the same difference give the same
relative conclusion. Concrete first cell. -/
theorem difference_not_pair_example :
  sub ((fun _ : Nat => (0, -2)) 0) ((fun _ : Nat => (0, 0)) 0) =
    sub ((fun _ : Nat => (1, -1)) 0) ((fun _ : Nat => (1, 1)) 0) ∧
  ((fun _ : Nat => (0, -2)) 0, (fun _ : Nat => (0, 0)) 0) ≠
  ((fun _ : Nat => (1, -1)) 0, (fun _ : Nat => (1, 1)) 0) := by
decide

```

Scholium

Since the uniform description of areas indicates the centre to which the force that most affects the body is directed, and the body is retained in its orbit by a force toward that centre, and every circular motion is rightly said to be performed about the centre by whose force the body is drawn back from rectilinear motion and retained in its orbit: why should we not use the uniform description of areas, in what follows, as the index of the centre about which every circular motion in free spaces is performed?

No theorem of the Lean reconstruction cites this item.

Proposition IV. Theorem IV. *The centripetal forces of bodies describing different circles with uniform motion tend to the centres of those circles, and are to each other as the squares of the arcs described in the same time applied to the radii of the circles.*

In modern terms: for arcs s described in a common time on circles of radius r , $F \propto s^2/r$; that is, $F \propto v^2/r$.

Proof. Let the bodies B, b , revolving in the circumferences of the circles BD, bd , describe the arcs BD, bd in the same time. By their inherent force alone they would describe tangents BC, bc equal to these arcs; so it is the centripetal forces that perpetually draw the bodies back from the tangents to the circumferences, and they are to each other in the first ratio of the nascent spaces CD, cd . They tend to the centres of the circles by Theorem II, because the areas described by the radii are taken proportional to the times. Make the figure tkb similar to the figure DCB . By Lemma V the line CD is to the line kt as the arc BD to the arc bt ; by Lemma XI the nascent line tk is to the nascent line dc as bt^2 to bd^2 ; so *ex aequo* the nascent line DC is to the nascent line dc as $BD \cdot bt$ to bd^2 , or equivalently as $BD \cdot bt/Sb$ to bd^2/Sb , and so (the ratios bt/Sb and BD/SB being equal) as BD^2/SB to bd^2/Sb .

In modern terms: the departure from the tangent over the arc s is the sagitta $CD \approx s^2/(2r)$ (Lemma XI), and the force is as that departure in a given time (Lemma X, Corollary 2).

1713. A new proof replaces this one. The forces tend to the centres of the circles by Proposition II and Corollary 2 of Proposition I, and are to each other as the versed sines of the least arcs described in equal times (Corollary 4 of Proposition I), that is, as the squares of those arcs applied to the diameters of the circles (Lemma VII). Since these arcs are as the arcs described in any equal times, and the diameters as the radii, the forces are as the squares of arcs described in the same time applied to the radii.

Corollary 1. Hence the centripetal forces are as the squares of the velocities applied to the radii of the circles: $F \propto v^2/r$.

Corollary 2. And they are inversely as the squares of the periodic times applied to the radii: $F \propto r/T^2$. That is, in the language of the geometers, these forces are in the ratio compounded of the duplicate ratio of the velocities directly and the simple ratio of the radii inversely; and also in the ratio compounded of the simple ratio of the radii directly and the duplicate ratio of the periodic times inversely.

Corollary 3. So if the periodic times are equal, both the centripetal forces and the velocities are as the radii, and conversely.

Corollary 4. If the squares of the periodic times are as the radii, the centripetal forces are equal and the velocities are in the subduplicate ratio of the radii, and conversely: $T^2 \propto r \iff F = \text{const}, v \propto r^{1/2}$.

Corollary 5. If the squares of the periodic times are as the squares of the radii, the centripetal forces are inversely as the radii and the velocities equal, and conversely: $T \propto r \iff F \propto 1/r, v = \text{const}$.

Corollary 6. If the squares of the periodic times are as the cubes of the radii, the centripetal forces are inversely as the squares of the radii, and the velocities in the

subduplicate ratio of the radii; and conversely: $T^2 \propto r^3 \iff F \propto 1/r^2$. [As printed in 1687 the velocity clause lacks “inversely”; by Corollary 1 the consistent relation is $v \propto r^{-1/2}$.]

Corollary 7. All the same holds for the times, velocities and forces with which bodies describe similar parts of any similar figures having similarly placed centres, by applying the preceding proof to those cases.

1713. The corollaries are rewritten and extended to nine.

Corollary 1. Therefore, since those arcs are as the velocities of the bodies, the forces are as the squares of the velocities applied to the radii, or, as the geometers say, in the ratio compounded of the duplicate ratio of the velocities directly and the simple ratio of the radii inversely.

Corollary 2. And since the periodic times are in the ratio compounded of the radii directly and the velocities inversely, the forces are inversely as the squares of the periodic times applied to the radii, that is, in the ratio compounded of the radii directly and the duplicate ratio of the periodic times inversely: $F \propto r/T^2$.

Corollary 3. So if the periodic times are equal, and therefore the velocities are as the radii, the forces are also as the radii, and conversely.

Corollary 4. If both the periodic times and the velocities are in the subduplicate ratio of the radii, the forces are equal, and conversely.

Corollary 5. If the periodic times are as the radii, and therefore the velocities are equal, the forces are inversely as the radii, and conversely.

Corollary 6. If the periodic times are in the sesquiplicate ratio of the radii, and therefore the velocities inversely in the subduplicate ratio of the radii, the forces are inversely as the squares of the radii, and conversely: $T \propto r^{3/2} \iff v \propto r^{-1/2} \iff F \propto r^{-2}$. This corrects the 1687 velocity clause.

Corollary 7. And universally, if the periodic time is as any power R^n of the radius R , and therefore the velocity inversely as R^{n-1} , the centripetal force is inversely as R^{2n-1} , and conversely.

Corollary 8. The old Corollary 7, with the addition that it is applied by substituting the uniform description of areas for uniform motion, and the distances of the bodies from the centres for the radii.

Corollary 9. From the same proof it also follows that the arc which a body describes in any time, revolving uniformly in a circle under a given centripetal force, is a mean proportional between the diameter of the circle and the descent of the body falling under the same force in the same time: $s^2 = d \cdot h$.

1726. Corollaries 1 and 2 drop the “applied to the radii” formulation and keep only the compound ratios: in the ratio compounded of the duplicate ratio of the velocities directly and the simple ratio of the radii inversely; and in the ratio compounded of the radii directly and the duplicate ratio of the periodic times inversely.

Lean reconstruction: 15 theorems in 3 modules cite this item; statements and proofs follow, by module in import order.

NewtonLimitDynamics/Comparison/CircleCompare.lean — Proposition IV, finite circular comparison (TASKS.md order 7). Both editions compare the centripetal forces of bodies describing circles in equal times by the squares of the simultaneous arcs divided by the radii, $F \propto \text{arc}^2 / (r \cdot t^2)$. The exact finite circle fact behind every route is the sagitta-chord relation $s \cdot (2r - s) = (c/2)^2$ for a chord c with sagitta s in a circle of radius r ; the edition-local routes differ only in *which limiting lemma* turns that finite relation into $\text{arc}^2 / (r \cdot t^2)$: * 1687: Proposition II (force as sagitta over t^2), Lemma V (similar figures, duplicate ratio ...

```

/-- Equal-time forces, cross-multiplied, are proportional to the sagittae. -/
theorem force_ratio_is_sagitta_ratio {r r' : Fraction} (w : CircleChord r) (w' : CircleChord r')
  (t : Fraction) (ht : positive t) :
  equiv (mul (forceBySagitta w t ht) w'.s) (mul (forceBySagitta w' t ht) w.s) := by
  unfold forceBySagitta quotient mul equiv
  dsimp
  ac_nf
  try omega

```

NewtonLimitDynamics/Diagnostic/InverseCubeAreal.lean — Action diagnostic layer (research/action-arguments/Arg002). Modern reconstruction over natural-number magnitudes of two uniform circles compared by Proposition IV Cor. 1 (force as squared velocity over radius, stated as a cross-multiplied proportion). No historical proof uses these theorems.

```

/-- Under an inverse-cube comparison, circles related by Cor. 1 have the same areal velocity. -/
theorem inverseCube_equalAreal (F1 F2 R1 R2 v1 v2 : Nat) (hF : 0 < F2) (hR : 0 < R2)
  (h1 : corOneProportion F1 F2 R1 R2 v1 v2) (h3 : inverseCube F1 F2 R1 R2) :
  equalAreal R1 R2 v1 v2 := by
  unfold corOneProportion at h1
  unfold inverseCube at h3
  unfold equalAreal
  apply Nat.eq_of_mul_eq_mul_left (Nat.mul_pos hF hR)
  calc F2 * R2 * ((R1 * v1) * (R1 * v1))
    = (F2 * (v1 * v1) * R2) * (R1 * R1) := by ac_rfl
    = (F1 * (v2 * v2) * R1) * (R1 * R1) := by rw [h1]
    _ = (F1 * (R1 * R1 * R1)) * (v2 * v2) := by ac_rfl
    _ = (F2 * (R2 * R2 * R2)) * (v2 * v2) := by rw [h3]
    _ = F2 * R2 * ((R2 * v2) * (R2 * v2)) := by ac_rfl

/-- Conversely, a common areal velocity with Cor. 1 forces the inverse-cube comparison. -/
theorem equalAreal_inverseCube (F1 F2 R1 R2 v1 v2 : Nat) (hv : 0 < v2)
  (h1 : corOneProportion F1 F2 R1 R2 v1 v2) (hA : equalAreal R1 R2 v1 v2) :
  inverseCube F1 F2 R1 R2 := by
  unfold corOneProportion at h1
  unfold equalAreal at hA
  unfold inverseCube
  apply Nat.eq_of_mul_eq_mul_left (Nat.mul_pos hv hv)
  calc v2 * v2 * (F1 * (R1 * R1 * R1))
    = (F1 * (v2 * v2) * R1) * (R1 * R1) := by ac_rfl
    _ = (F2 * (v1 * v1) * R2) * (R1 * R1) := by rw [h1]
    _ = F2 * R2 * ((R1 * v1) * (R1 * v1)) := by ac_rfl
    _ = F2 * R2 * ((R2 * v2) * (R2 * v2)) := by rw [hA]
    _ = v2 * v2 * (F2 * (R2 * R2 * R2)) := by ac_rfl

/-- Contrast: under an inverse-square comparison ( $F_1 R_1^2 = F_2 R_2^2$ , 1713 Cor. 6), radii 1 and 4 with speeds 2 and 1 satisfy Cor. 1 but have unequal areal velocities 2 and 4. -/
theorem inverseSquare_areal_varies :
  corOneProportion 16 1 1 4 2 1 ∧ 16 * (1 * 1) = 1 * (4 * 4) ∧
  ¬ equalAreal 1 4 2 1 := by

```

```

unfold corOneProportion equalAreal
decide

```

NewtonLimitDynamics/Polygon/HarmonicStability.lean — Stability of the finite impulse construction for the linear central field $a(p) = -w \cdot p$ (force proportional to distance; compare Proposition IV Cor. 3, equal periods with forces as radii, 1687 NATP00077 par64, 1713 NATP00082 par75). With equal cells d , the construction conserves exactly the quadratic form $w|x|^2 + w \cdot d \cdot (x \cdot v) + |v|^2$, which equals $w|x + (d/2)v|^2 + (1 - w \cdot d^2/4)|v|^2$ (stated with $d = c + c$ to avoid numeric literals). For $w > 0$ and $w \cdot d^2 < 4$ this bounds every refined orbit. Modern rational reconstruction; no ODE theorem, limit or curve is used or produced.

```

theorem linearField_central (w : Fraction) : central (linearField w) := by
  intro p
  unfold linearField negF pointScale det Fraction.equiv Fraction.add Fraction.mul Fraction.ofInt
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg, Int.mul_one, Int.zero_mul]
  ac_nf
  omega

/-- One cell of duration `d` conserves the invariant built with the same `d`. -/
theorem cell_invariant (w d : Fraction) (s : Point × Point) :
  Fraction.equiv (invariant w d (cell (linearField w) d s)) (invariant w d s) := by
  unfold invariant dot cell linearField negF pointAdd pointScale Fraction.equiv Fraction.add
  Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Every equal-cell schedule conserves it. -/
theorem schedule_invariant (w d : Fraction) :
  (n : Nat) → (s : Point × Point) →
  Fraction.equiv (invariant w d (schedule (linearField w) (List.replicate n d) s))
    (invariant w d s)
  | 0, _ => Fraction.equiv_refl _
  | n + 1, s =>
    Fraction.equiv_trans (schedule_invariant w d n (cell (linearField w) d s))
      (cell_invariant w d s)

/-- Completed square, for cells of duration `d = c + c`: the invariant equals
  `w|x + c*v|^2 + (1 - w*c^2)|v|^2`. When `w ≥ 0` and `w*c^2 < 1` (that is,
  `w*d^2 < 4`) both coefficients are nonnegative, so every equal-cell orbit
  keeps `|v|^2` and `w|x + c*v|^2` bounded by its initial invariant
  (`schedule_speed_bound`, `schedule_position_bound`). -/
theorem invariant_square (w c : Fraction) (s : Point × Point) :
  Fraction.equiv (invariant w (Fraction.add c c) s)
    (Fraction.add
      (Fraction.mul w (dot (pointAdd s.1 (pointScale c s.2)) (pointAdd s.1 (pointScale c s.2))))
      (Fraction.mul (margin w c) (dot s.2 s.2))) := by
  unfold invariant margin dot negF pointAdd pointScale Fraction.equiv Fraction.add Fraction.mul
  Fraction.ofInt
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg, Int.mul_one, Int.one_mul]
  ac_nf
  omega

private theorem self_mul_nonneg (a : Int) : 0 ≤ a * a := by
  rcases Int.le_total 0 a with h | h
  · exact Int.mul_nonneg h h
  · have h' : 0 ≤ -a := by omega
    have k := Int.mul_nonneg h' h'
    rwa [Int.neg_mul_neg] at k

theorem dot_self_num_nonneg (p : Point) : 0 ≤ (dot p p).num := by
  unfold dot Fraction.add Fraction.mul
  exact Int.add_nonneg (Int.mul_nonneg (self_mul_nonneg _) (self_mul_nonneg _))

```

```

(Int.mul_nonneg (self_mul_nonneg _) (self_mul_nonneg _))

private theorem le_add_of_num_nonneg (A B : Fraction) (hA : 0 ≤ A.num) :
  Fraction.le B (Fraction.add A B) := by
  unfold Fraction.le Fraction.add
  dsimp
  have h := Int.mul_nonneg (Int.mul_nonneg hA (Int.le_of_lt B.den_pos)) (Int.le_of_lt B.den_pos)
  have e1 : B.num * (A.den * B.den) = B.num * A.den * B.den := by ac_rfl
  rw [Int.add_mul, e1]
  omega

/-- For `w ≥ 0`: `(1 - w*c^2)|v|^2` never exceeds the invariant. -/
theorem speed_bound (w c : Fraction) (hw : 0 ≤ w.num) (s : Point × Point) :
  Fraction.le (Fraction.mul (margin w c) (dot s.2 s.2)) (invariant w (Fraction.add c c) s) :=
  Fraction.le_equiv_right
    (le_add_of_num_nonneg _ _ (Int.mul_nonneg hw (dot_self_num_nonneg _)))
    (Fraction.equiv_symm (invariant_square w c s))

/-- For a nonnegative margin: `w|x + c*v|^2` never exceeds the invariant. -/
theorem position_bound (w c : Fraction) (hm : 0 ≤ (margin w c).num) (s : Point × Point) :
  Fraction.le (Fraction.mul w (dot (pointAdd s.1 (pointScale c s.2)) (pointAdd s.1 (pointScale c s.2))))
    (invariant w (Fraction.add c c) s) :=
  Fraction.le_equiv_right
    (Fraction.le_add_nonnegative _ _ (Int.mul_nonneg hm (dot_self_num_nonneg _)))
    (Fraction.equiv_symm (invariant_square w c s))

/-- Mesh-uniform discrete stability: along every equal-cell schedule of cell
duration `2c`, `(1 - w*c^2)|v_n|^2` stays below the initial invariant. -/
theorem schedule_speed_bound (w c : Fraction) (hw : 0 ≤ w.num) (n : Nat) (s : Point × Point) :
  Fraction.le
    (Fraction.mul (margin w c)
      (dot (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).2
        (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).2))
    (invariant w (Fraction.add c c) s) :=
  Fraction.le_equiv_right (speed_bound w c hw _) (schedule_invariant w (Fraction.add c c) n s)

theorem schedule_position_bound (w c : Fraction) (hm : 0 ≤ (margin w c).num) (n : Nat)
(s : Point × Point) :
  Fraction.le
    (Fraction.mul w
      (dot (pointAdd (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).1
        (pointScale c (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).2))
        (pointAdd (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).1
        (pointScale c (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).2))))
    (invariant w (Fraction.add c c) s) :=
  Fraction.le_equiv_right (position_bound w c hm _) (schedule_invariant w (Fraction.add c c) n s)

```

Scholium

The case of Corollary 6 holds in the celestial bodies (as Wren, Hooke and Halley also gathered separately), and so what concerns a centripetal force decreasing in the duplicate ratio of the distances from the centres is set out more fully in what follows.

The preceding proof also gives the proportion of a centripetal force to any known force, such as gravity. In the time the body traverses the arc *BC* the force impels it through the space *CD*, which at the very beginning of the motion equals the square of the arc *BD* applied to the diameter of the circle; and every body urged by the same force continued always in the same direction describes spaces in the duplicate ratio of the times. So the force, in the time the revolving body describes any given arc, would make the same body, going straight on, describe a space equal to the square of that arc applied to the diameter of the circle; it is therefore to gravity as that space is to the space a heavy body describes falling in the same time. By propositions of this kind Huygens, in his excellent treatise *De Horologio oscillatorio*, compared the force

of gravity with the centrifugal forces of revolving bodies.

In modern terms: $CD \approx s^2/d$ with $d = 2r$, so the force is to gravity as s^2/d is to the distance $\frac{1}{2}gt^2$ fallen in the same time.

1713. This paragraph is replaced. By the preceding Proposition and its corollaries one also obtains the proportion of a centripetal force to any known force, such as gravity: if a body revolves by its gravity in a circle concentric with the Earth, that gravity is its centripetal force, and from the descent of heavy bodies both the time of one revolution and the arc described in any given time are given, by Corollary 9. Huygens compared gravity with the centrifugal forces of revolving bodies by propositions of this kind.

The preceding can also be proved this way. In any circle inscribe a polygon of any number of sides. If a body moving along the sides with a given velocity is reflected from the circle at each vertex, the force with which it strikes the circle at each reflection is as its velocity; so the sum of the forces in a given time is as that velocity and the number of reflections jointly, that is (the polygon being given in kind) as the length described in that time and the same length applied to the radius, that is, as the square of that length applied to the radius; and so, if the polygon, its sides diminished indefinitely, coincides with the circle, as the square of the arc described in the given time applied to the radius. This is the force with which the body presses on the circle, and equal to it is the contrary force with which the circle continually repels the body toward the centre.

1713. “This is the *centrifugal force* with which the body presses on the circle.” **1726** replaces “the same length applied to the radius” by “the same length increased or diminished in the ratio of that length to the radius of the circle”.

In modern terms: on a regular n -gon the body makes about $\nu t n/(2\pi r)$ reflections in the time t , each of impulse about $m\nu \cdot 2\pi/n$, so the total impulse is about $m\nu^2 t/r$. For a given t the length described is as ν , so this is as $(\nu t)^2/r$, Newton’s square of the length applied to the radius.

No theorem of the Lean reconstruction cites this item.

Appendix A. Theorems anchored outside Sections I-II

4 theorems cite passages outside the rendered range (Proposition VI and its later-edition counterparts, the Section I Scholium on vanishing quantities as cited for Lemma X, and the De Motu comparison chain). Statements only.

NewtonLimitDynamics/Comparison/Routes.lean

```
theorem twice_positive (t : Fraction) (ht : positive t) : positive (twice t)
```

```
theorem symmetric_parabola_sagitta (acc time : Fraction) : equiv (midpointHeight (parabolaHeight acc
  ↪ (negate time)) (parabolaHeight acc time)) (parabolaHeight acc time)
```

```
theorem generated_sagitta_commute (displacement time : Fraction) (ht : positive time) : equiv (mul (ofInt
  ↪ 8) (deflectionRatio displacement (twice time) (twice_positive time ht))) (mul (ofInt 2)
  ↪ (deflectionRatio displacement time ht))
```

```
theorem constant_force_generated (acc time : Fraction) (ht : positive time) : equiv (mul (ofInt 2)
  ↳ (deflectionRatio (parabolaHeight acc time) time ht)) acc
```

Appendix B. The foundation: theorems with no source anchor

1002 theorems have no Newton anchor. They build the rational arithmetic, point algebra, finite estimates, Cauchy names and quotient values, binary time, square covers and the generic lifting of operations to completed values on which the anchored proofs stand. Statements only, by module in import order.

BarrowLib/Common/FiniteGrowth.lean — Elementary finite growth estimates with a common positive denominator. These rational arithmetic bounds supply no completion, limiting trajectory, geometric area or historical analytic theorem.

```
theorem weightSum_nonnegative (xs : List Int) (hx : Nonnegative xs) : 0 ≤ weightSum xs

theorem factorProduct_nonnegative (D : Int) (hD : 0 ≤ D) (xs : List Int) (hx : Nonnegative xs) : 0 ≤
  ↳ factorProduct D xs

theorem cofactor_step (D a S : Int) (ha : 0 ≤ a) (hS : 0 ≤ S) : (D + a) * (D - (a + S)) ≤ D * (D - S)

theorem cofactor_bound (D : Int) (hD : 0 < D) (xs : List Int) (hx : Nonnegative xs) : factorProduct D xs
  ↳ * (D - weightSum xs) ≤ D ^ (xs.length + 1)

theorem uniform_product_bound (D : Int) (hD : 0 < D) (xs : List Int) (hx : Nonnegative xs) (hsmall : 2 *
  ↳ weightSum xs ≤ D) : factorProduct D xs ≤ 2 * D ^ xs.length

theorem factor_eq_one_add (D : Int) (hD : 0 < D) (a : Int) : Fraction.equiv (factor D hD a) (Fraction.add
  ↳ (Fraction.ofInt 1) (a, D, hD))

theorem amplification_empty (D : Int) (hD : 0 < D) : Fraction.equiv (amplification D hD [])
  ↳ (Fraction.ofInt 1)

theorem amplification_cons (D : Int) (hD : 0 < D) (a : Int) (xs : List Int) : Fraction.equiv
  ↳ (amplification D hD (a :: xs)) (Fraction.mul (factor D hD a) (amplification D hD xs))

theorem amplification_nonnegative (D : Int) (hD : 0 < D) (xs : List Int) (hx : Nonnegative xs) : 0 ≤
  ↳ (amplification D hD xs).num

theorem uniform_amplification (D : Int) (hD : 0 < D) (xs : List Int) (hx : Nonnegative xs) (hsmall : 2 *
  ↳ weightSum xs ≤ D) : Fraction.le (amplification D hD xs) (Fraction.ofInt 2)

theorem weightSum_append (xs ys : List Int) : weightSum (xs ++ ys) = weightSum xs + weightSum ys

theorem factorProduct_append (D : Int) (xs ys : List Int) : factorProduct D (xs ++ ys) = factorProduct D
  ↳ xs * factorProduct D ys

theorem denominator_power_add (D : Int) (m n : Nat) : D ^ (m + n) = D ^ m * D ^ n

theorem amplification_append (D : Int) (hD : 0 < D) (xs ys : List Int) : Fraction.equiv (amplification D
  ↳ hD (xs ++ ys)) (Fraction.mul (amplification D hD xs) (amplification D hD ys))

theorem weightSum_replicate (a : Int) (n : Nat) : weightSum (List.replicate n a) = (n : Int) * a

theorem factorProduct_replicate (D a : Int) (n : Nat) : factorProduct D (List.replicate n a) = (D + a) ^ n

theorem zero_increments (D : Int) (hD : 0 < D) (n : Nat) : Fraction.equiv (amplification D hD
  ↳ (List.replicate n 0)) (Fraction.ofInt 1)

theorem boundary_sample : Fraction.equiv (amplification 4 (by decide) [1, 1]) (25, 16, by decide)

theorem missing_smallness_counterexample : ¬ Fraction.le (amplification 1 (by decide) [1, 1])
  ↳ (Fraction.ofInt 2)
```

BarrowLib/Common/Quadratic.lean

```
theorem near_and (g : Magnitudes Q) (P R : Q → Prop) (hp : Near g P) (hr : Near g R) : Near g (fun h => P
  ↳ h ∧ R h)

theorem enclosure_reconstruction (g : Magnitudes Q) (ratio lower upper : Q → Q) (c : Q) (hl : Ultimate g
  ↳ lower c) (hu : Ultimate g upper c) (hb : Near g (fun h => g.le (lower h) (ratio h) ∧ g.le (ratio h)
  ↳ (upper h))) : Ultimate g ratio c

theorem near_has_witness (g : Magnitudes Q) (P : Q → Prop) (h : Near g P) : ∃ x, g.positive x ∧ P x

theorem not_near_false (g : Magnitudes Q) : ¬ Near g (fun _ => False)
```

BarrowLib/Common/RationalExhaustion.lean — Rational exhaustion of a closed order bound, proved by the explicit half-gap witness. No completeness, real order or calculus theorem is imported.

```
theorem le_of_enlargements (a b : Fraction) (h : ∀ eps : Fraction, 0 < eps.num → le a (add b eps)) : le a b
```

BarrowLib/Common/RationalMagnitudes.lean

```
theorem abs_num_nonnegative (a : Fraction) : 0 ≤ a.abs.num

theorem le_abs (a : Fraction) : le a a.abs

theorem abs_equiv {a b : Fraction} (h : equiv a b) : equiv a.abs b.abs

theorem abs_mul (a b : Fraction) : equiv (mul a b).abs (mul a.abs b.abs)

theorem abs_of_nonnegative (a : Fraction) (ha : 0 ≤ a.num) : equiv a.abs a

theorem abs_eq_of_nonnegative (a : Fraction) (ha : 0 ≤ a.num) : a.abs = a

theorem abs_neg (a : Fraction) : equiv (←a.num, a.den, a.den_pos) : Fraction).abs a.abs

theorem abs_add_le (a b : Fraction) : le (add a b).abs (add a.abs b.abs)

private theorem transfer {a b c : Fraction} (h : le a b) (k : le b c) : le a c

private theorem mixed {a b c : Fraction} (h : lt a b) (k : le b c) : lt a c

private theorem mixed' {a b c : Fraction} (h : le a b) (k : lt b c) : lt a c

theorem half_add_self (a : Fraction) : equiv (add a.half a.half) a

theorem half_lt (a : Fraction) (h : positive a) : lt a.half a

theorem equiv_iff_mutual_le (a b : Fraction) : equiv a b ↔ le a b ∧ le b a

theorem equiv_refl (a : Fraction) : equiv a a

theorem equiv_symm {a b : Fraction} (h : equiv a b) : equiv b a

theorem equiv_trans {a b c : Fraction} (h : equiv a b) (k : equiv b c) : equiv a c

theorem le_of_equiv {a b : Fraction} (h : equiv a b) : le a b

theorem le_equiv_right {a b c : Fraction} (h : le a b) (k : equiv b c) : le a c

theorem le_equiv_left {a b c : Fraction} (h : equiv a b) (k : le b c) : le a c

theorem add_le_add_right {a b : Fraction} (h : le a b) (c : Fraction) : le (add a c) (add b c)

theorem mul_le_mul_positive {a b : Fraction} (h : le a b) (c : Fraction) (hc : positive c) : le (mul a c)
  ↳ (mul b c)

theorem mul_le_cancel_positive_right {a b : Fraction} (c : Fraction) (hc : positive c) (h : le (mul a c)
  ↳ (mul b c)) : le a b

theorem add_comm (a b : Fraction) : equiv (add a b) (add b a)
```

```

theorem add_le_add_left {a b : Fraction} (h : le a b) (c : Fraction) : le (add c a) (add c b)

theorem add_le_add {a b c d : Fraction} (h : le a b) (k : le c d) : le (add a c) (add b d)

theorem mul_le_mul_nonnegative {a b : Fraction} (h : le a b) (c : Fraction) (hc : 0 ≤ c.num) : le (mul a c) (mul b c)

theorem mul_comm (a b : Fraction) : equiv (mul a b) (mul b a)

theorem mul_le_mul_nonnegative_left {a b : Fraction} (h : le a b) (c : Fraction) (hc : 0 ≤ c.num) : le (mul c a) (mul c b)

theorem abs_add_strict_example : lt (add ((1, 2, by decide) : Fraction) (-1, 2, by decide)).abs (add ((1, 2, by decide) : Fraction).abs ((-1, 2, by decide) : Fraction).abs)

theorem square_ratio (k t : Fraction) (ht : positive t) : equiv (deflectionRatio (mul k (mul t t)) t ht) k

theorem add_assoc (a b c : Fraction) : equiv (add (add a b) c) (add a (add b c))

theorem mul_assoc (a b c : Fraction) : equiv (mul (mul a b) c) (mul a (mul b c))

theorem mul_add (a b c : Fraction) : equiv (mul a (add b c)) (add (mul a b) (mul a c))

theorem positive_mul (a b : Fraction) (ha : positive a) (hb : positive b) : positive (mul a b)

theorem positive_iff_zero_lt (a : Fraction) : positive a ↔ lt (ofInt 0) a

theorem ultimate_congr (f k : Fraction → Fraction) (c : Fraction) (he : ∀ t, positive t → equiv (f t) (k t)) (hk : Ultimate magnitudes k c) : Ultimate magnitudes f c

theorem triangle_normalized_limit (slope : Fraction → Fraction) (c : Fraction) (hs : Ultimate magnitudes (fun t => mul (slope t) (mul t t))) c

theorem triangle_area_ratio (slope time : Fraction) (ht : positive time) : equiv (deflectionRatio (triangleArea time (mul slope time)) time ht) (half slope)

theorem constructed_triangle_limit (slope : Fraction → Fraction) (c : Fraction) (hs : Ultimate magnitudes (fun t => half (slope t)) c) : Ultimate magnitudes (ratio (fun t => triangleArea t (mul (slope t) t))) c

theorem mul_equiv_left (k : Fraction) {a b : Fraction} (h : equiv a b) : equiv (mul k a) (mul k b)

theorem add_equiv_right (k : Fraction) {a b : Fraction} (h : equiv a b) : equiv (add a k) (add b k)

theorem add_equiv {a b c d : Fraction} (h : equiv a b) (k : equiv c d) : equiv (add a c) (add b d)

theorem mul_equiv {a b c d : Fraction} (h : equiv a b) (k : equiv c d) : equiv (mul a c) (mul b d)

theorem add_equiv_left (c : Fraction) {a b : Fraction} (h : equiv a b) : equiv (add c a) (add c b)

theorem mul_equiv_right (c : Fraction) {a b : Fraction} (h : equiv a b) : equiv (mul a c) (mul b c)

theorem le_add_nonnegative (a b : Fraction) (hb : 0 ≤ b.num) : le a (add a b)

theorem nonnegative_add (a b : Fraction) (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) : 0 ≤ (add a b).num

theorem nonnegative_mul (a b : Fraction) (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) : 0 ≤ (mul a b).num

theorem nonnegative_equiv {a b : Fraction} (he : equiv a b) (hb : 0 ≤ b.num) : 0 ≤ a.num

theorem mul_zero (d : Fraction) : equiv (mul d (ofInt 0)) (ofInt 0)

theorem add_zero (c : Fraction) : equiv (add c (ofInt 0)) c

theorem add_mul (a b c : Fraction) : equiv (mul (add a b) c) (add (mul a c) (mul b c))

theorem le_add_cancel_left (z a b : Fraction) (h : le (add z a) (add z b)) : le a b

theorem add_lt_add_left {a b : Fraction} (hab : lt a b) (c : Fraction) : lt (add c a) (add c b)

```

```
theorem add_lt_add_right {a b : Fraction} (hab : lt a b) (c : Fraction) : lt (add a c) (add b c)
```

```
theorem add_lt_add {a b c d : Fraction} (hab : lt a b) (hcd : lt c d) : lt (add a c) (add b d)
```

BarrowLib/Polygon/AccelerationEstimates.lean — Actual finite velocity remainders relative to the force at a cell's starting point. Position displacement and force comparison derive the error; no derivative, completed motion or force equation is supplied.

```
theorem position_increment (a : Point → Point) (h : Fraction) (s : Point × Point) : Fraction.equiv
  ↳ (pointDistance (cell a h s).1 s.1) (Fraction.mul h.abs (pointNorm s.2))
```

```
theorem position_displacement (a : Point → Point) (h : Fraction) (s : Point × Point) (V : Fraction) (hh :
  ↳ 0 ≤ h.num) (N n : Nat) (hn : n ≤ N) (hv : ∀ i, i < N → Fraction.le (pointNorm (run a h s i).2) V) :
  ↳ Fraction.le (pointDistance (run a h s n).1 s.1) (Fraction.mul (time h n) V)
```

```
theorem velocity_remainder_from_samples (a : Point → Point) (h : Fraction) (s : Point × Point) (C :
  ↳ Fraction) (hh : 0 ≤ h.num) (N : Nat) (hs : ∀ i, i < N → Fraction.le (pointDistance (a (run a h s
  ↳ (i+1)).1) (a s.1)) C) : ∀ n, n ≤ N → Fraction.le (pointDistance (run a h s n).2 (predictedVelocity a
  ↳ h s n)) (Fraction.mul (time h n) C)
```

```
theorem force_variation_at (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V : Fraction) (hh
  ↳ : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num) (N i : Nat) (hi : i < N) (hc : Fraction.le
  ↳ (pointDistance (a (run a h s (i+1)).1) (a s.1)) (Fraction.add (Fraction.mul L (pointDistance (run a h
  ↳ s (i+1)).1 s.1)) E)) (hv : ∀ k, k < N → Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le
  ↳ (pointDistance (a (run a h s (i+1)).1) (a s.1)) (source L E (time h N) V)
```

```
theorem force_variation (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V : Fraction) (hh : 0
  ↳ ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num) (hc : comparisonContract a a L E) (N i : Nat) (hi : i < N)
  ↳ (hv : ∀ k, k < N → Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le (pointDistance (a (run a h
  ↳ s (i+1)).1) (a s.1)) (source L E (time h N) V)
```

```
theorem velocity_remainder_at (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V : Fraction)
  ↳ (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num) (n : Nat) (hc : ∀ k, k < n → Fraction.le
  ↳ (pointDistance (a (run a h s (k+1)).1) (a s.1)) (Fraction.add (Fraction.mul L (pointDistance (run a h
  ↳ s (k+1)).1 s.1)) E)) (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le
  ↳ (pointDistance (run a h s n).2 (predictedVelocity a h s n)) (Fraction.mul (time h n) (source L E
  ↳ (time h n) V))
```

```
theorem velocity_remainder (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V : Fraction) (hh
  ↳ : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num) (hc : comparisonContract a a L E) (n : Nat) (hv : ∀ k,
  ↳ k < n → Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le (pointDistance (run a h s n).2
  ↳ (predictedVelocity a h s n)) (Fraction.mul (time h n) (source L E (time h n) V))
```

```
theorem acceleration_secant_bound_at (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V :
  ↳ Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num) (n : Nat) (ht : 0 < (time h n).num) (hc :
  ↳ ∀ k, k < n → Fraction.le (pointDistance (a (run a h s (k+1)).1) (a s.1)) (Fraction.add (Fraction.mul L
  ↳ (pointDistance (run a h s (k+1)).1 s.1)) E)) (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2)
  ↳ V) : Fraction.le (pointDistance (pointScale (TimeCalibration.inverse (time h n) ht) (pointSub (run a
  ↳ h s n).2 s.2)) (a s.1)) (source L E (time h n) V)
```

```
theorem acceleration_secant_bound (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V :
  ↳ Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num) (hc : comparisonContract a a L E) (n :
  ↳ Nat) (ht : 0 < (time h n).num) (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  ↳ Fraction.le (pointDistance (pointScale (TimeCalibration.inverse (time h n) ht) (pointSub (run a h s
  ↳ n).2 s.2)) (a s.1)) (source L E (time h n) V)
```

```
theorem acceleration_secant_equivalent_time_at (a : Point → Point) (h : Fraction) (s : Point × Point) (L
  ↳ E V u : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num) (n : Nat) (ht : 0 < (time h
  ↳ n).num) (hc : ∀ k, k < n → Fraction.le (pointDistance (a (run a h s (k+1)).1) (a s.1)) (Fraction.add
  ↳ (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1)) E)) (hu : 0 < u.num) (he : Fraction.equiv u
  ↳ (time h n)) (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le (pointDistance
  ↳ (pointScale (TimeCalibration.inverse u hu) (pointSub (run a h s n).2 s.2)) (a s.1)) (source L E u V)
```

```
theorem acceleration_secant_equivalent_time (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V
  ↳ u : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num) (hc : comparisonContract a a L E) (n
  ↳ : Nat) (ht : 0 < (time h n).num) (hu : 0 < u.num) (he : Fraction.equiv u (time h n)) (hv : ∀ k, k < n →
  ↳ Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le (pointDistance (pointScale
  ↳ (TimeCalibration.inverse u hu) (pointSub (run a h s n).2 s.2)) (a s.1)) (source L E u V)
```

```

theorem two_cell_acceleration_control : Fraction.equiv (pointDistance (pointScale
  ↪ (TimeCalibration.inverse (time controlHalf 2) (by decide)) (pointSub (run pointNeg controlHalf
  ↪ controlState 2).2 controlState.2)) (pointNeg controlState.1)) (1,8,by decide)

theorem zero_bound_rejects_acceleration_control : ¬ Fraction.le (pointDistance (pointScale
  ↪ (TimeCalibration.inverse (time controlHalf 2) (by decide)) (pointSub (run pointNeg controlHalf
  ↪ controlState 2).2 controlState.2)) (pointNeg controlState.1)) (Fraction.ofInt 0)

```

BarrowLib/Polygon/AffineBoundary.lean — Affine edges meeting at the same rational vertex assign the same completed point to equivalent time addresses on their common boundary.

```

theorem edge_boundary_names (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (x v y u : Point) (m :
  ↪ Nat) (hbase : Fraction.equiv (timeApprox c T m) (Fraction.add (timeApprox b T m) (duration T m)))
  ↪ (hpoint : pointEquiv y (pointAdd x (pointScale (duration T m) v))) (htime : AddressEquiv T hT b c) :
  ↪ NameEquiv (edgeName b T hT x v m) (edgeName c T hT y u m)

```

BarrowLib/Polygon/AffineValues.lean

```

theorem affine_zero_phase (x v : Point) (t : Fraction) (ht : t.num = 0) : stateEquiv (affineState x v t)
  ↪ (x, zeroPoint)

theorem affine_difference (x v : Point) (t u : Fraction) : stateEquiv (stateSub (affineState x v t)
  ↪ (affineState x v u)) (pointScale (durationDifference u t) v, zeroPoint)

theorem affine_distance (x v : Point) (t u : Fraction) : Fraction.equiv (distance (affineState x v t)
  ↪ (affineState x v u)) (Fraction.mul (durationDifference u t).abs (pointNorm v))

theorem shift_difference (c t u : Fraction) : Fraction.equiv (durationDifference (durationDifference c u)
  ↪ (durationDifference c t)) (durationDifference u t)

theorem affine_vertex_distance (x v : Point) (t : Fraction) : Fraction.equiv (distance (affineState x v
  ↪ t) (x, zeroPoint)) (Fraction.mul t.abs (pointNorm v))

theorem phase_abs_bound (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m j : Nat) : Fraction.le
  ↪ (durationDifference (timeApprox b T m) (timeApprox b T (m+j))).abs (duration T m)

theorem phase_nonnegative (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) : (j : Nat) → 0 ≤
  ↪ (durationDifference (timeApprox b T m) (timeApprox b T (m+j))).num

theorem phase_interval (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m j : Nat) : 0 ≤
  ↪ (durationDifference (timeApprox b T m) (timeApprox b T (m+j))).num ∧ Fraction.le (durationDifference
  ↪ (timeApprox b T m) (timeApprox b T (m+j))) (duration T m)

theorem edge_vertex_bound (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (x v : Point) (m : Nat) : Within
  ↪ (realize (edgeName b T hT x v m)) (embed (x, zeroPoint)) (Fraction.mul (duration T m) (pointNorm v))

theorem edgeName_same_start (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (x v : Point) (m : Nat)
  ↪ (hbase : timeApprox b T m = timeApprox c T m) (htime : AddressEquiv T hT b c) : NameEquiv (edgeName b
  ↪ T hT x v m) (edgeName c T hT x v m)

```

BarrowLib/Polygon/BinaryCells.lean — Rational coarse-cell intervals and the adjacent-cell alternatives for equivalent binary addresses. These facts contain no mechanical premises.

```

theorem count_duration_monotone (T : Fraction) (m : Nat) (hT : 0 ≤ T.num) (i k : Nat) (hik : i ≤ k) :
  ↪ Fraction.le (Fraction.mul (Fraction.ofInt (i:Int)) (duration T m)) (Fraction.mul (Fraction.ofInt
  ↪ (k:Int)) (duration T m))

theorem coarse_upper (b : Nat → Bool) (T : Fraction) (m : Nat) : Fraction.equiv (Fraction.add (timeApprox
  ↪ b T m) (duration T m)) (Fraction.mul (Fraction.ofInt ((ticks b m+1):Int))) (duration T m))

theorem time_interval (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m j : Nat) : Fraction.le
  ↪ (timeApprox b T m) (timeApprox b T (m+j)) ∧ Fraction.le (timeApprox b T (m+j)) (Fraction.add
  ↪ (timeApprox b T m) (duration T m))

```

```

theorem coarse_two_step (b : Nat → Bool) (T : Fraction) (m : Nat) : Fraction.equiv (Fraction.add
  ↳ (Fraction.add (timeApprox b T m) (duration T m)) (duration T m)) (Fraction.mul (Fraction.ofInt
  ↳ ((ticks b m+2:Int))) (duration T m))

theorem separated_cell_time_gap (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m j : Nat) (hgap :
  ↳ ticks b m + 2 ≤ ticks c m) : Fraction.le (duration T m) (durationDifference (timeApprox b T (m+j))
  ↳ (timeApprox c T (m+j))).abs

theorem address_equiv_no_separated_cells (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (hpos : 0 <
  ↳ T.num) (m : Nat) (ht : AddressEquiv T hT b c) : ¬ ticks b m + 2 ≤ ticks c m

theorem address_equiv_cell_cases (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (hpos : 0 < T.num) (m
  ↳ : Nat) (ht : AddressEquiv T hT b c) : ticks b m = ticks c m ∨ ticks b m + 1 = ticks c m ∨ ticks c m +
  ↳ 1 = ticks b m

```

BarrowLib/Polygon/BinaryEndpoints.lean — Constructed time coordinates and endpoints, with retained public names. No motion or harmonic coefficient is a premise.

```

theorem timeCoordinate_injective (T : Fraction) (hT : 0 ≤ T.num) : ∀ x y : BinaryTime T hT,
  ↳ timeCoordinate T hT x = timeCoordinate T hT y → x = y

theorem timeWithin_address (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (R : Fraction) : TimeWithin
  ↳ T hT (Quotient.mk _ b) (Quotient.mk _ c) R ↔ NameBound (BinaryTime.timeName b T hT)
  ↳ (BinaryTime.timeName c T hT) R

theorem left_time_state_equiv (T : Fraction) (j : Nat) : stateEquiv (timeState leftAddress T j)
  ↳ (scalarState (Fraction.ofInt 0))

theorem left_time_coordinate (T : Fraction) (hT : 0 ≤ T.num) : timeCoordinate T hT (leftTime T hT) =
  ↳ embed (scalarState (Fraction.ofInt 0))

theorem right_ticks (j : Nat) : ticks rightAddress j + 1 = blocks j

theorem right_time_difference (T : Fraction) (j : Nat) : Fraction.equiv (durationDifference (timeApprox
  ↳ rightAddress T j) T) (duration T j)

theorem right_time_distance (T : Fraction) (j : Nat) (hT : 0 ≤ T.num) : Fraction.equiv (distance
  ↳ (timeState rightAddress T j) (scalarState T)) (duration T j)

theorem right_time_coordinate (T : Fraction) (hT : 0 ≤ T.num) : timeCoordinate T hT (rightTime T hT) =
  ↳ embed (scalarState T)

```

BarrowLib/Polygon/BinaryTime.lean — Binary-address time names built from the same dyadic tick counts as the actual harmonic prefixes. The quotient below is a constructed binary-time domain; no identification with an external real interval is asserted.

```

theorem scalarState_distance (t u : Fraction) : Fraction.equiv (distance (scalarState t) (scalarState u))
  ↳ (durationDifference u t).abs

theorem timeState_distance (b c : Nat → Bool) (T : Fraction) (j : Nat) : Fraction.equiv (distance
  ↳ (timeState b T j) (timeState c T j)) (durationDifference (timeApprox c T j) (timeApprox b T j)).abs

theorem time_step_difference (b : Nat → Bool) (T : Fraction) (j : Nat) : Fraction.equiv
  ↳ (durationDifference (timeApprox b T j) (timeApprox b T (j + 1))) (Fraction.mul (Fraction.ofInt (bit b
  ↳ j : Int)) (duration T (j + 1)))

theorem adjacent_time_bound (b : Nat → Bool) (T : Fraction) (j : Nat) (hT : 0 ≤ T.num) : Fraction.le
  ↳ (distance (timeState b T (j + 1)) (timeState b T j)) (duration T (j + 1))

theorem tail_double (T : Fraction) (j : Nat) : Fraction.equiv (Fraction.add (duration T j) (duration T
  ↳ j)) (doubleTail T j)

theorem finite_gap_time (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) : (k j : Nat) → Fraction.le
  ↳ (distance (timeState b T (j + k)) (timeState b T j)) (duration T j)

theorem two_sided_time (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (N m n : Nat) (hm : N ≤ m) (hn : N
  ↳ ≤ n) : Fraction.le (distance (timeState b T m) (timeState b T n)) (doubleTail T N)

```

```

theorem doubleTail_lt_tolerance (T eps : Fraction) (hT : 0 ≤ T.num) (heps : 0 < eps.num) : Fraction.lt
  ↳ (doubleTail T (modulus T eps)) eps

theorem time_cauchy (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) : ∀ eps : Fraction, 0 < eps.num → ∃
  ↳ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n → Fraction.lt (distance (timeState b T m) (timeState b T n)) eps

theorem addressEquiv_refl (T : Fraction) (hT : 0 ≤ T.num) (b : Nat → Bool) : AddressEquiv T hT b b

theorem addressEquiv_symm (T : Fraction) (hT : 0 ≤ T.num) {b c : Nat → Bool} (h : AddressEquiv T hT b c)
  ↳ : AddressEquiv T hT c b

theorem addressEquiv_trans (T : Fraction) (hT : 0 ≤ T.num) {a b c : Nat → Bool} (hab : AddressEquiv T hT
  ↳ a b) (hbc : AddressEquiv T hT b c) : AddressEquiv T hT a c

```

BarrowLib/Polygon/BoundedCuts.lean — Cauchy realization of nonnegative bounded closed rational cuts by explicit interval bisection. No scalar completeness field or desired Cauchy condition is supplied; the shrinking intervals prove it.

```

theorem lower_mem (c : Cut) : ∀ n, c.lower (interval c n).1

theorem upper_bound (c : Cut) : ∀ n q, c.lower q → Fraction.le q (interval c n).2

theorem interval_order (c : Cut) (n : Nat) : Fraction.le (interval c n).1 (interval c n).2

theorem half_tail (A : Fraction) (n : Nat) : Fraction.equiv (GeometricTail.tailCap A n).half
  ↳ (GeometricTail.tailCap A (n+1))

theorem width_cap (c : Cut) : ∀ n, Fraction.equiv (durationDifference (interval c n).1 (interval c n).2)
  ↳ (GeometricTail.tailCap c.bound n)

theorem adjacent_bound (c : Cut) (n : Nat) : Fraction.le (FiniteEstimates.stateDistance (scalarState
  ↳ (interval c (n+1)).1) (scalarState (interval c n).1)) (GeometricTail.tailCap c.bound (n+1))

theorem name_realizes_cut (c : Cut) (q : Fraction) : NameBelow q (name c) ↔ c.lower q

theorem value_realizes_cut (c : Cut) (q : Fraction) : Below q (value c).val ↔ c.lower q

theorem value_nonnegative (c : Cut) : Below (Fraction.ofInt 0) (value c).val

theorem value_bound (c : Cut) (q : Fraction) (hq : Below q (value c).val) : Fraction.le q c.bound

theorem lower_nonnegative (c : Cut) : ∀ n, 0 ≤ (interval c n).1.num

theorem value_within_zero (c : Cut) : Within (value c).val (embed (scalarState (Fraction.ofInt 0)))
  ↳ c.bound

theorem abs_width_cap (c : Cut) (n : Nat) : Fraction.equiv (durationDifference (interval c n).1 (interval
  ↳ c n).2).abs (GeometricTail.tailCap c.bound n)

theorem names_gap (c d : Cut) (heq : ∀ q, c.lower q ↔ d.lower q) (n : Nat) : Fraction.le
  ↳ (FiniteEstimates.stateDistance (scalarState (interval c n).1) (scalarState (interval d n).1))
  ↳ (GeometricTail.tailCap (Fraction.add c.bound d.bound) n)

theorem name_equiv_of_lower_iff (c d : Cut) (heq : ∀ q, c.lower q ↔ d.lower q) : NameEquiv (name c) (name
  ↳ d)

theorem value_eq_of_lower_iff (c d : Cut) (heq : ∀ q, c.lower q ↔ d.lower q) : value c = value d

theorem value_zero_of_bound_zero (c : Cut) (hz : c.bound.num = 0) : (value c).val = embed (scalarState
  ↳ (Fraction.ofInt 0))

theorem value_of_rational_cut (c : Cut) (r : Fraction) (hr : ∀ q, c.lower q ↔ Fraction.le q r) : (value
  ↳ c).val = embed (scalarState r)

theorem rationalCut_value (r B : Fraction) (hr : 0 ≤ r.num) (hB : 0 ≤ B.num) (hrB : Fraction.le r B) :
  ↳ (value (rationalCut r B hr hB hrB)).val = embed (scalarState r)

theorem one_third_control_value : (value firstControl).val = embed (scalarState oneThird)

```

```
theorem one_third_different_budgets : value firstControl = value secondControl
```

BarrowLib/Polygon/BoundedIteration.lean — Finite position and velocity bounds for iteration of an arbitrary triangular point map with bounded sampled values. No regularity, trajectory, derivative or ODE result is used. The sampled-value bound must be derived on the region or supplied explicitly; the iterates themselves are recursively constructed.

```
theorem zero_duration_run (a : Point → Point) (h : Fraction) (hh : h.num = 0) (s : Point × Point) : (n : Nat) → stateEquiv (run a h s n) s
```

```
theorem run_commute (a : Point → Point) (h : Fraction) (s : Point × Point) (n : Nat) : run a h (cell a h s) n = cell a h (run a h s n)
```

```
theorem time_nonnegative (h : Fraction) (hh : 0 ≤ h.num) (n : Nat) : 0 ≤ (time h n).num
```

```
theorem time_monotone (h : Fraction) (hh : 0 ≤ h.num) (i n : Nat) (hin : i ≤ n) : Fraction.le (time h i) (time h n)
```

```
theorem velocity_bound (a : Point → Point) (h : Fraction) (s : Point × Point) (B : Fraction) (hh : 0 ≤ h.num) (n : Nat) (hb : BoundedSamples a h s B n) : Fraction.le (pointNorm (run a h s n).2) (Fraction.add (pointNorm s.2) (Fraction.mul (time h n) B))
```

```
theorem position_cap_step (h P V B : Fraction) (n : Nat) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) : Fraction.le (Fraction.add (Fraction.add P (Fraction.add (Fraction.mul (time h n) V) (Fraction.mul (Fraction.mul (time h n) (time h n) B))) (Fraction.mul h (Fraction.add V (Fraction.mul (time h n) B)))) (Fraction.add P (Fraction.add (Fraction.mul (time h (n+1)) V) (Fraction.mul (Fraction.mul (time h (n+1)) (time h (n+1)) B))))
```

```
theorem position_bound (a : Point → Point) (h : Fraction) (s : Point × Point) (B : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat) (hb : BoundedSamples a h s B n) : Fraction.le (pointNorm (run a h s n).1) (positionCap h s B n)
```

```
theorem velocity_bound_at_time (a : Point → Point) (h : Fraction) (s : Point × Point) (B T : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat) (hb : BoundedSamples a h s B n) (ht : Fraction.le (time h n) T) : Fraction.le (pointNorm (run a h s n).2) (Fraction.add (pointNorm s.2) (Fraction.mul T B))
```

```
theorem position_bound_at_time (a : Point → Point) (h : Fraction) (s : Point × Point) (B T : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (hT : 0 ≤ T.num) (n : Nat) (hb : BoundedSamples a h s B n) (ht : Fraction.le (time h n) T) : Fraction.le (pointNorm (run a h s n).1) (uniformPositionCap T s B)
```

```
theorem state_bound_at_time (a : Point → Point) (h : Fraction) (s : Point × Point) (B T : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (hT : 0 ≤ T.num) (n : Nat) (hb : BoundedSamples a h s B n) (ht : Fraction.le (time h n) T) : Fraction.le (stateNorm (run a h s n)) (Fraction.add (uniformPositionCap T s B) (Fraction.add (pointNorm s.2) (Fraction.mul T B)))
```

```
theorem run_add (a : Point → Point) (h : Fraction) (s : Point × Point) (n : Nat) : (k : Nat) → run a h s (n+k) = run a h (run a h s n) k
```

```
theorem boundedSamples_restart (a : Point → Point) (h : Fraction) (s : Point × Point) (B : Fraction) (N n : Nat) (hnk : n+k ≤ N) (hb : BoundedSamples a h s B n) : BoundedSamples a h (run a h s n) B k
```

BarrowLib/Polygon/CalibratedGrowth.lean — Finite calibrated bounds from linear growth of an arbitrary rational map. No arrival bound or confinement premise is supplied. The recurrence uses the existing finite amplification and source-budget proofs, rather than an ODE or an integral inequality.

```
theorem cell_norm_bound_at (tau : Fraction) (ht : 0 < tau.num) (a : Point → Point) (h L E : Fraction) (s : Point × Point) (hL : 0 ≤ L.num) (hg : Fraction.le (pointNorm (a (cell a h s).1)) (Fraction.add (Fraction.mul L (pointNorm (cell a h s).1)) E)) : Fraction.le (norm tau (cell a h s)) (Fraction.add (Fraction.mul (amplification tau h L ht) (norm tau s)) (Fraction.mul (Fraction.mul tau h.abs) E))
```

```
theorem cell_norm_bound (tau : Fraction) (ht : 0 < tau.num) (a : Point → Point) (h L E : Fraction) (s : Point × Point) (hL : 0 ≤ L.num) (hg : Growth a L E) : Fraction.le (norm tau (cell a h s)) (Fraction.add (Fraction.mul (amplification tau h L ht) (norm tau s)) (Fraction.mul (Fraction.mul tau h.abs) E))
```

```

theorem run_norm_budget_at (tau : Fraction) (ht : 0 < tau.num) (a : Point → Point) (h L E : Fraction) (s :
  ↪ Point × Point) (hL : 0 ≤ L.num) : (n : Nat) → (hg : ∀ k, k < n → Fraction.le (pointNorm (a
  ↪ (BoundedIteration.run a h s (k+1)).1)) (Fraction.add (Fraction.mul L (pointNorm (BoundedIteration.run
  ↪ a h s (k+1)).1)) E)) → Fraction.le (norm tau (BoundedIteration.run a h s n)) (Fraction.add
  ↪ (Fraction.mul (fpower (amplification tau h L ht) n) (norm tau s)) (FiniteRecurrence.sourceBudget
  ↪ (amplification tau h L ht) (Fraction.mul (Fraction.mul tau h.abs) E) n))

theorem run_norm_budget (tau : Fraction) (ht : 0 < tau.num) (a : Point → Point) (h L E : Fraction) (s :
  ↪ Point × Point) (hL : 0 ≤ L.num) (hg : Growth a L E) : (n : Nat) → Fraction.le (norm tau
  ↪ (BoundedIteration.run a h s n)) (Fraction.add (Fraction.mul (fpower (amplification tau h L ht) n)
  ↪ (norm tau s)) (FiniteRecurrence.sourceBudget (amplification tau h L ht) (Fraction.mul (Fraction.mul
  ↪ tau h.abs) E) n))

theorem cap_rescale (c tau T E : Fraction) (s : Point × Point) (hc : 0 < c.num) : Fraction.equiv (cap
  ↪ (Fraction.mul c tau) (Fraction.mul c T) (rescaleConstant c E hc) (rescaleState c hc s)) (cap tau T E
  ↪ s)

theorem cap_nonnegative (tau T E : Fraction) (s : Point × Point) (ht : 0 ≤ tau.num) (hT : 0 ≤ T.num) (hE
  ↪ : 0 ≤ E.num) : 0 ≤ (cap tau T E s).num

theorem driftCap_budget (tau T L E : Fraction) (s : Point × Point) (ht : 0 < tau.num) (hT : 0 ≤ T.num)
  ↪ (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hw : Fraction.le (Fraction.mul T (rate tau L ht)) (1,2,by decide))
  ↪ : Fraction.le (Fraction.add (pointNorm s.1) (Fraction.mul T (Fraction.add (pointNorm s.2)
  ↪ (Fraction.mul T (Fraction.add (Fraction.mul L (driftCap tau T E s ht)) E)))) (driftCap tau T E s ht))

theorem run_norm_le_cap_at (tau : Fraction) (ht : 0 < tau.num) (a : Point → Point) (h T L E : Fraction)
  ↪ (s : Point × Point) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (n : Nat) (hg : ∀ k, k < n → Fraction.le
  ↪ (pointNorm (a (BoundedIteration.run a h s (k+1)).1)) (Fraction.add (Fraction.mul L (pointNorm
  ↪ (BoundedIteration.run a h s (k+1)).1)) E)) (he : Fraction.le (Fraction.mul (Fraction.ofInt (n : Int))
  ↪ h.abs) T) (hw : Fraction.le (Fraction.mul T (rate tau L ht)) (1,2,by decide)) : Fraction.le (norm tau
  ↪ (BoundedIteration.run a h s n)) (cap tau T E s)

theorem run_norm_le_cap (tau : Fraction) (ht : 0 < tau.num) (a : Point → Point) (h T L E : Fraction) (s :
  ↪ Point × Point) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hg : Growth a L E) (n : Nat) (he : Fraction.le
  ↪ (Fraction.mul (Fraction.ofInt (n : Int)) h.abs) T) (hw : Fraction.le (Fraction.mul T (rate tau L ht))
  ↪ (1,2,by decide)) : Fraction.le (norm tau (BoundedIteration.run a h s n)) (cap tau T E s)

```

BarrowLib/Polygon/CalibratedRefinement.lean — Calibrated actual coarse/two-half-cell accumulation. All maps are rational point maps; sample/arrival bounds are explicit finite premises.

```

theorem localSource_nonnegative (tau h L E B V : Fraction) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hE : 0 ≤
  ↪ E.num) (hB : 0 ≤ B.num) (hV : 0 ≤ V.num) : 0 ≤ (localSource tau h L E B V).num

theorem blockSource_nonnegative (tau h L E B V : Fraction) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hE : 0 ≤
  ↪ E.num) (hB : 0 ≤ B.num) (hV : 0 ≤ V.num) : 0 ≤ (blockSource tau h L E B V ht).num

theorem twoHalf_local_error_at (tau : Fraction) (ht : 0 < tau.num) (b : Point → Point) (h L E B V :
  ↪ Fraction) (t : Point × Point) (hL : 0 ≤ L.num) (hfirst : Fraction.le (pointDistance (b (cell b h
  ↪ t).1) (b (oneFull b h t).1)) (Fraction.add (Fraction.mul L (pointDistance (cell b h t).1 (oneFull b h
  ↪ t).1)) E)) (hsecond : Fraction.le (pointDistance (b (twoHalf b h t).1) (b (oneFull b h t).1))
  ↪ (Fraction.add (Fraction.mul L (pointDistance (twoHalf b h t).1 (oneFull b h t).1)) E)) (hB :
  ↪ Fraction.le (pointNorm (b (cell b h t).1)) B) (hV : Fraction.le (pointNorm t.2) V) : Fraction.le
  ↪ (TimeCalibration.distance tau (twoHalf b h t) (oneFull b h t)) (localSource tau h L E B V)

theorem twoHalf_local_error (tau : Fraction) (ht : 0 < tau.num) (b : Point → Point) (h L E B V :
  ↪ Fraction) (t : Point × Point) (hL : 0 ≤ L.num) (hlocal : comparisonContract b b L E) (hB :
  ↪ Fraction.le (pointNorm (b (cell b h t).1)) B) (hV : Fraction.le (pointNorm t.2) V) : Fraction.le
  ↪ (TimeCalibration.distance tau (twoHalf b h t) (oneFull b h t)) (localSource tau h L E B V)

theorem cross_block_error_at (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E B V :
  ↪ Fraction) (s t : Point × Point) (hL : 0 ≤ L.num) (hc : BlockComparisons a b h L E s t) (hB :
  ↪ Fraction.le (pointNorm (b (cell b h t).1)) B) (hV : Fraction.le (pointNorm t.2) V) : Fraction.le
  ↪ (TimeCalibration.distance tau (twoHalf a h s) (oneFull b h t)) (Fraction.add (Fraction.mul
  ↪ (blockFactor tau h L ht) (TimeCalibration.distance tau s t)) (blockSource tau h L E B V ht))

```

```

theorem cross_block_error (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E B V :
  ↳ Fraction) (s t : Point × Point) (hL : 0 ≤ L.num) (hcross : comparisonContract a b L E) (hlocal :
  ↳ comparisonContract b b L E) (hB : Fraction.le (pointNorm (b (cell b h t).1)) B) (hV : Fraction.le
  ↳ (pointNorm t.2) V) : Fraction.le (TimeCalibration.distance tau (twoHalf a h s) (oneFull b h t))
  ↳ (Fraction.add (Fraction.mul (blockFactor tau h L ht) (TimeCalibration.distance tau s t)) (blockSource
  ↳ tau h L E B V ht))

```

```

theorem actual_error_le_source_at (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E B V :
  ↳ Fraction) (s : Point × Point) (hL : 0 ≤ L.num) : (n : Nat) → (hc : ∀ k, k < n → BlockComparisons a b h
  ↳ L E (fineAt a h s k) (coarseAt b h s k)) → (hB : ∀ k, k < n → Fraction.le (pointNorm (b (cell b h
  ↳ (coarseAt b h s k).1)) B) → (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) →
  ↳ Fraction.le (TimeCalibration.distance tau (fineAt a h s n) (coarseAt b h s n))
  ↳ (FiniteRecurrence.sourceBudget (blockFactor tau h L ht) (blockSource tau h L E B V ht) n)

```

```

theorem actual_error_le_source (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E B V :
  ↳ Fraction) (s : Point × Point) (hL : 0 ≤ L.num) (hcross : comparisonContract a b L E) (hlocal :
  ↳ comparisonContract b b L E) : (n : Nat) → (hB : ∀ k, k < n → Fraction.le (pointNorm (b (cell b h
  ↳ (coarseAt b h s k).1)) B) → (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) →
  ↳ Fraction.le (TimeCalibration.distance tau (fineAt a h s n) (coarseAt b h s n))
  ↳ (FiniteRecurrence.sourceBudget (blockFactor tau h L ht) (blockSource tau h L E B V ht) n)

```

```

theorem actual_uniform_error_at (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E B V :
  ↳ Fraction) (s : Point × Point) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hB0 : 0 ≤ B.num) (hV0 : 0 ≤ V.num)
  ↳ (n : Nat) (hs : TimeCalibration.Window tau h L ht (2*n)) (hc : ∀ k, k < n → BlockComparisons a b h L
  ↳ E (fineAt a h s k) (coarseAt b h s k)) (hB : ∀ k, k < n → Fraction.le (pointNorm (b (cell b h
  ↳ (coarseAt b h s k).1)) B) (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) :
  ↳ Fraction.le (TimeCalibration.distance tau (fineAt a h s n) (coarseAt b h s n)) (Fraction.mul
  ↳ (Fraction.ofInt (2*(n : Int))) (blockSource tau h L E B V ht))

```

```

theorem actual_uniform_error (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E B V :
  ↳ Fraction) (s : Point × Point) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hB0 : 0 ≤ B.num) (hV0 : 0 ≤ V.num)
  ↳ (hcross : comparisonContract a b L E) (hlocal : comparisonContract b b L E) (n : Nat) (hs :
  ↳ TimeCalibration.Window tau h L ht (2*n)) (hB : ∀ k, k < n → Fraction.le (pointNorm (b (cell b h
  ↳ (coarseAt b h s k).1)) B) (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) :
  ↳ Fraction.le (TimeCalibration.distance tau (fineAt a h s n) (coarseAt b h s n)) (Fraction.mul
  ↳ (Fraction.ofInt (2*(n : Int))) (blockSource tau h L E B V ht))

```

```

theorem blockSource_bound (tau : Fraction) (ht : 0 < tau.num) (h T L E B V : Fraction) (hL : 0 ≤ L.num)
  ↳ (hE : 0 ≤ E.num) (hB : 0 ≤ B.num) (hHT : Fraction.le h. abs T) (hK : Fraction.le
  ↳ (TimeCalibration.amplification tau h L ht) (Fraction.ofInt 2)) : Fraction.le (blockSource tau h L E B
  ↳ V ht) (Fraction.add (Fraction.mul (Fraction.mul h. abs h. abs) (consistencyCoefficient tau T L B V))
  ↳ (Fraction.mul (Fraction.ofInt 5) (Fraction.mul (Fraction.mul tau h. abs E))))

```

```

theorem coarseAt_eq_run (a : Point → Point) (h : Fraction) (s : Point × Point) : (n : Nat) → coarseAt a h
  ↳ s n = BoundedIteration.run a (Fraction.add h h) s n

```

```

theorem fineAt_eq_run (a : Point → Point) (h : Fraction) (s : Point × Point) : (n : Nat) → fineAt a h s n
  ↳ = BoundedIteration.run a h s (2*n)

```

BarrowLib/Polygon/CauchyValues.lean

```

theorem position_bounded_tail (a : EndpointCauchyName) : ∃ R : Fraction, 0 ≤ R.num ∧ ∃ N : Nat, ∀ j, N ≤ j
  ↳ → Fraction.le (pointNorm (a.approx j).1) R

```

```

theorem mesh_position_product_vanishes (a : EndpointCauchyName) (eps : Fraction) (heps : 0 < eps.num) : ∃
  ↳ N : Nat, ∀ j, N ≤ j → Fraction.lt (Fraction.mul (duration (Fraction.ofInt 1) j) (pointNorm (a.approx
  ↳ j).1)) eps

```

```

private theorem equiv_zero_num (a : Fraction) (h : Fraction.equiv a (Fraction.ofInt 0)) : a.num = 0

```

```

private theorem zero_lt_positive (eps : Fraction) (heps : 0 < eps.num) : Fraction.lt (Fraction.ofInt 0)
  ↳ eps

```

```

theorem distance_self_lt (a : Point × Point) (eps : Fraction) (heps : 0 < eps.num) : Fraction.lt
  ↳ (distance a a) eps

```

```

theorem nameEquiv_refl (a : EndpointCauchyName) : NameEquiv a a

```

```

theorem nameEquiv_symm {a b : EndpointCauchyName} (hab : NameEquiv a b) : NameEquiv b a

```

```

private theorem half_add_equiv (eps : Fraction) : Fraction.equiv (Fraction.add eps.half eps.half) eps
private theorem half_lt (eps : Fraction) (heps : 0 < eps.num) : Fraction.lt eps.half eps

theorem nameEquiv_trans {a b c : EndpointCauchyName} (hab : NameEquiv a b) (hbc : NameEquiv b c) :
  ↪ NameEquiv a c

private theorem add_zero_split (a b : Fraction) (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) (h : Fraction.equiv
  ↪ (Fraction.add a b) (Fraction.ofInt 0)) : a.num = 0 ∧ b.num = 0

private theorem pointNorm_zero_coords (p : Point) (h : (pointNorm p).num = 0) : p.1.num = 0 ∧ p.2.num = 0

private theorem fraction_sub_zero_equiv (a b : Fraction) (h : (Fraction.add a {-b.num, b.den,
  ↪ b.den_pos}).num = 0) : Fraction.equiv a b

theorem distance_zero_iff_stateEquiv (a b : Point × Point) : Fraction.equiv (distance a b)
  ↪ (Fraction.ofInt 0) ↔ stateEquiv a b

theorem nameEquiv_of_levelwise_stateEquiv (a b : EndpointCauchyName) (h : ∀ n, stateEquiv (a.approx n)
  ↪ (b.approx n)) : NameEquiv a b

theorem constantName_equiv_iff (a b : Point × Point) : NameEquiv (constantName a) (constantName b) ↔
  ↪ stateEquiv a b

theorem embed_eq_iff_stateEquiv (a b : Point × Point) : embed a = embed b ↔ stateEquiv a b

theorem add_lt_add_left {a b : Fraction} (hab : Fraction.lt a b) (c : Fraction) : Fraction.lt
  ↪ (Fraction.add c a) (Fraction.add c b)

private theorem add_lt_add_right {a b : Fraction} (hab : Fraction.lt a b) (c : Fraction) : Fraction.lt
  ↪ (Fraction.add a c) (Fraction.add b c)

private theorem lt_equiv_left {a b c : Fraction} (hab : Fraction.equiv a b) (hbc : Fraction.lt b c) :
  ↪ Fraction.lt a c

theorem lt_equiv_right {a b c : Fraction} (hab : Fraction.lt a b) (hbc : Fraction.equiv b c) :
  ↪ Fraction.lt a c

private theorem three_quarters_lt (R eps : Fraction) (heps : 0 < eps.num) : Fraction.lt (Fraction.add
  ↪ (Fraction.add (Fraction.add R eps.half.half) eps.half.half) eps.half.half) (Fraction.add R eps)

theorem nameBound_symm {a b : EndpointCauchyName} {R : Fraction} (h : NameBound a b R) : NameBound b a R

private theorem distance_three (a a' b b' : Point × Point) : Fraction.le (distance a' b') (Fraction.add
  ↪ (distance a' a) (Fraction.add (distance a b) (distance b b'))))

private theorem three_quarters_reordered (R eps : Fraction) (heps : 0 < eps.num) : Fraction.lt
  ↪ (Fraction.add eps.half.half (Fraction.add (Fraction.add R eps.half.half) eps.half.half))
  ↪ (Fraction.add R eps)

private theorem nameBound_transport {a a' b b' : EndpointCauchyName} {R : Fraction} (ha : NameEquiv a a')
  ↪ (hb : NameEquiv b b') : NameBound a b R → NameBound a' b' R

theorem nameBound_congr {a a' b b' : EndpointCauchyName} {R : Fraction} (ha : NameEquiv a a') (hb :
  ↪ NameEquiv b b') : NameBound a b R ↔ NameBound a' b' R

theorem within_realize (a b : EndpointCauchyName) (R : Fraction) : Within (realize a) (realize b) R ↔
  ↪ NameBound a b R

theorem within_symm (x y : Value) (R : Fraction) (h : Within x y R) : Within y x R

private theorem two_quarters_lt (R S eps : Fraction) (heps : 0 < eps.num) : Fraction.lt (Fraction.add
  ↪ (Fraction.add R eps.half.half) (Fraction.add S eps.half.half)) (Fraction.add (Fraction.add R S) eps)

theorem nameBound_triangle {a b c : EndpointCauchyName} {R S : Fraction} (hab : NameBound a b R) (hbc :
  ↪ NameBound b c S) : NameBound a c (Fraction.add R S)

theorem within_triangle (x y z : Value) (R S : Fraction) (hxy : Within x y R) (hyz : Within y z S) :
  ↪ Within x z (Fraction.add R S)

```

```

private theorem lt_self_add_positive (R eps : Fraction) (heps : 0 < eps.num) : Fraction.lt R
  ↪ (Fraction.add R eps)

theorem nameBound_of_eventual_le (a b : EndpointCauchyName) (R : Fraction) (N : Nat) (h : ∀ n : Nat, N ≤
  ↪ n → Fraction.le (distance (a.approx n) (b.approx n)) R) : NameBound a b R

theorem constant_approximants_converge (a : EndpointCauchyName) (eps : Fraction) (heps : 0 < eps.num) : ∃
  ↪ N : Nat, ∀ m : Nat, N ≤ m → Within (embed (a.approx m)) (realize a) eps

theorem nameBound_zero_iff (a b : EndpointCauchyName) : NameBound a b (Fraction.ofInt 0) ↔ NameEquiv a b

theorem within_zero_iff (x y : Value) : Within x y (Fraction.ofInt 0) ↔ x = y

theorem le_add_cancel_left (z a b : Fraction) (h : Fraction.le (Fraction.add z a) (Fraction.add z b)) :
  ↪ Fraction.le a b

```

BarrowLib/Polygon/CompletionGeometry.lean — Elementary closure and closed coordinate enclosures in the explicit Cauchy plane. Closure uses all positive rational tolerances; no external topology, measure, integral or curve is a premise.

```

theorem within_of_thickenings (x y : Value) (R : Fraction) (h : ∀ eps : Fraction, 0 < eps.num → Within x
  ↪ y (Fraction.add R eps)) : Within x y R

theorem within_embedded_iff (a b : Point × Point) (R : Fraction) : Within (embed a) (embed b) R ↔
  ↪ Fraction.le (distance a b) R

theorem position_distance_zero (s : Point × Point) : Fraction.equiv (distance (positionState s)
  ↪ (zeroPoint, zeroPoint)) (pointNorm s.1)

theorem position_band_realize (a : EndpointCauchyName) (r R : Fraction) (h : ∀ n, Fraction.le r
  ↪ (pointNorm (a.approx n).1) ∧ Fraction.le (pointNorm (a.approx n).1) R) : Within (positionValue
  ↪ (realize a)) (embed (zeroPoint, zeroPoint)) R ∧ ∀ D, Within (positionValue (realize a)) (embed
  ↪ (zeroPoint, zeroPoint)) D → Fraction.le r D

theorem closure_contains (A : PositionValue → Prop) (x : PositionValue) (hx : A x) : Closure A x

theorem closure_mono (A B : PositionValue → Prop) (h : ∀ x, A x → B x) (x : PositionValue) (hx : Closure
  ↪ A x) : Closure B x

theorem closure_idempotent (A : PositionValue → Prop) (x : PositionValue) : Closure (Closure A) x ↔
  ↪ Closure A x

theorem closure_image_bound (A : PositionValue → Prop) (x : PositionValue) (hx : Closure A x) (f : Value
  ↪ → Value) (hf : ∀ u v R, Within u v R → Within (f u) (f v) R) (centre : Value) (R : Fraction) (hA : ∀
  ↪ y, A y → Within (f y.val) centre R) : Within (f x.val) centre R

theorem closure_square (A : PositionValue → Prop) (x : PositionValue) (hx : Closure A x) (centre : Point)
  ↪ (R : NonnegativeRadius) (hA : ∀ y, A y → CoordinateSquare centre R y) : CoordinateSquare centre R x

theorem square_of_ball_bound (x : PositionValue) (centre : Point) (R : NonnegativeRadius) (h : Within
  ↪ x.val (embed (centre, zeroPoint)) R.val) : CoordinateSquare centre R x

```

BarrowLib/Polygon/ConvexCover.lean — Finite rational convex enclosures in the chosen coordinate L1 magnitude. The square is an explicit point set; its area is a covering budget, not an assertion about the area of a union of patches or a limiting trajectory.

```

theorem complement_nonnegative (a : Fraction) (ha : UnitInterval a) : 0 ≤ (complement a).num

theorem complement_abs (a : Fraction) (ha : UnitInterval a) : Fraction.equiv (complement a).abs
  ↪ (complement a)

theorem interval_abs (a : Fraction) (ha : UnitInterval a) : Fraction.equiv a.abs a

theorem weights_sum_one (a : Fraction) : Fraction.equiv (Fraction.add (complement a) a) (Fraction.ofInt
  ↪ 1)

theorem pointSub_self_zero (p : Point) : Fraction.equiv (pointNorm (pointSub p p)) (Fraction.ofInt 0)

```

```

theorem pointSub_chain (a b c : Point) : pointEquiv (pointSub a c) (pointAdd (pointSub a b) (pointSub b
  ↪ c))

theorem pointSub_triangle (a b c : Point) : Fraction.le (pointNorm (pointSub a c)) (Fraction.add
  ↪ (pointNorm (pointSub a b)) (pointNorm (pointSub b c)))

theorem drift_offset (h : Fraction) (x v : Point) : pointEquiv (pointSub (pointAdd x (pointScale h v)) x)
  ↪ (pointScale h v)

theorem lerp_offset (a : Fraction) (anchor p q : Point) : pointEquiv (pointSub (lerp a p q) anchor)
  ↪ (pointAdd (pointScale (complement a) (pointSub p anchor)) (pointScale a (pointSub q anchor)))

theorem lerp_ball_bound (a : Fraction) (ha : UnitInterval a) (anchor p q : Point) (R : Fraction) (hp :
  ↪ Fraction.le (pointNorm (pointSub p anchor)) R) (hq : Fraction.le (pointNorm (pointSub q anchor)) R) :
  ↪ Fraction.le (pointNorm (pointSub (lerp a p q) anchor)) R

theorem ball_inside_square (anchor p : Point) (R : Fraction) (h : Fraction.le (pointNorm (pointSub p
  ↪ anchor)) R) : SquareContains anchor R p

theorem matchedPatch_ball (theta lambda : Fraction) (ht : UnitInterval theta) (hl : UnitInterval lambda)
  ↪ (anchor c0 c1 f0 f1 : Point) (R : Fraction) (hc0 : Fraction.le (pointNorm (pointSub c0 anchor)) R)
  ↪ (hc1 : Fraction.le (pointNorm (pointSub c1 anchor)) R) (hf0 : Fraction.le (pointNorm (pointSub f0
  ↪ anchor)) R) (hf1 : Fraction.le (pointNorm (pointSub f1 anchor)) R) : Fraction.le (pointNorm (pointSub
  ↪ (matchedPatch theta lambda c0 c1 f0 f1) anchor)) R

theorem matchedPatch_square (theta lambda : Fraction) (ht : UnitInterval theta) (hl : UnitInterval
  ↪ lambda) (anchor c0 c1 f0 f1 : Point) (R : Fraction) (hc0 : Fraction.le (pointNorm (pointSub c0
  ↪ anchor)) R) (hc1 : Fraction.le (pointNorm (pointSub c1 anchor)) R) (hf0 : Fraction.le (pointNorm
  ↪ (pointSub f0 anchor)) R) (hf1 : Fraction.le (pointNorm (pointSub f1 anchor)) R) : SquareContains
  ↪ anchor R (matchedPatch theta lambda c0 c1 f0 f1)

theorem complement_interval (a : Fraction) (ha : UnitInterval a) : UnitInterval (complement a)

theorem lerp_zero (p q : Point) : pointEquiv (lerp (Fraction.ofInt 0) p q) p

theorem lerp_one (p q : Point) : pointEquiv (lerp (Fraction.ofInt 1) p q) q

theorem lerp_swap (a : Fraction) (p q : Point) : pointEquiv (lerp a p q) (lerp (complement a) q p)

```

BarrowLib/Polygon/ConvexValues.lean

```

theorem lerp_distance (a : Fraction) (ha : UnitInterval a) (p q p' q' : Point) : Fraction.le
  ↪ (pointDistance (lerp a p q) (lerp a p' q')) (Fraction.add (pointDistance p p') (pointDistance q q'))

theorem convexState_distance (a : Fraction) (ha : UnitInterval a) (s t s' t' : Point × Point) :
  ↪ Fraction.le (distance (convexState a s t) (convexState a s' t')) (Fraction.add (distance s s')
  ↪ (distance t t'))

theorem convexName_equiv (a : Fraction) (ha : UnitInterval a) (s t s' t' : EndpointCauchyName) (hs :
  ↪ NameEquiv s s') (ht : NameEquiv t t') : NameEquiv (convexName a ha s t) (convexName a ha s' t')

theorem convexState_anchor_bound (a : Fraction) (ha : UnitInterval a) (s t : Point × Point) (centre :
  ↪ Point) (R : Fraction) (hs : Fraction.le (distance s (centre, zeroPoint)) R) (ht : Fraction.le
  ↪ (distance t (centre, zeroPoint)) R) : Fraction.le (distance (convexState a s t) (centre, zeroPoint)) R

theorem convexName_ball (a : Fraction) (ha : UnitInterval a) (s t : EndpointCauchyName) (centre : Point)
  ↪ (R : Fraction) (hs : NameBound s (constantName (centre, zeroPoint)) R) (ht : NameBound t (constantName
  ↪ (centre, zeroPoint)) R) : NameBound (convexName a ha s t) (constantName (centre, zeroPoint)) R

theorem convexValue_ball (a : Fraction) (ha : UnitInterval a) (x y : Value) (centre : Point) (R :
  ↪ Fraction) (hx : Within x (embed (centre, zeroPoint)) R) (hy : Within y (embed (centre, zeroPoint)) R) :
  ↪ Within (convexValue a ha x y) (embed (centre, zeroPoint)) R

theorem convexValue_position (a : Fraction) (ha : UnitInterval a) (x y : Value) : positionValue
  ↪ (convexValue a ha x y) = convexValue a ha x y

theorem first_convex_state (a : Fraction) (s t : Point × Point) : stateEquiv (firstState (convexState a s
  ↪ t)) (convexState a (firstState s) (firstState t))

```

```

theorem second_convex_state (a : Fraction) (s t : Point × Point) : stateEquiv (secondState (convexState a
  ↪ s t)) (convexState a (secondState s) (secondState t))

theorem firstValue_convex (a : Fraction) (ha : UnitInterval a) (x y : Value) : firstValue (convexValue a
  ↪ ha x y) = convexValue a ha (firstValue x) (firstValue y)

theorem secondValue_convex (a : Fraction) (ha : UnitInterval a) (x y : Value) : secondValue (convexValue
  ↪ a ha x y) = convexValue a ha (secondValue x) (secondValue y)

theorem convexPosition_square (a : Fraction) (ha : UnitInterval a) (x y : PositionValue) (centre : Point)
  ↪ (R : NonnegativeRadius) (hx : CoordinateSquare centre R x) (hy : CoordinateSquare centre R y) :
  ↪ CoordinateSquare centre R (convexPosition a ha x y)

theorem convexValue_zero (ha : UnitInterval (Fraction.ofInt 0)) (x y : Value) : convexValue
  ↪ (Fraction.ofInt 0) ha x y = positionValue x

theorem convexValue_one (ha : UnitInterval (Fraction.ofInt 1)) (x y : Value) : convexValue
  ↪ (Fraction.ofInt 1) ha x y = positionValue y

theorem convexPosition_zero (ha : UnitInterval (Fraction.ofInt 0)) (x y : PositionValue) : convexPosition
  ↪ (Fraction.ofInt 0) ha x y = x

theorem convexPosition_one (ha : UnitInterval (Fraction.ofInt 1)) (x y : PositionValue) : convexPosition
  ↪ (Fraction.ofInt 1) ha x y = y

theorem convexValue_swap (a : Fraction) (ha : UnitInterval a) (x y : Value) : convexValue a ha x y =
  ↪ convexValue (complement a) (complement_interval a ha) y x

theorem convexPosition_swap (a : Fraction) (ha : UnitInterval a) (x y : PositionValue) : convexPosition a
  ↪ ha x y = convexPosition (complement a) (complement_interval a ha) y x

```

BarrowLib/Polygon/DyadicArithmetic.lean

```

theorem blocks_succ (j : Nat) : blocks (j + 1) = blocks j + blocks j

theorem duration_halving (T : Fraction) (j : Nat) : Fraction.equiv (duration T j) (Fraction.add (duration
  ↪ T (j + 1)) (duration T (j + 1)))

theorem duration_congr {T U : Fraction} (h : Fraction.equiv T U) (j : Nat) : Fraction.equiv (duration T
  ↪ j) (duration U j)

theorem blocks_duration (T : Fraction) (m : Nat) : Fraction.equiv (Fraction.mul (Fraction.ofInt (blocks
  ↪ m:Int)) (duration T m)) T

theorem two_pow_ge_succ (N : Nat) : (N : Int) + 1 ≤ (2 : Int) ^ N

theorem neg_equiv {a b : Fraction} (h : Fraction.equiv a b) : Fraction.equiv (-a.num, a.den, a.den_pos)
  ↪ (-b.num, b.den, b.den_pos)

theorem bit_le_one (b : Nat → Bool) (j : Nat) : bit b j ≤ 1

theorem ticks_lt_blocks (b : Nat → Bool) : (j : Nat) → ticks b j < blocks j

theorem ticks_le_blocks (b : Nat → Bool) (j : Nat) : ticks b j ≤ blocks j

theorem ticks_next (b : Nat → Bool) (j : Nat) : ticks b (j + 1) = ticks b j + ticks b j + bit b j

theorem durationDifference_chain (a b c : Fraction) : Fraction.equiv (durationDifference a c)
  ↪ (Fraction.add (durationDifference a b) (durationDifference b c))

theorem add_difference_cancel (a b : Fraction) : Fraction.equiv (Fraction.add a (durationDifference a b))
  ↪ b

theorem difference_nonnegative_iff (a b : Fraction) : 0 ≤ (durationDifference a b).num ↔ Fraction.le a b

theorem difference_add_bound (a b H : Fraction) (h : Fraction.le (durationDifference a b) H) :
  ↪ Fraction.le b (Fraction.add a H)

theorem difference_congr {a b c d : Fraction} (hac : Fraction.equiv a c) (hbd : Fraction.equiv b d) :
  ↪ Fraction.equiv (durationDifference a b) (durationDifference c d)

```

```

theorem phase_end_difference (a H t : Fraction) : Fraction.equiv (durationDifference (durationDifference
  ↪ a t) H) (durationDifference t (Fraction.add a H))

theorem difference_interval_gaps (a e c : Fraction) (ha : Fraction.le a e) (hc : Fraction.le e c) :
  ↪ Fraction.le (durationDifference a e).abs (durationDifference a c).abs ∧ Fraction.le
  ↪ (durationDifference e c).abs (durationDifference a c).abs

theorem countTime_difference (T : Fraction) (j n k : Nat) : Fraction.equiv (durationDifference (countTime
  ↪ T j n) (countTime T j (n + k))) (Fraction.mul (Fraction.ofInt (k : Int)) (duration T j))

theorem countTime_abs_difference (T : Fraction) (j n k : Nat) (hT : 0 ≤ T.num) : Fraction.equiv
  ↪ (durationDifference (countTime T j n) (countTime T j (n + k))).abs (Fraction.mul (Fraction.ofInt (k :
  ↪ Int)) (duration T j))

theorem durationDifference_abs_symm (a b : Fraction) : Fraction.equiv (durationDifference a b).abs
  ↪ (durationDifference b a).abs

theorem all_zero_ticks (j : Nat) : ticks (fun _ => false) j = 0

```

BarrowLib/Polygon/DyadicNodes.lean — Actual dyadic nodes of the constructed binary-time domain, including the right endpoint. A truncation lies within one cell of its time value.

```

theorem blocks_add (m j : Nat) : blocks (m+j)=blocks m*blocks j

theorem finiteAddress_later_ticks (m k j : Nat) (hk : k<blocks m) : ticks (finiteAddress m k)
  ↪ (m+j)=k*blocks j

theorem grid_time (T : Fraction) (m j : Nat) : Fraction.equiv (Fraction.mul (Fraction.ofInt (blocks j :
  ↪ Int)) (duration T (m+j))) (duration T m)

theorem finiteAddress_time (T : Fraction) (m k j : Nat) (hk : k<blocks m) : Fraction.equiv (timeApprox
  ↪ (finiteAddress m k) T (m+j)) (countTime T m k)

theorem node_time_coordinate (T : Fraction) (hT : 0 ≤ T.num) (m k : Nat) (hk : k≤blocks m) :
  ↪ timeCoordinate T hT (nodeTime T hT m k) = embed (scalarState (countTime T m k))

theorem truncation_time_within (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) : TimeWithin T
  ↪ hT (nodeTime T hT m (ticks b m)) (Quotient.mk _ b) (duration T m)

```

BarrowLib/Polygon/EquivalentDuration.lean — Actual triangular cells at value-equivalent rational durations. A sampled map may distinguish point representatives; comparison retains its additive E.

```

theorem pointDistance_zero_of_equiv {p q : Point} (hpq : pointEquiv p q) : Fraction.equiv (pointDistance
  ↪ p q) (Fraction.ofInt 0)

theorem cell_position_equiv {d e : Fraction} (a : Point → Point) (s : Point × Point) (hde :
  ↪ Fraction.equiv d e) : pointEquiv (cell a d s).1 (cell a e s).1

theorem cell_position_perturbation (a b : Point → Point) (d e : Fraction) (s t : Point × Point) (hde :
  ↪ Fraction.equiv d e) : Fraction.le (pointDistance (cell a d s).1 (cell b e t).1) (Fraction.add
  ↪ (pointDistance s.1 t.1) (Fraction.mul d.abs (pointDistance s.2 t.2)))

theorem cell_velocity_perturbation_at (a b : Point → Point) (d e L E : Fraction) (s t : Point × Point)
  ↪ (hde : Fraction.equiv d e) (hc : Fraction.le (pointDistance (a (cell a d s).1) (b (cell b e t).1))
  ↪ (Fraction.add (Fraction.mul L (pointDistance (cell a d s).1 (cell b e t).1)) E)) : Fraction.le
  ↪ (pointDistance (cell a d s).2 (cell b e t).2) (Fraction.add (pointDistance s.2 t.2) (Fraction.mul
  ↪ d.abs (Fraction.add (Fraction.mul L (pointDistance (cell a d s).1 (cell b e t).1)) E)))

theorem cell_velocity_perturbation (a b : Point → Point) (d e L E : Fraction) (s t : Point × Point) (hde
  ↪ : Fraction.equiv d e) (hc : comparisonContract a b L E) : Fraction.le (pointDistance (cell a d s).2
  ↪ (cell b e t).2) (Fraction.add (pointDistance s.2 t.2) (Fraction.mul d.abs (Fraction.add (Fraction.mul
  ↪ L (pointDistance (cell a d s).1 (cell b e t).1)) E)))

```

```

theorem cell_amplification_at (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (d e L E :
  ↳ Fraction) (s t : Point × Point) (hde : Fraction.equiv d e) (hL : 0 ≤ L.num) (hc : Fraction.le
  ↳ (pointDistance (a (cell a d s).1) (b (cell b e t).1)) (Fraction.add (Fraction.mul L (pointDistance
  ↳ (cell a d s).1 (cell b e t).1)) E)) : Fraction.le (TimeCalibration.distance tau (cell a d s) (cell b
  ↳ e t)) (Fraction.add (Fraction.mul (TimeCalibration.amplification tau d L ht)
  ↳ (TimeCalibration.distance tau s t)) (Fraction.mul (Fraction.mul tau d.abs) E))

theorem cell_amplification (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (d e L E : Fraction)
  ↳ (s t : Point × Point) (hde : Fraction.equiv d e) (hL : 0 ≤ L.num) (hc : comparisonContract a b L E) :
  ↳ Fraction.le (TimeCalibration.distance tau (cell a d s) (cell b e t)) (Fraction.add (Fraction.mul
  ↳ (TimeCalibration.amplification tau d L ht) (TimeCalibration.distance tau s t)) (Fraction.mul
  ↳ (Fraction.mul tau d.abs) E))

theorem run_distance_le_source_at (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (d e L E :
  ↳ Fraction) (s : Point × Point) (hde : Fraction.equiv d e) (hL : 0 ≤ L.num) : (n : Nat) → (hc : ∀ k, k
  ↳ < n → Fraction.le (pointDistance (a (cell a d (BoundedIteration.run a d s k)).1) (b (cell b e
  ↳ (BoundedIteration.run b e s k)).1)) (Fraction.add (Fraction.mul L (pointDistance (cell a d
  ↳ (BoundedIteration.run a d s k)).1 (cell b e (BoundedIteration.run b e s k)).1)) E)) → Fraction.le
  ↳ (TimeCalibration.distance tau (BoundedIteration.run a d s n) (BoundedIteration.run b e s n))
  ↳ (FiniteRecurrence.sourceBudget (TimeCalibration.amplification tau d L ht) (Fraction.mul
  ↳ (Fraction.mul tau d.abs) E) n)

theorem run_distance_le_source (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (d e L E :
  ↳ Fraction) (s : Point × Point) (hde : Fraction.equiv d e) (hL : 0 ≤ L.num) (hc : comparisonContract a
  ↳ b L E) : (n : Nat) → Fraction.le (TimeCalibration.distance tau (BoundedIteration.run a d s n)
  ↳ (BoundedIteration.run b e s n)) (FiniteRecurrence.sourceBudget (TimeCalibration.amplification tau d L
  ↳ ht) (Fraction.mul (Fraction.mul tau d.abs) E) n)

theorem run_uniform_error_at (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (d e L E :
  ↳ Fraction) (s : Point × Point) (hde : Fraction.equiv d e) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (n : Nat)
  ↳ (hs : TimeCalibration.Window tau d L ht n) (hc : ∀ k, k < n → Fraction.le (pointDistance (a (cell a d
  ↳ (BoundedIteration.run a d s k)).1) (b (cell b e (BoundedIteration.run b e s k)).1)) (Fraction.add
  ↳ (Fraction.mul L (pointDistance (cell a d (BoundedIteration.run a d s k)).1 (cell b e
  ↳ (BoundedIteration.run b e s k)).1)) E)) : Fraction.le (TimeCalibration.distance tau
  ↳ (BoundedIteration.run a d s n) (BoundedIteration.run b e s n)) (Fraction.mul (Fraction.ofInt (2*(n :
  ↳ Int))) (Fraction.mul (Fraction.mul tau d.abs) E))

theorem run_uniform_error (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (d e L E : Fraction)
  ↳ (s : Point × Point) (hde : Fraction.equiv d e) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hc :
  ↳ comparisonContract a b L E) (n : Nat) (hs : TimeCalibration.Window tau d L ht n) : Fraction.le
  ↳ (TimeCalibration.distance tau (BoundedIteration.run a d s n) (BoundedIteration.run b e s n))
  ↳ (Fraction.mul (Fraction.ofInt (2*(n : Int))) (Fraction.mul (Fraction.mul tau d.abs) E))

theorem representative_sensitive_control : Fraction.equiv (Fraction.ofInt 1) ((2,2,by decide) : Fraction)
  ↳ ∧ Fraction.equiv (TimeCalibration.distance (Fraction.ofInt 2) (cell representativeMap (Fraction.ofInt
  ↳ 1) rest) (cell representativeMap (2,2,by decide) rest)) (Fraction.ofInt 2) ∧ ~ stateEquiv (cell
  ↳ representativeMap (Fraction.ofInt 1) rest) (cell representativeMap (2,2,by decide) rest)

BarrowLib/Polygon/FiniteAccumulation.lean — Finite accumulation for actual coarse
cells and pairs of half cells. The point map is arbitrary; applications to Newtonian
central fields impose centrality separately. All estimates are coordinate L1 estimates
on rational states.

theorem localBudget_le_uniform (h L E B V : Fraction) (s : Point × Point) (hL : 0 ≤ L.num) (hV :
  ↳ Fraction.le (pointNorm s.2) V) : Fraction.le (localBudget h L E B s) (uniformLocalBudget h L E B V)

theorem amplification_nonnegative (h L : Fraction) (hL : 0 ≤ L.num) : 0 ≤ (amplification L h).num

private theorem one_le_one_add (a : Fraction) (ha : 0 ≤ a.num) : Fraction.le (Fraction.ofInt 1)
  ↳ (Fraction.add (Fraction.ofInt 1) a)

theorem one_le_amplification (h L : Fraction) (hL : 0 ≤ L.num) : Fraction.le (Fraction.ofInt 1)
  ↳ (amplification L h)

theorem blockFactor_nonnegative (h L : Fraction) (hL : 0 ≤ L.num) : 0 ≤ (blockFactor h L).num

theorem one_le_blockFactor (h L : Fraction) (hL : 0 ≤ L.num) : Fraction.le (Fraction.ofInt 1)
  ↳ (blockFactor h L)

```

```

theorem factorPower_nonnegative (r : Fraction) (hr : 0 ≤ r.num) : (n : Nat) → 0 ≤ (factorPower r n).num

theorem one_le_factorPower (r : Fraction) (hr : 0 ≤ r.num) (h1 : Fraction.le (Fraction.ofInt 1) r) : (n :
  ↳ Nat) → Fraction.le (Fraction.ofInt 1) (factorPower r n)

theorem uniformBlockSource_nonnegative (h L E B V : Fraction) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hB : 0 ≤
  ↳ B.num) (hV : 0 ≤ V.num) : 0 ≤ (uniformBlockSource h L E B V).num

theorem cross_twoHalf_perturbation (a b : Point → Point) (h L E : Fraction) (s t : Point × Point) (hL : 0
  ↳ ≤ L.num) (hc : comparisonContract a b L E) : Fraction.le (stateDistance (twoHalf a h s) (twoHalf b h
  ↳ t)) (twoStepBudget h L E (stateDistance s t))

theorem twoHalf_perturbation (a : Point → Point) (h L E : Fraction) (s t : Point × Point) (hL : 0 ≤
  ↳ L.num) (hc : comparisonContract a a L E) : Fraction.le (stateDistance (twoHalf a h s) (twoHalf a h
  ↳ t)) (twoStepBudget h L E (stateDistance s t))

theorem cross_block_error (a b : Point → Point) (h L E B : Fraction) (s t : Point × Point) (hL : 0 ≤
  ↳ L.num) (hcross : comparisonContract a b L E) (hlocal : comparisonContract b b L E) (hB : Fraction.le
  ↳ (pointNorm (b (cell b h t).1)) B) : Fraction.le (stateDistance (twoHalf a h s) (oneFull b h t))
  ↳ (Fraction.add (twoStepBudget h L E (stateDistance s t)) (localBudget h L E B t))

theorem block_error (a : Point → Point) (h L E B : Fraction) (s t : Point × Point) (hL : 0 ≤ L.num) (hc :
  ↳ comparisonContract a a L E) (hB : Fraction.le (pointNorm (a (cell a h t).1)) B) : Fraction.le
  ↳ (stateDistance (twoHalf a h s) (oneFull a h t)) (Fraction.add (twoStepBudget h L E (stateDistance s
  ↳ t)) (localBudget h L E B t))

theorem cross_actual_error_le_budget (a b : Point → Point) (h L E B : Fraction) (s : Point × Point) (hL :
  ↳ 0 ≤ L.num) (hcross : comparisonContract a b L E) (hlocal : comparisonContract b b L E) : (n : Nat) →
  ↳ (hB : ∀ k, k < n → Fraction.le (pointNorm (b (cell b h (coarseAt b h s k)).1)) B) → Fraction.le
  ↳ (stateDistance (fineAt a h s n) (coarseAt b h s n)) (errorBudget b h L E B s n)

theorem actual_error_le_budget (a : Point → Point) (h L E B : Fraction) (s : Point × Point) (hL : 0 ≤
  ↳ L.num) (hc : comparisonContract a a L E) : (n : Nat) → (hB : ∀ k, k < n → Fraction.le (pointNorm (a
  ↳ (cell a h (coarseAt a h s k)).1)) B) → Fraction.le (stateDistance (fineAt a h s n) (coarseAt a h s
  ↳ n)) (errorBudget a h L E B s n)

theorem errorBudget_le_constant (a : Point → Point) (h L E B V : Fraction) (s : Point × Point) (hL : 0 ≤
  ↳ L.num) : (n : Nat) → (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt a h s k).2) V) → Fraction.le
  ↳ (errorBudget a h L E B s n) (constantErrorBudget h L E B V n)

theorem cross_actual_error_le_constant_budget (a b : Point → Point) (h L E B V : Fraction) (s : Point ×
  ↳ Point) (hL : 0 ≤ L.num) (hcross : comparisonContract a b L E) (hlocal : comparisonContract b b L E)
  ↳ (n : Nat) (hB : ∀ k, k < n → Fraction.le (pointNorm (b (cell b h (coarseAt b h s k)).1)) B) (hV : ∀
  ↳ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) : Fraction.le (stateDistance (fineAt a h s
  ↳ n) (coarseAt b h s n)) (constantErrorBudget h L E B V n)

theorem actual_error_le_constant_budget (a : Point → Point) (h L E B V : Fraction) (s : Point × Point) (hL
  ↳ : 0 ≤ L.num) (hc : comparisonContract a a L E) (n : Nat) (hB : ∀ k, k < n → Fraction.le (pointNorm (a
  ↳ (cell a h (coarseAt a h s k)).1)) B) (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt a h s k).2)
  ↳ V) : Fraction.le (stateDistance (fineAt a h s n) (coarseAt a h s n)) (constantErrorBudget h L E B V n)

theorem constant_budget_power_bound (h L E B V : Fraction) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hB : 0 ≤
  ↳ B.num) (hV : 0 ≤ V.num) : (n : Nat) → Fraction.le (constantErrorBudget h L E B V n) (Fraction.mul
  ↳ (count n) (Fraction.mul (uniformBlockSource h L E B V) (factorPower (blockFactor h L) n)))

theorem cross_actual_error_le_two_count_source (a b : Point → Point) (h L E B V : Fraction) (s : Point ×
  ↳ Point) (n : Nat) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hBnonneg : 0 ≤ B.num) (hVnonneg : 0 ≤ V.num)
  ↳ (hcross : comparisonContract a b L E) (hlocal : comparisonContract b b L E) (hB : ∀ k, k < n →
  ↳ Fraction.le (pointNorm (b (cell b h (coarseAt b h s k)).1)) B) (hV : ∀ k, k < n → Fraction.le
  ↳ (pointNorm (coarseAt b h s k).2) V) (hpower : Fraction.le (factorPower (blockFactor h L) n)
  ↳ (Fraction.ofInt 2)) : Fraction.le (stateDistance (fineAt a h s n) (coarseAt b h s n)) (Fraction.mul
  ↳ (Fraction.ofInt 2) (Fraction.mul (count n) (uniformBlockSource h L E B V)))

```

```

theorem actual_error_le_two_count_source (a : Point → Point) (h L E B V : Fraction) (s : Point × Point)
  ↪ (n : Nat) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hBnonneg : 0 ≤ B.num) (hVnonneg : 0 ≤ V.num) (hc :
  ↪ comparisonContract a a L E) (hB : ∀ k, k < n → Fraction.le (pointNorm (a (cell a h (coarseAt a h s
  ↪ k)).1)) B) (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt a h s k).2) V) (hpow : Fraction.le
  ↪ (factorPower (blockFactor h L) n) (Fraction.ofInt 2)) : Fraction.le (stateDistance (fineAt a h s n)
  ↪ (coarseAt a h s n)) (Fraction.mul (Fraction.ofInt 2) (Fraction.mul (count n) (uniformBlockSource h L
  ↪ E B V)))

theorem factorPower_fpower (r : Fraction) : (n : Nat) → Fraction.equiv (factorPower r n)
  ↪ (HarmonicAccumulation.fpower r n)

theorem blockFactor_power_le_two (h L : Fraction) (n : Nat) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hs :
  ↪ SmallWindow h L n) : Fraction.le (factorPower (blockFactor h L) n) (Fraction.ofInt 2)

theorem cross_actual_uniform_error (a b : Point → Point) (h L E B V : Fraction) (s : Point × Point) (n :
  ↪ Nat) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hBnonneg : 0 ≤ B.num) (hVnonneg : 0 ≤ V.num)
  ↪ (hs : SmallWindow h L n) (hcross : comparisonContract a b L E) (hlocal : comparisonContract b b L E)
  ↪ (hB : ∀ k, k < n → Fraction.le (pointNorm (b (cell b h (coarseAt b h s k)).1)) B) (hV : ∀ k, k < n →
  ↪ Fraction.le (pointNorm (coarseAt b h s k).2) V) : Fraction.le (stateDistance (fineAt a h s n)
  ↪ (coarseAt b h s n)) (Fraction.mul (Fraction.ofInt 2) (Fraction.mul (count n) (uniformBlockSource h L
  ↪ E B V)))

theorem actual_uniform_error (a : Point → Point) (h L E B V : Fraction) (s : Point × Point) (n : Nat) (hh :
  ↪ 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hBnonneg : 0 ≤ B.num) (hVnonneg : 0 ≤ V.num) (hs :
  ↪ SmallWindow h L n) (hc : comparisonContract a a L E) (hB : ∀ k, k < n → Fraction.le (pointNorm (a
  ↪ (cell a h (coarseAt a h s k)).1)) B) (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt a h s k).2)
  ↪ V) : Fraction.le (stateDistance (fineAt a h s n) (coarseAt a h s n)) (Fraction.mul (Fraction.ofInt 2)
  ↪ (Fraction.mul (count n) (uniformBlockSource h L E B V)))

theorem cross_constant_contract : comparisonContract sampleDown sampleDownTwo sampleZero (Fraction.ofInt
  ↪ 1)

theorem coarse_constant_contract : comparisonContract sampleDownTwo sampleDownTwo sampleZero
  ↪ (Fraction.ofInt 1)

theorem zero_block_control (a : Point → Point) (h : Fraction) (s : Point × Point) : Fraction.equiv
  ↪ (stateDistance (fineAt a h s 0) (coarseAt a h s 0)) sampleZero

theorem zero_duration_block_control : Fraction.equiv (stateDistance (fineAt sampleDown sampleZero
  ↪ sampleState 1) (coarseAt sampleDown sampleZero sampleState 1)) sampleZero

theorem constant_block_control : Fraction.equiv (stateDistance (fineAt sampleDown sampleHalf sampleState
  ↪ 1) (coarseAt sampleDown sampleHalf sampleState 1)) sampleQuarter

theorem constant_block_budget_control : Fraction.equiv (errorBudget sampleDown sampleHalf sampleZero
  ↪ sampleZero (Fraction.ofInt 1) sampleState 1) sampleQuarter

theorem cross_constant_block_control : Fraction.equiv (stateDistance (fineAt sampleDown sampleHalf
  ↪ sampleState 1) (coarseAt sampleDownTwo sampleHalf sampleState 1)) sampleFiveQuarters

theorem cross_constant_block_nonzero : ¬ Fraction.le (stateDistance (fineAt sampleDown sampleHalf
  ↪ sampleState 1) (coarseAt sampleDownTwo sampleHalf sampleState 1)) sampleZero

theorem cross_constant_requires_sample_error : ¬ Fraction.le (stateDistance (fineAt sampleDown
  ↪ sampleHalf sampleState 1) (coarseAt sampleDownTwo sampleHalf sampleState 1)) (errorBudget
  ↪ sampleDownTwo sampleHalf sampleZero sampleZero (Fraction.ofInt 2) sampleState 1)

```

BarrowLib/Polygon/FiniteAddress.lean — Explicit terminating binary addresses for every integer numerator below 2^m . This is finite arithmetic, with no external real interval assumed.

```

theorem finiteAddress_tail : (m k j : Nat) → m ≤ j → finiteAddress m k j = false

theorem ticks_congr_before (b c : Nat → Bool) : (m : Nat) → (∀ i, i < m → b i = c i) → ticks b m = ticks c m

theorem finiteAddress_ticks : (m k : Nat) → k < blocks m → ticks (finiteAddress m k) m = k

```

BarrowLib/Polygon/FiniteEstimates.lean — Finite coordinate estimates for an arbitrary

rational point map. The coordinate L1 magnitude is a chosen algebraic gauge; no trajectory or force law is asserted.

```

theorem pointNorm_le_distance_add (p q : Point) : Fraction.le (pointNorm p) (Fraction.add (pointDistance
  ↪ p q) (pointNorm q))

theorem pointDistance_triangle (p q r : Point) : Fraction.le (pointDistance p r) (Fraction.add
  ↪ (pointDistance p q) (pointDistance q r))

theorem pointDistance_symm (p q : Point) : Fraction.equiv (pointDistance p q) (pointDistance q p)

theorem comparisonContract_reverse (a b : Point → Point) (L E : Fraction) (hc : ∀ p q, Fraction.le
  ↪ (pointDistance (a p) (b q)) (Fraction.add (Fraction.mul L (pointDistance p q)) E)) : ∀ p q,
  ↪ Fraction.le (pointDistance (b p) (a q)) (Fraction.add (Fraction.mul L (pointDistance p q)) E)

theorem stateDistance_triangle (s t u : Point × Point) : Fraction.le (stateDistance s u) (Fraction.add
  ↪ (stateDistance s t) (stateDistance t u))

theorem stateDistance_symm (s t : Point × Point) : Fraction.equiv (stateDistance s t) (stateDistance t s)

theorem pointDistance_self_zero (p : Point) : Fraction.equiv (pointDistance p p) (Fraction.ofInt 0)

theorem stateDistance_self_zero (s : Point × Point) : Fraction.equiv (stateDistance s s) (Fraction.ofInt
  ↪ 0)

theorem pointDistance_equiv {p p' q q' : Point} (hp : pointEquiv p p') (hq : pointEquiv q q') :
  ↪ Fraction.equiv (pointDistance p q) (pointDistance p' q')

private theorem point_difference_add (p q r t : Point) : pointEquiv (pointSub (pointAdd p r) (pointAdd q
  ↪ t)) (pointAdd (pointSub p q) (pointSub r t))

private theorem point_difference_scale (h : Fraction) (p q : Point) : pointEquiv (pointSub (pointScale h
  ↪ p) (pointScale h q)) (pointScale h (pointSub p q))

theorem difference_add_bound (p q r t : Point) : Fraction.le (pointDistance (pointAdd p r) (pointAdd q
  ↪ t)) (Fraction.add (pointDistance p q) (pointDistance r t))

theorem pointDistance_neg (p q : Point) : Fraction.equiv (pointDistance (pointNeg p) (pointNeg q))
  ↪ (pointDistance p q)

theorem difference_sub_bound (p q r t : Point) : Fraction.le (pointDistance (pointSub p r) (pointSub q
  ↪ t)) (Fraction.add (pointDistance p q) (pointDistance r t))

theorem difference_scale (h : Fraction) (p q : Point) : Fraction.equiv (pointDistance (pointScale h p)
  ↪ (pointScale h q)) (Fraction.mul h.abs (pointDistance p q))

theorem component_amplification (P V H L E : Fraction) (hP : 0 ≤ P.num) (hV : 0 ≤ V.num) (hH : 0 ≤ H.num)
  ↪ (hL : 0 ≤ L.num) : Fraction.le (Fraction.add (Fraction.add P (Fraction.mul H V)) (Fraction.add V
  ↪ (Fraction.mul H (Fraction.add (Fraction.mul L (Fraction.add P (Fraction.mul H V))) E))))
  ↪ (Fraction.add (Fraction.mul (Fraction.mul (Fraction.add (Fraction.ofInt 1) H) (Fraction.add
  ↪ (Fraction.ofInt 1) (Fraction.mul H L))) (Fraction.add P V)) (Fraction.mul H E))

theorem cell_position_growth (a : Point → Point) (h : Fraction) (s : Point × Point) : Fraction.le
  ↪ (pointNorm (cell a h s).1) (Fraction.add (pointNorm s.1) (Fraction.mul h.abs (pointNorm s.2)))

theorem cell_velocity_growth (a : Point → Point) (h B : Fraction) (s : Point × Point) (hB : Fraction.le
  ↪ (pointNorm (a (cell a h s).1)) B) : Fraction.le (pointNorm (cell a h s).2) (Fraction.add (pointNorm
  ↪ s.2) (Fraction.mul h.abs B))

theorem cell_state_growth (a : Point → Point) (h B : Fraction) (s : Point × Point) (hB : Fraction.le
  ↪ (pointNorm (a (cell a h s).1)) B) : Fraction.le (stateNorm (cell a h s)) (Fraction.add (Fraction.add
  ↪ (pointNorm s.1) (Fraction.mul h.abs (pointNorm s.2))) (Fraction.add (pointNorm s.2) (Fraction.mul
  ↪ h.abs B)))

theorem cell_position_perturbation (a b : Point → Point) (h : Fraction) (s t : Point × Point) :
  ↪ Fraction.le (pointDistance (cell a h s).1 (cell b h t).1) (Fraction.add (pointDistance s.1 t.1)
  ↪ (Fraction.mul h.abs (pointDistance s.2 t.2)))

```

```

theorem cell_velocity_perturbation_at (a b : Point → Point) (h L E : Fraction) (s t : Point × Point) (hc :
  ↪ : Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1)) (Fraction.add (Fraction.mul L
  ↪ (pointDistance (cell a h s).1 (cell b h t).1)) E)) : Fraction.le (pointDistance (cell a h s).2 (cell
  ↪ b h t).2) (Fraction.add (pointDistance s.2 t.2) (Fraction.mul h.abs (Fraction.add (Fraction.mul L
  ↪ (pointDistance (cell a h s).1 (cell b h t).1)) E))))

theorem cell_velocity_perturbation (a b : Point → Point) (h L E : Fraction) (s t : Point × Point) (hc :
  ↪ comparisonContract a b L E) : Fraction.le (pointDistance (cell a h s).2 (cell b h t).2) (Fraction.add
  ↪ (pointDistance s.2 t.2) (Fraction.mul h.abs (Fraction.add (Fraction.mul L (pointDistance (cell a h
  ↪ s).1 (cell b h t).1)) E))))

theorem cell_state_perturbation_at (a b : Point → Point) (h L E : Fraction) (s t : Point × Point) (hc :
  ↪ Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1)) (Fraction.add (Fraction.mul L
  ↪ (pointDistance (cell a h s).1 (cell b h t).1)) E)) : Fraction.le (stateDistance (cell a h s) (cell b
  ↪ h t)) (Fraction.add (Fraction.add (pointDistance s.1 t.1) (Fraction.mul h.abs (pointDistance s.2
  ↪ t.2))) (Fraction.add (pointDistance s.2 t.2) (Fraction.mul h.abs (Fraction.add (Fraction.mul L
  ↪ (pointDistance (cell a h s).1 (cell b h t).1)) E))))))

theorem cell_state_perturbation (a b : Point → Point) (h L E : Fraction) (s t : Point × Point) (hc :
  ↪ comparisonContract a b L E) : Fraction.le (stateDistance (cell a h s) (cell b h t)) (Fraction.add
  ↪ (Fraction.add (pointDistance s.1 t.1) (Fraction.mul h.abs (pointDistance s.2 t.2))) (Fraction.add
  ↪ (pointDistance s.2 t.2) (Fraction.mul h.abs (Fraction.add (Fraction.mul L (pointDistance (cell a h
  ↪ s).1 (cell b h t).1)) E))))))

theorem cell_state_perturbation_closed_at (a b : Point → Point) (h L E : Fraction) (s t : Point × Point)
  ↪ (hL : 0 ≤ L.num) (hc : Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1))
  ↪ (Fraction.add (Fraction.mul L (pointDistance (cell a h s).1 (cell b h t).1)) E)) : Fraction.le
  ↪ (stateDistance (cell a h s) (cell b h t)) (Fraction.add (Fraction.add (pointDistance s.1 t.1)
  ↪ (Fraction.mul h.abs (pointDistance s.2 t.2))) (Fraction.add (pointDistance s.2 t.2) (Fraction.mul
  ↪ h.abs (Fraction.add (Fraction.mul L (Fraction.add (pointDistance s.1 t.1) (Fraction.mul h.abs
  ↪ (pointDistance s.2 t.2)))) E))))))

theorem cell_state_perturbation_closed (a b : Point → Point) (h L E : Fraction) (s t : Point × Point) (hL :
  ↪ 0 ≤ L.num) (hc : comparisonContract a b L E) : Fraction.le (stateDistance (cell a h s) (cell b h
  ↪ t)) (Fraction.add (Fraction.add (pointDistance s.1 t.1) (Fraction.mul h.abs (pointDistance s.2 t.2)))
  ↪ (Fraction.add (pointDistance s.2 t.2) (Fraction.mul h.abs (Fraction.add (Fraction.mul L (Fraction.add
  ↪ (pointDistance s.1 t.1) (Fraction.mul h.abs (pointDistance s.2 t.2)))) E))))))

theorem cell_amplification_at (a b : Point → Point) (h L E : Fraction) (s t : Point × Point) (hL : 0 ≤
  ↪ L.num) (hc : Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1)) (Fraction.add
  ↪ (Fraction.mul L (pointDistance (cell a h s).1 (cell b h t).1)) E)) : Fraction.le (stateDistance (cell
  ↪ a h s) (cell b h t)) (Fraction.add (Fraction.mul (amplification L h) (stateDistance s t))
  ↪ (Fraction.mul h.abs E))

theorem cell_amplification (a b : Point → Point) (h L E : Fraction) (s t : Point × Point) (hL : 0 ≤
  ↪ L.num) (hc : comparisonContract a b L E) : Fraction.le (stateDistance (cell a h s) (cell b h t))
  ↪ (Fraction.add (Fraction.mul (amplification L h) (stateDistance s t)) (Fraction.mul h.abs E))

theorem twoHalf_position_identity (a : Point → Point) (h : Fraction) (s : Point × Point) : pointEquiv
  ↪ (twoHalf a h s).1 (pointAdd (oneFull a h s).1 (pointScale (Fraction.mul h h) (a (cell a h s).1)))

private theorem twoHalf_position_difference (a : Point → Point) (h : Fraction) (s : Point × Point) :
  ↪ pointEquiv (pointSub (twoHalf a h s).1 (oneFull a h s).1) (pointScale (Fraction.mul h h) (a (cell a h
  ↪ s).1))

theorem twoHalf_position_error (a : Point → Point) (h B : Fraction) (s : Point × Point) (hB : Fraction.le
  ↪ (pointNorm (a (cell a h s).1)) B) : Fraction.le (pointDistance (twoHalf a h s).1 (oneFull a h s).1)
  ↪ (Fraction.mul (Fraction.mul h h).abs B)

private theorem first_to_full_difference (a : Point → Point) (h : Fraction) (s : Point × Point) :
  ↪ pointEquiv (pointSub (cell a h s).1 (oneFull a h s).1) (pointScale h (pointNeg s.2))

theorem first_to_full_distance (a : Point → Point) (h : Fraction) (s : Point × Point) : Fraction.equiv
  ↪ (pointDistance (cell a h s).1 (oneFull a h s).1) (Fraction.mul h.abs (pointNorm s.2))

private theorem twoHalf_velocity_difference (a : Point → Point) (h : Fraction) (s : Point × Point) :
  ↪ pointEquiv (pointSub (twoHalf a h s).2 (oneFull a h s).2) (pointScale h (pointAdd (pointSub (a (cell
  ↪ a h s).1) (a (oneFull a h s).1)) (pointSub (a (twoHalf a h s).1) (a (oneFull a h s).1))))))

```

```

theorem twoHalf_velocity_sample_error (a : Point → Point) (h E1 E2 : Fraction) (s : Point × Point) (h1 :
  ↪ Fraction.le (pointDistance (a (cell a h s).1) (a (oneFull a h s).1)) E1) (h2 : Fraction.le
  ↪ (pointDistance (a (twoHalf a h s).1) (a (oneFull a h s).1)) E2) : Fraction.le (pointDistance (twoHalf
  ↪ a h s).2 (oneFull a h s).2) (Fraction.mul h.abs (Fraction.add E1 E2))

theorem twoHalf_velocity_error (a : Point → Point) (h L E : Fraction) (s : Point × Point) (hc :
  ↪ comparisonContract a a L E) : Fraction.le (pointDistance (twoHalf a h s).2 (oneFull a h s).2)
  ↪ (Fraction.mul h.abs (Fraction.add (Fraction.add (Fraction.mul L (pointDistance (cell a h s).1
  ↪ (oneFull a h s).1)) E) (Fraction.add (Fraction.mul L (pointDistance (twoHalf a h s).1 (oneFull a h
  ↪ s).1)) E)))

theorem twoHalf_velocity_error_closed_at (a : Point → Point) (h L E B : Fraction) (s : Point × Point) (hL :
  ↪ 0 ≤ L.num) (hB : Fraction.le (pointNorm (a (cell a h s).1)) B) (h1 : Fraction.le (pointDistance (a
  ↪ (cell a h s).1) (a (oneFull a h s).1)) (Fraction.add (Fraction.mul L (pointDistance (cell a h s).1
  ↪ (oneFull a h s).1)) E)) (h2 : Fraction.le (pointDistance (a (twoHalf a h s).1) (a (oneFull a h s).1))
  ↪ (Fraction.add (Fraction.mul L (pointDistance (twoHalf a h s).1 (oneFull a h s).1)) E)) : Fraction.le
  ↪ (pointDistance (twoHalf a h s).2 (oneFull a h s).2) (Fraction.mul h.abs (Fraction.add (Fraction.add
  ↪ (Fraction.mul L (Fraction.mul h.abs (pointNorm s.2))) E) (Fraction.add (Fraction.mul L (Fraction.mul
  ↪ (Fraction.mul h h).abs B)) E)))

theorem twoHalf_velocity_error_closed (a : Point → Point) (h L E B : Fraction) (s : Point × Point) (hL :
  ↪ 0 ≤ L.num) (hB : Fraction.le (pointNorm (a (cell a h s).1)) B) (hc : comparisonContract a a L E) :
  ↪ Fraction.le (pointDistance (twoHalf a h s).2 (oneFull a h s).2) (Fraction.mul h.abs (Fraction.add
  ↪ (Fraction.add (Fraction.mul L (Fraction.mul h.abs (pointNorm s.2))) E) (Fraction.add (Fraction.mul L
  ↪ (Fraction.mul (Fraction.mul h h).abs B)) E)))

theorem twoHalf_state_error (a : Point → Point) (h L E B : Fraction) (s : Point × Point) (hB : Fraction.le
  ↪ (pointNorm (a (cell a h s).1)) B) (hc : comparisonContract a a L E) : Fraction.le (stateDistance
  ↪ (twoHalf a h s) (oneFull a h s)) (Fraction.add (Fraction.mul (Fraction.mul h h).abs B) (Fraction.mul
  ↪ h.abs (Fraction.add (Fraction.add (Fraction.mul L (pointDistance (cell a h s).1 (oneFull a h s).1))
  ↪ E) (Fraction.add (Fraction.mul L (pointDistance (twoHalf a h s).1 (oneFull a h s).1)) E))))

theorem twoHalf_state_error_closed (a : Point → Point) (h L E B : Fraction) (s : Point × Point) (hL : 0 ≤
  ↪ L.num) (hB : Fraction.le (pointNorm (a (cell a h s).1)) B) (hc : comparisonContract a a L E) :
  ↪ Fraction.le (stateDistance (twoHalf a h s) (oneFull a h s)) (Fraction.add (Fraction.mul (Fraction.mul
  ↪ h h).abs B) (Fraction.mul h.abs (Fraction.add (Fraction.add (Fraction.mul L (Fraction.mul h h).abs
  ↪ (pointNorm s.2))) E) (Fraction.add (Fraction.mul L (Fraction.mul (Fraction.mul h h).abs B)) E))))

theorem zero_duration_cell (a : Point → Point) (h : Fraction) (hh : h.num = 0) (s : Point × Point) :
  ↪ stateEquiv (cell a h s) s

theorem linear_sample_control : stateEquiv (cell controlLinear controlEighth controlStart)
  ↪ ((Fraction.ofInt 1, controlEighth), ((-1, 8, by decide), (63, 64, by decide)))

theorem constant_sample_control : stateEquiv (cell controlA controlHalf controlState) ((controlZero,
  ↪ controlZero), (controlZero, (-1, 2, by decide)))

theorem unequal_sample_control : Fraction.equiv (stateDistance (cell controlA controlHalf controlState)
  ↪ (cell controlB controlHalf controlState)) controlHalf

theorem unequal_sample_control_nonzero : ¬ Fraction.le (stateDistance (cell controlA controlHalf
  ↪ controlState) (cell controlB controlHalf controlState)) controlZero

theorem zero_duration_control (a : Point → Point) (s : Point × Point) : stateEquiv (cell a controlZero s)
  ↪ s

```

BarrowLib/Polygon/FiniteFactorProducts.lean — Elementary repeated two-factor growth, with arbitrary rational increments. The common-denominator proof is shared by calibrated and unit-gauge estimates. No motion, limit or differential equation is a premise.

```

theorem pairWeights_nonnegative (P Q : Fraction) (hP : 0 ≤ P.num) (hQ : 0 ≤ Q.num) : (n : Nat) →
  ↪ FiniteGrowth.Nonnegative (pairWeights P Q n)

theorem pairWeights_sum (P Q : Fraction) : (n : Nat) → FiniteGrowth.weightSum (pairWeights P Q n) = (n :
  ↪ Int) * (P.num * Q.den + Q.num * P.den)

```

```

theorem pair_product (P Q : Fraction) : Fraction.equiv (FiniteGrowth.amplification (P.den * Q.den)
  ↪ (Int.mul_pos P.den_pos Q.den_pos) [P.num * Q.den, Q.num * P.den]) (Fraction.mul (Fraction.add
  ↪ (Fraction.ofInt 1) P) (Fraction.add (Fraction.ofInt 1) Q))

theorem repeated_pair_product (P Q : Fraction) : (n : Nat) → Fraction.equiv (FiniteGrowth.amplification
  ↪ (P.den * Q.den) (Int.mul_pos P.den_pos Q.den_pos) (pairWeights P Q n)) (fpower (Fraction.mul
  ↪ (Fraction.add (Fraction.ofInt 1) P) (Fraction.add (Fraction.ofInt 1) Q)) n)

theorem repeated_pair_le_two (P Q : Fraction) (n : Nat) (hP : 0 ≤ P.num) (hQ : 0 ≤ Q.num) (hs :
  ↪ Fraction.le (Fraction.mul (Fraction.ofInt (n : Int)) (Fraction.add P Q)) (1,2,by decide)) :
  ↪ Fraction.le (fpower (Fraction.mul (Fraction.add (Fraction.ofInt 1) P) (Fraction.add (Fraction.ofInt
  ↪ 1) Q)) n) (Fraction.ofInt 2)

theorem fpower_congr {r q : Fraction} (hrq : Fraction.equiv r q) : (n : Nat) → Fraction.equiv (fpower r
  ↪ n) (fpower q n)

theorem fpower_square (r : Fraction) : (n : Nat) → Fraction.equiv (fpower (Fraction.mul r r) n) (fpower r
  ↪ (2*n))

```

BarrowLib/Polygon/FinitePower.lean

```

theorem fpower_nonnegative (a : Fraction) (ha : 0 ≤ a.num) : (n : Nat) → 0 ≤ (fpower a n).num

```

BarrowLib/Polygon/FiniteRecurrence.lean — Finite rational recurrence with a constant nonnegative source.

```

theorem factorPower_fpower (r : Fraction) : (n : Nat) → Fraction.equiv (FiniteAccumulation.factorPower r
  ↪ n) (fpower r n)

theorem one_le_fpower (r : Fraction) (hr : 0 ≤ r.num) (hone : Fraction.le (Fraction.ofInt 1) r) (n : Nat)
  ↪ : Fraction.le (Fraction.ofInt 1) (fpower r n)

theorem fpower_add (r : Fraction) (m : Nat) : (n : Nat) → Fraction.equiv (fpower r (m + n)) (Fraction.mul
  ↪ (fpower r m) (fpower r n))

theorem fpower_congr {r q : Fraction} (hrq : Fraction.equiv r q) : (n : Nat) → Fraction.equiv (fpower r
  ↪ n) (fpower q n)

theorem fpower_integer_blocks (r : Fraction) (k : Nat) : (N : Nat) → Fraction.equiv (fpower (fpower r k)
  ↪ N) (fpower r (k * N))

theorem fpower_prefix_le (r : Fraction) (hr : 0 ≤ r.num) (hone : Fraction.le (Fraction.ofInt 1) r) (i N :
  ↪ Nat) (hi : i ≤ N) : Fraction.le (fpower r i) (fpower r N)

theorem sourceBudget_nonnegative (r C : Fraction) (hr : 0 ≤ r.num) (hC : 0 ≤ C.num) : (n : Nat) → 0 ≤
  ↪ (sourceBudget r C n).num

theorem sourceBudget_power (r C : Fraction) (hr : 0 ≤ r.num) (hC : 0 ≤ C.num) (hone : Fraction.le
  ↪ (Fraction.ofInt 1) r) : (n : Nat) → Fraction.le (sourceBudget r C n) (Fraction.mul (Fraction.ofInt (n
  ↪ : Int)) (Fraction.mul C (fpower r n)))

theorem sourceBudget_two_count (r C : Fraction) (n : Nat) (hr : 0 ≤ r.num) (hC : 0 ≤ C.num) (hone :
  ↪ Fraction.le (Fraction.ofInt 1) r) (hp : Fraction.le (fpower r n) (Fraction.ofInt 2)) : Fraction.le
  ↪ (sourceBudget r C n) (Fraction.mul (Fraction.ofInt (2*(n : Int))) C)

```

BarrowLib/Polygon/FiniteSequenceGap.lean — A finite telescoping estimate for any rational-valued distance with a zero diagonal and triangle inequality. No map or limiting premise is used.

```

theorem finite_gap (D : (Point × Point) → (Point × Point) → Fraction) (hz : ∀ s, Fraction.equiv (D s s)
  ↪ (Fraction.ofInt 0)) (htri : ∀ s t u, Fraction.le (D s u) (Fraction.add (D s t) (D t u))) (f : Nat →
  ↪ Point × Point) (N : Nat) (C : Fraction) (hstep : ∀ i, i < N → Fraction.le (D (f (i+1)) (f i)) C) : (n
  ↪ k : Nat) → n+k ≤ N → Fraction.le (D (f (n+k)) (f n)) (Fraction.mul (Fraction.ofInt (k : Int)) C)

```

BarrowLib/Polygon/GeometricTail.lean — Rational geometric-tail estimates for an arbitrary sequence of states. The coefficient and adjacent-error premise are supplied separately.

```

theorem tail_halving (A : Fraction) (j : Nat) : Fraction.equiv (Fraction.add (tailCap A (j + 1)) (tailCap
  ↪ A (j + 1))) (tailCap A j)

theorem tail_double (A : Fraction) (j : Nat) : Fraction.equiv (Fraction.add (tailCap A j) (tailCap A j))
  ↪ (doubleTail A j)

theorem tail_add (A B : Fraction) (n : Nat) : Fraction.equiv (Fraction.add (tailCap A n) (tailCap B n))
  ↪ (tailCap (Fraction.add A B) n)

theorem finite_gap (x : Nat → Point × Point) (A : Fraction) (hA : 0 ≤ A.num) (hadj : ∀ j, Fraction.le
  ↪ (stateDistance (x (j + 1)) (x j)) (tailCap A (j + 1))) : (k j : Nat) → Fraction.le (stateDistance (x
  ↪ (j + k)) (x j)) (tailCap A j)

theorem two_sided (x : Nat → Point × Point) (A : Fraction) (hA : 0 ≤ A.num) (hadj : ∀ j, Fraction.le
  ↪ (stateDistance (x (j + 1)) (x j)) (tailCap A (j + 1))) (N m n : Nat) (hm : N ≤ m) (hn : N ≤ n) :
  ↪ Fraction.le (stateDistance (x m) (x n)) (doubleTail A N)

theorem doubleTail_lt_tolerance (A eps : Fraction) (hA : 0 ≤ A.num) (heps : 0 < eps.num) : Fraction.lt
  ↪ (doubleTail A (modulus A eps)) eps

theorem duration_eventually_small (T eps : Fraction) (hT : 0 ≤ T.num) (heps : 0 < eps.num) : ∃ N : Nat, ∀
  ↪ j : Nat, N ≤ j → Fraction.lt (duration T j) eps

```

BarrowLib/Polygon/IntegerRefinement.lean — Elementary binary tick arithmetic for a finite address followed by zero bits.

```

theorem ticks_zero_tail (b : Nat → Bool) (m : Nat) (hz : ∀ j, m ≤ j → b j = false) : (j : Nat) → ticks b
  ↪ (m + j) = ticks b m * blocks j

theorem duration_nested (T : Fraction) (m j : Nat) : Fraction.equiv (duration (duration T m) j) (duration
  ↪ T (m + j))

```

BarrowLib/Polygon/IntegerSchedule.lean — Elementary rational duration arithmetic for an integer number of cells.

```

theorem integerDuration_closed (h : Fraction) : (k : Nat) → Fraction.equiv (integerDuration h k)
  ↪ (Fraction.mul (Fraction.ofInt (k : Int)) h)

theorem integerDuration_nonnegative (h : Fraction) (hh : 0 ≤ h.num) : (k : Nat) → 0 ≤ (integerDuration h
  ↪ k).num

theorem integerDuration_le (h : Fraction) (hh : 0 ≤ h.num) (i k : Nat) (hik : i ≤ k) : Fraction.le
  ↪ (integerDuration h i) (integerDuration h k)

theorem half_totalTime (d : Fraction) (N : Nat) : Fraction.equiv (totalTime d.half N) (fullTime d N)

theorem integer_fullTime (h : Fraction) (k N : Nat) : Fraction.equiv (fullTime h (k * N)) (fullTime
  ↪ (integerDuration h k) N)

```

BarrowLib/Polygon/IntegerTime.lean — Rational binary-time and integer-duration bridges. Retained namespaces are compatibility names; no harmonic field or mechanical premise occurs.

```

theorem dyadic_integer_duration (b : Nat → Bool) (T : Fraction) (m j : Nat) : Fraction.equiv
  ↪ (integerDuration (duration T (m+j)) (ticks b m)) (duration (timeApprox b T m) j)

theorem dyadic_fullTime (b : Nat → Bool) (T : Fraction) (m j : Nat) : Fraction.equiv (fullTime
  ↪ (integerDuration (duration T (m+j)) (ticks b m)) (blocks j)) (timeApprox b T m)

theorem timeApprox_nonnegative (b : Nat → Bool) (T : Fraction) (m : Nat) (hT : 0 ≤ T.num) : 0 ≤
  ↪ (timeApprox b T m).num

```

BarrowLib/Polygon/KinematicEstimates.lean — Finite drift/kick remainders at actual sampled arrivals. The $O(t^2)$ position remainder and $O(t)$ velocity change are derived from the recurrence; no derivative, curve, integral or ODE theorem is a premise.

```

theorem inertial_zero (h : Fraction) (s : Point × Point) : pointEquiv (inertialPosition h s 0) s.1

```

```

theorem inertial_step (h : Fraction) (s : Point × Point) (n : Nat) : pointEquiv (inertialPosition h s
  ↪ (n+1)) (pointAdd (inertialPosition h s n) (pointScale h s.2))

theorem velocity_increment (a : Point → Point) (h : Fraction) (s : Point × Point) : Fraction.equiv
  ↪ (pointDistance (cell a h s).2 s.2) (Fraction.mul h.abs (pointNorm (a (cell a h s).1)))

theorem velocity_displacement (a : Point → Point) (h : Fraction) (s : Point × Point) (B : Fraction) (hh :
  ↪ 0 ≤ h.num) (n : Nat) (hb : BoundedSamples a h s B n) : Fraction.le (pointDistance (run a h s n).2
  ↪ s.2) (Fraction.mul (time h n) B)

theorem quadratic_step (h B : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat) : Fraction.le
  ↪ (Fraction.add (Fraction.mul (Fraction.mul (time h n) (time h n)) B) (Fraction.mul h (Fraction.mul
  ↪ (time h n) B))) (Fraction.mul (Fraction.mul (time h (n+1)) (time h (n+1))) B)

theorem position_remainder (a : Point → Point) (h : Fraction) (s : Point × Point) (B : Fraction) (hh : 0
  ↪ ≤ h.num) (hB : 0 ≤ B.num) (n : Nat) (hb : BoundedSamples a h s B n) : Fraction.le (pointDistance (run
  ↪ a h s n).1 (inertialPosition h s n)) (Fraction.mul (Fraction.mul (time h n) (time h n)) B)

theorem secant_identity (t : Fraction) (ht : 0 < t.num) (p x v : Point) : pointEquiv (pointSub
  ↪ (pointScale (TimeCalibration.inverse t ht) (pointSub p x)) v) (pointScale (TimeCalibration.inverse t
  ↪ ht) (pointSub p (pointAdd x (pointScale t v))))

theorem position_secant_bound (a : Point → Point) (h : Fraction) (s : Point × Point) (B : Fraction) (hh :
  ↪ 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat) (ht : 0 < (time h n).num) (hb : BoundedSamples a h s B n) :
  ↪ Fraction.le (pointDistance (pointScale (TimeCalibration.inverse (time h n) ht) (pointSub (run a h s
  ↪ n).1 s.1)) s.2) (Fraction.mul (time h n) B)

theorem position_secant_equivalent_time (a : Point → Point) (h : Fraction) (s : Point × Point) (B u :
  ↪ Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat) (ht : 0 < (time h n).num) (hu : 0 < u.num) (he :
  ↪ Fraction.equiv u (time h n)) (hb : BoundedSamples a h s B n) : Fraction.le (pointDistance (pointScale
  ↪ (TimeCalibration.inverse u hu) (pointSub (run a h s n).1 s.1)) s.2) (Fraction.mul u B)

theorem two_cell_secant_control : Fraction.equiv (pointDistance (pointScale (TimeCalibration.inverse
  ↪ (time controlHalf 2) (by decide)) (pointSub (run controlForce controlHalf controlState 2).1
  ↪ controlState.1)) controlState.2) {1,4,by decide}

theorem zero_bound_rejects_secant_control : ~ Fraction.le (pointDistance (pointScale
  ↪ (TimeCalibration.inverse (time controlHalf 2) (by decide)) (pointSub (run controlForce controlHalf
  ↪ controlState 2).1 controlState.1)) controlState.2) (Fraction.ofInt 0)

```

BarrowLib/Polygon/MatchedRegion.lean — Unsigned matched regions for two maps on the constructed binary-time domain. Each dyadic cell closes all simultaneous rational connectors; their finite union counts overlaps once. Closed square enclosures and geometric cover budgets are independent of the construction of either map.

```

theorem cellPatch_square (T : Fraction) (hT : 0 ≤ T.num) (p g : BinaryTime T hT → PositionValue) (m :
  ↪ Nat) (centres : Nat → Point) (R : NonnegativeRadius) (h : ∀ b : Nat → Bool, CoordinateSquare (centres
  ↪ (ticks b m)) R (p (Quotient.mk _ b)) ∧ CoordinateSquare (centres (ticks b m)) R (g (Quotient.mk _
  ↪ b))) (k : Nat) (x : PositionValue) (hx : cellPatch T hT p g m k x) : CoordinateSquare (centres k) R x

theorem closed_cell_square (T : Fraction) (hT : 0 ≤ T.num) (p g : BinaryTime T hT → PositionValue) (m :
  ↪ Nat) (centres : Nat → Point) (R : NonnegativeRadius) (h : ∀ b : Nat → Bool, CoordinateSquare (centres
  ↪ (ticks b m)) R (p (Quotient.mk _ b)) ∧ CoordinateSquare (centres (ticks b m)) R (g (Quotient.mk _
  ↪ b))) (k : Nat) (x : PositionValue) (hx : Closure (cellPatch T hT p g m k) x) : CoordinateSquare
  ↪ (centres k) R x

theorem connector_in_region (T : Fraction) (hT : 0 ≤ T.num) (p g : BinaryTime T hT → PositionValue) (m :
  ↪ Nat) (t : BinaryTime T hT) (a : Fraction) (ha : UnitInterval a) : Region T hT p g m (convexPosition a
  ↪ ha (p t) (g t))

theorem reversed_connector_in_region (T : Fraction) (hT : 0 ≤ T.num) (p g : BinaryTime T hT →
  ↪ PositionValue) (m : Nat) (t : BinaryTime T hT) (a : Fraction) (ha : UnitInterval a) : Region T hT p g
  ↪ m (convexPosition a ha (g t) (p t))

theorem polygon_in_region (T : Fraction) (hT : 0 ≤ T.num) (p g : BinaryTime T hT → PositionValue) (m :
  ↪ Nat) (t : BinaryTime T hT) : Region T hT p g m (p t)

theorem curve_in_region (T : Fraction) (hT : 0 ≤ T.num) (p g : BinaryTime T hT → PositionValue) (m : Nat)
  ↪ (t : BinaryTime T hT) : Region T hT p g m (g t)

```

```

theorem uniform_budget_geometric (centres : Nat → Point) (R : NonnegativeRadius) (m : Nat) (C : Fraction)
  ↳ (hr : Fraction.equiv R.val (duration C m)) : Fraction.equiv (sumBudget (fun k => (centres k,R)))
  ↳ (blocks m)) (duration (squareArea C) m)

```

BarrowLib/Polygon/PairingValues.lean — Dot products and determinants of completed positions, with one shared Cauchy and representative-independence proof. Finite bilinear differences and magnitude bounds are the input; bounded Cauchy tails are derived. SampledValues covers globally Lipschitz maps, whereas these pairings are Lipschitz only on bounded sets. No integral or derivative is a primitive.

```

theorem dot_abs_le_product (p q : Point) : Fraction.le (dot p q).abs (Fraction.mul (pointNorm p)
  ↳ (pointNorm q))

theorem dot_congr {p p' q q' : Point} (hp : pointEquiv p p') (hq : pointEquiv q q') : Fraction.equiv (dot
  ↳ p q) (dot p' q')

theorem difference_bound (f : Form) (p q r s : Point) : Fraction.le (durationDifference (f.apply p q)
  ↳ (f.apply r s)).abs (Fraction.add (Fraction.mul (pointDistance p r) (pointNorm q)) (Fraction.mul
  ↳ (pointNorm r) (pointDistance q s)))

theorem pairing_state_bound (f : Form) (R : Fraction) (hR : 0 ≤ R.num) (s t u v : Point × Point) (hu :
  ↳ Fraction.le (pointNorm u.1) R) (ht : Fraction.le (pointNorm t.1) R) : Fraction.le (distance
  ↳ (pairingState f s t) (pairingState f u v)) (Fraction.mul (Fraction.add (distance s u) (distance t v))
  ↳ R)

theorem fixed_left_bound (f : Form) (s t u : Point × Point) : Fraction.le (distance (pairingState f s t)
  ↳ (pairingState f s u)) (Fraction.mul (distance t u) (pointNorm s.1))

private theorem pairing_small (f : Form) (R eps : Fraction) (hR : 0 ≤ R.num) (s t u v : Point × Point)
  ↳ (hu : Fraction.le (pointNorm u.1) R) (ht : Fraction.le (pointNorm t.1) R) (hsu : Fraction.lt
  ↳ (distance s u) (factorDelta R eps.half hR)) (htv : Fraction.lt (distance t v) (factorDelta R eps.half
  ↳ hR)) : Fraction.lt (distance (pairingState f s t) (pairingState f u v)) eps

theorem pairingName_equiv (f : Form) (a b a' b' : EndpointCauchyName) (ha : NameEquiv a a') (hb :
  ↳ NameEquiv b b') : NameEquiv (pairingName f a b) (pairingName f a' b')

theorem pairingValue_embed (f : Form) (s t : Point × Point) : pairingValue f (embed s) (embed t) = embed
  ↳ (pairingState f s t)

theorem pairingValue_scalar (f : Form) (x y : Value) : PositionValues.firstValue (pairingValue f x y) =
  ↳ pairingValue f x y

theorem scaled_pairing_approximant (f : Form) (c : Fraction) (a b : EndpointCauchyName) (j : Nat) :
  ↳ stateEquiv ((secantName c (pairingName f a b) (constantName
  ↳ (PositionValues.zeroPoint, PositionValues.zeroPoint))).approx j) (scalarState (Fraction.mul c
  ↳ (f.apply (a.approx j).1 (b.approx j).1)))

theorem pairing_name_bound_right (f : Form) (a b c : EndpointCauchyName) (R S : Fraction) (hR : 0 ≤
  ↳ R.num) (M : Nat) (ha : ∀ j, M ≤ j → Fraction.le (pointNorm (a.approx j).1) R) (hbc : NameBound b c S) :
  ↳ NameBound (pairingName f a b) (pairingName f a c) (Fraction.mul S R)

```

BarrowLib/Polygon/PointAlgebra.lean

```

theorem pointEquiv_symm {p q : Point} (h : pointEquiv p q) : pointEquiv q p

theorem pointEquiv_trans {p q r : Point} (h : pointEquiv p q) (k : pointEquiv q r) : pointEquiv p r

theorem pointAdd_congr {p p' q q' : Point} (hp : pointEquiv p p') (hq : pointEquiv q q') : pointEquiv
  ↳ (pointAdd p q) (pointAdd p' q')

theorem pointScale_congr (d : Fraction) {p q : Point} (h : pointEquiv p q) : pointEquiv (pointScale d p)
  ↳ (pointScale d q)

theorem pointScale_ratio_congr {d e : Fraction} {p q : Point} (hde : Fraction.equiv d e) (hpq :
  ↳ pointEquiv p q) : pointEquiv (pointScale d p) (pointScale e q)

theorem pointNeg_congr {p q : Point} (hpq : pointEquiv p q) : pointEquiv (pointNeg p) (pointNeg q)

```

```
theorem pointSub_congr {p p' q q' : Point} (hpp' : pointEquiv p p') (hqq' : pointEquiv q q') : pointEquiv
  ↪ (pointSub p q) (pointSub p' q')
```

```
theorem pointSub_zero (p : Point) : pointEquiv (pointSub p (Fraction.ofInt 0, Fraction.ofInt 0)) p
```

```
theorem det_add_right (x v w : Point) : Fraction.equiv (det x (pointAdd v w)) (Fraction.add (det x v)
  ↪ (det x w))
```

BarrowLib/Polygon/PointBounds.lean — Coordinate L1 magnitudes for finite rational polygon data. These diagnostics depend on the chosen coordinates; they are neither Euclidean area nor a physical position/velocity norm without a calibration of their units.

```
theorem pointNorm_nonnegative (p : Point) : 0 ≤ (pointNorm p).num
```

```
theorem stateNorm_nonnegative (s : Point × Point) : 0 ≤ (stateNorm s).num
```

```
theorem pointNorm_equiv {p q : Point} (h : pointEquiv p q) : Fraction.equiv (pointNorm p) (pointNorm q)
```

```
theorem pointNorm_neg (p : Point) : Fraction.equiv (pointNorm (pointNeg p)) (pointNorm p)
```

```
theorem stateNorm_equiv {s t : Point × Point} (h : stateEquiv s t) : Fraction.equiv (stateNorm s)
  ↪ (stateNorm t)
```

```
theorem pointNorm_add_le (p q : Point) : Fraction.le (pointNorm (pointAdd p q)) (Fraction.add (pointNorm
  ↪ p) (pointNorm q))
```

```
theorem pointNorm_scale (d : Fraction) (p : Point) : Fraction.equiv (pointNorm (pointScale d p))
  ↪ (Fraction.mul d.abs (pointNorm p))
```

```
theorem stateNorm_add_le (s t : Point × Point) : Fraction.le (stateNorm (pointAdd s.1 t.1, pointAdd s.2
  ↪ t.2)) (Fraction.add (stateNorm s) (stateNorm t))
```

```
theorem stateNorm_scale (d : Fraction) (s : Point × Point) : Fraction.equiv (stateNorm (pointScale d s.1,
  ↪ pointScale d s.2)) (Fraction.mul d.abs (stateNorm s))
```

```
theorem point_le_state (s : Point × Point) : Fraction.le (pointNorm s.1) (stateNorm s)
```

```
theorem velocity_le_state (s : Point × Point) : Fraction.le (pointNorm s.2) (stateNorm s)
```

BarrowLib/Polygon/PolygonValues.lean — A polygon on the constructed binary-time domain from actual finite vertices with proved position joining. Alias invariance, zero-time values and affine vertex bounds are shared independently of the map producing vertices. There is no supplied curve, motion equation or convergence field.

```
theorem polygon_same_cell (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) (v : VertexChain T
  ↪ m) (hcell : ticks b m = ticks c m) (hTtime : AddressEquiv T hT b c) : polygonPosition b T hT m v =
  ↪ polygonPosition c T hT m v
```

```
theorem polygon_adjacent_cells (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) (v :
  ↪ VertexChain T m) (hcell : ticks b m + 1 = ticks c m) (hTtime : AddressEquiv T hT b c) : polygonPosition b
  ↪ T hT m v = polygonPosition c T hT m v
```

```
theorem zero_window_vertices (T : Fraction) (m : Nat) (v : VertexChain T m) (hz : T.num = 0) : ∀ k,
  ↪ pointEquiv (v.state k).1 (v.state 0).1
```

```
theorem polygon_zero_window (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) (v : VertexChain T
  ↪ m) (hz : T.num = 0) : polygonPosition b T hT m v = asPosition (embed ((v.state
  ↪ 0).1, AffineValues.zeroPoint))
```

```
theorem polygon_address_independent (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) (v :
  ↪ VertexChain T m) (hTtime : AddressEquiv T hT b c) : polygonPosition b T hT m v = polygonPosition c T
  ↪ hT m v
```

```
theorem polygon_vertex_bound (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) (v : VertexChain T
  ↪ m) (V : Fraction) (hv : Fraction.le (pointNorm (v.state (ticks b m)).2) V) : Within (polygonPosition
  ↪ b T hT m v).val (positionValue (embed (v.state (ticks b m)))) (Fraction.mul (duration T m) V)
```

```

theorem polygonMap_left (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) (v : VertexChain T m) : polygonMap T hT
  ↪ m v (leftTime T hT) = embedPosition (v.state 0).1

```

BarrowLib/Polygon/PositionValues.lean

```

private theorem distance_decompose (a b : Point × Point) : distance a b = Fraction.add (Fraction.add
  ↪ (xGap a b) (yGap a b)) (Fraction.add (vxGap a b) (vyGap a b))

private theorem projected_distance (a b : Point × Point) : Fraction.equiv (distance (positionState a)
  ↪ (positionState b)) (Fraction.add (xGap a b) (yGap a b))

private theorem first_distance (a b : Point × Point) : Fraction.equiv (distance (firstState a)
  ↪ (firstState b)) (xGap a b)

private theorem second_distance (a b : Point × Point) : Fraction.equiv (distance (secondState a)
  ↪ (secondState b)) (yGap a b)

theorem position_nonexpansive (a b : Point × Point) : Fraction.le (distance (positionState a)
  ↪ (positionState b)) (distance a b)

theorem first_nonexpansive (a b : Point × Point) : Fraction.le (distance (firstState a) (firstState b))
  ↪ (distance a b)

theorem second_nonexpansive (a b : Point × Point) : Fraction.le (distance (secondState a) (secondState
  ↪ b)) (distance a b)

theorem mapName_equiv (f : Point × Point → Point × Point) (hLip : ∀ a b, Fraction.le (distance (f a) (f
  ↪ b)) (distance a b)) {a b : EndpointCauchyName} (hab : NameEquiv a b) : NameEquiv (mapName f hLip a)
  ↪ (mapName f hLip b)

theorem mapValue_embed (f : Point × Point → Point × Point) (hLip : ∀ a b, Fraction.le (distance (f a) (f
  ↪ b)) (distance a b)) (s : Point × Point) : mapValue f hLip (embed s) = embed (f s)

theorem mapName_bound (f : Point × Point → Point × Point) (hLip : ∀ a b, Fraction.le (distance (f a) (f
  ↪ b)) (distance a b)) (a b : EndpointCauchyName) (R : Fraction) (h : NameBound a b R) : NameBound
  ↪ (mapName f hLip a) (mapName f hLip b) R

theorem mapValue_within (f : Point × Point → Point × Point) (hLip : ∀ a b, Fraction.le (distance (f a) (f
  ↪ b)) (distance a b)) (x y : Value) (R : Fraction) (h : Within x y R) : Within (mapValue f hLip x)
  ↪ (mapValue f hLip y) R

theorem positionValue_embed (s : Point × Point) : positionValue (embed s) = embed (positionState s)

theorem firstValue_embed (s : Point × Point) : firstValue (embed s) = embed (firstState s)

theorem secondValue_embed (s : Point × Point) : secondValue (embed s) = embed (secondState s)

theorem positionValue_idempotent (v : Value) : positionValue (positionValue v) = positionValue v

theorem positionValue_within (x y : Value) (R : Fraction) (h : Within x y R) : Within (positionValue x)
  ↪ (positionValue y) R

theorem firstValue_within (x y : Value) (R : Fraction) (h : Within x y R) : Within (firstValue x)
  ↪ (firstValue y) R

theorem secondValue_within (x y : Value) (R : Fraction) (h : Within x y R) : Within (secondValue x)
  ↪ (secondValue y) R

theorem firstValue_positionValue (v : Value) : firstValue (positionValue v) = firstValue v

theorem secondValue_positionValue (v : Value) : secondValue (positionValue v) = secondValue v

theorem square_of_eventual_coordinate_bounds (a : EndpointCauchyName) (centre : Point) (radius :
  ↪ NonnegativeRadius) (N : Nat) (hx : ∀ n : Nat, N ≤ n → Fraction.le (distance (firstState (a.approx n))
  ↪ (firstState (centre, zeroPoint))) radius.val) (hy : ∀ n : Nat, N ≤ n → Fraction.le (distance
  ↪ (secondState (a.approx n)) (secondState (centre, zeroPoint))) radius.val) : CoordinateSquare centre
  ↪ radius (asPosition (realize a))

```

```

theorem square_of_rational_coordinate_bounds (p centre : Point) (radius : NonnegativeRadius) (hx :
  ↪ Fraction.le (distance (firstState (p, zeroPoint)) (firstState (centre, zeroPoint))) radius.val) (hy :
  ↪ Fraction.le (distance (secondState (p, zeroPoint)) (secondState (centre, zeroPoint))) radius.val) :
  ↪ CoordinateSquare centre radius (embedPosition p)

```

BarrowLib/Polygon/QuadraticEstimates.lean — Exact finite quadratic comparison for drift-then-kick iteration. The constant-map position coefficient is $t^*(t-h)/2$, rather than $t^2/2$. Variable maps have a derived cubic/error remainder; the remaining half-mesh bias is explicit. No curve, derivative or integral is a premise.

```

theorem quadratic_time_congr (t u : Fraction) (s : Point × Point) (a : Point) (he : Fraction.equiv t u) :
  ↪ pointEquiv (quadraticPosition t s a) (quadraticPosition u s a)

```

```

theorem discrete_zero (h : Fraction) (s : Point × Point) (a : Point) : pointEquiv (discretePosition h s a
  ↪ 0) s.1

```

```

theorem discrete_step (h : Fraction) (s : Point × Point) (a : Point) (n : Nat) : pointEquiv
  ↪ (discretePosition h s a (n+1)) (pointAdd (discretePosition h s a n) (pointScale h (inertialPosition h
  ↪ (s.2,a) n)))

```

```

theorem constant_run_formula (a : Point) (h : Fraction) (s : Point × Point) (n : Nat) : stateEquiv (run
  ↪ (fun _ => a) h s n) (discretePosition h s a n, inertialPosition h (s.2,a) n)

```

```

theorem position_remainder_from_samples (a : Point → Point) (h : Fraction) (s : Point × Point) (C :
  ↪ Fraction) (hh : 0 ≤ h.num) (hC : 0 ≤ C.num) (N : Nat) (hs : ∀ i, i < N → Fraction.le (pointDistance (a
  ↪ (run a h s (i+1)).1) (a s.1)) C) : ∀ n, n ≤ N → Fraction.le (pointDistance (run a h s n).1
  ↪ (discretePosition h s (a s.1) n)) (Fraction.mul (Fraction.mul (time h n) (time h n)) C)

```

```

theorem position_remainder_at (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V : Fraction)
  ↪ (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num) (n : Nat) (hc : ∀ k, k < n →
  ↪ Fraction.le (pointDistance (a (run a h s (k+1)).1) (a s.1)) (Fraction.add (Fraction.mul L
  ↪ (pointDistance (run a h s (k+1)).1 s.1) E)) (hV : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2)
  ↪ V) : Fraction.le (pointDistance (run a h s n).1 (discretePosition h s (a s.1) n)) (Fraction.mul
  ↪ (Fraction.mul (time h n) (time h n)) (source L E (time h n) V))

```

```

theorem position_remainder (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V : Fraction) (hh
  ↪ : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num) (hc : comparisonContract a a L E) (n :
  ↪ Nat) (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le (pointDistance (run a
  ↪ h s n).1 (discretePosition h s (a s.1) n)) (Fraction.mul (Fraction.mul (time h n) (time h n)) (source
  ↪ L E (time h n) V))

```

```

theorem discrete_quadratic_offset (h : Fraction) (s : Point × Point) (a : Point) (n : Nat) : pointEquiv
  ↪ (pointSub (quadraticPosition (time h n) s a) (discretePosition h s a n)) (pointScale (Fraction.mul
  ↪ (time h n) h).half a)

```

```

theorem half_mesh_bias (h : Fraction) (s : Point × Point) (a : Point) (hh : 0 ≤ h.num) (n : Nat) :
  ↪ Fraction.equiv (pointDistance (discretePosition h s a n) (quadraticPosition (time h n) s a))
  ↪ (Fraction.mul (Fraction.mul (time h n) h).half (pointNorm a))

```

```

theorem position_quadratic_remainder_at (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V :
  ↪ Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num) (n : Nat) (hc : ∀ k, k < n
  ↪ → Fraction.le (pointDistance (a (run a h s (k+1)).1) (a s.1)) (Fraction.add (Fraction.mul L
  ↪ (pointDistance (run a h s (k+1)).1 s.1) E)) (hV : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2)
  ↪ V) : Fraction.le (pointDistance (run a h s n).1 (quadraticPosition (time h n) s (a s.1)))
  ↪ (Fraction.add (Fraction.mul (Fraction.mul (time h n) (time h n)) (source L E (time h n) V))
  ↪ (Fraction.mul (Fraction.mul (time h n) h).half (pointNorm (a s.1))))

```

```

theorem position_quadratic_remainder (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V :
  ↪ Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num) (hc :
  ↪ comparisonContract a a L E) (n : Nat) (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  ↪ Fraction.le (pointDistance (run a h s n).1 (quadraticPosition (time h n) s (a s.1))) (Fraction.add
  ↪ (Fraction.mul (Fraction.mul (time h n) (time h n)) (source L E (time h n) V)) (Fraction.mul
  ↪ (Fraction.mul (time h n) h).half (pointNorm (a s.1))))

```

```

theorem two_cell_half_mesh_control : Fraction.equiv (pointDistance (run (fun _ => controlForce)
  ↪ controlHalf controlState 2).1 (quadraticPosition (time controlHalf 2) controlState controlForce))
  ↪ {1,4,by decide}

```

```
theorem exact_quadratic_rejects_finite_control : ¬ pointEquiv (run (fun _ => controlForce) controlHalf
  ↪ controlState 2).1 (quadraticPosition (time controlHalf 2) controlState controlForce)
```

BarrowLib/Polygon/QuadraticPotentialValues.lean — Quadratic scalar values and their normalized tangent-continuation increments. The polynomial remainder is finite algebra, and its Cauchy completion uses the shared pairing and secant operators. No potential-force relation or desired leading asymptotic is supplied as a premise.

```
theorem quadraticValue_embed (c : Fraction) (s : Point × Point) : quadraticValue c (embed s) = embed
  ↪ (scalarState (quadratic c s.1))
```

```
theorem normalized_remainder_identity (c h : Fraction) (ht : 0 < h.num) (s u : Point × Point) (a : Point)
  ↪ : Fraction.equiv (durationDifference (Fraction.mul c (dot a s.1)) (normalizedStep c h ht s u))
  ↪ (Fraction.mul c (Fraction.add (dot (pointSub (QuadraticSecants.secondState h ht s u).1 a) s.1)
  ↪ (Fraction.add (Fraction.mul h (dot (QuadraticSecants.secondState h ht s u).1 s.2)) (Fraction.mul
  ↪ (Fraction.mul h h).half.half (dot (QuadraticSecants.secondState h ht s u).1
  ↪ (QuadraticSecants.secondState h ht s u).1))))))
```

```
theorem normalized_remainder_bound (c h P V Z D : Fraction) (ht : 0 < h.num) (s u : Point × Point) (a :
  ↪ Point) (hP : 0 ≤ P.num) (hV : 0 ≤ V.num) (hZ : 0 ≤ Z.num) (hp : Fraction.le (pointNorm s.1) P) (hv :
  ↪ Fraction.le (pointNorm s.2) V) (hz : Fraction.le (pointNorm (QuadraticSecants.secondState h ht s
  ↪ u).1) Z) (hd : Fraction.le (pointDistance (QuadraticSecants.secondState h ht s u).1 a) D) :
  ↪ Fraction.le (durationDifference (Fraction.mul c (dot a s.1)) (normalizedStep c h ht s u)).abs
  ↪ (Fraction.mul c.abs (Fraction.add (Fraction.mul D P) (Fraction.add (Fraction.mul h (Fraction.mul Z
  ↪ V)) (Fraction.mul (Fraction.mul h h).half.half (Fraction.mul Z Z))))))
```

```
theorem remainderCoefficient_nonnegative (c T P V Z U : Fraction) (hT : 0 ≤ T.num) (hP : 0 ≤ P.num) (hV :
  ↪ 0 ≤ V.num) (hZ : 0 ≤ Z.num) (hU : 0 ≤ U.num) : 0 ≤ (remainderCoefficient c T P V Z U).num
```

```
theorem uniform_frame_bound (c h T P V Z U r A : Fraction) (ht : 0 < h.num) (s u : Point × Point) (a :
  ↪ Point) (hP : 0 ≤ P.num) (hV : 0 ≤ V.num) (hZ : 0 ≤ Z.num) (hwindow : Fraction.le h T) (hp :
  ↪ Fraction.le (pointNorm s.1) P) (hv : Fraction.le (pointNorm s.2) V) (hz : Fraction.le (pointNorm
  ↪ (QuadraticSecants.secondState h ht s u).1) Z) (hd : Fraction.le (pointDistance
  ↪ (QuadraticSecants.secondState h ht s u).1 a) (Fraction.add (Fraction.mul h U) (Fraction.mul r A))) :
  ↪ Fraction.le (durationDifference (Fraction.mul c (dot a s.1)) (normalizedStep c h ht s u)).abs
  ↪ (Fraction.add (Fraction.mul h (remainderCoefficient c T P V Z U)) (Fraction.mul r
  ↪ (roundingCoefficient c P A)))
```

```
theorem normalized_step_approximant (c h : Fraction) (ht : 0 < h.num) (a b : EndpointCauchyName) (j :
  ↪ Nat) : stateEquiv ((normalizedStepName c h ht a b).approx j) (scalarState (normalizedStep c h ht
  ↪ (a.approx j) (b.approx j)))
```

BarrowLib/Polygon/QuadraticSecants.lean — Normalized second-order position departure, composed from the existing proved secant operators. Actual finite quadratic remainders yield force error $2(LtV+E)+(h/t)|a(x_0)|$. The half-mesh bias remains explicit.

```
theorem second_identity (t : Fraction) (ht : 0 < t.num) (s u : Point × Point) (a : Point) : pointEquiv
  ↪ (pointSub (secondState t ht s u).1 a) (pointScale (Fraction.mul (Fraction.ofInt 2) (Fraction.mul
  ↪ (TimeCalibration.inverse t ht) (TimeCalibration.inverse t ht))) (pointSub u.1 (quadraticPosition t s
  ↪ a)))
```

```
theorem second_state_time_congr (t u : Fraction) (ht : 0 < t.num) (hu : 0 < u.num) (he : Fraction.equiv t
  ↪ u) (s v : Point × Point) : pointEquiv (secondState t ht s v).1 (secondState u hu s v).1
```

```
theorem finite_second_bound_at (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V : Fraction)
  ↪ (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num) (n : Nat) (ht : 0 < (time h
  ↪ n).num) (hc : ∀ k, k < n → Fraction.le (pointDistance (a (run a h s (k+1))).1) (a s.1)) (Fraction.add
  ↪ (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1)) E)) (hv : ∀ k, k < n → Fraction.le (pointNorm
  ↪ (run a h s k).2) V) : Fraction.le (pointDistance (secondState (time h n) ht s (run a h s n)).1) (a
  ↪ s.1)) (Fraction.add (Fraction.mul (Fraction.ofInt 2) (source L E (time h n) V)) (Fraction.mul
  ↪ (Fraction.mul h (TimeCalibration.inverse (time h n) ht)) (pointNorm (a s.1))))
```

```

theorem finite_second_bound (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V : Fraction) (hh
  ↪ : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num) (hc : comparisonContract a a L E) (n :
  ↪ Nat) (ht : 0 < (time h n).num) (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  ↪ Fraction.le (pointDistance (secondState (time h n) ht s (run a h s n)).1 (a s.1)) (Fraction.add
  ↪ (Fraction.mul (Fraction.ofInt 2) (source L E (time h n) V)) (Fraction.mul (Fraction.mul h
  ↪ (TimeCalibration.inverse (time h n) ht)) (pointNorm (a s.1))))

```

```

theorem finite_second_equivalent_time_at (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V u :
  ↪ Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num) (n : Nat) (ht : 0 < (time
  ↪ h n).num) (hc : ∀ k, k < n → Fraction.le (pointDistance (a (run a h s (k+1))).1) (a s.1)) (Fraction.add
  ↪ (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1)) E)) (hu : 0 < u.num) (he : Fraction.equiv u
  ↪ (time h n)) (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le (pointDistance
  ↪ (secondState u hu s (run a h s n)).1 (a s.1)) (Fraction.add (Fraction.mul (Fraction.ofInt 2) (source
  ↪ L E u V)) (Fraction.mul (Fraction.mul h (TimeCalibration.inverse u hu)) (pointNorm (a s.1))))

```

```

theorem finite_second_equivalent_time (a : Point → Point) (h : Fraction) (s : Point × Point) (L E V u :
  ↪ Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num) (hc : comparisonContract
  ↪ a a L E) (n : Nat) (ht : 0 < (time h n).num) (hu : 0 < u.num) (he : Fraction.equiv u (time h n)) (hv :
  ↪ ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) : Fraction.le (pointDistance (secondState u hu
  ↪ s (run a h s n)).1 (a s.1)) (Fraction.add (Fraction.mul (Fraction.ofInt 2) (source L E u V))
  ↪ (Fraction.mul (Fraction.mul h (TimeCalibration.inverse u hu)) (pointNorm (a s.1))))

```

BarrowLib/Polygon/RationalIntervals.lean — Elementary rational interval bisection and gap identities.

```

theorem half_equiv {a b : Fraction} (h : Fraction.equiv a b) : Fraction.equiv a.half b.half

```

```

theorem half_le {a b : Fraction} (h : Fraction.le a b) : Fraction.le a.half b.half

```

```

theorem half_double (a : Fraction) : Fraction.equiv (Fraction.add a a).half a

```

```

theorem midpoint_between (a b : Fraction) (h : Fraction.le a b) : Fraction.le a (midpoint a b) ∧
  ↪ Fraction.le (midpoint a b) b

```

```

theorem midpoint_lower_gap (a b : Fraction) : Fraction.equiv (durationDifference a (midpoint a b))
  ↪ (durationDifference a b).half

```

```

theorem midpoint_upper_gap (a b : Fraction) : Fraction.equiv (durationDifference (midpoint a b) b)
  ↪ (durationDifference a b).half

```

BarrowLib/Polygon/SampledValues.lean — Completion of uniformly coherent rational map samples on a named region. A completed input has a regional Cauchy representative; comparisons use only certified approximants. Representative independence is proved before choosing a representative, without a total-map or completed-continuity premise.

```

theorem admissible_realize (region : (Point × Point) → Prop) (a : EndpointCauchyName) (ha : ∀ n, region
  ↪ (a.approx n)) : Admissible region (realize a)

```

```

theorem admissible_embed (region : (Point × Point) → Prop) (s : Point × Point) (hs : region s) :
  ↪ Admissible region (embed s)

```

```

theorem admissible_true (x : Value) : Admissible (fun _ => True) x

```

```

theorem admissibleName_mem (region : (Point × Point) → Prop) (x : Value) (hx : Admissible region x) (n :
  ↪ Nat) : region ((admissibleName region x hx).approx n)

```

```

theorem admissibleName_realize (region : (Point × Point) → Prop) (x : Value) (hx : Admissible region x) :
  ↪ realize (admissibleName region x hx) = x

```

```

theorem scaled_add_small (C eps d e : Fraction) (hC : 0 ≤ C.num) (hd : 0 ≤ d.num) (hdelta : Fraction.lt d
  ↪ (factorDelta C eps.half hC)) (he : Fraction.lt e eps.half) : Fraction.lt (Fraction.add (Fraction.mul
  ↪ d C) e) eps

```

```

theorem sampledName_equiv (f : Family) (a b : EndpointCauchyName) (ha : ∀ n, f.region (a.approx n)) (hb :
  ↪ ∀ n, f.region (b.approx n)) (hab : NameEquiv a b) : NameEquiv (sampledName f a ha) (sampledName f b
  ↪ hb)

```

```

theorem sampledValue_realize (f : Family) (a : EndpointCauchyName) (ha : ∀ n, f.region (a.approx n)) (hx :
  ↪ Admissible f.region (realize a)) : sampledValue f (realize a) hx = realize (sampledName f a ha)

```

```

theorem sampledValue_congr (f : Family) (x y : Value) (hx : Admissible f.region x) (hy : Admissible
  ↪ f.region y) (hxy : x=y) : sampledValue f x hx = sampledValue f y hy

theorem sampledName_offset_equiv (f : Family) (a : EndpointCauchyName) (ha : ∀ n, f.region (a.approx n))
  ↪ (m : Nat) : NameEquiv (sampledName f a ha) (sampledName (offsetFamily f m) a ha)

theorem sampledValue_offset (f : Family) (x : Value) (hx : Admissible f.region x) (m : Nat) :
  ↪ sampledValue (offsetFamily f m) x hx = sampledValue f x hx

theorem nameBound_of_vanishing_error (a b : EndpointCauchyName) (R : Fraction) (e : Nat → Fraction) (he :
  ↪ ∀ eps : Fraction, 0 < eps.num → ∃ N : Nat, ∀ n, N ≤ n → Fraction.lt (e n) eps) (hlevel : ∀ n,
  ↪ Fraction.le (distance (a.approx n) (b.approx n)) (Fraction.add R (e n))) : NameBound a b R

theorem nameBound_scale_error (a b ta tb : EndpointCauchyName) (C R : Fraction) (hC : 0 ≤ C.num) (e : Nat
  ↪ → Fraction) (he : ∀ eps : Fraction, 0 < eps.num → ∃ N : Nat, ∀ n, N ≤ n → Fraction.lt (e n) eps)
  ↪ (hlevel : ∀ n, Fraction.le (distance (a.approx n) (b.approx n)) (Fraction.add (Fraction.mul (distance
  ↪ (ta.approx n) (tb.approx n)) C) (e n))) (hnear : NameBound ta tb R) : NameBound a b (Fraction.mul R C)

theorem sampledValue_within (f : Family) (x y : Value) (hx : Admissible f.region x) (hy : Admissible
  ↪ f.region y) (R : Fraction) (hxy : Within x y R) : Within (sampledValue f x hx) (sampledValue f y hy)
  ↪ (Fraction.mul R f.coefficient)

theorem nameBound_affine (a b ta tb : EndpointCauchyName) (C R E : Fraction) (hC : 0 ≤ C.num) (M : Nat)
  ↪ (hlevel : ∀ n, M ≤ n → Fraction.le (distance (a.approx n) (b.approx n)) (Fraction.add (Fraction.mul
  ↪ (distance (ta.approx n) (tb.approx n)) C) E)) (hnear : NameBound ta tb R) : NameBound a b
  ↪ (Fraction.add (Fraction.mul R C) E)

theorem sampled_approximant_bound (f : Family) (a : EndpointCauchyName) (ha : ∀ n, f.region (a.approx n))
  ↪ (hx : Admissible f.region (realize a)) (m : Nat) (R : Fraction) (hnear : Within (embed (a.approx m))
  ↪ (realize a) R) : Within (embed (f.sample m (a.approx m))) (sampledValue f (realize a) hx)
  ↪ (Fraction.add (Fraction.mul R f.coefficient) (f.error m))

```

BarrowLib/Polygon/ScalarOrder.lean — A closed rational lower comparison for the first scalar coordinate of a constructed Cauchy value. Representative invariance is proved before lifting.

```

theorem first_coordinate_gap (a b : Point × Point) : Fraction.le (durationDifference b.1.1 a.1.1).abs
  ↪ (distance a b)

theorem nameBelow_transport (q : Fraction) (a b : EndpointCauchyName) (hab : NameEquiv a b) (ha :
  ↪ NameBelow q a) : NameBelow q b

theorem nameBelow_congr (q : Fraction) (a b : EndpointCauchyName) (hab : NameEquiv a b) : NameBelow q a ↔
  ↪ NameBelow q b

theorem below_realize (q : Fraction) (a : EndpointCauchyName) : Below q (realize a) ↔ NameBelow q a

theorem below_embed_iff (q : Fraction) (s : Point × Point) : Below q (embed s) ↔ Fraction.le q s.1.1

theorem below_downward (p q : Fraction) (v : Value) (hpq : Fraction.le p q) (hq : Below q v) : Below p v

```

BarrowLib/Polygon/ScaledTolerance.lean — Elementary rational tolerances for scaling and monotonicity of generic closed quotient bounds. The retained namespace is an import-compatible name; no harmonic parameter or mechanical result is a premise.

```

theorem factorDenominator_positive (C : Fraction) (hC : 0 ≤ C.num) : 0 < (factorDenominator C).num

theorem factorDelta_positive (C eps : Fraction) (hC : 0 ≤ C.num) (heps : 0 < eps.num) : 0 < (factorDelta
  ↪ C eps hC).num

theorem factor_control (C eps d : Fraction) (hC : 0 ≤ C.num) (hd : 0 ≤ d.num) (hdelta : Fraction.lt d
  ↪ (factorDelta C eps hC)) : Fraction.lt (Fraction.mul d C) eps

theorem factor_delta_weak (C eps : Fraction) (hC : 0 ≤ C.num) (heps : 0 ≤ eps.num) : Fraction.le
  ↪ (Fraction.mul (factorDelta C eps hC) C) eps

theorem nameBound_mono (a b : EndpointCauchyName) (R S : Fraction) (hRS : Fraction.le R S) (h : NameBound
  ↪ a b R) : NameBound a b S

```

```
theorem within_mono (x y : Value) (R S : Fraction) (hRS : Fraction.le R S) (h : Within x y R) : Within x
  ↪ y S
```

```
theorem nameBound_scale (a b ta tb : EndpointCauchyName) (C R : Fraction) (hC : 0 ≤ C.num) (hlevel : ∀ n
  ↪ : Nat, Fraction.le (distance (a.approx n) (b.approx n)) (Fraction.mul (distance (ta.approx n)
  ↪ (tb.approx n)) C)) (hnear : NameBound ta tb R) : NameBound a b (Fraction.mul R C)
```

BarrowLib/Polygon/SecantValues.lean — Rationally scaled position differences and velocity projections of constructed Cauchy values. All Cauchy and representative-invariance proofs precede quotient lifting; no rate-of-change premise is supplied.

```
theorem pointState_distance (p q : Point) : Fraction.equiv (distance (pointState p) (pointState q))
  ↪ (pointDistance p q)
```

```
theorem velocity_nonexpansive (s t : Point × Point) : Fraction.le (distance (velocityState s)
  ↪ (velocityState t)) (distance s t)
```

```
theorem velocityValue_within (x y : Value) (R : Fraction) (h : Within x y R) : Within (velocityValue x)
  ↪ (velocityValue y) R
```

```
theorem velocityValue_embed (s : Point × Point) : velocityValue (embed s) = embed (velocityState s)
```

```
theorem secant_distance_bound (q : Fraction) (a b c d : Point × Point) : Fraction.le (distance
  ↪ (secantState q a b) (secantState q c d)) (Fraction.mul q.abs (Fraction.add (distance a c) (distance b
  ↪ d)))
```

```
theorem two_scaled_small (q eps r s : Fraction) (_heps : 0 < eps.num) (hr : 0 ≤ r.num) (hs : 0 ≤ s.num)
  ↪ (h1 : Fraction.lt r (factorDelta q.abs eps.half (Fraction.abs_num_nonnegative q))) (h2 : Fraction.lt
  ↪ s (factorDelta q.abs eps.half (Fraction.abs_num_nonnegative q))) : Fraction.lt (Fraction.mul q.abs
  ↪ (Fraction.add r s)) eps
```

```
theorem secantName_equiv (q : Fraction) (a b a' b' : EndpointCauchyName) (ha : NameEquiv a a') (hb :
  ↪ NameEquiv b b') : NameEquiv (secantName q a b) (secantName q a' b')
```

```
theorem secantValue_realize (q : Fraction) (a b : EndpointCauchyName) : secantValue q (realize a)
  ↪ (realize b) = realize (secantName q a b)
```

```
theorem secantValue_embed (q : Fraction) (s t : Point × Point) : secantValue q (embed s) (embed t) =
  ↪ embed (secantState q s t)
```

```
theorem shiftedName_equiv (a : EndpointCauchyName) (m : Nat) : NameEquiv (shiftedName a m) a
```

```
theorem shiftedValue (a : EndpointCauchyName) (m : Nat) : realize (shiftedName a m) = realize a
```

BarrowLib/Polygon/SquareContentValues.lean — The elementary all-cover outer-content cut is realized as a Cauchy scalar by shrinking rational intervals. Different proved initial covers produce the same value. An ordinary measure or inner-area identification remains separate.

```
theorem contentValue_lower_cut (A : PositionValue → Prop) (c : Cover A) (q : Fraction) : Below q
  ↪ (contentValue A c).val ↔ LowerContent A q
```

```
theorem contentValue_nonnegative (A : PositionValue → Prop) (c : Cover A) : Below (Fraction.ofInt 0)
  ↪ (contentValue A c).val
```

```
theorem contentValue_within_zero (A : PositionValue → Prop) (c : Cover A) : Within (contentValue A c).val
  ↪ (embed (scalarState (Fraction.ofInt 0))) c.budget
```

```
theorem contentValue_independent_cover (A : PositionValue → Prop) (c d : Cover A) : contentValue A c =
  ↪ contentValue A d
```

```
theorem contentValue_region_congr (A B : PositionValue → Prop) (h : ∀ x, A x ↔ B x) (c : Cover A) (d :
  ↪ Cover B) : contentValue A c = contentValue B d
```

```
theorem contentValue_any_cover_bound (A : PositionValue → Prop) (c d : Cover A) : Within (contentValue A
  ↪ c).val (embed (scalarState (Fraction.ofInt 0))) d.budget
```

```
theorem empty_value_zero : (contentValue (fun _ => False) emptyCover).val = embed (scalarState
  ↪ (Fraction.ofInt 0))
```

```
theorem singleton_value_zero (p : Point) : (contentValue (fun x => x = embedPosition p) (singletonCover
  ↪ p)).val = embed (scalarState (Fraction.ofInt 0))
```

BarrowLib/Polygon/SquareOuterContent.lean — Nonnegative elementary outer content in the completed coordinate plane. A cover is a finite family of closed rational squares. Its budget counts squares with multiplicity; the region is a point set, so overlapping or crossing lobes have no signed cancellation. Content is the closed lower cut of the infimum of all covering budgets, rather than an arbitrary chosen cover budget.

```
theorem squareArea_nonnegative (R : Fraction) (hR : 0 ≤ R.num) : 0 ≤ (squareArea R).num
```

```
theorem sumBudget_nonnegative (squares : Nat → Square) : ∀ n, 0 ≤ (sumBudget squares n).num
```

```
theorem budget_nonnegative {A : PositionValue → Prop} (c : Cover A) : 0 ≤ c.budget.num
```

```
theorem uniform_budget (centres : Nat → Point) (R : NonnegativeRadius) : ∀ n, Fraction.equiv (sumBudget
  ↪ (fun k => (centres k, R)) n) (Fraction.mul (Fraction.ofInt (n : Int)) (squareArea R.val))
```

```
theorem content_zero_lower (A : PositionValue → Prop) : LowerContent A (Fraction.ofInt 0)
```

```
theorem content_downward (A : PositionValue → Prop) (p q : Fraction) (hpq : Fraction.le p q) (hq :
  ↪ LowerContent A q) : LowerContent A p
```

```
theorem content_closed (A : PositionValue → Prop) (q : Fraction) (h : ∀ eps : Fraction, 0 < eps.num → ∃
  ↪ p, LowerContent A p ∧ Fraction.le q (Fraction.add p eps)) : LowerContent A q
```

```
theorem content_cover_bound (A : PositionValue → Prop) (c : Cover A) (q : Fraction) (hq : LowerContent A
  ↪ q) : Fraction.le q c.budget
```

```
theorem content_mono (A B : PositionValue → Prop) (hAB : ∀ x, A x → B x) (q : Fraction) (hq :
  ↪ LowerContent A q) : LowerContent B q
```

```
theorem content_union_includes (A B : PositionValue → Prop) (q : Fraction) (h : LowerContent A q v
  ↪ LowerContent B q) : LowerContent (fun x => A x ∨ B x) q
```

```
theorem empty_content (q : Fraction) : LowerContent (fun _ => False) q ↔ Fraction.le q (Fraction.ofInt 0)
```

```
theorem singleton_content (p : Point) (q : Fraction) : LowerContent (fun x => x = embedPosition p) q ↔
  ↪ Fraction.le q (Fraction.ofInt 0)
```

BarrowLib/Polygon/StateDistance.lean

```
private theorem pointNeg_congr {p q : Point} (hp : pointEquiv p q) : pointEquiv (pointNeg p) (pointNeg q)
```

```
private theorem pointSub_congr {p p' q q' : Point} (hp : pointEquiv p p') (hq : pointEquiv q q') :
  ↪ pointEquiv (pointSub p q) (pointSub p' q')
```

```
theorem stateSub_congr {s s' t t' : Point × Point} (hs : stateEquiv s s') (ht : stateEquiv t t') :
  ↪ stateEquiv (stateSub s t) (stateSub s' t')
```

```
theorem stateSub_norm_symm (a b : Point × Point) : Fraction.equiv (stateNorm (stateSub a b)) (stateNorm
  ↪ (stateSub b a))
```

```
theorem stateSub_triangle (a b c : Point × Point) : Fraction.le (stateNorm (stateSub a c)) (Fraction.add
  ↪ (stateNorm (stateSub a b)) (stateNorm (stateSub b c)))
```

```
theorem stateSub_self_norm_zero (s : Point × Point) : Fraction.equiv (stateNorm (stateSub s s))
  ↪ (Fraction.ofInt 0)
```

BarrowLib/Polygon/TailValues.lean — A shared closed-value bound from an actual name's adjacent geometric estimate. Applications supply their derived finite estimate and coefficient.

```

theorem approximant_bound (a : EndpointCauchyName) (A : Fraction) (hA : 0 ≤ A.num) (hadj : ∀ j,
  ↳ Fraction.le (distance (a.approx (j+1)) (a.approx j)) (GeometricTail.tailCap A (j+1))) (m : Nat) :
  ↳ Within (embed (a.approx m)) (realize a) (GeometricTail.tailCap A m)

```

BarrowLib/Polygon/TangentTriangleValues.lean — The signed doubled triangle of a completed point, its tangent continuation and an actual next completed point. Shared pairings and secants construct it; finite algebra identifies its H^{-3} normalization with half the determinant of velocity and normalized second departure. It is not an unsigned lobe or area of a matched region.

```

theorem triangleValue_embed (h : Fraction) (s u : Point × Point) : triangleValue h (embed s) (embed u) =
  ↳ embed (scalarState (TriangleBounds.triangleTwice s.1 (pointAdd s.1 (pointScale h s.2)) u.1))

```

```

theorem normalized_triangle_identity (h : Fraction) (ht : 0 < h.num) (x y : Value) :
  ↳ normalizedTriangleValue h ht x y = secantValue (Fraction.ofInt 1).half (pairingValue detForm
  ↳ (velocityValue x) (QuadraticSecants.secondValue h ht x y)) (embed (zeroPoint, zeroPoint))

```

BarrowLib/Polygon/TimeCalibration.lean — Calibrated coordinate gauges and finite triangular-map estimates. A positive rational time calibration is free data. Rescaling the time unit preserves the weighted gauge, dimensionless growth and sampling budgets. Cauchy convergence is independent of this choice. This modern elementary infrastructure assumes no motion, force law, potential or universal scale.

```

theorem inverse_positive (tau : Fraction) (ht : 0 < tau.num) : 0 < (inverse tau ht).num

```

```

theorem inverse_product (tau : Fraction) (ht : 0 < tau.num) : Fraction.equiv (Fraction.mul tau (inverse
  ↳ tau ht)) (Fraction.ofInt 1)

```

```

theorem inverse_congr {tau rho : Fraction} (ht : 0 < tau.num) (hr : 0 < rho.num) (h : Fraction.equiv tau
  ↳ rho) : Fraction.equiv (inverse tau ht) (inverse rho hr)

```

```

theorem norm_nonnegative (tau : Fraction) (s : Point × Point) (ht : 0 ≤ tau.num) : 0 ≤ (norm tau s).num

```

```

theorem position_le_norm (tau : Fraction) (s : Point × Point) (ht : 0 ≤ tau.num) : Fraction.le (pointNorm
  ↳ s.1) (norm tau s)

```

```

theorem velocity_le_norm_inverse (tau : Fraction) (s : Point × Point) (ht : 0 < tau.num) : Fraction.le
  ↳ (pointNorm s.2) (Fraction.mul (norm tau s) (inverse tau ht))

```

```

theorem cell_increment (tau h : Fraction) (a : Point → Point) (s : Point × Point) : Fraction.equiv
  ↳ (distance tau (cell a h s) s) (Fraction.mul h.abs (Fraction.add (pointNorm s.2) (Fraction.mul tau
  ↳ (pointNorm (a (cell a h s).1)))))

```

```

theorem cell_increment_bound (tau h B V : Fraction) (a : Point → Point) (s : Point × Point) (ht : 0 ≤
  ↳ tau.num) (hv : Fraction.le (pointNorm s.2) V) (ha : Fraction.le (pointNorm (a (cell a h s).1) B) :
  ↳ Fraction.le (distance tau (cell a h s) s) (Fraction.mul h.abs (Fraction.add V (Fraction.mul tau B)))

```

```

theorem distance_self_zero (tau : Fraction) (s : Point × Point) : Fraction.equiv (distance tau s s)
  ↳ (Fraction.ofInt 0)

```

```

theorem distance_symm (tau : Fraction) (s t : Point × Point) : Fraction.equiv (distance tau s t)
  ↳ (distance tau t s)

```

```

theorem distance_triangle (tau : Fraction) (ht : 0 < tau.num) (s t u : Point × Point) : Fraction.le
  ↳ (distance tau s u) (Fraction.add (distance tau s t) (distance tau t u))

```

```

theorem norm_rescale (c tau : Fraction) (s : Point × Point) (hc : 0 < c.num) : Fraction.equiv (norm
  ↳ (Fraction.mul c tau) (rescaleState c hc s)) (norm tau s)

```

```

theorem amplification_rescale (c tau h L : Fraction) (hc : 0 < c.num) (ht : 0 < tau.num) : Fraction.equiv
  ↳ (amplification (Fraction.mul c tau) (Fraction.mul c h) (rescaleConstant c L hc) (Int.mul_pos hc ht))
  ↳ (amplification tau h L ht)

```

```

theorem window_rescale (c tau T L : Fraction) (hc : 0 < c.num) (ht : 0 < tau.num) : Fraction.equiv
  ↳ (Fraction.mul (Fraction.mul c T) (rate (Fraction.mul c tau) (rescaleConstant c L hc) (Int.mul_pos hc
  ↳ ht))) (Fraction.mul T (rate tau L ht))

```

```

theorem component_amplification (tau P V H L E : Fraction) (ht : 0 < tau.num) (hP : 0 ≤ P.num) (hV : 0 ≤
  V.num) (hH : 0 ≤ H.num) (hL : 0 ≤ L.num) : Fraction.le (Fraction.add (Fraction.add P (Fraction.mul H
  V)) (Fraction.mul tau (Fraction.add V (Fraction.mul H (Fraction.add (Fraction.mul L (Fraction.add P
  (Fraction.mul H V)) E)))) (Fraction.add (Fraction.mul (amplification tau H L ht) (Fraction.add P
  (Fraction.mul tau V))) (Fraction.mul (Fraction.mul tau H) E))

theorem cell_amplification_at (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E :
  Fraction) (s t : Point × Point) (hL : 0 ≤ L.num) (hc : Fraction.le (pointDistance (a (cell a h s).1)
  (b (cell b h t).1)) (Fraction.add (Fraction.mul L (pointDistance (cell a h s).1 (cell b h t).1)) E))
  : Fraction.le (distance tau (cell a h s) (cell b h t)) (Fraction.add (Fraction.mul (amplification tau
  h L ht) (distance tau s t)) (Fraction.mul (Fraction.mul tau h.abs) E))

theorem cell_amplification (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E : Fraction)
  (s t : Point × Point) (hL : 0 ≤ L.num) (hc : comparisonContract a b L E) : Fraction.le (distance tau
  (cell a h s) (cell b h t)) (Fraction.add (Fraction.mul (amplification tau h L ht) (distance tau s t))
  (Fraction.mul (Fraction.mul tau h.abs) E))

theorem sampling_term_rescale (c tau h E : Fraction) (hc : 0 < c.num) : Fraction.equiv (Fraction.mul
  (Fraction.mul (Fraction.mul c tau) (Fraction.mul c h).abs) (rescaleConstant c E hc)) (Fraction.mul
  (Fraction.mul tau h.abs) E)

theorem distance_rescale (c tau : Fraction) (s t : Point × Point) (hc : 0 < c.num) : Fraction.equiv
  (distance (Fraction.mul c tau) (rescaleState c hc s) (rescaleState c hc t)) (distance tau s t)

theorem norm_unit_calibration (s : Point × Point) : Fraction.equiv (norm (Fraction.ofInt 1) s) (stateNorm
  s)

theorem distance_le_uncalibrated (tau : Fraction) (ht : 0 < tau.num) (s t : Point × Point) : Fraction.le
  (distance tau s t) (Fraction.mul (Fraction.add (Fraction.ofInt 1) tau) (stateDistance s t))

theorem uncalibrated_le_distance (tau : Fraction) (ht : 0 < tau.num) (s t : Point × Point) : Fraction.le
  (stateDistance s t) (Fraction.mul (Fraction.add (Fraction.ofInt 1) (inverse tau ht)) (distance tau s
  t))

theorem calibrated_tolerance (tau eps : Fraction) (ht : 0 < tau.num) (s t : Point × Point) (hd :
  Fraction.lt (stateDistance s t) (HarmonicTimeRealization.factorDelta (Fraction.add (Fraction.ofInt
  1) tau) eps (Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt ht)))) : Fraction.lt (distance
  tau s t) eps

theorem unit_tolerance (tau eps : Fraction) (ht : 0 < tau.num) (s t : Point × Point) (hd : Fraction.lt
  (distance tau s t) (HarmonicTimeRealization.factorDelta (Fraction.add (Fraction.ofInt 1) (inverse tau
  ht)) eps (Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos)))) : Fraction.lt
  (stateDistance s t) eps

theorem distance_unit_calibration (s t : Point × Point) : Fraction.equiv (distance (Fraction.ofInt 1) s
  t) (stateDistance s t)

theorem cauchy_calibration_iff (tau : Fraction) (ht : 0 < tau.num) (a : Nat → Point × Point) :
  CalibratedCauchy tau a ↔ CalibratedCauchy (Fraction.ofInt 1) a

theorem nameEquiv_calibration_iff (tau : Fraction) (ht : 0 < tau.num) (a b :
  HarmonicDyadic.EndpointCauchyName) : CalibratedEquivalent tau a.approx b.approx ↔
  CauchyValues.NameEquiv a b

theorem amplification_nonnegative (tau h L : Fraction) (ht : 0 < tau.num) (hL : 0 ≤ L.num) : 0 ≤
  (amplification tau h L ht).num

theorem one_le_amplification (tau h L : Fraction) (ht : 0 < tau.num) (hL : 0 ≤ L.num) : Fraction.le
  (Fraction.ofInt 1) (amplification tau h L ht)

theorem rate_nonnegative (tau L : Fraction) (ht : 0 < tau.num) (hL : 0 ≤ L.num) : 0 ≤ (rate tau L ht).num

theorem window_mono (tau h L : Fraction) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (n N : Nat) (hn : n ≤ N) (hs
  : Window tau h L ht N) : Window tau h L ht n

theorem window_of_elapsed (tau h T L : Fraction) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (n : Nat) (he :
  Fraction.le (Fraction.mul (Fraction.ofInt (n : Int)) h.abs) T) (hs : Fraction.le (Fraction.mul T
  (rate tau L ht)) (1,2,by decide)) : Window tau h L ht n

```

```

theorem amplification_expansion (tau h L : Fraction) (ht : 0 < tau.num) : Fraction.equiv (amplification
  ↪ tau h L ht) (Fraction.add (Fraction.ofInt 1) (Fraction.add (Fraction.mul h.abs (rate tau L ht))
  ↪ (Fraction.mul (Fraction.mul h.abs h.abs) L)))

theorem amplification_power_le_two (tau h L : Fraction) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (n : Nat) (hs
  ↪ : Window tau h L ht n) : Fraction.le (HarmonicAccumulation.fpower (amplification tau h L ht) n)
  ↪ (Fraction.ofInt 2)

theorem run_distance_le_source (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E :
  ↪ Fraction) (s : Point × Point) (hL : 0 ≤ L.num) (hc : comparisonContract a b L E) : (n : Nat) →
  ↪ Fraction.le (distance tau (BoundedIteration.run a h s n) (BoundedIteration.run b h s n))
  ↪ (FiniteRecurrence.sourceBudget (amplification tau h L ht) (Fraction.mul (Fraction.mul tau h.abs) E)
  ↪ n)

theorem run_uniform_discrepancy (tau : Fraction) (ht : 0 < tau.num) (a b : Point → Point) (h L E :
  ↪ Fraction) (s : Point × Point) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hc : comparisonContract a b L E) (n
  ↪ : Nat) (hs : Window tau h L ht n) : Fraction.le (distance tau (BoundedIteration.run a h s n)
  ↪ (BoundedIteration.run b h s n)) (Fraction.mul (Fraction.ofInt (2 * (n : Int)))) (Fraction.mul
  ↪ (Fraction.mul tau h.abs) E))

theorem cauchy_rescale (c tau : Fraction) (hc : 0 < c.num) (a : Nat → Point × Point) : CalibratedCauchy
  ↪ (Fraction.mul c tau) (fun n => rescaleState c hc (a n)) ↔ CalibratedCauchy tau a

theorem window_rescale_iff (c tau h L : Fraction) (hc : 0 < c.num) (ht : 0 < tau.num) (n : Nat) : Window
  ↪ (Fraction.mul c tau) (Fraction.mul c h) (rescaleConstant c L hc) (Int.mul_pos hc ht) n ↔ Window tau h
  ↪ L ht n

theorem rescaling_control : Fraction.equiv (norm two sampleState) (norm (Fraction.mul three two)
  ↪ (rescaleState three (by decide) sampleState)) ∧ Fraction.equiv (amplification two quarter
  ↪ (Fraction.ofInt 1) (by decide)) (amplification (Fraction.mul three two) (Fraction.mul three quarter)
  ↪ (rescaleConstant three (Fraction.ofInt 1) (by decide)) (by decide))

theorem uncalibrated_norm_changes_control : ¬ Fraction.equiv (stateNorm sampleState) (stateNorm
  ↪ (rescaleState two (by decide) sampleState))

theorem zero_lipschitz_control : Fraction.equiv (amplification two half (Fraction.ofInt 0) (by decide))
  ↪ (5,4,by decide)

```

BarrowLib/Polygon/TriangleBounds.lean — Unsigned doubled triangle magnitudes for finite rational path patches. These estimates concern area between compared paths, separately from their Kepler swept areas. A sum counts patches with multiplicity; interpreting it as a union's area still needs the geometric decomposition or enclosure. No limiting trajectory is supplied or constructed here.

```

theorem det_abs_le_product (p q : Point) : Fraction.le (det p q).abs (Fraction.mul (pointNorm p)
  ↪ (pointNorm q))

theorem radius_lower_of_areal_bound (p v : Point) (r V : Fraction) (hV : 0 < V.num) (hv : Fraction.le
  ↪ (pointNorm v) V) (ha : Fraction.le (Fraction.mul r V) (det p v).abs) : Fraction.le r (pointNorm p)

theorem triangleMagnitude_nonnegative (p q r : Point) : 0 ≤ (triangleMagnitude p q r).num

theorem triangleMagnitude_le_product (p q r : Point) : Fraction.le (triangleMagnitude p q r)
  ↪ (Fraction.mul (pointNorm (pointSub q p)) (pointNorm (pointSub r p)))

theorem triangleTwice_consecutive (p q r : Point) : Fraction.equiv (triangleTwice p q r) (det (pointSub q
  ↪ p) (pointSub r q))

theorem triangleMagnitude_le_consecutive (p q r : Point) : Fraction.le (triangleMagnitude p q r)
  ↪ (Fraction.mul (pointNorm (pointSub q p)) (pointNorm (pointSub r q)))

theorem triangleTwice_translation (origin p q r : Point) : Fraction.equiv (triangleTwice (pointSub p
  ↪ origin) (pointSub q origin) (pointSub r origin)) (triangleTwice p q r)

theorem triangleMagnitude_translation (origin p q r : Point) : Fraction.equiv (triangleMagnitude
  ↪ (pointSub p origin) (pointSub q origin) (pointSub r origin)) (triangleMagnitude p q r)

```

```

theorem triangleTwice_swap (p q r : Point) : Fraction.equiv (triangleTwice p r q) ((-(triangleTwice p q
↪ r).num, (triangleTwice p q r).den, (triangleTwice p q r).den_pos) : Fraction)

theorem triangleMagnitude_swap (p q r : Point) : Fraction.equiv (triangleMagnitude p r q)
↪ (triangleMagnitude p q r)

theorem sample_signed_positive : Fraction.equiv (triangleTwice p q r) one

theorem sample_signed_negative : Fraction.equiv (triangleTwice p r q) (Fraction.ofInt (-1))

theorem sample_opposite_unsigned_sum : Fraction.equiv (Fraction.add (triangleMagnitude p q r)
↪ (triangleMagnitude p r q)) (Fraction.ofInt 2)

theorem sample_opposite_signed_sum : Fraction.equiv (Fraction.add (triangleTwice p q r) (triangleTwice p
↪ r q)) zero

theorem sample_unsigned_not_signed : ¬ Fraction.equiv (triangleMagnitude p r q) (triangleTwice p r q)

```

NewtonLimitDynamics/Diagnostic/ConstructedHarmonicPotential.lean — The harmonic polynomial potential evaluated at the actual constructed curve and its tangent continuation. Its normalized per-cell drop tends to $-m \cdot \dot{a} \cdot a / 2$. Finite second-order and half-mesh errors are retained until proved vanishing. This law test does not construct general radial potentials or identify the deflection triangle with D_{mesh} .

```

theorem finite_work_remainder (mass w : Fraction) (p q : Point) : Fraction.equiv (Fraction.add
↪ (durationDifference (quadratic (Fraction.mul mass w).half p) (quadratic (Fraction.mul mass w).half
↪ q)) (Fraction.mul mass (dot (linearField w p) (pointSub q p)))) (Fraction.mul (Fraction.mul mass
↪ w).half (dot (pointSub q p) (pointSub q p)))

private theorem frame_nonnegative (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) : 0 ≤ (frame w T
↪ s).P.num ∧ 0 ≤ (frame w T s).V.num ∧ 0 ≤ (frame w T s).A.num ∧ 0 ≤ (frame w T s).U.num ∧ 0 ≤ (frame w
↪ T s).Z.num

theorem coefficient_nonnegative (mass w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) : 0 ≤
↪ (coefficient mass w T s).num

private theorem work_approximant (mass w : Fraction) (a : EndpointCauchyName) (j : Nat) : stateEquiv
↪ ((workName mass w a).approx j) (scalarState (Fraction.mul (Fraction.mul mass w).half (dot
↪ (linearField w (a.approx j)).1) (a.approx j)).1)))

theorem force_work_eq_energy (mass w : Fraction) (x : Value) : secantValue (Fraction.mul mass w).half
↪ (pairingValue dotForm (HarmonicCompletedForce.linearValue w x) x) (embed (zeroPoint, zeroPoint)) =
↪ forceEnergyValue mass (HarmonicCompletedForce.linearValue w x)

private theorem node_frame_bounds (w T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hT : 0 ≤ T.num)
↪ (hs : DyadicSmallTime w T) (ht : 0 < T.num) (m k : Nat) (hk : k+1 ≤ blocks m) (j : Nat) : let d

theorem normalized_potential_cell_bound (mass w T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hT :
↪ 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num) (m k : Nat) (hk : k+1 ≤ blocks m) : Within
↪ (normalizedStepValue (Fraction.mul mass w).half (duration T m) ht
↪ (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m k))
↪ (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (k+1)))) (forceEnergyValue mass
↪ (HarmonicCompletedForce.linearValue w (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT
↪ m k)))) (Fraction.mul (duration T m) (coefficient mass w T s))

theorem normalized_potential_steps_converge (mass w T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num)
↪ (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num) (eps : Fraction) (heps : 0 < eps.num) :
↪ ∃ N : Nat, ∀ m, N ≤ m → ∀ k, k+1 ≤ blocks m → Within (normalizedStepValue (Fraction.mul mass w).half
↪ (duration T m) ht (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m k))
↪ (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (k+1)))) (forceEnergyValue mass
↪ (HarmonicCompletedForce.linearValue w (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT
↪ m k)))) eps

theorem quadratic_prediction_control : let h : Fraction

```

NewtonLimitDynamics/Diagnostic/DeflectionPotential.lean — Exact finite deflection-triangle/potential identities. These compare the next polygon point to its inertial

continuation, not a polygon to a completed curve. The kick at B changes the next position by $h^2 a(B)$. This diagnostic is neither a historical proof nor a universal action-constant argument.

```

theorem deflection_triangle (h : Fraction) (B v a : Point) : Fraction.equiv (triangleTwice B (inertial h
  ↪ B v) (deflected h B v a)) (Fraction.mul (Fraction.mul (Fraction.mul h h) h) (det v a))

theorem unsigned_deflection_triangle (h : Fraction) (B v a : Point) : Fraction.equiv (triangleMagnitude B
  ↪ (inertial h B v) (deflected h B v a)) (Fraction.mul (Fraction.mul (Fraction.mul h h) h) (det v a)).abs

theorem galilean_potential_step (m g h : Fraction) (B v : Point) : Fraction.equiv (potentialStep
  ↪ (linearPotential m g) (inertial h B v) (deflected h B v (fallAcceleration g))) (negF (Fraction.mul
  ↪ (Fraction.mul m (Fraction.mul g g)) (Fraction.mul h h)))

theorem galilean_area_potential_cross_relation (m g h : Fraction) (B v : Point) : Fraction.equiv
  ↪ (Fraction.mul (Fraction.mul m g) (triangleTwice B (inertial h B v) (deflected h B v (fallAcceleration
  ↪ g)))) (Fraction.mul (Fraction.mul h v.1) (potentialStep (linearPotential m g) (inertial h B v)
  ↪ (deflected h B v (fallAcceleration g))))

theorem harmonic_potential_step (m w h : Fraction) (B v : Point) : Fraction.equiv (potentialStep
  ↪ (quadraticPotential m w) (inertial h B v) (deflected h B v (linearField w B))) (Fraction.add (negF
  ↪ (Fraction.mul (Fraction.mul m (Fraction.mul w w)) (Fraction.mul (Fraction.mul h h) (dot B B))))
  ↪ (Fraction.add (negF (Fraction.mul (Fraction.mul m (Fraction.mul w w)) (Fraction.mul (Fraction.mul
  ↪ (Fraction.mul h h) h) (dot B v)))) (Fraction.mul (Fraction.mul m (Fraction.mul (Fraction.mul w w) w)
  ↪ (Fraction.mul (Fraction.mul (Fraction.mul h h) (Fraction.mul h h) (dot B B))).half))

theorem harmonic_leading_term_not_exact : let one

```

NewtonLimitDynamics/Diagnostic/QuadraticEndpointPotential.lean — The half-coefficient deflection triangle and Galilean potential relation at actual constructed rational endpoint values. The completed endpoint is derived from the constant-force polygons. This is not D_{mesh} or a universal action scale, and no general curved potential expansion is assumed.

```

theorem quadratic_deflection_triangle (h : Fraction) (B v a : Point) : Fraction.equiv (triangleTwice B
  ↪ (inertial h B v) (quadraticPosition h (B,v) a)) (Fraction.mul (Fraction.mul (Fraction.mul h h) h)
  ↪ (det v a)).half

theorem galilean_quadratic_potential_step (m g h : Fraction) (B v : Point) : Fraction.equiv
  ↪ (potentialStep (linearPotential m g) (inertial h B v) (quadraticPosition h (B,v) (fallAcceleration
  ↪ g))) (negF (Fraction.mul (Fraction.mul m (Fraction.mul g g)) (Fraction.mul h h))).half

theorem galilean_quadratic_cross_relation (m g h : Fraction) (B v : Point) : Fraction.equiv (Fraction.mul
  ↪ (Fraction.mul m g) (triangleTwice B (inertial h B v) (quadraticPosition h (B,v) (fallAcceleration
  ↪ g)))) (Fraction.mul (Fraction.mul h v.1) (potentialStep (linearPotential m g) (inertial h B v)
  ↪ (quadraticPosition h (B,v) (fallAcceleration g))))

theorem galilean_constructed_endpoint_relation (m g h : Fraction) (B v : Point) (hh : 0 ≤ h.num) : ∃ C :
  ↪ Point, asPosition (endpointValue (fallAcceleration g) h (B,v) hh)=embedPosition C ∧ Fraction.equiv
  ↪ (triangleTwice B (inertial h B v) C) (Fraction.mul (Fraction.mul (Fraction.mul h h) h) (det v
  ↪ (fallAcceleration g))).half ∧ Fraction.equiv (potentialStep (linearPotential m g) (inertial h B v) C)
  ↪ (negF (Fraction.mul (Fraction.mul m (Fraction.mul g g)) (Fraction.mul h h))).half ∧ Fraction.equiv
  ↪ (Fraction.mul (Fraction.mul m g) (triangleTwice B (inertial h B v) C)) (Fraction.mul (Fraction.mul h
  ↪ v.1) (potentialStep (linearPotential m g) (inertial h B v) C))

```

NewtonLimitDynamics/Polygon/CalibratedForces.lean — Modern calibrated instances of the general finite force estimates. The free calibration fixes no physical action constant and adds no historical premise. General convergence, confinement and restart remain separate.

```

theorem sampled_calibrated_discrepancy (o : Oracle) (tau h L : Fraction) (ht : 0 < tau.num) (hL :
  ↳ LipschitzOn o L) (i j : Nat) (hij : i ≤ j) (s : Point × Point) (n : Nat) (hR : ∀ k, k < n → o.region
  ↳ (BoundedIteration.run (o.sample i) h s (k+1)).1 ∧ o.region (BoundedIteration.run (o.sample j) h s
  ↳ (k+1)).1) (hs : TimeCalibration.Window tau h L ht n) : Fraction.le (TimeCalibration.distance tau
  ↳ (schedule (o.sample i) (List.replicate n h) s) (schedule (o.sample j) (List.replicate n h) s))
  ↳ (Fraction.mul (Fraction.ofInt (2 * (n : Int))) (Fraction.mul (Fraction.mul tau h.abs) (Fraction.add
  ↳ (Fraction.add (o.error i) (o.error i)) (o.error i))))

theorem harmonic_calibrated_cell (tau w h : Fraction) (ht : 0 < tau.num) (s t : Point × Point) :
  ↳ Fraction.le (TimeCalibration.distance tau (cell (linearField w) h s) (cell (linearField w) h t))
  ↳ (Fraction.add (Fraction.mul (TimeCalibration.amplification tau h w.abs ht) (TimeCalibration.distance
  ↳ tau s t)) (Fraction.mul (Fraction.mul tau h.abs) (Fraction.ofInt 0)))

theorem parallel_calibrated_cell (tau h : Fraction) (ht : 0 < tau.num) (a : Point) (s t : Point × Point)
  ↳ : Fraction.le (TimeCalibration.distance tau (cell (fun _ => a) h s) (cell (fun _ => a) h t))
  ↳ (Fraction.add (Fraction.mul (TimeCalibration.amplification tau h (Fraction.ofInt 0) ht)
  ↳ (TimeCalibration.distance tau s t)) (Fraction.mul (Fraction.mul tau h.abs) (Fraction.ofInt 0)))

theorem harmonic_cell_rescale (c w h : Fraction) (hc : 0 < c.num) (s : Point × Point) : stateEquiv
  ↳ (TimeCalibration.rescaleState c hc (cell (linearField w) h s)) (cell (linearField
  ↳ (TimeCalibration.rescaleConstant c w hc)) (Fraction.mul c h) (TimeCalibration.rescaleState c hc s))

```

NewtonLimitDynamics/Polygon/CauchyValues.lean

```

theorem timeValue_bound (w : Fraction) (s : Point × Point) (T U : ShortRationalTime w) : Within (timeValue
  ↳ w s U) (timeValue w s T) (Fraction.mul (timeLipschitz w s) (durationDifference T.val U.val).abs)

theorem binaryValue_prefix_bound (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  ↳ (hs : DyadicSmallTime w T) (m : Nat) : Within (embed (prefixState b w T s m)) (binaryValue b w T s hT
  ↳ hs) (HarmonicBinaryPrefix.tailCap w T s m)

theorem rational_time_uniform_value_bound (w : Fraction) (s : Point × Point) (eps : Fraction) (heps : 0 <
  ↳ eps.num) (T U : ShortRationalTime w) (hnear : Fraction.lt (durationDifference T.val U.val).abs
  ↳ (timeDelta w s eps.half)) : Within (timeValue w s U) (timeValue w s T) eps.half

theorem sample_endpoint_zero_distance : Fraction.equiv (distance (endpoint sampleOne sampleQuarter
  ↳ sampleState 0) sampleState) (9, 16, by decide)

theorem sample_tail_zero : Fraction.equiv (HarmonicDyadic.tailCap sampleOne sampleQuarter sampleState 0)
  ↳ sampleTail

theorem sample_lower_all_levels (j : Nat) : Fraction.le sampleLower (distance (endpoint sampleOne
  ↳ sampleQuarter sampleState j) sampleState)

theorem sample_endpoint_value_ne_initial : endpointValue sampleOne sampleQuarter sampleState (by decide)
  ↳ (by unfold DyadicSmallTime Fraction.le; decide) ≠ embed sampleState

```

NewtonLimitDynamics/Polygon/CompletedForce.lean — Extend the actual sampled central force to completed positions with a certified regional Cauchy representative. No whole-plane domain is required. The same mesh precision used by the motion is retained. Error exhaustion and Lipschitz comparisons construct the extension and its bounds; no desired acceleration value or curve force equation is supplied.

```

theorem precisionError_le_sampleError (o : CentralOracle) (E0 : Fraction) (hE : 0 < E0.num) (j : Nat) :
  ↳ Fraction.le (o.error (precision o.toOracle E0 hE j)) (sampleError o E0 hE j)

theorem sampleError_tail (o : CentralOracle) (E0 : Fraction) (hE : 0 < E0.num) (j : Nat) : Fraction.le
  ↳ (sampleError o E0 hE j) (duration (errorCoefficient E0) j)

theorem sampleError_vanishes (o : CentralOracle) (E0 : Fraction) (hE : 0 < E0.num) (eps : Fraction) (heps :
  ↳ 0 < eps.num) : ∃ N : Nat, ∀ j, N ≤ j → Fraction.lt (sampleError o E0 hE j) eps

theorem forceValue_realize (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num) (hL : LipschitzOn
  ↳ o.toOracle L) (a : EndpointCauchyName) (ha : ∀ n, o.region (a.approx n).1) (hx :
  ↳ SampledValues.Admissible (fun q => o.region q.1) (realize a)) : forceValue o E0 L hE hL (realize a)
  ↳ hx = realize (forceName o E0 L hE hL a ha)

```

```

theorem forceValue_within (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num) (hL : LipschitzOn
  ↳ o.toOracle L) (x y : Value) (hx : SampledValues.Admissible (fun q => o.region q.1) x) (hy :
  ↳ SampledValues.Admissible (fun q => o.region q.1) y) (R : Fraction) (hxy : Within x y R) : Within
  ↳ (forceValue o E0 L hE hL x hx) (forceValue o E0 L hE hL y hy) (Fraction.mul R L)

theorem position_admissible (o : CentralOracle) (x : Value) (hx : SampledValues.Admissible (fun q =>
  ↳ o.region q.1) x) : SampledValues.Admissible (fun q => o.region q.1) (PositionValues.positionValue x)

theorem force_input_position (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num) (hL : LipschitzOn
  ↳ o.toOracle L) (x : Value) (hx : SampledValues.Admissible (fun q => o.region q.1) x) : forceValue o E0
  ↳ L hE hL (PositionValues.positionValue x) (position_admissible o x hx) = forceValue o E0 L hE hL x hx

theorem force_output_position (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num) (hL : LipschitzOn
  ↳ o.toOracle L) (x : Value) (hx : SampledValues.Admissible (fun q => o.region q.1) x) :
  ↳ PositionValues.positionValue (forceValue o E0 L hE hL x hx) = forceValue o E0 L hE hL x hx

theorem force_rational_agreement (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num) (hL :
  ↳ LipschitzOn o.toOracle L) (s : Point × Point) (hs : o.region s.1) : forceValue o E0 L hE hL (embed s)
  ↳ (SampledValues.admissible_embed _ s hs) = accelerationValue o.toOracle s.1 hs

theorem force_precision_independent (o : CentralOracle) (E0 E1 L L' : Fraction) (hE : 0 < E0.num) (hE' :
  ↳ 0 < E1.num) (hL : LipschitzOn o.toOracle L) (hL' : LipschitzOn o.toOracle L') (x : Value) (hx :
  ↳ SampledValues.Admissible (fun q => o.region q.1) x) : forceValue o E0 L hE hL x hx = forceValue o E1
  ↳ L' hE' hL' x hx

theorem prefix_force_bound (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j : Nat) : Within
  ↳ (embed (accelerationState (field o E0 hE j (GeneralForcePrefix.prefixState b o E0 T s hE j).1)))
  ↳ (forceValue o E0 L hE d.lipschitz (GeneralForceTime.gammaValue o E0 T tau L B s hE d (Quotient.mk _
  ↳ b))) (GeneralForceTime.gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b))) (duration
  ↳ (curveErrorCoefficient (GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive) E0 L)
  ↳ j)

theorem prefix_force_uniform_convergence (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps : Fraction)
  ↳ (heps : 0 < eps.num) : ∃ N : Nat, ∀ j, N ≤ j → ∀ b : Nat → Bool, Within (embed (accelerationState
  ↳ (field o E0 hE j (GeneralForcePrefix.prefixState b o E0 T s hE j).1))) (forceValue o E0 L hE
  ↳ d.lipschitz (GeneralForceTime.gammaValue o E0 T tau L B s hE d (Quotient.mk _ b)))
  ↳ (GeneralForceTime.gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b))) eps

NewtonLimitDynamics/Polygon/ForceClasses.lean — Uniform rational approximations to
possibly irrational accelerations on an explicit region. These are contracts on the
force data, not assertions that an impulse polygon converges. Realized sample val-
ues are constructed Cauchy quotients. Historical editions and action diagnostics are
not premises here.

theorem sample_origin_zero (o : CentralOracle) (n : Nat) : pointEquiv (o.sample n (Fraction.ofInt
  ↳ 0, Fraction.ofInt 0)) (Fraction.ofInt 0, Fraction.ofInt 0)

theorem sample_central (o : CentralOracle) (n : Nat) : central (o.sample n)

theorem sampled_finite_area_law (o : CentralOracle) (n : Nat) (ds : List Fraction) (s : Point × Point) :
  ↳ Fraction.equiv (swept (o.sample n) ds s) (Fraction.mul (elapsed ds) (momentum s))

theorem acceleration_approximants_converge (o : Oracle) (p : Point) (hp : o.region p) (eps : Fraction)
  ↳ (heps : 0 < eps.num) : ∃ N : Nat, ∀ n, N ≤ n → Within (embed (accelerationState (o.sample n p)))
  ↳ (accelerationValue o p hp) eps

theorem sample_linear_growth (o : CentralOracle) (L : Fraction) (hL : LipschitzOn o.toOracle L) (hzero :
  ↳ o.region (Fraction.ofInt 0, Fraction.ofInt 0)) (n : Nat) (p : Point) (hp : o.region p) : Fraction.le
  ↳ (pointNorm (o.sample n p)) (Fraction.add (Fraction.mul L (pointNorm p)) (Fraction.add (o.error n)
  ↳ (o.error n)))

theorem acceleration_distance (p q : Point) : Fraction.equiv (distance (accelerationState p)
  ↳ (accelerationState q)) (FiniteEstimates.pointDistance p q)

theorem sample_point_error (o : Oracle) (i j : Nat) (hij : i ≤ j) (p : Point) (hp : o.region p) :
  ↳ Fraction.le (FiniteEstimates.pointDistance (o.sample i p) (o.sample j p)) (o.error i)

```

```

theorem samples_comparison_contract (o : Oracle) (L : Fraction) (hL : LipschitzOn o L) (i j : Nat) (hij :
  ↪ i ≤ j) (p q : Point) (hp : o.region p) (hq : o.region q) : Fraction.le (FiniteEstimates.pointDistance
  ↪ (o.sample i p) (o.sample j q)) (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance p q))
  ↪ (Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i)))

theorem exact_accelerationValue (a : Field) (p : Point) : accelerationValue (exactOracle a) p True.intro
  ↪ = embed (accelerationState (a p))

theorem harmonic_distance_only (w : Fraction) (hw : 0 ≤ w.num) : DistanceOnly (harmonicOracle w hw)

theorem harmonic_comparison_contract (w : Fraction) : FiniteEstimates.comparisonContract (linearField w)
  ↪ (linearField w) w.abs (Fraction.ofInt 0)

theorem harmonic_lipschitz_on (w : Fraction) (hw : 0 ≤ w.num) : LipschitzOn (harmonicOracle w
  ↪ hw).toOracle w.abs

theorem harmonic_class_a_force (w : Fraction) (hw : 0 ≤ w.num) : ClassAForce (harmonicOracle w hw)

theorem parallel_cell (a : Point) (h : Fraction) (s : Point × Point) : cell ((parallelOracle a).sample 0)
  ↪ h s = endKick h s a

theorem parallel_transverse_cell (a : Point) (h : Fraction) (s : Point × Point) : Fraction.equiv (det
  ↪ (cell (fun _ => a) h s).2 a) (det s.2 a)

theorem parallel_transverse_schedule (a : Point) (ds : List Fraction) (s : Point × Point) :
  ↪ Fraction.equiv (det (schedule (fun _ => a) ds s).2 a) (det s.2 a)

theorem run_eq_schedule (a : Field) (h : Fraction) (s : Point × Point) (n : Nat) : BoundedIteration.run a
  ↪ h s n = schedule a (List.replicate n h) s

theorem bounded_samples_of_confined (o : Oracle) (j : Nat) (h : Fraction) (s : Point × Point) (B :
  ↪ Fraction) (n : Nat) (hB : BoundedOn o B) (hR : ∀ i : Nat, i < n → o.region (BoundedIteration.run
  ↪ (o.sample j) h s (i+1)).1) : BoundedIteration.BoundedSamples (o.sample j) h s B n

theorem sampled_polygon_velocity_bound (o : Oracle) (j : Nat) (h : Fraction) (s : Point × Point) (B :
  ↪ Fraction) (n : Nat) (hh : 0 ≤ h.num) (hB : BoundedOn o B) (hR : ∀ i : Nat, i < n → o.region
  ↪ (BoundedIteration.run (o.sample j) h s (i+1)).1) : Fraction.le (pointNorm (schedule (o.sample j)
  ↪ (List.replicate n h) s).2) (Fraction.add (pointNorm s.2) (Fraction.mul (BoundedIteration.time h n)
  ↪ B))

theorem sampled_polygon_position_bound (o : Oracle) (j : Nat) (h : Fraction) (s : Point × Point) (B :
  ↪ Fraction) (n : Nat) (hh : 0 ≤ h.num) (hB : BoundedOn o B) (hR : ∀ i : Nat, i < n → o.region
  ↪ (BoundedIteration.run (o.sample j) h s (i+1)).1) : Fraction.le (pointNorm (schedule (o.sample j)
  ↪ (List.replicate n h) s).1) (BoundedIteration.positionCap h s B n)

theorem sampled_polygon_state_bound (o : Oracle) (j : Nat) (h : Fraction) (s : Point × Point) (B T :
  ↪ Fraction) (n : Nat) (hh : 0 ≤ h.num) (hT : 0 ≤ T.num) (hB : BoundedOn o B) (ht : Fraction.le
  ↪ (BoundedIteration.time h n) T) (hR : ∀ i : Nat, i < n → o.region (BoundedIteration.run (o.sample j) h
  ↪ s (i+1)).1) : Fraction.le (stateNorm (schedule (o.sample j) (List.replicate n h) s)) (Fraction.add
  ↪ (BoundedIteration.uniformPositionCap T s B) (Fraction.add (pointNorm s.2) (Fraction.mul T B)))

theorem continuous_local_refinement (o : Oracle) (hC : ContinuousOn o) (eps : Fraction) (heps : 0 <
  ↪ eps.num) : ∃ delta : Fraction, 0 < delta.num ∧ ∃ N : Nat, ∀ (j : Nat) (h B : Fraction) (s : Point ×
  ↪ Point), N ≤ j → BoundedOn o B → o.region (FiniteEstimates.cell (o.sample j) h s).1 → o.region
  ↪ (FiniteEstimates.oneFull (o.sample j) h s).1 → o.region (FiniteEstimates.twoHalf (o.sample j) h s).1
  ↪ → Fraction.lt (Fraction.mul h.abs (pointNorm s.2)) delta → Fraction.lt (Fraction.mul (Fraction.mul h
  ↪ h).abs B) delta → Fraction.le (FiniteEstimates.stateDistance (FiniteEstimates.twoHalf (o.sample j) h
  ↪ s) (FiniteEstimates.oneFull (o.sample j) h s)) (Fraction.add (Fraction.mul (Fraction.mul h h).abs B)
  ↪ (Fraction.mul h.abs (Fraction.add eps eps)))

```

```

theorem sampled_uniform_refinement (o : Oracle) (j : Nat) (h L B V : Fraction) (s : Point × Point) (n :
  ↳ Nat) (hh : 0 ≤ h.num) (hL : LipschitzOn o L) (hR : ∀ k, k < n → let t := FiniteAccumulation.coarseAt
  ↳ (o.sample j) h s k let u := FiniteAccumulation.fineAt (o.sample (j+1)) h s k o.region
  ↳ (FiniteEstimates.cell (o.sample (j+1)) h u).1 ∧ o.region (FiniteEstimates.twoHalf (o.sample (j+1)) h
  ↳ u).1 ∧ o.region (FiniteEstimates.cell (o.sample j) h t).1 ∧ o.region (FiniteEstimates.twoHalf
  ↳ (o.sample j) h t).1 ∧ o.region (FiniteEstimates.oneFull (o.sample j) h t).1) (hB : BoundedOn o B) (hV
  ↳ : 0 ≤ V.num) (hs : FiniteAccumulation.SmallWindow h L n) (hvel : ∀ k, k < n → Fraction.le (pointNorm
  ↳ (FiniteAccumulation.coarseAt (o.sample j) h s k).2) V) : Fraction.le (FiniteEstimates.stateDistance
  ↳ (FiniteAccumulation.fineAt (o.sample (j+1)) h s n) (FiniteAccumulation.coarseAt (o.sample j) h s n))
  ↳ (Fraction.mul (Fraction.ofInt 2) (Fraction.mul (FiniteAccumulation.count n)
  ↳ (FiniteAccumulation.uniformBlockSource h L (Fraction.add (Fraction.add (o.error j) (o.error j))
  ↳ (o.error j)) B V)))

```

NewtonLimitDynamics/Polygon/GeneralForceAccelerationSecants.lean — Completed dyadic velocity secants converge to the completed force at the constructed positions. Actual finite velocity remainders and force sampling error derive the estimate; no acceleration equation is a premise. The result is a bracketing dyadic secant criterion, not unrestricted differentiation.

```

theorem node_force_value (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↳ E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat) :
  ↳ CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (nodeTime T
  ↳ d.time_nonnegative m k)) (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))
  ↳ = CompletedForce.forceValue o E0 L hE d.lipschitz (realize (nodeName o E0 T tau L B s hE d m k))
  ↳ (node_admissible o E0 T tau L B s hE d m k)

```

```

theorem cell_acceleration_secant_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  ↳ (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m k :
  ↳ Nat) (hk : k+1 ≤ blocks m) : Within (cellAccelerationSecant o E0 T tau L B s hE d hT m k)
  ↳ (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (nodeTime T
  ↳ d.time_nonnegative m k)) (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m
  ↳ k))) (Fraction.mul L (Fraction.mul (duration T m) (velocityCap T B s)))

```

```

theorem accelerationCoefficient_nonnegative (T tau L B : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  ↳ (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) : 0 ≤ (accelerationCoefficient T tau L B s ht).num

```

```

theorem bracketing_acceleration_secant_bound (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B :
  ↳ Fraction) (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s
  ↳ hE) (hT : 0 < T.num) (m : Nat) : Within (cellAccelerationSecant o E0 T tau L B s hE d hT m (ticks b
  ↳ m)) (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (Quotient.mk _
  ↳ b)) (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b))) (Fraction.mul (duration T m)
  ↳ (accelerationCoefficient T tau L B s d.calibration_positive))

```

```

theorem dyadic_acceleration_uniform_identification (o : CentralOracle) (E0 T tau L B : Fraction) (s :
  ↳ Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 <
  ↳ T.num) (eps : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool, Within
  ↳ (cellAccelerationSecant o E0 T tau L B s hE d hT m (ticks b m)) (CompletedForce.forceValue o E0 L hE
  ↳ d.lipschitz (gammaValue o E0 T tau L B s hE d (Quotient.mk _ b)) (gamma_admissible o E0 T tau L B s
  ↳ hE d (Quotient.mk _ b))) eps

```

NewtonLimitDynamics/Polygon/GeneralForceEndpoint.lean — Construct fixed-time motion values from actual sampled central polygons. Lipschitz comparison and sample bounds hold only on a certified band. The partial-time invariant derives actual/coarse and both shadow membership before force sampling; no whole-plane or supplied confinement trace is used. No motion-Cauchy, trajectory, partition-independence or derivative premise is supplied.

```

theorem run_band (d : Conditions o E0 T tau L B s hE) (j : Nat) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat)
  ↳ (hn : Fraction.le (BoundedIteration.time h n) T) : RegionConfinement.Band d.inner_radius
  ↳ d.outer_radius (BoundedIteration.run (field o E0 hE j) h s n).1

```

```

theorem run_region (d : Conditions o E0 T tau L B s hE) (j : Nat) (h : Fraction) (hh : 0 ≤ h.num) (n :
  ↳ Nat) (hn : Fraction.le (BoundedIteration.time h n) T) : o.region (BoundedIteration.run (field o E0 hE
  ↳ j) h s n).1

```

```

theorem actual_samples (d : Conditions o E0 T tau L B s hE) (j : Nat) : BoundedIteration.BoundedSamples
  ↪ (field o E0 hE j) (duration T j) s B (blocks j)

theorem coarse_samples (d : Conditions o E0 T tau L B s hE) (j : Nat) : BoundedIteration.BoundedSamples
  ↪ (field o E0 hE j) (Fraction.add (duration T (j+1)) (duration T (j+1))) s B (blocks j)

theorem shadow_samples (d : Conditions o E0 T tau L B s hE) (j k : Nat) (hk : k < blocks j) : Fraction.le
  ↪ (pointNorm (field o E0 hE j (FiniteEstimates.cell (field o E0 hE j) (duration T (j+1)))
  ↪ (FiniteAccumulation.coarseAt (field o E0 hE j) (duration T (j+1)) s k)).1)) B

theorem sampleError_nonnegative (o : CentralOracle) (E0 : Fraction) (hE : 0 < E0.num) (j : Nat) : 0 ≤
  ↪ (sampleError o E0 hE j).num

theorem cross_contract (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j : Nat) (p q : Point) (hp : o.region p) (hq : o.region
  ↪ q) : Fraction.le (FiniteEstimates.pointDistance (field o E0 hE (j+1) p) (field o E0 hE j q))
  ↪ (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance p q)) (sampleError o E0 hE j))

theorem local_contract (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j : Nat) (p q : Point) (hp : o.region p) (hq : o.region
  ↪ q) : Fraction.le (FiniteEstimates.pointDistance (field o E0 hE j p) (field o E0 hE j q))
  ↪ (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance p q)) (sampleError o E0 hE j))

theorem coarse_velocity (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j k : Nat) (hk : k < blocks j) : Fraction.le (pointNorm
  ↪ (FiniteAccumulation.coarseAt (field o E0 hE j) (duration T (j+1)) s k).2) (velocityCap T B s)

theorem fine_window (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : Conditions o E0 T tau L B s hE) (j : Nat) : TimeCalibration.Window tau (duration T (j+1)) L
  ↪ d.calibration_positive (2*blocks j)

theorem coarse_window (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : Conditions o E0 T tau L B s hE) (j : Nat) : TimeCalibration.Window tau (Fraction.add (duration T
  ↪ (j+1)) (duration T (j+1))) L d.calibration_positive (blocks j)

theorem paired_finite_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j n : Nat) (hn : n ≤ blocks j) : Fraction.le
  ↪ (TimeCalibration.distance tau (BoundedIteration.run (field o E0 hE (j+1)) (duration T (j+1)) s (2*n))
  ↪ (BoundedIteration.run (field o E0 hE j) (duration T j) s n)) (Fraction.add (Fraction.mul
  ↪ (Fraction.ofInt (2*(blocks j : Int))) (CalibratedRefinement.blockSource tau (duration T (j+1)) L
  ↪ (sampleError o E0 hE j) B (velocityCap T B s) d.calibration_positive)) (Fraction.mul (Fraction.ofInt
  ↪ (2*(blocks j : Int))) (Fraction.mul (Fraction.mul tau (Fraction.add (duration T (j+1)) (duration T
  ↪ (j+1))).abs) (sampleError o E0 hE j))))

theorem adjacent_finite_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j : Nat) : Fraction.le (TimeCalibration.distance tau
  ↪ (endpoint o E0 T hE s (j+1)) (endpoint o E0 T hE s j)) (Fraction.add (Fraction.mul (Fraction.ofInt
  ↪ (2*(blocks j : Int))) (CalibratedRefinement.blockSource tau (duration T (j+1)) L (sampleError o E0 hE
  ↪ j) B (velocityCap T B s) d.calibration_positive)) (Fraction.mul (Fraction.ofInt (2*(blocks j : Int)))
  ↪ (Fraction.mul (Fraction.mul tau (Fraction.add (duration T (j+1)) (duration T (j+1))).abs)
  ↪ (sampleError o E0 hE j))))

theorem paired_mesh_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j n : Nat) (hn : n ≤ blocks j) : Fraction.le
  ↪ (TimeCalibration.distance tau (BoundedIteration.run (field o E0 hE (j+1)) (duration T (j+1)) s (2*n))
  ↪ (BoundedIteration.run (field o E0 hE j) (duration T j) s n)) (Fraction.add (Fraction.mul (Fraction.mul
  ↪ T (duration T (j+1))) (CalibratedRefinement.consistencyCoefficient tau T L B (velocityCap T B s)))
  ↪ (Fraction.mul (Fraction.ofInt 7) (Fraction.mul (Fraction.mul tau T) (sampleError o E0 hE j))))

theorem adjacent_mesh_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j : Nat) : Fraction.le (TimeCalibration.distance tau
  ↪ (endpoint o E0 T hE s (j+1)) (endpoint o E0 T hE s j)) (Fraction.add (Fraction.mul (Fraction.mul T
  ↪ (duration T (j+1))) (CalibratedRefinement.consistencyCoefficient tau T L B (velocityCap T B s)))
  ↪ (Fraction.mul (Fraction.ofInt 7) (Fraction.mul (Fraction.mul tau T) (sampleError o E0 hE j))))

theorem weightedCoefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) : 0 ≤ (weightedCoefficient E0 T
  ↪ tau L B s).num

```

```

theorem paired_weighted_tail (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j n : Nat) (hn : n ≤ blocks j) : Fraction.le
  ↪ (TimeCalibration.distance tau (BoundedIteration.run (field o E0 hE (j+1)) (duration T (j+1)) s (2*n))
  ↪ (BoundedIteration.run (field o E0 hE j) (duration T j) s n)) (GeometricTail.tailCap
  ↪ (weightedCoefficient E0 T tau L B s) (j+1))

```

```

theorem adjacent_weighted_tail (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0
  ↪ < E0.num) (d : Conditions o E0 T tau L B s hE) (j : Nat) : Fraction.le (TimeCalibration.distance tau
  ↪ (endpoint o E0 T hE s (j+1)) (endpoint o E0 T hE s j)) (GeometricTail.tailCap (weightedCoefficient E0
  ↪ T tau L B s) (j+1))

```

```

theorem coefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num) (hT : 0 ≤
  ↪ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) : 0 ≤ (coefficient E0 T tau L B s ht).num

```

```

theorem adjacent_tail (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : Conditions o E0 T tau L B s hE) (j : Nat) : Fraction.le (FiniteEstimates.stateDistance (endpoint
  ↪ o E0 T hE s (j+1)) (endpoint o E0 T hE s j)) (GeometricTail.tailCap (coefficient E0 T tau L B s
  ↪ d.calibration_positive) (j+1))

```

```

theorem approximants_converge (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (eps : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ n, N
  ↪ ≤ n → Within (embed (endpoint o E0 T hE s n)) (endpointValue o E0 T tau L B s hE d) eps

```

```

theorem zero_time_endpoint (o : CentralOracle) (E0 T : Fraction) (hE : 0 < E0.num) (s : Point × Point)
  ↪ (hT : T.num = 0) (j : Nat) : stateEquiv (endpoint o E0 T hE s j) s

```

```

theorem zero_time_value (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : Conditions o E0 T tau L B s hE) (hT : T.num = 0) : endpointValue o E0 T tau L B s hE d = embed s

```

NewtonLimitDynamics/Polygon/GeneralForceGrowth.lean — Derive the motion-sample bounds on a finite ball from regional linear force growth and the calibrated window. The ball budget is closed before sampling, using only initial-state magnitudes and force data. Actual and shadow confinement then follow from the shared partial-time invariant. Singular laws use the annular constructor rather than growth at the origin.

```

theorem shadowCap_nonnegative (E0 T tau : Fraction) (s : Point × Point) (hE : 0 < E0.num) (hT : 0 ≤
  ↪ T.num) (ht : 0 < tau.num) : 0 ≤ (shadowCap E0 T tau s ht).num

```

```

theorem field_growth (o : CentralOracle) (E0 T tau L : Fraction) (s : Point × Point) (hE : 0 < E0.num) (d
  ↪ : Data o E0 T tau L s) (j : Nat) (p : Point) (hp : o.region p) : Fraction.le (pointNorm (field o E0
  ↪ hE j p)) (Fraction.add (Fraction.mul L (pointNorm p)) (Fraction.mul (Fraction.ofInt 2) E0))

```

```

theorem forceCap_nonnegative (o : CentralOracle) (E0 T tau L : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Data o E0 T tau L s) : 0 ≤ (forceCap E0 T tau L s d.calibration_positive).num

```

```

theorem field_bound_on_ball (o : CentralOracle) (E0 T tau L : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Data o E0 T tau L s) (j : Nat) (p : Point) (hp : RegionConfinement.Band (Fraction.ofInt
  ↪ 0) (shadowCap E0 T tau s d.calibration_positive) p) : Fraction.le (pointNorm (field o E0 hE j p))
  ↪ (forceCap E0 T tau L s d.calibration_positive)

```

```

theorem run_state_bound (o : CentralOracle) (E0 T tau L : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : Data o E0 T tau L s) (j : Nat) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat) (hn : Fraction.le
  ↪ (BoundedIteration.time h n) T) : Fraction.le (TimeCalibration.norm tau (BoundedIteration.run (field o
  ↪ E0 hE j) h s n)) (CalibratedGrowth.cap tau T (Fraction.mul (Fraction.ofInt 2) E0) s)

```

```

theorem bounded_samples (o : CentralOracle) (E0 T tau L : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : Data o E0 T tau L s) (j : Nat) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat) (hn : Fraction.le
  ↪ (BoundedIteration.time h n) T) : BoundedIteration.BoundedSamples (field o E0 hE j) h s (forceCap E0 T
  ↪ tau L s d.calibration_positive) n

```

NewtonLimitDynamics/Polygon/GeneralForcePathContent.lean — Canonical Cauchy scalar outer content of the actual general matched polygon/curve region. The all-cover cut, enclosure and geometric decay are derived; the choice of initial cover affects no completed content value.

```

theorem D_meshValue_lower_cut (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (q :
  ↪ Fraction) : Below q (D_meshValue o E0 T tau L B s hE d m).val ↔ D_mesh o E0 T tau L B s hE d m q

theorem D_meshValue_nonnegative (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) : Below
  ↪ (Fraction.ofInt 0) (D_meshValue o E0 T tau L B s hE d m).val

theorem D_meshValue_budget_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) : Within
  ↪ (D_meshValue o E0 T tau L B s hE d m).val (embed (scalarState (Fraction.ofInt 0))) (duration
  ↪ (budgetCoefficient E0 T tau L B s d.calibration_positive) m)

theorem D_meshValue_tends_zero (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps : Fraction)
  ↪ (heps : 0 < eps.num) : ∃ N, ∀ m, N ≤ m → Within (D_meshValue o E0 T tau L B s hE d m).val (embed
  ↪ (scalarState (Fraction.ofInt 0))) eps

theorem D_meshValue_independent_cover (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s :
  ↪ Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (c
  ↪ : Cover (Region o E0 T tau L B s hE d m)) : D_meshValue o E0 T tau L B s hE d m = contentValue (Region
  ↪ o E0 T tau L B s hE d m) c

theorem D_meshValue_zero_window (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (hz :
  ↪ T.num=0) : (D_meshValue o E0 T tau L B s hE d m).val = embed (scalarState (Fraction.ofInt 0))

```

NewtonLimitDynamics/Polygon/GeneralForcePathRegion.lean — The actual unsigned region between the sampled central-force polygon and its constructed curve. The shared matched-region geometry retains all cell closures and the final connector. Covers are derived from actual vertex and prefix bounds; no area or enclosure of a supplied curve is a premise.

```

theorem simultaneous_endpoints_square (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B :
  ↪ Fraction) (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s
  ↪ hE) (m : Nat) : CoordinateSquare (cellStart o E0 T s hE m (ticks b m)) (coverRadius o E0 T tau L B s
  ↪ hE d m) (polygonMap o E0 T s hE d.time_nonnegative m (Quotient.mk _ b)) ∧ CoordinateSquare (cellStart
  ↪ o E0 T s hE m (ticks b m)) (coverRadius o E0 T tau L B s hE d m) (gammaPosition o E0 T tau L B s hE d
  ↪ (Quotient.mk _ b))

theorem connector_in_region (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (t :
  ↪ BinaryTime T d.time_nonnegative) (a : Fraction) (ha : UnitInterval a) : Region o E0 T tau L B s hE d
  ↪ m (convexPosition a ha (polygonMap o E0 T s hE d.time_nonnegative m t)) (gammaPosition o E0 T tau L B
  ↪ s hE d t))

theorem final_connector_in_region (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (a :
  ↪ Fraction) (ha : UnitInterval a) : Region o E0 T tau L B s hE d m (convexPosition a ha (polygonMap o
  ↪ E0 T s hE d.time_nonnegative m (rightTime T d.time_nonnegative))) (asPosition (endpointValue o E0 T
  ↪ tau L B s hE d.toConditions)))

theorem budgetCoefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num) (hT
  ↪ : 0 ≤ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) : 0 ≤ (budgetCoefficient E0 T tau L
  ↪ B s ht).num

theorem actual_budget_geometric (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  ↪ Fraction.equiv (actualCover o E0 T tau L B s hE d m).budget (duration (budgetCoefficient E0 T tau L B
  ↪ s d.calibration_positive) m)

theorem D_mesh_nonnegative (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  ↪ (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) : D_mesh o E0 T
  ↪ tau L B s hE d m (Fraction.ofInt 0)

```

```

theorem D_mesh_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE :
  ↪ 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (q : Fraction) (hq :
  ↪ D_mesh o E0 T tau L B s hE d m q) : Fraction.le q (duration (budgetCoefficient E0 T tau L B s
  ↪ d.calibration_positive) m)

```

```

theorem D_mesh_tends_zero (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  ↪ (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps : Fraction) (heps : 0
  ↪ < eps.num) : ∃ N, ∀ m, N ≤ m → ∀ q, D_mesh o E0 T tau L B s hE d m q → Fraction.lt q eps

```

```

theorem budget_zero_window (E0 T tau L B : Fraction) (s : Point × Point) (ht : 0 < tau.num) (m : Nat) (hz
  ↪ : T.num=0) : Fraction.equiv (duration (budgetCoefficient E0 T tau L B s ht) m) (Fraction.ofInt 0)

```

```

theorem D_mesh_zero_window (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  ↪ (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (hz : T.num=0) (q
  ↪ : Fraction) : D_mesh o E0 T tau L B s hE d m q ↔ Fraction.le q (Fraction.ofInt 0)

```

NewtonLimitDynamics/Polygon/GeneralForcePolygonCurve.lean — An actual coarse polygon quotient for the general sampled central-force construction. Shared finite-vertex geometry proves all aliases; the actual velocity cap and prefix tail give a uniform whole-edge curve comparison.

```

theorem edgeCoefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num) (hT :
  ↪ 0 ≤ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) : 0 ≤ (edgeCoefficient E0 T tau L B s
  ↪ ht).num

```

```

theorem edgeRadius_geometric (E0 T tau L B : Fraction) (s : Point × Point) (ht : 0 < tau.num) (m : Nat) :
  ↪ Fraction.equiv (Fraction.add (Fraction.mul (duration T m) (velocityCap T B s)) (GeometricTail.tailCap
  ↪ (GeneralForcePrefix.coefficient E0 T tau L B s ht) m)) (duration (edgeCoefficient E0 T tau L B s ht)
  ↪ m)

```

```

theorem polygon_vertex_bound (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  ↪ (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m :
  ↪ Nat) : Within (polygonMap o E0 T s hE d.time_nonnegative m (Quotient.mk _ b)).val (positionValue
  ↪ (embed (prefixState b o E0 T s hE m))) (Fraction.mul (duration T m) (velocityCap T B s))

```

```

theorem whole_edge_bound (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s :
  ↪ Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  ↪ Within (polygonMap o E0 T s hE d.time_nonnegative m (Quotient.mk _ b)).val (gammaPosition o E0 T tau L
  ↪ B s hE d (Quotient.mk _ b)).val (duration (edgeCoefficient E0 T tau L B s d.calibration_positive) m)

```

```

theorem polygonMap_whole_edge_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point
  ↪ × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (t :
  ↪ BinaryTime T d.time_nonnegative) : Within (polygonMap o E0 T s hE d.time_nonnegative m t).val
  ↪ (gammaPosition o E0 T tau L B s hE d t).val (duration (edgeCoefficient E0 T tau L B s
  ↪ d.calibration_positive) m)

```

```

theorem polygonMap_uniform_convergence (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s :
  ↪ Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps :
  ↪ Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ t : BinaryTime T d.time_nonnegative, Within
  ↪ (polygonMap o E0 T s hE d.time_nonnegative m t).val (gammaPosition o E0 T tau L B s hE d t).val eps

```

```

theorem polygonMap_left (o : ForceClasses.CentralOracle) (E0 T : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (hT : 0 ≤ T.num) (m : Nat) : polygonMap o E0 T s hE hT m (leftTime T hT) = embedPosition s.1

```

```

theorem shared_initial_endpoint (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  ↪ polygonMap o E0 T s hE d.time_nonnegative m (leftTime T d.time_nonnegative) = gammaPosition o E0 T
  ↪ tau L B s hE d (leftTime T d.time_nonnegative)

```

NewtonLimitDynamics/Polygon/GeneralForcePrecision.lean — A monotone mesh-precision choice extracted from an oracle's force error.

```

theorem target_positive (E0 : Fraction) (hE : 0 < E0.num) (j : Nat) : 0 < (GeometricTail.tailCap E0 j).num

```

```

theorem threshold_error (o : Oracle) (E0 : Fraction) (hE : 0 < E0.num) (j n : Nat) (hn : threshold o E0
  ↪ hE j ≤ n) : Fraction.lt (o.error n) (GeometricTail.tailCap E0 j)

```

```
theorem precision_successor (o : Oracle) (E0 : Fraction) (hE : 0 < E0.num) (j : Nat) : precision o E0 hE
  ↪ j ≤ precision o E0 hE (j + 1)
```

```
theorem precision_monotone (o : Oracle) (E0 : Fraction) (hE : 0 < E0.num) {i j : Nat} (hi j : i ≤ j) :
  ↪ precision o E0 hE i ≤ precision o E0 hE j
```

```
theorem precision_threshold (o : Oracle) (E0 : Fraction) (hE : 0 < E0.num) : (j : Nat) → threshold o E0
  ↪ hE j ≤ precision o E0 hE j
```

```
theorem precision_error (o : Oracle) (E0 : Fraction) (hE : 0 < E0.num) (j : Nat) : Fraction.lt (o.error
  ↪ (precision o E0 hE j)) (GeometricTail.tailCap E0 j)
```

NewtonLimitDynamics/Polygon/GeneralForcePrefix.lean — Actual prefixes of one dyadic sampled central-force family. The Cauchy estimate is derived from paired refinement and the actual optional cell. Force bounds are on the represented runs, and no motion-name premise is supplied.

```
theorem actual_samples (d : Conditions o E0 T tau L B s hE) (j : Nat) : BoundedIteration.BoundedSamples
  ↪ (field o E0 hE j) (duration T j) s B (blocks j)
```

```
theorem count_time_le (T : Fraction) (hT : 0 ≤ T.num) (j n : Nat) (hn : n ≤ blocks j) : Fraction.le
  ↪ (BoundedIteration.time (duration T j) n) T
```

```
theorem count_velocity (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j n : Nat) (hn : n ≤ blocks j) : Fraction.le (pointNorm
  ↪ (countState o E0 T s hE j n).2) (velocityCap T B s)
```

```
theorem count_region (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : Conditions o E0 T tau L B s hE) (j n : Nat) (hn : n ≤ blocks j) : o.region (countState o E0 T s
  ↪ hE j n).1
```

```
theorem restarted_comparison (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j n k : Nat) (hn : n+k ≤ blocks j) (i : Nat) (hi : i < k)
  ↪ : Fraction.le (FiniteEstimates.pointDistance (field o E0 hE j (BoundedIteration.run (field o E0 hE j)
  ↪ (duration T j) (countState o E0 T s hE j n) (i+1)).1) (field o E0 hE j (countState o E0 T s hE j
  ↪ n).1)) (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance (BoundedIteration.run (field o
  ↪ E0 hE j) (duration T j) (countState o E0 T s hE j n) (i+1)).1 (countState o E0 T s hE j n).1))
  ↪ (sampleError o E0 hE j))
```

```
theorem speedCap_nonnegative (T tau B : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (ht : 0 < tau.num)
  ↪ (hB : 0 ≤ B.num) : 0 ≤ (speedCap T tau B s).num
```

```
theorem count_step_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : Conditions o E0 T tau L B s hE) (j n : Nat) (hn : n < blocks j) : Fraction.le
  ↪ (TimeCalibration.distance tau (countState o E0 T s hE j (n+1)) (countState o E0 T s hE j n))
  ↪ (Fraction.mul (duration T j) (speedCap T tau B s))
```

```
theorem prefix_next (b : Nat → Bool) (o : CentralOracle) (E0 T : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (j : Nat) : prefixState b o E0 T s hE (j+1) = if b j then countState o E0 T s hE (j+1)
  ↪ (2*ticks b j+1) else countState o E0 T s hE (j+1) (2*ticks b j)
```

```
theorem weightedCoefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) : 0 ≤ (weightedCoefficient E0 T
  ↪ tau L B s).num
```

```
theorem adjacent_weighted_tail (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point
  ↪ × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE) (j : Nat) : Fraction.le
  ↪ (TimeCalibration.distance tau (prefixState b o E0 T s hE (j+1)) (prefixState b o E0 T s hE j))
  ↪ (GeometricTail.tailCap (weightedCoefficient E0 T tau L B s) (j+1))
```

```
theorem coefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num) (hT : 0 ≤
  ↪ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) : 0 ≤ (coefficient E0 T tau L B s ht).num
```

```
theorem adjacent_tail (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  ↪ (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE) (j : Nat) : Fraction.le
  ↪ (FiniteEstimates.stateDistance (prefixState b o E0 T s hE (j+1)) (prefixState b o E0 T s hE j))
  ↪ (GeometricTail.tailCap (coefficient E0 T tau L B s d.calibration_positive) (j+1))
```

```
theorem zero_time_prefix (b : Nat → Bool) (o : CentralOracle) (E0 T : Fraction) (s : Point × Point) (hE :
  ↪ 0 < E0.num) (hT : T.num = 0) (j : Nat) : stateEquiv (prefixState b o E0 T s hE j) s
```

NewtonLimitDynamics/Polygon/GeneralForceQuadraticSecants.lean — The normalized departure from the constructed tangent over a dyadic cell converges to the completed force. Finite quadratic remainders and proved Cauchy-name boundedness remove the actual half-mesh and sampling errors. No Taylor expansion, derivative or desired second-order equation is supplied.

```
theorem node_second_sample_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE :
  ↪ 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m k : Nat) (hk
  ↪ : k+1 ≤ blocks m) (j : Nat) : Fraction.le (FiniteEstimates.pointDistance (QuadraticSecants.secondState
  ↪ (duration T m) hT ((nodeName o E0 T tau L B s hE d m k).approx j) ((nodeName o E0 T tau L B s hE d m
  ↪ (k+1)).approx j)).1 (field o E0 hE (m+j) ((nodeName o E0 T tau L B s hE d m k).approx j).1))
  ↪ (Fraction.add (Fraction.mul (Fraction.ofInt 2) (Fraction.mul L (Fraction.mul (duration T m)
  ↪ (velocityCap T B s)))) (Fraction.add (Fraction.mul (sampleError o E0 hE (m+j)) (Fraction.ofInt 2))
  ↪ (Fraction.mul (duration (Fraction.ofInt 1) j) (pointNorm (field o E0 hE (m+j) ((nodeName o E0 T tau L
  ↪ B s hE d m k).approx j).1))))))
```

```
theorem cell_second_secant_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE :
  ↪ 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m k : Nat) (hk
  ↪ : k+1 ≤ blocks m) : Within (cellSecondSecant o E0 T tau L B s hE d hT m k) (CompletedForce.forceValue o
  ↪ E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))
  ↪ (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))) (Fraction.mul
  ↪ (Fraction.ofInt 2) (Fraction.mul L (Fraction.mul (duration T m) (velocityCap T B s))))
```

```
theorem secondCoefficient_nonnegative (T tau L B : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (ht : 0
  ↪ < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) : 0 ≤ (secondCoefficient T tau L B s ht).num
```

```
theorem bracketing_second_secant_bound (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction) (s
  ↪ : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 <
  ↪ T.num) (m : Nat) : Within (cellSecondSecant o E0 T tau L B s hE d hT m (ticks b m))
  ↪ (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))
  ↪ (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b))) (Fraction.mul (duration T m)
  ↪ (secondCoefficient T tau L B s d.calibration_positive))
```

```
theorem dyadic_second_uniform_identification (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↪ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (eps
  ↪ : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool, Within (cellSecondSecant o
  ↪ E0 T tau L B s hE d hT m (ticks b m)) (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o
  ↪ E0 T tau L B s hE d (Quotient.mk _ b)) (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b))) eps
```

NewtonLimitDynamics/Polygon/GeneralForceSecants.lean — Position secants of the actual constructed central-force map converge to its constructed velocity along the bracketing dyadic cells. The finite remainder is derived and transferred through Cauchy values. This does not assert an unrestricted derivative or identify acceleration with force.

```
theorem node_approx (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat) (hk : k ≤ blocks m) (j : Nat) :
  ↪ (nodeName o E0 T tau L B s hE d m k).approx j = GeneralForcePrefix.countState o E0 T s hE (m+j)
  ↪ (k*blocks j)
```

```
theorem node_value (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num) (d
  ↪ : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat) : realize (nodeName o E0 T tau L B s
  ↪ hE d m k) = GeneralForceTime.gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)
```

```
theorem node_region (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  ↪ (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k j : Nat) : o.region ((nodeName o E0 T
  ↪ tau L B s hE d m k).approx j).1
```

```
theorem node_admissible (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↪ E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat) :
  ↪ SampledValues.Admissible (fun q => o.region q.1) (realize (nodeName o E0 T tau L B s hE d m k))
```

```

theorem cell_secant_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↳ E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m k : Nat) (hk :
  ↳ k+1≤blocks m) : Within (cellSecant o E0 T tau L B s hE d hT m k) (velocityValue
  ↳ (GeneralForceTime.gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)))
  ↳ (Fraction.mul (duration T m) B)

theorem rateCoefficient_nonnegative (T tau B : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (ht : 0 <
  ↳ tau.num) (hB : 0 ≤ B.num) : 0 ≤ (rateCoefficient T tau B s ht).num

theorem bracketing_secant_bound (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point
  ↳ × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  ↳ (m : Nat) : Within (cellSecant o E0 T tau L B s hE d hT m (ticks b m)) (velocityValue
  ↳ (GeneralForceTime.gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))) (Fraction.mul (duration T m)
  ↳ (rateCoefficient T tau B s d.calibration_positive))

theorem dyadic_velocity_uniform_identification (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  ↳ (eps : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N≤m → ∀ b : Nat → Bool, Within (cellSecant o
  ↳ E0 T tau L B s hE d hT m (ticks b m)) (velocityValue (GeneralForceTime.gammaValue o E0 T tau L B s hE
  ↳ d (Quotient.mk _ b))) eps

theorem dyadic_velocity_identification (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction) (s
  ↳ : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 <
  ↳ T.num) (eps : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N≤m → Within (cellSecant o E0 T tau L
  ↳ B s hE d hT m (ticks b m)) (velocityValue (GeneralForceTime.gammaValue o E0 T tau L B s hE d
  ↳ (Quotient.mk _ b))) eps

```

NewtonLimitDynamics/Polygon/GeneralForceTangentTriangle.lean — Leading signed doubled tangent-deflection triangle on the actual constructed sampled Lipschitz central-force curve. Completed determinants transfer the proved second-order position bound, without assuming an area expansion. The triangle is distinct from the matched region and its D_{mesh} .

```

theorem coefficient_nonnegative (T L B : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hL : 0 ≤ L.num)
  ↳ (hB : 0 ≤ B.num) : 0 ≤ (coefficient T L B s).num

theorem cell_triangle_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 <
  ↳ E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m k : Nat) (hk :
  ↳ k+1≤blocks m) : Within (normalizedTriangleValue (duration T m) hT (gammaValue o E0 T tau L B s hE d
  ↳ (nodeTime T d.time_nonnegative m k)) (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative
  ↳ m (k+1)))) (secantValue (Fraction.ofInt 1).half (pairingValue detForm (velocityValue (gammaValue o E0
  ↳ T tau L B s hE d (nodeTime T d.time_nonnegative m k))) (CompletedForce.forceValue o E0 L hE
  ↳ d.lipschitz (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)) (gamma_admissible
  ↳ o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)))) (embed (zeroPoint, zeroPoint)))
  ↳ (Fraction.mul (duration T m) (coefficient T L B s))

theorem normalized_triangles_converge (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  ↳ (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (eps :
  ↳ Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N≤m → ∀ k, k+1≤blocks m → Within
  ↳ (normalizedTriangleValue (duration T m) hT (gammaValue o E0 T tau L B s hE d (nodeTime T
  ↳ d.time_nonnegative m k)) (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m (k+1))))
  ↳ (secantValue (Fraction.ofInt 1).half (pairingValue detForm (velocityValue (gammaValue o E0 T tau L B
  ↳ s hE d (nodeTime T d.time_nonnegative m k))) (CompletedForce.forceValue o E0 L hE d.lipschitz
  ↳ (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)) (gamma_admissible o E0 T tau L
  ↳ B s hE d (nodeTime T d.time_nonnegative m k)))) (embed (zeroPoint, zeroPoint))) eps

```

NewtonLimitDynamics/Polygon/GeneralForceTime.lean — A local general sampled central-force time map, constructed from actual prefixes. Same-grid drift/kick bounds prove representative invariance before the time quotient is lifted. This is a binary-time domain, not a supplied real trajectory or a derivative/ODE identification.

```

theorem stateTimeFactor_nonnegative (T tau B : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (ht : 0 <
  ↳ tau.num) (hB : 0 ≤ B.num) : 0 ≤ (stateTimeFactor T tau B s ht).num

```

```

theorem count_gap (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0
  ↳ < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j n k : Nat) (hnk : n+k ≤ blocks
  ↳ j) : Fraction.le (TimeCalibration.distance tau (countState o E0 T s hE j (n+k)) (countState o E0 T s
  ↳ hE j n)) (Fraction.mul (Fraction.ofInt (k : Int)) (Fraction.mul (duration T j) (speedCap T tau B s)))

theorem count_ordered_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j n k : Nat) (hnk :
  ↳ n+k ≤ blocks j) : Fraction.le (TimeCalibration.distance tau (countState o E0 T s hE j (n+k))
  ↳ (countState o E0 T s hE j n)) (Fraction.mul (durationDifference (countTime T j n) (countTime T j
  ↳ (n+k))).abs (speedCap T tau B s))

theorem count_same_grid_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j m n : Nat) (hm :
  ↳ m ≤ blocks j) (hn : n ≤ blocks j) : Fraction.le (TimeCalibration.distance tau (countState o E0 T s hE
  ↳ j m) (countState o E0 T s hE j n)) (Fraction.mul (durationDifference (countTime T j n) (countTime T j
  ↳ m))).abs (speedCap T tau B s))

theorem prefix_time_bound (b c : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s
  ↳ : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j : Nat)
  ↳ : Fraction.le (distance (prefixState b o E0 T s hE j) (prefixState c o E0 T s hE j)) (Fraction.mul
  ↳ (distance (timeState b T j) (timeState c T j)) (stateTimeFactor T tau B s d.calibration_positive))

theorem address_state_equiv (b c : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  ↳ (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hbc :
  ↳ AddressEquiv T d.time_nonnegative b c) : NameEquiv (prefixName b o E0 T tau L B s hE d) (prefixName c
  ↳ o E0 T tau L B s hE d)

theorem prefix_band (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s :
  ↳ Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j : Nat) :
  ↳ RegionConfinement.Band d.inner_radius d.outer_radius (prefixState b o E0 T s hE j).1

theorem prefix_region (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s :
  ↳ Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j : Nat) :
  ↳ o.region (prefixState b o E0 T s hE j).1

theorem gamma_admissible (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  ↳ (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (x : BinaryTime T
  ↳ d.time_nonnegative) : SampledValues.Admissible (fun q => o.region q.1) (gammaValue o E0 T tau L B s
  ↳ hE d x)

theorem gamma_band (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0
  ↳ < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (x : BinaryTime T
  ↳ d.time_nonnegative) : Within (PositionValues.positionValue (gammaValue o E0 T tau L B s hE d x))
  ↳ (embed (PositionValues.zeroPoint, PositionValues.zeroPoint)) d.outer_radius ∧ ∀ D, Within
  ↳ (PositionValues.positionValue (gammaValue o E0 T tau L B s hE d x)) (embed
  ↳ (PositionValues.zeroPoint, PositionValues.zeroPoint)) D → Fraction.le d.inner_radius D

theorem gamma_conditions_independent (o : ForceClasses.CentralOracle) (E0 T tau L B tau' L' B' :
  ↳ Fraction) (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s
  ↳ hE) (e : GeneralForcePrefix.Conditions o E0 T tau' L' B' s hE) (x : BinaryTime T d.time_nonnegative)
  ↳ : gammaValue o E0 T tau L B s hE d x = gammaValue o E0 T tau' L' B' s hE e x

theorem gammaValue_address (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s
  ↳ : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) :
  ↳ gammaValue o E0 T tau L B s hE d (Quotient.mk _ b) = realize (prefixName b o E0 T tau L B s hE d)

theorem prefix_value_bound (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s
  ↳ : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat)
  ↳ : Within (embed (prefixState b o E0 T s hE m)) (gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))
  ↳ (GeometricTail.tailCap (GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive) m)

theorem prefix_uniform_convergence (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point
  ↳ × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps : Fraction)
  ↳ (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool, Within (embed (prefixState b o E0 T s
  ↳ hE m)) (gammaValue o E0 T tau L B s hE d (Quotient.mk _ b)) eps

```

```

theorem gamma_within (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point) (hE :
  ↳ 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (x y : BinaryTime T
  ↳ d.time_nonnegative) (R : Fraction) (hxy : TimeWithin T d.time_nonnegative x y R) : Within (gammaValue
  ↳ o E0 T tau L B s hE d x) (gammaValue o E0 T tau L B s hE d y) (Fraction.mul R (stateTimeFactor T tau
  ↳ B s d.calibration_positive))

```

```

theorem timeTolerance_positive (T tau B : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (ht : 0 <
  ↳ tau.num) (hB : 0 ≤ B.num) (eps : Fraction) (heps : 0 < eps.num) : 0 < (timeTolerance T tau B s hT ht
  ↳ hB eps).num

```

```

theorem gamma_uniform_continuity (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps : Fraction)
  ↳ (heps : 0 < eps.num) (x y : BinaryTime T d.time_nonnegative) (hxy : TimeWithin T d.time_nonnegative x
  ↳ y (timeTolerance T tau B s d.time_nonnegative d.calibration_positive d.bound_nonnegative eps)) :
  ↳ Within (gammaValue o E0 T tau L B s hE d x) (gammaValue o E0 T tau L B s hE d y) eps.half

```

```

theorem gammaPosition_within (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (x y : BinaryTime T
  ↳ d.time_nonnegative) (R : Fraction) (hxy : TimeWithin T d.time_nonnegative x y R) : Within
  ↳ (gammaPosition o E0 T tau L B s hE d x).val (gammaPosition o E0 T tau L B s hE d y).val (Fraction.mul
  ↳ R (stateTimeFactor T tau B s d.calibration_positive))

```

```

theorem left_endpoint_value (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) : gammaValue o E0 T
  ↳ tau L B s hE d (leftTime T d.time_nonnegative) = embed s

```

```

theorem zero_time_value (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  ↳ (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : T.num = 0) (x :
  ↳ BinaryTime T d.time_nonnegative) : gammaValue o E0 T tau L B s hE d x = embed s

```

```

theorem right_endpoint_value (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction) (s : Point ×
  ↳ Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) : gammaValue o E0 T
  ↳ tau L B s hE d (rightTime T d.time_nonnegative) = endpointValue o E0 T tau L B s hE d.toConditions

```

NewtonLimitDynamics/Polygon/HarmonicAccelerationSecants.lean — The retained harmonic curve's completed velocity secants converge to its completed linear central force, as an instance of the general result.

```

theorem cellAccelerationSecant_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 <
  ↳ E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num) (m k : Nat) :
  ↳ GeneralForceAccelerationSecants.cellAccelerationSecant (harmonicOracle w hw) E0 T (Fraction.ofInt 1)
  ↳ w.abs (HarmonicGeneralEndpoint.harmonicBound w s) s hE (HarmonicGeneralTime.conditions w E0 T s hw hE
  ↳ hT hs) ht m k = cellAccelerationSecant w T s hT hs ht m k

```

```

theorem acceleration_secants_converge (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 <
  ↳ E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num) (eps : Fraction) (heps : 0 <
  ↳ eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool, Within (cellAccelerationSecant w T s hT hs ht m
  ↳ (ticks b m)) (HarmonicCompletedForce.linearValue w (HarmonicTimeRealization.gammaValue w T s hT hs
  ↳ (Quotient.mk _ b))) eps

```

NewtonLimitDynamics/Polygon/HarmonicCompletedForce.lean — The general completed force specializes to the retained linear central law at all completed positions, and its actual polygon force samples converge uniformly along the retained harmonic curve.

```

theorem completed_linear_force (w E0 : Fraction) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hL : LipschitzOn
  ↳ (harmonicOracle w hw).toOracle w.abs) (x : Value) (hx : SampledValues.Admissible (fun q =>
  ↳ (harmonicOracle w hw).region q.1) x) : CompletedForce.forceValue (harmonicOracle w hw) E0 w.abs hE hL
  ↳ x hx = linearValue w x

```

```

theorem retained_curve_force_samples_converge (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE
  ↳ : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (eps : Fraction) (heps : 0 < eps.num) : ∃ N
  ↳ : Nat, ∀ j, N ≤ j → ∀ b : Nat → Bool, Within (embed (accelerationState (linearField w (prefixState b
  ↳ w T s j).1))) (linearValue w (HarmonicTimeRealization.gammaValue w T s hT hs (Quotient.mk _ b))) eps

```

NewtonLimitDynamics/Polygon/HarmonicConstructionAgreement.lean — Agreement of endpoint and global-prefix harmonic names at dyadic times.

```

private theorem factor_nonnegative (w : Fraction) : 0 ≤ (Fraction.add (Fraction.ofInt 1) w.abs).num

```

```

theorem subduration_small (w T : Fraction) (m : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  ↪ DyadicSmallTime w (duration T m)

theorem unit_tail_level (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m j : Nat) (htick : ticks
  ↪ b m = 1) (hz : ∀ i, m ≤ i → b i = false) : stateEquiv (endpoint w (duration T m) s j) (prefixState b w
  ↪ T s (m + j))

theorem unit_tail_names (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m : Nat) (hT : 0 ≤ T.num)
  ↪ (hs : DyadicSmallTime w T) (htick : ticks b m = 1) (hz : ∀ i, m ≤ i → b i = false) : NameEquiv
  ↪ (endpointName w (duration T m) s hT (subduration_small w T m hT hs)) (prefixName b w T s hT hs)

theorem unit_tail_value (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m : Nat) (hT : 0 ≤ T.num)
  ↪ (hs : DyadicSmallTime w T) (htick : ticks b m = 1) (hz : ∀ i, m ≤ i → b i = false) : timeValue w s
  ↪ (duration T m, hT, subduration_small w T m hT hs) = gammaValue w T s hT hs (Quotient.mk _ b)

theorem full_window_value (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) : timeValue w s (T, hT, hs) = gammaValue w T s hT hs (rightTime T hT)

theorem zero_window_value (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (hz : T.num = 0) : timeValue w s (T, hT, hs) = gammaValue w T s hT hs (Quotient.mk
  ↪ _ b)

theorem three_tick_finite_control : Fraction.equiv (distance (endpoint one threeSixteenths testState 0)
  ↪ (prefixState threeQuarterAddress one quarter testState 2)) (426975, 16777216, by decide)

theorem three_tick_finite_identity_false : ¬ stateEquiv (endpoint one threeSixteenths testState 0)
  ↪ (prefixState threeQuarterAddress one quarter testState 2)

```

NewtonLimitDynamics/Polygon/HarmonicDyadicAgreement.lean — E/G agreement at every finite binary dyadic time, from an actual integer-subdivision comparison whose error decreases geometrically.

```

theorem timeApprox_small (b : Nat → Bool) (w T : Fraction) (m : Nat) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) : DyadicSmallTime w (timeApprox b T m)

theorem dyadic_integer_small (b : Nat → Bool) (w T : Fraction) (m j : Nat) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) : FullSmallTime w (integerDuration (duration T (m+j))) (ticks b m) (blocks j)

theorem dyadicIntegerCoefficient_nonnegative (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m :
  ↪ Nat) (hT : 0 ≤ T.num) : 0 ≤ (dyadicIntegerCoefficient b w T s m).num

theorem dyadic_integer_error_cap (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m j : Nat) :
  ↪ Fraction.equiv (Fraction.mul (Fraction.ofInt 2) (Fraction.mul (Fraction.ofInt (blocks j : Int))
  ↪ (blockSource w (duration T (m+j)) (ticks b m) s))) (duration (dyadicIntegerCoefficient b w T s m) j)

theorem dyadic_fine_prefix (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m j : Nat) (hz : ∀ i, m
  ↪ ≤ i → b i = false) : fineBlocks w (duration T (m+j)) (ticks b m) s (blocks j) = prefixState b w T s
  ↪ (m+j)

theorem dyadic_coarse_endpoint (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m j : Nat) :
  ↪ stateEquiv (coarseBlocks w (duration T (m+j)) (ticks b m) s (blocks j)) (endpoint w (timeApprox b T
  ↪ m) s j)

theorem finite_tail_level_error (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m j : Nat) (hT : 0
  ↪ ≤ T.num) (hs : DyadicSmallTime w T) (hk : 0 < ticks b m) (hz : ∀ i, m ≤ i → b i = false) : Fraction.le
  ↪ (distance (endpoint w (timeApprox b T m) s j) (prefixState b w T s (m+j))) (duration
  ↪ (dyadicIntegerCoefficient b w T s m) j)

theorem finite_tail_names (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m : Nat) (hT : 0 ≤ T.num)
  ↪ (hs : DyadicSmallTime w T) (hz : ∀ i, m ≤ i → b i = false) : NameEquiv (endpointName w (timeApprox b
  ↪ T m) s (timeApprox_nonnegative b T m hT) (timeApprox_small b w T m hT hs)) (prefixName b w T s hT hs)

theorem finite_tail_value (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (m : Nat) (hT : 0 ≤
  ↪ T.num) (hs : DyadicSmallTime w T) (hz : ∀ i, m ≤ i → b i = false) : CauchyValues.timeValue w s
  ↪ (timeApprox b T m, timeApprox_nonnegative b T m hT, timeApprox_small b w T m hT hs) = gammaValue w T s
  ↪ hT hs (Quotient.mk _ b)

```

```

theorem dyadicTime_exact (w T : Fraction) (k m : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (hk :
  ↪ k < blocks m) : (dyadicTime w T k m hT hs hk).val = Fraction.mul (Fraction.ofInt (k:Int)) (duration T m)

theorem dyadic_value (w T : Fraction) (s : Point × Point) (k m : Nat) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (hk : k < blocks m) : CauchyValues.timeValue w s (dyadicTime w T k m hT hs hk) =
  ↪ gammaValue w T s hT hs (Quotient.mk _ (finiteAddress m k))

theorem timeValue_equiv_parameter (w : Fraction) (s : Point × Point) (t u : ShortRationalTime w) (ht :
  ↪ Fraction.equiv t.val u.val) : CauchyValues.timeValue w s t = CauchyValues.timeValue w s u

theorem full_dyadic_value_at_represented_time (w T : Fraction) (s : Point × Point) (m : Nat) (hT : 0 ≤
  ↪ T.num) (hs : DyadicSmallTime w T) (t : ShortRationalTime w) (ht : Fraction.equiv t.val (Fraction.mul
  ↪ (Fraction.ofInt (blocks m:Int)) (duration T m))) : CauchyValues.timeValue w s t = gammaValue w T s hT
  ↪ hs (rightTime T hT)

theorem dyadic_value_at_represented_time (w T : Fraction) (s : Point × Point) (k m : Nat) (hT : 0 ≤
  ↪ T.num) (hs : DyadicSmallTime w T) (hk : k < blocks m) (t : ShortRationalTime w) (ht : Fraction.equiv
  ↪ t.val (Fraction.mul (Fraction.ofInt (k:Int)) (duration T m))) : CauchyValues.timeValue w s t =
  ↪ gammaValue w T s hT hs (Quotient.mk _ (finiteAddress m k))

theorem three_tick_value_agreement (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) : CauchyValues.timeValue w s (dyadicTime w T 3 2 hT hs (by decide)) = gammaValue
  ↪ w T s hT hs (Quotient.mk _ (finiteAddress 2 3))

theorem three_tick_address_control : ticks (finiteAddress 2 3) 2 = 3 ∧ finiteAddress 2 3 2 = false ∧
  ↪ finiteAddress 2 3 5 = false

```

NewtonLimitDynamics/Polygon/HarmonicGeneralEndpoint.lean — The harmonic family is an actual instance of the general sampled endpoint construction. Its finite acceleration bounds are derived on the short family, not postulated globally. The old endpoint names and values remain available.

```

theorem harmonic_field (w E0 : Fraction) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (j : Nat) :
  ↪ GeneralForceEndpoint.field (harmonicOracle w hw) E0 hE j = linearField w

theorem harmonicBound_nonnegative (w : Fraction) (s : Point × Point) : 0 ≤ (harmonicBound w s).num

theorem linear_sample_norm (w : Fraction) (p : Point) : Fraction.equiv (pointNorm (linearField w p))
  ↪ (Fraction.mul w.abs (pointNorm p))

theorem linear_sample_bound (w : Fraction) (s : Point × Point) (p : Point) (hp : Fraction.le (pointNorm
  ↪ p) (Fraction.mul (Fraction.ofInt 4) (stateNorm s))) : Fraction.le (pointNorm (linearField w p))
  ↪ (harmonicBound w s)

theorem full_run_state_bound (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime
  ↪ w T) (j k : Nat) (hk : k ≤ blocks j) : Fraction.le (stateNorm (BoundedIteration.run (linearField w)
  ↪ (Fraction.add (duration T (j+1)) (duration T (j+1))) s k)) (Fraction.mul (Fraction.ofInt 2)
  ↪ (stateNorm s))

theorem two_to_four (s : Point × Point) : Fraction.le (Fraction.mul (Fraction.ofInt 2) (stateNorm s))
  ↪ (Fraction.mul (Fraction.ofInt 4) (stateNorm s))

theorem time_le_one (w T : Fraction) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) : Fraction.le T
  ↪ (Fraction.ofInt 1)

theorem shadow_position_bound (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime
  ↪ w T) (j k : Nat) (hk : k < blocks j) : Fraction.le (pointNorm (FiniteEstimates.cell (linearField w)
  ↪ (duration T (j+1)) (FiniteAccumulation.coarseAt (linearField w) (duration T (j+1)) s k)).1)
  ↪ (Fraction.mul (Fraction.ofInt 4) (stateNorm s))

theorem harmonic_band_budget (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime
  ↪ w T) : Fraction.le (Fraction.add (pointNorm s.1) (Fraction.mul T (GeneralForceEndpoint.velocityCap T
  ↪ (harmonicBound w s) s))) (Fraction.mul (Fraction.ofInt 4) (stateNorm s))

theorem harmonic_endpoint_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num)
  ↪ (j : Nat) : GeneralForceEndpoint.endpoint (harmonicOracle w hw) E0 T hE s j = HarmonicDyadic.endpoint
  ↪ w T s j

```

```

theorem harmonic_name_equiv (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num)
  ↳ (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) : CauchyValues.NameEquiv
  ↳ (GeneralForceEndpoint.endpointName (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs (harmonicBound
  ↳ w s) s hE (conditions w E0 T s hw hE hT hs)) (HarmonicDyadic.endpointName w T s hT hs)

```

```

theorem harmonic_value_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT
  ↳ : 0 ≤ T.num) (hs : DyadicSmallTime w T) : GeneralForceEndpoint.endpointValue (harmonicOracle w hw) E0
  ↳ T (Fraction.ofInt 1) w.abs (harmonicBound w s) s hE (conditions w E0 T s hw hE hT hs) =
  ↳ CauchyValues.realize (HarmonicDyadic.endpointName w T s hT hs)

```

NewtonLimitDynamics/Polygon/HarmonicGeneralPathContent.lean — The actual general harmonic matched region equals the retained region. Their scalar contents agree despite different proved cover coefficients.

```

theorem harmonic_region_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT
  ↳ : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) (x : PositionValue) : GeneralForcePathRegion.Region
  ↳ (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs (harmonicBound w s) s hE (conditions w E0 T s hw
  ↳ hE hT hs) m x ↔ HarmonicPathRegion.Region w T s hT hs m x

```

```

theorem harmonic_content_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num)
  ↳ (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) : GeneralForcePathContent.D_meshValue
  ↳ (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs (harmonicBound w s) s hE (conditions w E0 T s hw
  ↳ hE hT hs) m = HarmonicPathContent.D_meshValue w T s hT hs m

```

NewtonLimitDynamics/Polygon/HarmonicGeneralPolygon.lean — Exact equality of the general sampled polygon instance and the retained harmonic polygon map. It requires no curve or small-window premise.

```

theorem harmonic_polygonMap_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num)
  ↳ (hT : 0 ≤ T.num) (m : Nat) (t : BinaryTime.BinaryTime T hT) : GeneralForcePolygonCurve.polygonMap
  ↳ (harmonicOracle w hw) E0 T s hE hT m t = HarmonicPolygonCurve.polygonMap w T s hT m t

```

NewtonLimitDynamics/Polygon/HarmonicGeneralTime.lean — Exact harmonic instance of the constructed general time map. All actual sample bounds are derived on the retained short window. The old harmonic declarations remain available, and no new historical edge is asserted.

```

theorem actual_run_state_bound (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↳ DyadicSmallTime w T) (j k : Nat) (hk : k ≤ blocks j) : Fraction.le (stateNorm (BoundedIteration.run
  ↳ (linearField w) (duration T j) s k)) (Fraction.mul (Fraction.ofInt 2) (stateNorm s))

```

```

theorem harmonic_prefix_eq (b : Nat → Bool) (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE
  ↳ : 0 < E0.num) (j : Nat) : GeneralForcePrefix.prefixState b (harmonicOracle w hw) E0 T s hE j =
  ↳ HarmonicBinaryPrefix.prefixState b w T s j

```

```

theorem harmonic_name_equiv (b : Nat → Bool) (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE
  ↳ : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) : CauchyValues.NameEquiv
  ↳ (GeneralForcePrefix.prefixName b (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs (harmonicBound w
  ↳ s) s hE (conditions w E0 T s hw hE hT hs)) (HarmonicBinaryPrefix.prefixName b w T s hT hs)

```

```

theorem harmonic_value_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT
  ↳ : 0 ≤ T.num) (hs : DyadicSmallTime w T) (x : BinaryTime.BinaryTime T hT) :
  ↳ GeneralForceTime.gammaValue (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs (harmonicBound w s) s
  ↳ hE (conditions w E0 T s hw hE hT hs) x = HarmonicTimeRealization.gammaValue w T s hT hs x

```

```

theorem harmonic_position_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num)
  ↳ (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (x : BinaryTime.BinaryTime T hT) :
  ↳ GeneralForceTime.gammaPosition (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs (harmonicBound w
  ↳ s) s hE (conditions w E0 T s hw hE hT hs) x = PositionValues.gammaPosition w T s hT hs x

```

NewtonLimitDynamics/Polygon/HarmonicIntegerSubdivision.lean — Finite unequal subdivision identities for the actual harmonic end-kick cell.

```

theorem split_position_identity (w a b : Fraction) (s : Point × Point) : pointEquiv (stateSub (cell
  ↳ (linearField w) b (cell (linearField w) a s)) (cell (linearField w) (Fraction.add a b) s)).1
  ↳ (pointScale (Fraction.mul a b) (linearField w (cell (linearField w) a s)).1))

```

```

theorem split_velocity_identity (w a b : Fraction) (s : Point × Point) : pointEquiv (stateSub (cell
  ↪ (linearField w) b (cell (linearField w) a s)) (cell (linearField w) (Fraction.add a b) s)).2
  ↪ (pointAdd (pointScale (negF (Fraction.mul a b)) (linearField w s.2)) (pointScale (Fraction.mul b
  ↪ (Fraction.mul a b)) (linearField w (linearField w (cell (linearField w) a s).1))))))

theorem split_state_sample_bound (w a b : Fraction) (s : Point × Point) : Fraction.le (stateNorm
  ↪ (stateSub (cell (linearField w) b (cell (linearField w) a s)) (cell (linearField w) (Fraction.add a
  ↪ b) s))) (Fraction.add (Fraction.mul (Fraction.mul a b).abs (pointNorm (linearField w (cell
  ↪ (linearField w) a s).1))) (Fraction.add (Fraction.mul (Fraction.mul a b).abs (pointNorm (linearField
  ↪ w s.2))) (Fraction.mul (Fraction.mul b (Fraction.mul a b)).abs (pointNorm (linearField w (linearField
  ↪ w (cell (linearField w) a s).1))))))

theorem linearField_norm (w : Fraction) (p : Point) : Fraction.equiv (pointNorm (linearField w p))
  ↪ (Fraction.mul w.abs (pointNorm p))

theorem split_state_harmonic_bound (w a b : Fraction) (s : Point × Point) : Fraction.le (stateNorm
  ↪ (stateSub (cell (linearField w) b (cell (linearField w) a s)) (cell (linearField w) (Fraction.add a
  ↪ b) s))) (Fraction.add (Fraction.mul (Fraction.mul a b).abs (Fraction.mul w.abs (pointNorm (cell
  ↪ (linearField w) a s).1))) (Fraction.add (Fraction.mul (Fraction.mul a b).abs (Fraction.mul w.abs
  ↪ (pointNorm s.2))) (Fraction.mul (Fraction.mul b (Fraction.mul a b)).abs (Fraction.mul w.abs
  ↪ (Fraction.mul w.abs (pointNorm (cell (linearField w) a s).1))))))

theorem split_state_uniform_bound (w a b M : Fraction) (s : Point × Point) (hb : Fraction.le b.abs
  ↪ (Fraction.ofInt 1)) (hP : Fraction.le (pointNorm (cell (linearField w) a s).1) M) (hV : Fraction.le
  ↪ (pointNorm s.2) M) : Fraction.le (stateNorm (stateSub (cell (linearField w) b (cell (linearField w) a
  ↪ s)) (cell (linearField w) (Fraction.add a b) s))) (uniformSplitBudget w a b M)

theorem first_arrival_le_state (w a : Fraction) (s : Point × Point) (ha : Fraction.le a.abs
  ↪ (Fraction.ofInt 1)) : Fraction.le (pointNorm (cell (linearField w) a s).1) (stateNorm s)

theorem split_state_short_bound (w a b : Fraction) (s : Point × Point) (ha : Fraction.le a.abs
  ↪ (Fraction.ofInt 1)) (hb : Fraction.le b.abs (Fraction.ofInt 1)) : Fraction.le (stateNorm (stateSub
  ↪ (cell (linearField w) b (cell (linearField w) a s)) (cell (linearField w) (Fraction.add a b) s)))
  ↪ (uniformSplitBudget w a b (stateNorm s))

theorem uniformSplitBudget_factor (w a b : Fraction) (s : Point × Point) : Fraction.equiv
  ↪ (uniformSplitBudget w a b (stateNorm s)) (Fraction.mul (Fraction.mul a b).abs (splitFactor w s))

theorem splitFactor_nonnegative (w : Fraction) (s : Point × Point) : 0 ≤ (splitFactor w s).num

theorem integer_duration_product_abs (h : Fraction) (k : Nat) (hh : 0 ≤ h.num) : Fraction.equiv
  ↪ (Fraction.mul (IntegerSchedule.integerDuration h k) h).abs (Fraction.mul (Fraction.ofInt (k : Int))
  ↪ (Fraction.mul h h))

theorem kappaCap_nonnegative (w : Fraction) : 0 ≤ (kappaCap w).num

theorem kappa_le_cap (w h : Fraction) (hh : Fraction.le h.abs (Fraction.ofInt 1)) : Fraction.le (kappa w
  ↪ h) (kappaCap w)

theorem integer_error_step (w h : Fraction) (s : Point × Point) (k : Nat) : Fraction.le (stateNorm
  ↪ (stateSub (integerFine w h s (k + 1)) (integerCoarse w h s (k + 1)))) (Fraction.add (Fraction.mul
  ↪ (kappa w h) (stateNorm (stateSub (integerFine w h s k) (integerCoarse w h s k)))) (integerLocalBudget
  ↪ w h s k))

theorem integer_error_step_quadratic (w h : Fraction) (s : Point × Point) (k : Nat) (hh : 0 ≤ h.num) (ha
  ↪ : Fraction.le (integerDuration h k).abs (Fraction.ofInt 1)) (hb : Fraction.le h.abs (Fraction.ofInt
  ↪ 1)) : Fraction.le (stateNorm (stateSub (integerFine w h s (k + 1)) (integerCoarse w h s (k + 1))))
  ↪ (Fraction.add (Fraction.mul (kappa w h) (stateNorm (stateSub (integerFine w h s k) (integerCoarse w h
  ↪ s k)))) (Fraction.mul (Fraction.mul (Fraction.ofInt (k : Int)) (Fraction.mul h h)) (splitFactor w s)))

theorem integer_error_le_budget (w h : Fraction) (s : Point × Point) : (k : Nat) → Fraction.le (stateNorm
  ↪ (stateSub (integerFine w h s k) (integerCoarse w h s k))) (integerErrorBudget w h s k)

theorem quadraticCap_nonnegative (w : Fraction) (s : Point × Point) : (k : Nat) → 0 ≤ (quadraticCap w s
  ↪ k).num

theorem integer_error_quadratic (w h : Fraction) (s : Point × Point) (hh : 0 ≤ h.num) (hb : Fraction.le
  ↪ h.abs (Fraction.ofInt 1)) : (k : Nat) → (∀ i, i ≤ k → Fraction.le (integerDuration h i).abs
  ↪ (Fraction.ofInt 1)) → Fraction.le (stateNorm (stateSub (integerFine w h s k) (integerCoarse w h s
  ↪ k))) (Fraction.mul (Fraction.mul h h) (quadraticCap w s k))

```

```

theorem integerFine_perturbation (w h : Fraction) (s t : Point × Point) : (k : Nat) → Fraction.le
  ↳ (stateNorm (stateSub (integerFine w h s k) (integerFine w h t k))) (Fraction.mul (fpower (kappa w h)
  ↳ k) (stateNorm (stateSub s t)))

theorem block_error_step (w h : Fraction) (k : Nat) (s : Point × Point) (N : Nat) (hh : 0 ≤ h.num) (hb :
  ↳ Fraction.le h.abs (Fraction.ofInt 1)) (hshort : ∀ i, i ≤ k → Fraction.le (integerDuration h i).abs
  ↳ (Fraction.ofInt 1)) : Fraction.le (stateNorm (stateSub (fineBlocks w h k s (N + 1)) (coarseBlocks w h
  ↳ k s (N + 1)))) (Fraction.add (Fraction.mul (fpower (kappa w h) k) (stateNorm (stateSub (fineBlocks w
  ↳ h k s N) (coarseBlocks w h k s N)))) (Fraction.mul (Fraction.mul h h) (quadraticCap w (coarseBlocks w
  ↳ h k s N) k)))

theorem splitFactor_le (w : Fraction) (s t : Point × Point) (hst : Fraction.le (stateNorm s) (stateNorm
  ↳ t)) : Fraction.le (splitFactor w s) (splitFactor w t)

theorem splitFactor_double (w : Fraction) (s t : Point × Point) (hst : Fraction.le (stateNorm s)
  ↳ (Fraction.mul (Fraction.ofInt 2) (stateNorm t))) : Fraction.le (splitFactor w s) (Fraction.mul
  ↳ (Fraction.ofInt 2) (splitFactor w t))

theorem quadraticCap_double (w : Fraction) (s t : Point × Point) (hst : Fraction.le (stateNorm s)
  ↳ (Fraction.mul (Fraction.ofInt 2) (stateNorm t))) : (k : Nat) → Fraction.le (quadraticCap w s k)
  ↳ (Fraction.mul (Fraction.ofInt 2) (quadraticCap w t k))

theorem coarseBlocks_norm_bound (w h : Fraction) (k : Nat) (s : Point × Point) : (N : Nat) → Fraction.le
  ↳ (stateNorm (coarseBlocks w h k s N)) (Fraction.mul (fpower (kappa w (integerDuration h k)) N)
  ↳ (stateNorm s))

theorem coarseBlocks_state_le_two (w h : Fraction) (k : Nat) (s : Point × Point) (N : Nat) (hpower :
  ↳ Fraction.le (fpower (kappa w (integerDuration h k)) N) (Fraction.ofInt 2)) : (i : Nat) → i ≤ N →
  ↳ Fraction.le (stateNorm (coarseBlocks w h k s i)) (Fraction.mul (Fraction.ofInt 2) (stateNorm s))

theorem blockSource_nonnegative (w h : Fraction) (k : Nat) (s : Point × Point) (hh : 0 ≤ h.num) : 0 ≤
  ↳ (blockSource w h k s).num

theorem block_error_le_budget (w h : Fraction) (k : Nat) (s : Point × Point) (N : Nat) (hh : 0 ≤ h.num)
  ↳ (hb : Fraction.le h.abs (Fraction.ofInt 1)) (hshort : ∀ j, j ≤ k → Fraction.le (integerDuration h
  ↳ j).abs (Fraction.ofInt 1)) (hcoarsePower : Fraction.le (fpower (kappa w (integerDuration h k)) N)
  ↳ (Fraction.ofInt 2)) : (i : Nat) → i ≤ N → Fraction.le (stateNorm (stateSub (fineBlocks w h k s i)
  ↳ (coarseBlocks w h k s i))) (sourceBudget (fpower (kappa w h) k) (blockSource w h k s) i)

theorem kappa_duration_congr (w a b : Fraction) (hab : Fraction.equiv a b) : Fraction.equiv (kappa w a)
  ↳ (kappa w b)

theorem full_power_le_two (w d : Fraction) (N : Nat) (hd : 0 ≤ d.num) (hs : FullSmallTime w d N) :
  ↳ Fraction.le (fpower (kappa w d) N) (Fraction.ofInt 2)

theorem block_power_le_two (w h : Fraction) (k N : Nat) (hh : 0 ≤ h.num) (hs : FullSmallTime w
  ↳ (integerDuration h k) N) : Fraction.le (fpower (fpower (kappa w h) k) N) (Fraction.ofInt 2)

theorem accumulated_integer_error (w h : Fraction) (k N : Nat) (s : Point × Point) (hh : 0 ≤ h.num) (hb :
  ↳ Fraction.le h.abs (Fraction.ofInt 1)) (hshort : ∀ j, j ≤ k → Fraction.le (integerDuration h j).abs
  ↳ (Fraction.ofInt 1)) (hs : FullSmallTime w (integerDuration h k) N) : Fraction.le (stateNorm (stateSub
  ↳ (fineBlocks w h k s N) (coarseBlocks w h k s N))) (Fraction.mul (Fraction.ofInt 2) (Fraction.mul
  ↳ (Fraction.ofInt (N : Int)) (blockSource w h k s)))

theorem full_window_duration_le_one (w d : Fraction) (N : Nat) (hd : 0 ≤ d.num) (hN : 0 < N) (hs :
  ↳ FullSmallTime w d N) : Fraction.le d (Fraction.ofInt 1)

theorem integer_window_short (w h : Fraction) (k N : Nat) (hh : 0 ≤ h.num) (hk : 0 < k) (hN : 0 < N) (hs
  ↳ : FullSmallTime w (integerDuration h k) N) : Fraction.le h.abs (Fraction.ofInt 1) ∧ ∀ j, j ≤ k →
  ↳ Fraction.le (integerDuration h j).abs (Fraction.ofInt 1)

theorem accumulated_integer_error_positive (w h : Fraction) (k N : Nat) (s : Point × Point) (hh : 0 ≤
  ↳ h.num) (hk : 0 < k) (hN : 0 < N) (hs : FullSmallTime w (integerDuration h k) N) : Fraction.le
  ↳ (stateNorm (stateSub (fineBlocks w h k s N) (coarseBlocks w h k s N))) (Fraction.mul (Fraction.ofInt
  ↳ 2) (Fraction.mul (Fraction.ofInt (N : Int)) (blockSource w h k s)))

theorem integerFine_eq_run (w h : Fraction) (s : Point × Point) : (n : Nat) → integerFine w h s n =
  ↳ BoundedIteration.run (linearField w) h s n

```

```
theorem fineBlocks_eq_schedule (w h : Fraction) (k : Nat) (s : Point × Point) : (N : Nat) → fineBlocks w
  ↪ h k s N = integerFine w h s (k*N)
```

```
theorem coarseBlocks_eq_schedule (w h : Fraction) (k : Nat) (s : Point × Point) : (N : Nat) →
  ↪ coarseBlocks w h k s N = integerFine w (integerDuration h k) s N
```

NewtonLimitDynamics/Polygon/HarmonicPathContent.lean — Cauchy scalar outer content of the actual constructed harmonic matched region. The all-cover infimum is represented exactly; the scalar is independent of the initial covering budget. Ordinary Euclidean area and P5 are separate.

```
theorem D_meshValue_lower_cut (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime
  ↪ w T) (m : Nat) (q : Fraction) : Below q (D_meshValue w T s hT hs m).val ↔ D_mesh w T s hT hs m q
```

```
theorem D_meshValue_nonnegative (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) : Below (Fraction.ofInt 0) (D_meshValue w T s hT hs m).val
```

```
theorem D_meshValue_budget_bound (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) : Within (D_meshValue w T s hT hs m).val (embed (scalarState
  ↪ (Fraction.ofInt 0))) (duration (budgetCoefficient w T s) m)
```

```
theorem D_meshValue_tends_zero (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (eps : Fraction) (heps : 0 < eps.num) : ∃ N, ∀ m, N ≤ m → Within (D_meshValue w
  ↪ T s hT hs m).val (embed (scalarState (Fraction.ofInt 0))) eps
```

```
theorem D_meshValue_independent_cover (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) (c : Cover (Region w T s hT hs m)) : D_meshValue w T s hT hs m =
  ↪ contentValue (Region w T s hT hs m) c
```

```
theorem D_meshValue_zero_window (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) (hz : T.num = 0) : (D_meshValue w T s hT hs m).val = embed
  ↪ (scalarState (Fraction.ofInt 0))
```

NewtonLimitDynamics/Polygon/HarmonicPathRegion.lean — The actual nonnegative matched region between the constructed harmonic curve and the coarse polygon. Each cell uses simultaneous polygon/curve positions and all rational convex connectors, then closure in the completed plane. The union counts overlapping and crossing lobes once, without signed cancellation. The final endpoint connector is included in the last cell.

```
theorem edgeRadius_nonnegative (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat) : 0 ≤
  ↪ (edgeRadius w T s m).num
```

```
theorem simultaneous_endpoints_square (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤
  ↪ T.num) (hs : DyadicSmallTime w T) (m : Nat) : CoordinateSquare (cellStart w T s m (ticks b m))
  ↪ (coverRadius w T s hT m) (polygonMap w T s hT m (Quotient.mk _ b)) ∧ CoordinateSquare (cellStart w T
  ↪ s m (ticks b m)) (coverRadius w T s hT m) (gammaPosition w T s hT hs (Quotient.mk _ b))
```

```
theorem cellPatch_square (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  ↪ (m k : Nat) (x : PositionValue) (hx : cellPatch w T s hT hs m k x) : CoordinateSquare (cellStart w T
  ↪ s m k) (coverRadius w T s hT m) x
```

```
theorem closed_cell_square (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) (m k : Nat) (x : PositionValue) (hx : Closure (cellPatch w T s hT hs m k) x) : CoordinateSquare
  ↪ (cellStart w T s m k) (coverRadius w T s hT m) x
```

```
theorem connector_in_region (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) (m : Nat) (t : BinaryTime T hT) (a : Fraction) (ha : UnitInterval a) : Region w T s hT hs m
  ↪ (convexPosition a ha (polygonMap w T s hT m t) (gammaPosition w T s hT hs t))
```

```
theorem reversed_connector_in_region (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) (t : BinaryTime T hT) (a : Fraction) (ha : UnitInterval a) : Region w
  ↪ T s hT hs m (convexPosition a ha (gammaPosition w T s hT hs t) (polygonMap w T s hT m t))
```

```
theorem polygon_in_region (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) (m : Nat) (t : BinaryTime T hT) : Region w T s hT hs m (polygonMap w T s hT m t)
```

```

theorem curve_in_region (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  ↪ (m : Nat) (t : BinaryTime T hT) : Region w T s hT hs m (gammaPosition w T s hT hs t)

theorem shared_initial_endpoint (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) : polygonMap w T s hT m (leftTime T hT) = gammaPosition w T s hT hs
  ↪ (leftTime T hT)

theorem final_connector_in_region (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) (a : Fraction) (ha : UnitInterval a) : Region w T s hT hs m
  ↪ (convexPosition a ha (polygonMap w T s hT m (rightTime T hT)) (asPosition (endpointValue w T s hT
  ↪ hs)))

theorem budgetCoefficient_nonnegative (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) : 0 ≤
  ↪ (budgetCoefficient w T s).num

theorem actual_budget_geometric (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) : Fraction.equiv (actualCover w T s hT hs m).budget (duration
  ↪ (budgetCoefficient w T s) m)

theorem D_mesh_nonnegative (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) (m : Nat) : D_mesh w T s hT hs m (Fraction.ofInt 0)

theorem D_mesh_bound (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m
  ↪ : Nat) (q : Fraction) (hq : D_mesh w T s hT hs m q) : Fraction.le q (duration (budgetCoefficient w T
  ↪ s) m)

theorem D_mesh_tends_zero (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) (eps : Fraction) (heps : 0 < eps.num) : ∃ N, ∀ m, N ≤ m → ∀ q, D_mesh w T s hT hs m q → Fraction.lt
  ↪ q eps

theorem D_mesh_zero_window (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) (m : Nat) (hz : T.num = 0) (q : Fraction) : D_mesh w T s hT hs m q ↔ Fraction.le q (Fraction.ofInt
  ↪ 0)

```

NewtonLimitDynamics/Polygon/HarmonicPolygonCurve.lean — Actual within-cell polygon positions and their distance from the constructed harmonic position value, for every binary address. The affine names use later times in the same cell. No curve or adjacent-error condition is supplied. Address independence includes same-cell and shared-boundary aliases.

```

theorem edgeCoefficient_nonnegative (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) : 0 ≤
  ↪ (edgeCoefficient w T s).num

theorem edgeRadius_geometric (w T : Fraction) (s : Point × Point) (m : Nat) : Fraction.equiv (edgeRadius
  ↪ w T s m) (duration (edgeCoefficient w T s) m)

theorem edgeRadius_eventually_small (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (eps :
  ↪ Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → Fraction.lt (edgeRadius w T s m) eps

theorem polygon_phase_interval (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m j : Nat) : 0 ≤
  ↪ (HarmonicTimeComparison.durationDifference (timeApprox b T m) (timeApprox b T (m+j))).num ∧
  ↪ Fraction.le (HarmonicTimeComparison.durationDifference (timeApprox b T m) (timeApprox b T (m+j)))
  ↪ (duration T m)

theorem polygon_vertex_bound (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) : Within (polygonPosition b w T s hT m).val (positionValue (embed
  ↪ (prefixState b w T s m))) (Fraction.mul (duration T m) (Fraction.mul (Fraction.ofInt 2) (stateNorm
  ↪ s)))

theorem whole_edge_bound (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (m : Nat) : Within (polygonPosition b w T s hT m).val (gammaPosition w T s hT hs
  ↪ (Quotient.mk _ b)).val (edgeRadius w T s m)

theorem uniform_whole_edge_convergence (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (eps : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
  ↪ Within (polygonPosition b w T s hT m).val (gammaPosition w T s hT hs (Quotient.mk _ b)).val eps

```

```

theorem polygon_same_cell_address_independent (b c : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat) (hcell : ticks b m = ticks c m) (htime : AddressEquiv T hT b c) :
  ↪ polygonPosition b w T s hT m = polygonPosition c w T s hT m

theorem polygon_adjacent_address_independent (b c : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat) (hcell : ticks b m + 1 = ticks c m) (htime : AddressEquiv T hT b c) :
  ↪ polygonPosition b w T s hT m = polygonPosition c w T s hT m

theorem polygon_zero_window (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat) (hz : T.num = 0) : polygonPosition b w T s hT m = asPosition (embed (s.1, AffineValues.zeroPoint))

theorem polygon_address_independent (b c : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat) (htime : AddressEquiv T hT b c) : polygonPosition b w T s hT m = polygonPosition c w T s hT m

theorem polygonMap_whole_edge_bound (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) (t : BinaryTime T hT) : Within (polygonMap w T s hT m t).val (gammaPosition w T s hT hs t).val (edgeRadius w T s m)

theorem polygonMap_uniform_convergence (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (eps : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ t : BinaryTime T hT, Within (polygonMap w T s hT m t).val (gammaPosition w T s hT hs t).val eps

theorem half_time_polygon_alias (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat) :
  ↪ polygonPosition firstAlias w T s hT m = polygonPosition secondAlias w T s hT m

theorem half_time_distinct_coarse_cells : ticks firstAlias 1 = 1 ∧ ticks secondAlias 1 = 0

theorem polygonMap_left (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat) : polygonMap w T s hT m (leftTime T hT) = embedPosition s.1

```

NewtonLimitDynamics/Polygon/HarmonicQuadraticSecants.lean — The retained harmonic curve inherits the constructed normalized second-order position departure criterion, with all sample bounds derived.

```

theorem second_secants_converge (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num) (eps : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool, Within (QuadraticSecants.secondValue (duration T m) ht (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (ticks b m)))) (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (ticks b m + 1)))) (HarmonicCompletedForce.linearValue w (HarmonicTimeRealization.gammaValue w T s hT hs (Quotient.mk _ b))) eps

```

NewtonLimitDynamics/Polygon/HarmonicSecants.lean — The retained harmonic curve's position-secants result is a corollary of the actual general central-force construction. No derivative primitive or new historical dependency is used.

```

theorem cellSecant_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num) (m k : Nat) : GeneralForceSecants.cellSecant (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs (HarmonicGeneralEndpoint.harmonicBound w s) s hE (HarmonicGeneralTime.conditions w E0 T s hw hE hT hs) ht m k = cellSecant w T s hT hs ht m k

theorem velocity_secants_converge (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num) (eps : Fraction) (heps : 0 < eps.num) : ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool, Within (cellSecant w T s hT hs ht m (ticks b m)) (velocityValue (HarmonicTimeRealization.gammaValue w T s hT hs (Quotient.mk _ b))) eps

```

NewtonLimitDynamics/Polygon/ParallelQuadraticEndpoint.lean — Actual centre-at-infinity polygon endpoints converge to the explicit quadratic state at every nonnegative rational time. Cauchy names are derived from the finite half-mesh error, with no supplied curve or small window.

```

theorem coefficient_nonnegative (a : Point) (T : Fraction) (hT : 0 ≤ T.num) : 0 ≤ (coefficient a T).num

theorem full_elapsed (T : Fraction) (j : Nat) : Fraction.equiv (time (duration T j) (blocks j)) T

```

```
theorem endpoint_error (a : Point) (T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (j : Nat) :
  ↪ Fraction.equiv (distance (endpoint a T s j) (quadraticState a T s)) (duration (coefficient a T) j)
```

```
theorem endpointValue_eq (a : Point) (T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) : endpointValue
  ↪ a T s hT = embed (quadraticState a T s)
```

NewtonLimitDynamics/Polygon/PositionValues.lean — Planar positions of the constructed harmonic state values. All magnitudes are coordinate L1 diagnostics. Closed coordinate squares below are point sets in this quotient plane, not state-space regions or supplied Euclidean area.

```
theorem gammaPosition_within (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime
  ↪ w T) (x y : BinaryTime T hT) (R : Fraction) (hR : 0 ≤ R.num) (hxy : TimeWithin T hT x y R) : Within
  ↪ (gammaPosition w T s hT hs x).val (gammaPosition w T s hT hs y).val (Fraction.mul R (stateTimeFactor
  ↪ w s))
```

```
theorem gammaPosition_uniform_continuity (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs :
  ↪ DyadicSmallTime w T) (eps : Fraction) (heps : 0 < eps.num) (x y : BinaryTime T hT) (hxy : TimeWithin
  ↪ T hT x y (timeTolerance w s eps)) : Within (gammaPosition w T s hT hs x).val (gammaPosition w T s hT
  ↪ hs y).val eps.half
```

```
theorem gammaPosition_alias (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) : gammaPosition w T s hT hs (Quotient.mk _ firstAlias) = gammaPosition w T s hT hs (Quotient.mk _
  ↪ secondAlias)
```

```
theorem gammaPosition_left (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) : gammaPosition w T s hT hs (leftTime T hT) = embedPosition s.l
```

```
theorem gammaPosition_right (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w
  ↪ T) : gammaPosition w T s hT hs (rightTime T hT) = asPosition (endpointValue w T s hT hs)
```

```
theorem gammaPosition_zero_time (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  ↪ (hs : DyadicSmallTime w T) (hzero : T.num = 0) : gammaPosition w T s hT hs (Quotient.mk _ b) =
  ↪ embedPosition s.l
```

```
theorem gammaPosition_zero_state_norm (b : Nat → Bool) (w T : Fraction) (s : Point × Point) (hT : 0 ≤
  ↪ T.num) (hs : DyadicSmallTime w T) (hzero : (stateNorm s).num = 0) : gammaPosition w T s hT hs
  ↪ (Quotient.mk _ b) = embedPosition s.l
```

```
theorem sample_corner_in_square : CoordinateSquare sampleCentre sampleRadius (embedPosition sampleCorner)
```

```
theorem sample_corner_L1_distance_two : Fraction.equiv (pointNorm (pointSub sampleCorner sampleCentre))
  ↪ (Fraction.ofInt 2)
```

```
theorem sample_position_level_one : Fraction.equiv (distance (positionState (endpoint sampleOne
  ↪ sampleQuarter sampleState 1)) (positionState sampleState)) (135, 512, by decide)
```

```
theorem sample_tail_one : Fraction.equiv (HarmonicDyadic.tailCap sampleOne sampleQuarter sampleState 1)
  ↪ sampleTail
```

```
theorem sample_position_lower_all_levels (j : Nat) (hj : 1 ≤ j) : Fraction.le sampleLower (distance
  ↪ (positionState (endpoint sampleOne sampleQuarter sampleState j)) (positionState sampleState))
```

```
theorem sample_endpoint_position_ne_initial : positionValue (endpointValue sampleOne sampleQuarter
  ↪ sampleState (by decide) (by unfold DyadicSmallTime Fraction.le; decide)) ≠ embed (positionState
  ↪ sampleState)
```

```
theorem sample_gammaPosition_right_ne_left : gammaPosition sampleOne sampleQuarter sampleState (by
  ↪ decide) (by unfold DyadicSmallTime Fraction.le; decide) (rightTime sampleQuarter (by decide)) ≠
  ↪ gammaPosition sampleOne sampleQuarter sampleState (by decide) (by unfold DyadicSmallTime Fraction.le;
  ↪ decide) (leftTime sampleQuarter (by decide))
```

NewtonLimitDynamics/Polygon/RegionConfinement.lean — Finite regional confinement before force sampling. One partial-time invariant controls actual runs and restarted shadow cells. The only force bound is on the named region; no whole-plane regularity, supplied motion or completed quantity is used. Kepler sampling and localization of the existing completion are separate remaining parts of handoff A.6.

```

theorem velocityCap_nonnegative (T B : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (hB : 0 ≤ B.num) :
  ↪ 0 ≤ (velocityCap T B s).num

theorem duration_le_window (T : Fraction) (hT : 0 ≤ T.num) (j : Nat) : Fraction.le (duration T j) T

theorem fine_time (T : Fraction) (j : Nat) : Fraction.equiv (BoundedIteration.time (duration T (j+1))
  ↪ (2*blocks j)) T

theorem coarse_time (T : Fraction) (j : Nat) : Fraction.equiv (BoundedIteration.time (Fraction.add
  ↪ (duration T (j+1)) (duration T (j+1))) (blocks j)) T

theorem initial_invariant (T B : Fraction) (s0 : Point × Point) : Invariant T B s0 (Fraction.ofInt 0) s0

theorem band_of_bounds (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) (d : Frame
  ↪ region T B r R s0) (p v : Point) (hp : Fraction.le (pointNorm p) R) (hv : Fraction.le (pointNorm v)
  ↪ (velocityCap T B s0)) (hm : Fraction.equiv (det p v) (CentralSchedule.momentum s0)) : Band r R p

theorem invariant_band (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) (d : Frame
  ↪ region T B r R s0) (t : Fraction) (q : Point × Point) (ht : Fraction.le t T) (hq : Invariant T B s0 t
  ↪ q) : Band r R q.1

theorem arrival_band (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) (d : Frame region T
  ↪ B r R s0) (a : Point → Point) (t h : Fraction) (q : Point × Point) (hh : 0 ≤ h.num) (ht : Fraction.le
  ↪ (Fraction.add t h) T) (hq : Invariant T B s0 t q) : Band r R (FiniteEstimates.cell a h q).1

theorem advance (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) (d : Frame region T B r
  ↪ R s0) (a : Point → Point) (hc : CentralSchedule.central a) (hb : ∀ p, Band r R p → Fraction.le
  ↪ (pointNorm (a p)) B) (t h : Fraction) (q : Point × Point) (hh : 0 ≤ h.num) (ht : Fraction.le
  ↪ (Fraction.add t h) T) (hq : Invariant T B s0 t q) : Invariant T B s0 (Fraction.add t h)
  ↪ (FiniteEstimates.cell a h q)

theorem run_invariant (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) (d : Frame region
  ↪ T B r R s0) (a : Point → Point) (hc : CentralSchedule.central a) (hb : ∀ p, Band r R p → Fraction.le
  ↪ (pointNorm (a p)) B) (h : Fraction) (hh : 0 ≤ h.num) : (n : Nat) → Fraction.le (BoundedIteration.time
  ↪ h n) T → Invariant T B s0 (BoundedIteration.time h n) (BoundedIteration.run a h s0 n)

theorem run_band (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) (d : Frame region T B
  ↪ r R s0) (a : Point → Point) (hc : CentralSchedule.central a) (hb : ∀ p, Band r R p → Fraction.le
  ↪ (pointNorm (a p)) B) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat) (hn : Fraction.le
  ↪ (BoundedIteration.time h n) T) : Band r R (BoundedIteration.run a h s0 n).1

theorem run_bounded_samples (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) (d : Frame
  ↪ region T B r R s0) (a : Point → Point) (hc : CentralSchedule.central a) (hb : ∀ p, Band r R p →
  ↪ Fraction.le (pointNorm (a p)) B) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat) (hn : Fraction.le
  ↪ (BoundedIteration.time h n) T) : BoundedIteration.BoundedSamples a h s0 B n

theorem shadow_bands (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) (d : Frame region
  ↪ T B r R s0) (a : Point → Point) (hc : CentralSchedule.central a) (hb : ∀ p, Band r R p → Fraction.le
  ↪ (pointNorm (a p)) B) (m k : Nat) (hk : k < HarmonicDyadic.blocks m) : Band r R (FiniteEstimates.cell
  ↪ a (HarmonicDyadic.duration T (m+1)) (FiniteAccumulation.coarseAt a (HarmonicDyadic.duration T (m+1))
  ↪ s0 k)).1 ∧ Band r R (FiniteEstimates.twoHalf a (HarmonicDyadic.duration T (m+1))
  ↪ (FiniteAccumulation.coarseAt a (HarmonicDyadic.duration T (m+1)) s0 k)).1

theorem sampled_cell_comparison (o : CentralOracle) (T B r R : Fraction) (s0 : Point × Point) (d : Frame
  ↪ o.region T B r R s0) (L tau : Fraction) (hL : Lipschitz0n o.toOracle L) (htau : 0 < tau.num) (i j :
  ↪ Nat) (hij : i ≤ j) (t u h : Fraction) (q z : Point × Point) (hh : 0 ≤ h.num) (ht : Fraction.le
  ↪ (Fraction.add t h) T) (hu : Fraction.le (Fraction.add u h) T) (hq : Invariant T B s0 t q) (hz :
  ↪ Invariant T B s0 u z) : Fraction.le (TimeCalibration.distance tau (FiniteEstimates.cell (o.sample i)
  ↪ h q) (FiniteEstimates.cell (o.sample j) h z)) (Fraction.add (Fraction.mul
  ↪ (TimeCalibration.amplification tau h L htau) (TimeCalibration.distance tau q z)) (Fraction.mul
  ↪ (Fraction.mul tau h.abs) (Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i))))

```

```

theorem sampled_refinement_bound (o : CentralOracle) (T B r R : Fraction) (s0 : Point × Point) (d : Frame
  ↳ o.region T B r R s0) (L tau : Fraction) (hL : LipschitzOn o.toOracle L) (htau : 0 < tau.num) (i j :
  ↳ Nat) (hij : i ≤ j) (hbi : ∀ p, Band r R p → Fraction.le (pointNorm (o.sample i p)) B) (hbj : ∀ p,
  ↳ Band r R p → Fraction.le (pointNorm (o.sample j p)) B) (m n : Nat) (hn : n ≤ HarmonicDyadic.blocks m)
  ↳ (hs : TimeCalibration.Window tau (HarmonicDyadic.duration T (m+1)) L htau (2*n)) : Fraction.le
  ↳ (TimeCalibration.distance tau (FiniteAccumulation.fineAt (o.sample j) (HarmonicDyadic.duration T
  ↳ (m+1)) s0 n) (FiniteAccumulation.coarseAt (o.sample i) (HarmonicDyadic.duration T (m+1)) s0 n))
  ↳ (Fraction.mul (Fraction.ofInt (2*(n : Int))) (CalibratedRefinement.blockSource tau
  ↳ (HarmonicDyadic.duration T (m+1)) L (Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i))
  ↳ B (velocityCap T B s0) htau))

theorem sampled_equivalent_duration_bound (o : CentralOracle) (T B r R : Fraction) (s0 : Point × Point) (f
  ↳ : Frame o.region T B r R s0) (L tau : Fraction) (hL : LipschitzOn o.toOracle L) (htau : 0 < tau.num)
  ↳ (i j : Nat) (hij : i ≤ j) (hbi : ∀ p, Band r R p → Fraction.le (pointNorm (o.sample i p)) B) (hbj : ∀
  ↳ p, Band r R p → Fraction.le (pointNorm (o.sample j p)) B) (d e : Fraction) (hde : Fraction.equiv d e)
  ↳ (hd : 0 ≤ d.num) (he : 0 ≤ e.num) (n : Nat) (hdt : Fraction.le (BoundedIteration.time d n) T) (het :
  ↳ Fraction.le (BoundedIteration.time e n) T) (hs : TimeCalibration.Window tau d L htau n) : Fraction.le
  ↳ (TimeCalibration.distance tau (BoundedIteration.run (o.sample i) d s0 n) (BoundedIteration.run
  ↳ (o.sample j) e s0 n)) (Fraction.mul (Fraction.ofInt (2*(n : Int))) (Fraction.mul (Fraction.mul tau
  ↳ d.abs) (Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i))))

```